EXPERT INSIGHT

AI Agents on AWS

Beginner's guide to building AI agents on AWS

Bunny Kaushik | Mona M

packt

OceanofPDF.com

AI Agents on AWS

Beginner's guide to building AI agents on AWS

Bunny Kaushik

Mona M

‹packt›

AI Agents on AWS

Copyright © 2026 Packt Publishing

All rights reserved . No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author nor Packt Publishing, or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

This book was written by Mona M and Bunny Kaushik. Generative AI tools were used only to assist with ideation, phrasing, and diagram drafts, and all technical content and code were created, verified, and tested by the author and Packt's editorial team. Packt does not accept AI-generated content that replaces expert authorship.

Portfolio Director: Gebin George
Relationship Lead: Sonia Chauhan
Project Manager: Prajakta Naik
Content Engineer: Afzal Shaikh
Technical Editor: Sumant Jadhav
Indexer: Manju Arasan
Production Designer: Deepak Chavan
Growth Lead: Nimisha Dua
First published: May 2026
Production reference: 1280426
Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square

Birmingham
B3 1RB, UK
ISBN 978-1-80638-721-2
www.packtpub.com

Contributors

About the authors

Bunny Kaushik is a Solutions Architect at AWS and an AI/ML specialist who works with the world's leading financial institutions to build scalable AI systems that deliver measurable business impact. He focuses on AI strategy, governance, and enterprise adoption, helping organizations transition from isolated prototypes to production-ready systems. He is also a published author, speaker, and active mentor to startups and professionals. He specializes in practical AI implementation, helping the next generation of builders navigate the transition from theory to production.

Mona is a Senior AI/ML Specialist Solutions Architect at AWS with over 15 years of experience spanning AI/ML, cloud, and software engineering. She specializes in designing and scaling production-grade AI systems, including LLMs, fine-tuning, inference optimization, and enterprise AI architecture across AWS and Google Cloud. Mona is the author of two bestselling books and also a published speaker and researcher who has contributed to leading industry conferences and academic work. She actively mentors professionals in AI and cloud.

About the reviewer

Sireesha Muppala, Ph.D. , is a visionary technology leader with over 25 years of experience driving digital transformation and unlocking the potential of cutting-edge innovations. As a Senior Solutions Architecture Leader at Amazon Web Services (AWS), Sireesha spearheads efforts to empower businesses with the transformative power of generative AI and other emerging technologies. She currently leads a team of dedicated technologists at AWS, focused on partnering with leading organizations in the Automotive and Manufacturing sectors to unlock the full potential of generative AI and other cutting-edge cloud-based technologies. Prior to AWS, she held leadership roles at organizations of all sizes across diverse industries, including Oracle, Blackhawk Network, and Primer AI.

Vishnu Elangovan is a Worldwide Agentic Specialist Solutions Architect at AWS, specializing in AI inference, agentic AI distributed systems, and enterprise generative AI applications. With over a decade of experience spanning traditional machine learning and generative AI, he brings deep technical expertise to helping organizations build and scale intelligent systems. Vishnu is a frequent speaker at leading AI conferences, a published thought leader across multiple platforms, and an invited book reviewer. He regularly authors AWS blogs, sharing practical insights on emerging AI architectures and patterns. Outside of work, he enjoys traveling, hiking, outdoor sports, and discovering family-owned coffee shops and wineries.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns— *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at `https://packt.link/8Oz6Y` or scan the QR code below.

OceanofPDF.com

Table of Contents

Preface

Free benefits with your book

Chapter 1: Understanding AI Agents on AWS

What is RAG and how does it work?

What are agents?

- Agent capabilities •

Exploring the AWS Agentic Stack

Building AI agents on AWS

Progressing from LLMs to multi-agent systems

Understanding agent protocols

- Model Context Protocol •
- Agent-to-Agent Protocol •

Introducing AWS Strands SDK

Building a "Hello World" agent

- Setting up your environment in Amazon SageMaker studio •
- Creating a simple agent •

Summary

Chapter 2: Building Agents with Tools

Technical requirements

Why do agents need tools?

- From function to tool •

When one tool isn't enough •
Building your first tool-enabled agent
The Strands Agents tool ecosystem
Pre-built community tools •
What's in the toolbox? •
Using multiple tools together •
AWS integration •
Building custom tools
When simple tools aren't enough •
The database connection problem •
Slow warehouse checks •
Choosing your approach •
Quick start with Strands Agent Builder •
Building agents on LangChain •
Summary

Chapter 3: Agent Memory

Prerequisites
What is agent memory
Types of agent memory •
Challenges with implementing agent memory •
Understanding agent workflows with memory •
Implementing memory systems with AWS services
AI agent/LLM •
Short-term memory (context) •
Structured long-term memory •
Semantic long-term memory (the RAG layer) •

Durable storage •

Memory orchestration •

Building a personalized agent with memory

Context management and memory optimization for agent memory

Memory optimization •

Introducing Amazon Bedrock AgentCore Memory

Architectural overview of Amazon Bedrock AgentCore Memory

Understanding Amazon Bedrock AgentCore short-term memory

Memory strategies, namespaces, and long-term retrieval in AgentCore Memory

Namespaces •

Memory strategies •

Memory extraction module •

Bringing it all together •

Hands-on lab: Using AgentCore Memory with LangGraph

How the integration works •

Run the agent •

Testing memory persistence •

Summary

Chapter 4: Advanced Agent Architecture Patterns

Technical requirements

When one agent isn't enough

The problem with single agents •

The multi-agent solution •

What are multi-agent patterns?

Your first multi-agent pattern: the supervisor-worker •

The trade-offs of the supervisor-worker pattern •
Make agents reusable: agent as a tool •
How data flows through multi-agent systems •
Understanding communication techniques
Sequential communication •
Parallel communication •
Collaborative communication •
Common orchestration patterns
Swarm patterns •
Graph pattern •
When multi-agent is not the answer
Summary

Chapter 5: Agent Communication

Technical requirements
Model Context Protocol
Why is a standardized protocol needed for tools? •
What problem does MCP solve? •
Understanding MCP in action •
Differnce between host and MCP client •
Managing lifecycle between client and server •
Understanding MCP server features •
Tools •
Resources •
Resource template •
Prompt templates •
Understanding MCP client features •

Sampling •

Roots •

Elicitation •

Building and using MCP servers •

Using pre-built MCP servers •

Building your own or custom MCP Server •

Hands-on example of building an MCP server and connecting to an MCP client •

Server.py code •

Client.py code •

Agent2Agent protocol

Understanding the difference between MCP vs A2A •

Why is a standardized protocol needed for agents? •

Understanding A2A by an example of a multi-agent system •

How A2A architecture works •

Understanding A2A data model •

Communication styles in A2A •

How to implement A2A •

Building a multi-agent system with A2A and MCP

Weather Agent Server •

Flights Agent Server •

Travel Orchestrator •

Summary

Chapter 6: Production Deployment and Enterprise Integration

Technical requirements

What actually changes when you go to production

Deployment path

AWS Lambda: Serverless and event-driven •

Amazon ECS: Always-on and container-driven •

From Lambda handler to web server •

What's actually happening on AWS •

When to reach for Amazon ECS •

Amazon Bedrock AgentCore

The AgentCore architecture •

AgentCore runtime •

Deployment walkthrough •

How runtime compares to Lambda and ECS •

AgentCore Gateway •

Targets •

Semantic search •

Authentication: Inbound and outbound •

AgentCore Identity •

Enterprise integration and industry use cases

Financial services: Fraud detection at transaction speed •

Healthcare: Multi-specialty surgical scheduling •

Manufacturing: Predictive maintenance on the factory floor •

E-Commerce: Intelligent customer support with specialized agents •

Summary

Chapter 7: Evaluation, Observability, and AI Governance

Technical requirements

Agent evaluation

Why agent evaluation matters? •

- Agent evaluation use-case •
- Relationship between agent observability and evaluation •
- Evaluation metrics for evaluators •
  - *Understating types of agent evaluation •*
  - *Agent evaluation at a glance •*

AgentCore Evaluations providing a managed service for evaluations

- Collecting traces for Agent evaluation via AgentCore Observability •
- Agentcore evaluation levels •
- Hands-on using AgentCore evaluations to evaluate travel assistant •
  - *Prerequisites: •*
  - *Step 1: Setup •*
  - *Step 2: Understand the agent (agent_app.py) •*
  - *Step 3: Deploy the agent •*
  - *Step 4: Create a custom evaluator •*
  - *Step 5: Define test prompts and invoke the agent •*
  - *Step 6: Wait for traces •*
  - *Step 7: Run evaluations (3 passes) •*
  - *Step 8: Summary table •*
  - *Step 9: Save results •*
  - *Key insight •*

Observability: Seeing what your agents do

- What you need to capture •
- The OpenTelemetry Foundation •
  - *Seeing traces locally with Jaeger •*
  - *Langfuse •*
- LangSmith: Observability from the LangChain Ecosystem •
  - *Connecting Strands to LangSmith •*

What LangSmith and Langfuse Offer •

AgentCore Observability

What gets captured •

Exporting to external platforms •

Putting it all together •

Observability anti-patterns •

AI Governance: Keeping Agents Within Bounds

Guardrails: Filtering model outputs •

Applying guardrails to a Strands agent •

AgentCore Policy: Deterministic control over tool calls •

How Cedar policies work •

Policies based on user identity •

Guardrails and policy •

Organizational AI governance: The human layer •

Audit trails •

Incident response for AI systems •

Before you go to production •

Summary

Chapter 8: Unlock Your Exclusive Benefits

Unlock this book's free benefits in 3 easy steps

Other Books You May Enjoy

Index

OceanofPDF.com

Preface

This book is a high-level, practical guide to building AI agents on AWS, taking you from core concepts to production-ready systems. It introduces the foundations of agentic AI and explains how autonomous agents, LLM agents, and intelligent agents think, reason, act, and collaborate. You will learn how to design single-agent systems, extend them with tool calling, function calling, and agent memory, and build more capable applications using Amazon Bedrock, Amazon Bedrock AgentCore, the Strands Agents SDK, and LangChain.

As the book progresses, it explores how to create stateful agents using Amazon Bedrock AgentCore Memory, enabling more personalized and context-aware experiences. It then expands into multi-agent systems, agent orchestration, and agentic workflows, showing how specialized agents can coordinate to solve complex tasks. You will also learn how emerging interoperability standards such as MCP (Model Context Protocol) and A2A (Agent-to-Agent) support tool integration, agent communication, and more modular, connected agent ecosystems.

The book also covers the essential production topics needed to move beyond prototypes. You will understand how to deploy and scale agents with Amazon Bedrock AgentCore and how to improve reliability through agent evaluation, observability, monitoring, governance, and guardrails. This book provides a strong, high-level understanding of how to design, deploy, and operationalize production-ready AI agents on AWS using modern architectures and real-world best practices.

By the end of this book, you will be able to design, build, and deploy intelligent, autonomous AI systems that operate reliably at scale on AWS.

Who this book is for

This book is for AI engineers, machine learning engineers, cloud architects, software engineers, and developers who want to build and deploy autonomous AI systems on AWS. It is also well suited for graduate and undergraduate students who want to enter the agentic AI space. Familiarity with Python, APIs, and basic AI/ML concepts is recommended, but the book is designed to make agentic AI accessible to readers who are still building their foundation.

What this book covers

Chapter 1 , *Understanding AI Agents on AWS* , lays the foundational groundwork for understanding AI agents on AWS. By deconstructing the evolution of agentic intelligence, it helps you see how systems progress from basic LLMs to sophisticated multi-agent collaborations. It also explores the future of interoperability through emerging protocols like MCP and A2A, which enable agents to share data and collaborate seamlessly across platforms.

Chapter 2 , *Building Agents with Tools* , shows you how to turn conversational agents into systems that actually do things. You will start by transforming simple Python functions into agent tools, then explore ready-made tools that can save you days of development time, and finally build custom tools for the unique problems your business faces.

Chapter 3 , *Agent Memory* , helps you build multi-agent systems that actually work in production. You will learn to use AWS services like AgentCore Memory and Mem0, and implement different types of memory, including short-term and long-term memory. It also covers context management strategies for building personalized agents that remember user preferences across sessions.

Chapter 4 , *Advanced Agent Architecture Patterns* , shows how agents can be orchestrated within multi-agent systems. It explores collaborative agent architectures and design patterns, where multiple agents work together to solve complex tasks

Chapter 5 , *Agent Communication* , introduces two important protocols widely used for agent communication: MCP and A2A. You'll learn how to standardize these communication protocols and make agentic systems easier to build, extend, and scale.

Chapter 6 , *Production Deployment and Enterprise Integration* , gives practical guidance on deploying agents into enterprise environments. It provides you enough hands-on examples to get your agents running in production. By the end of this chapter, you'll have a clear understanding of choosing the right deployment path and the patterns for connecting your agents to enterprise databases, APIs, and event-driven systems

Chapter 7 , *Evaluation, Observability, and AI Governance* , teaches how to evaluate agent behavior, apply security controls and guardrails, improve performance, and instrument agents for visibility across every step of execution. It also explores observability tools such as LangSmith, AgentCore Observability, and Langfuse, along with governance practices that help organizations manage risk, maintain compliance, and build confidence in AI-driven systems.

To get the most out of this book

This book is designed to be hands-on and implementation-focused, so having the right background and environment will help you get the most out of each chapter. Ensure that you have:

- Working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with access to Amazon Bedrock and permissions for S3, DynamoDB, and Lambda (for AWS integration examples)
- Familiarity with REST APIs, JSON, and Docker (helpful but not required)
- AWS CLI and SAM CLI installed (for the Lambda deployment walkthrough)
- Any IDE like Kiro, Claude Code, or Cursor for development (optional)

Download the example code files

The code bundle for the book is hosted on GitHub at `https://github.com/PacktPublishing/Agents-on-AWS` . We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing` . Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: `https://packt.link/gbp/978-1-80638-721-2` .

Conventions used

There are a number of text conventions used throughout this book.

`CodeInText` : Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "In the Jupyter Notebook, open the `math-agent-with-checkpointing.ipynb` file "

A block of code is set as follows:

```
Import LangGraph and LangChain components
# Import LangGraph and LangChain components
```

```
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.prebuilt import create_react_agent
# Import the AgentCoreMemorySaver that we will use as a checkpointer
```

Any command-line input or output is written as follows:

```
The EV sector shows strong growth with major players expanding. However, valuations are high and some
companies face production challenges. Consider established players like Tesla or Ford over newer
entrants. If you have high risk tolerance, the sector looks promising. If you prefer stability, wait
for valuations to normalize.
```

Bold : Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: " Each model connects to MCP once, and each tool also connects to MCP once, reducing the integrations to **N + M connections** ."

> **NOTE**
> Warnings or important notes appear like this.

> **TIP**
> Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback : If you have questions about any aspect of this book or have any general feedback, please email us at `customercare @packt.com` and mention the book's title in the subject of your message.

Errata : Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit `http://www.packt.com/submit-errata` , click **Submit Errata** , and fill in the form.

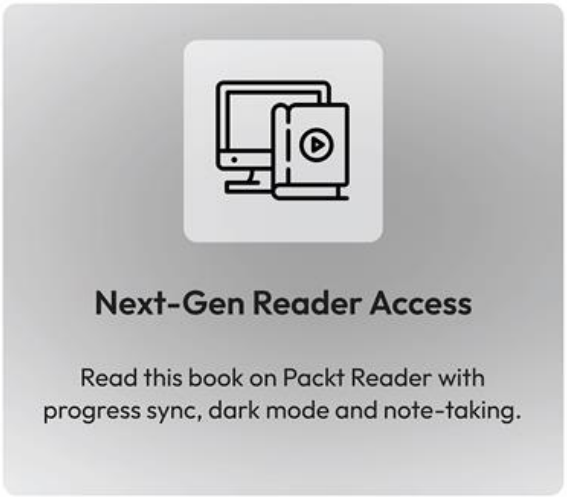
Piracy : If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author : If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `http://authors.packt.com/` .

Free benefits with your book

This book comes with free benefits to support your learning. Activate them now for instant access (see the " *How to Unlock* " section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



How to Unlock

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don ' t require one

Join our Discord and Reddit Space

You're not the only one navigating fragmented tools, constant updates, and unclear best practices. Join a growing community of professionals exchanging insights that don't make it into documentation.

Join our Discord at https://packt.link/z8ivB or scan the QR code below:	Follow us on Reddit at https://packt.link/0rExL or scan the QR code below:

OceanofPDF.com

1

Understanding AI Agents on AWS

Standard LLMs are powerful but come with limitations, such as knowledge cutoff and the inability to access private or real-time data. This raises the question of how we can provide LLMs with information they don't have, or enable them to interact with the real world. Two key approaches address this without fine-tuning or retraining the entire model: **Retrieval Augmented Generation (RAG)** , which gives the LLM relevant context from your data, and **AI agents** , which provide the LLM with the ability to reason and use tools to achieve goals.

You may have built a RAG system that can answer questions about you or your organization's data, only to find that your users want more. For example, they want the AI to actually do the work—book a meeting, update a database, or coordinate a complex workflow. Transitioning from a passive "knowledge retriever" to an active "agent" is where many developers struggle. This chapter is important because it establishes the architectural blueprint you need to move past simple chat interfaces and start building "actor" systems that provide real value.

You might find yourself asking: What is the actual difference between a standard chatbot and an agent? How do I move beyond simple prompts to systems that can execute tasks reliably? This chapter addresses these questions by stripping away the hype and focusing on the core architectural components that make an agent functional. Understanding these fundamentals is crucial. Without a solid grasp of how agents *think* and *act* , it is impossible to build reliable systems that deliver real-world value.

The journey typically begins with basic LLM capabilities, progresses to handling single-prompt interactions, then evolves into dialogue-based

systems such as chatbots, becomes more grounded with RAG-enabled conversations, and ultimately culminates in agents that can reason, use tools, and take action.

Our approach in this chapter is foundational and conceptual. If you are already an experienced AI engineer who has deployed multi-agent systems in production using frameworks like **LangGraph** or **CrewAI** , this chapter may feel like a refresher. If so, feel free to skim the definitions and jump ahead to *Chapter 2, Building Agents with Tools* , where we begin a deeper dive into building agents using the AWS Strands framework.

For most readers, however, starting here ensures that we are all using the same vocabulary and conceptual framework as we move forward. We begin by drawing a clear line between RAG systems and agents based on how they handle reasoning, context, and action. We then deconstruct the *b rain* of an agent to understand how it plans and remembers. Finally, we introduce the AWS Agentic Stack and Strands SDK, and walk through setting up your local development environment and building your first functional "Hello World" agent.

In this chapter we will cover the following key topics:

- What is RAG and how it works?
- What are agents and their capabilities?
- AWS Agentic Stack
- From LLMs to multi-agent systems
- Agent protocols
- AWS Strands SDK
- Building a "Hello World" agent

What is RAG and how does it work?

Retrieval augmented generation (**RAG**) is a technique that enhances the capabilities of LLMs by combining them with an information retrieval system as follows:

1. **Retrieve** : The system searches an external knowledge base (e.g., a database, an API, or a document library) for information relevant to the user's query.

2. **Augment** : The relevant information (context) is added to the original query and passed to the LLM in a detailed prompt.
3. **Generate** : The LLM uses the provided context to generate a factual, accurate, and relevant answer.

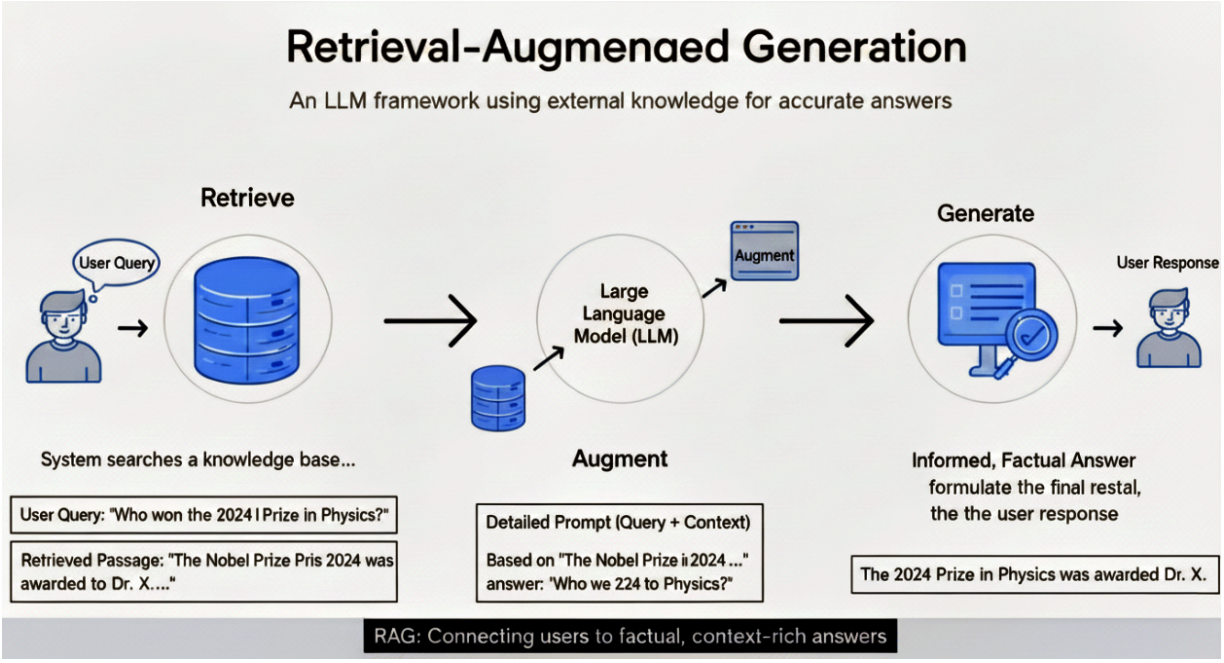


Figure 1.1: How Retrieval Augmented Generation works

For example, consider the **User Query** from the preceding *F igure 1.1* , `Who won the 2024 Nobel Prize in Physics?` To answer this, a RAG-based system follows the same three-step process described above:

- Retrieve: System queries a real-time news database for `2024 Nobel Prize Physics winner` . It finds: `The Nobel Prize in Physics 2024 was awarded to Dr. X. . . .`
- Augment: The system augments or adds the retrieved context with the actual query asked to create a detailed prompt (Retrieved context + Query).
- Generate: The LLM receives this fact and uses it to formulate the final, accurate response for the user.

The RAG pipeline shown in *Figure 1.2* walks through each step involved in building a complete RAG system, preparing data all the way to generating a final answer:

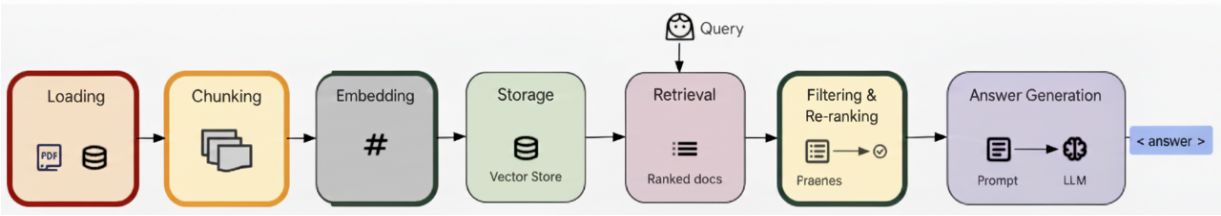


Figure 1.2: RAG pipeline

Let's walk through the pipeline step by step:

1. **Loading or ingesting data:** In this step, you will load or ingest the documents you want to query
2. **Chunking** : Start by splitting your document or documents into smaller chunks of text. This is important because LLMs have a limited **context window** , so they cannot process very large documents all at once. Chunking also helps reduce the **lost-in-the-middle** effect, where important details buried in the middle of a long passage may be overlooked. For example, in a story, the model might miss a character introduced halfway through the text if the entire document is treated as one large block. By breaking content into smaller pieces, the system can retrieve the most relevant chunks and provide the model with focused context for answering the question.
3. **Embedding** : Generate an embedding vector for each individual text chunk. These embeddings are the numerical representations of the text's meaning. This is done by an embedding model, for example, the Amazon Titan model.
4. **Storage:** Store each text chunk together with its embedding vector in a vector database or vector store. These systems are designed to support similarity search, which helps retrieve content that is semantically related to the user's query. Common examples include **Amazon OpenSearch Service** , **Pinecone** , **FAISS** , **Chroma** , and **PostgreSQL with pgvector** . The best option depends on your application's scale, performance needs, and architecture.
5. **Retrieval:** When a user asks a question, the system converts the question into an embedding and uses it to search the vector database for the **most relevant text chunks** . It then retrieves the top **k** results, where **k** simply means the number of chunks selected, such as the top 3,

top 5, or top 10 matches. These retrieved chunks are the pieces of text most similar in meaning to the user's query and are passed to the model as supporting context.

6. **Filtering and Reranking:** This is the process of selecting the most relevant information from a large set of documents to improve the quality of the generated output. Filtering and reranking enhance the quality of retrieved passages in RAG but are optional due to added complexity and computational costs.
7. **Answer generation:** After the most relevant documents or passages are identified from a knowledge base, they are passed to the language model, along with the original query. Answer generation is the final step where a language model crafts a coherent and relevant response based on the retrieved information.

In the next section, we'll take this concept further and explore how RAG becomes a core building block for AI agents.

What are agents?

The word *agent* simply refers to someone (or in this case, something) that performs a task on your behalf. AI agents are autonomous software systems that use artificial intelligence to reason, plan, and carry out tasks for humans or other systems. These tasks can range from compiling research and paying bills to planning a trip or managing enterprise applications.

Agents can make decisions, adapt to new information, and take action without needing explicit human instructions for every step. In many cases, they also use RAG as part of their workflow to retrieve and apply relevant information.

What truly makes agents powerful is their ability to think iteratively. They evaluate results, adjust their approach, and continue working toward a goal.

Let's consider the example below, where an agent acts as a smart travel planner:

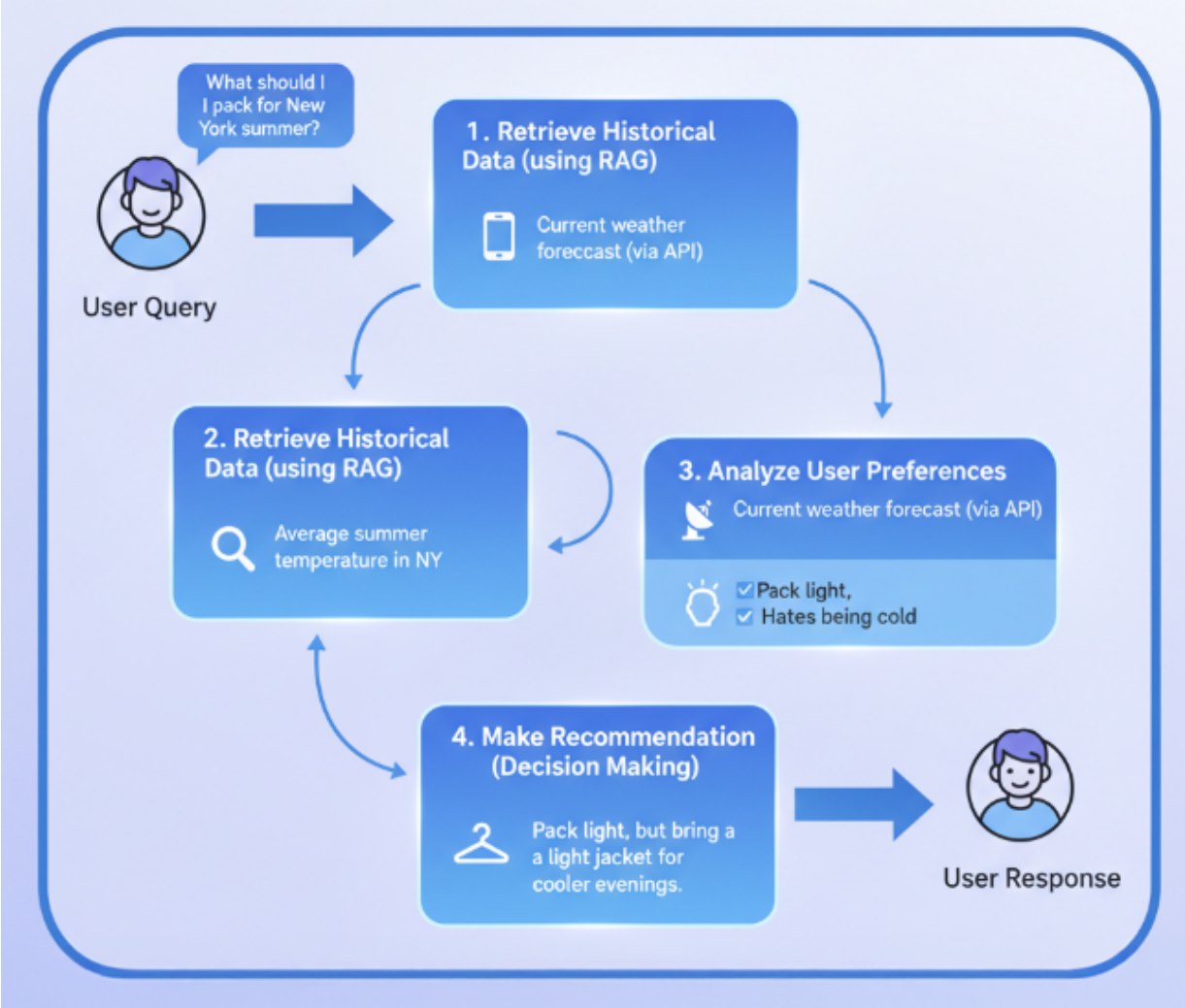


Figure 1.3: AI agent as a travel planner using agentic RAG

The AI Agent acts as a personal travel planner, taking a user's query `W hat should I pack for New York summer?` It uses RAG to retrieve historical weather data, checks real-time forecasts via an API, and analyzes user preferences like packing light but avoiding cold. Finally, it synthesizes all this information to make a personalized recommendation, such as `Pack light, but bring a light jacket for cooler evenings` , demonstrating its decision-making capabilities beyond simple information retrieval.

Agent capabilities

So far, we're aware of how RAG strengthens LLMs by adding a retrieval step that allows them to search a knowledge base and use the relevant context to generate more accurate answers. AI agents build on this foundation and take

it much further. They make decisions and take actions on their own, even performing complex, multi-step tasks for humans. They achieve this by:

1. **Planning:** Breaking down high-level objectives into executable tasks.
2. **Acting** : Utilizing tools and information (knowledge bases) to accomplish these tasks.
3. **Reasoning** : Observing the results of their actions and updating their plans until the goal is achieved.

For instance, an AI agent could fully automate the process of scheduling a meeting, including checking calendars, sending invites, booking rooms, following up on declines, and updating the meeting details—a task that would typically require a human executive assistant.

An agent consists of these components, as shown in the following figure:

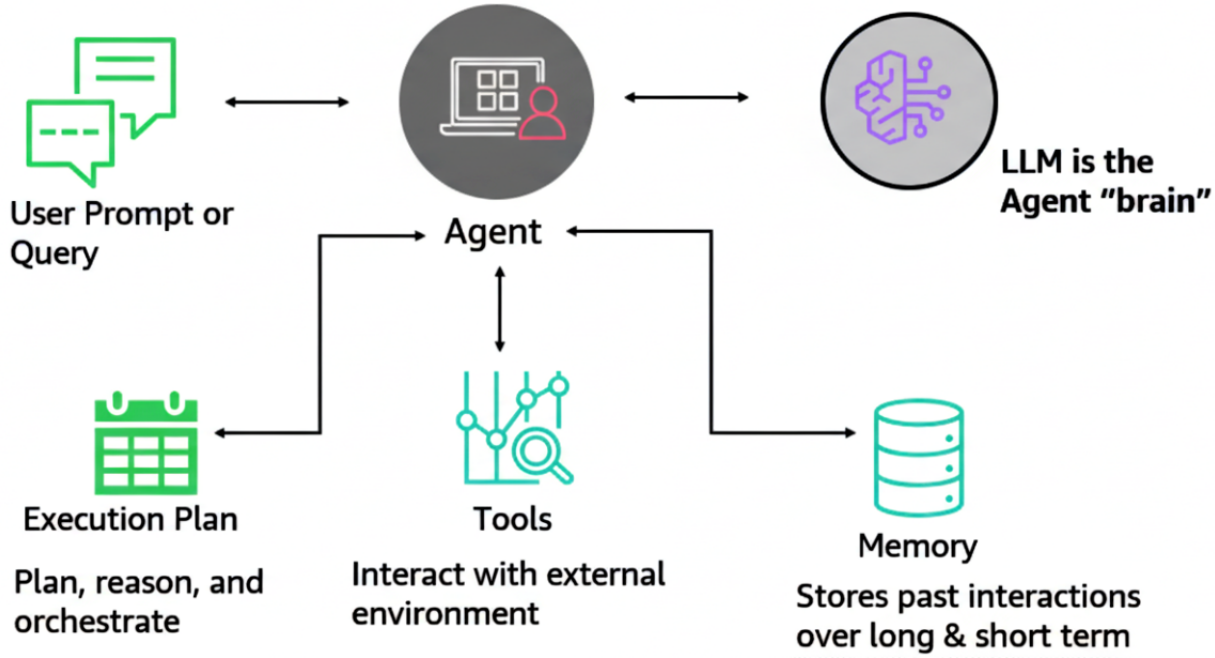


Figure 1.4: Agent components

Let's break down the key components that make up this architecture:

- **Tools** : These are external functions or APIs that the Agent can leverage to interact with the outside world beyond its internal knowledge. This could include search engines, databases, calculators, code interpreters, weather APIs, RAG, or any other specialized service that can perform

specific actions. We are going to cover this in detail in *Chapter 2* , *Building Agents with Tools* .

- **Memory** : Stores past interactions over long and short term, which allows it to remember conversations so that the agent can build context and understand the bigger picture. We are going to cover this in *Chapter 3* , *Agent Memory* .
- **Planning** : Agents analyze the problem, figure out the best sequence of actions to solve it, and decide which tools to use and when. It's not about random actions, but rather about strategic thinking and careful analysis to get the job done right.
- **LLM:** This is the core intelligence or *brain* of the agent. An agent consults its LLM (brain) to understand the query, generate potential responses, or determine a course of action such as which tool to call. This is a crucial, bidirectional relationship.

With this view of how agents are structured and how their core components work together, let's see how these concepts map to real-world systems within the AWS ecosystem.

Exploring the AWS Agentic Stack

First, we will introduce the AWS Agentic stack in this section and then we will cover how you can build agents on AWS. Understanding this stack is crucial because building effective AI agents requires matching the right tool to your specific use case. While our previous discussions focused on the theoretical mechanics of RAG and agentic workflows, moving into the AWS ecosystem allows us to ground these concepts in real-world architecture. The AWS ecosystem allows us to ground these concepts in real-world architecture. As of the publication of this book, the AWS agentic AI landscape spans a range of services designed for different users and levels of development effort. This spectrum includes easy-to-use solutions such as **Amazon Q** for business users, managed platforms such as **Amazon Bedrock** for developers who want flexibility without managing infrastructure, and highly customizable services such as **Amazon SageMaker** and specialized hardware for advanced builders who need deep control over their AI systems. The following diagram on AWS Agentic Stack represents this landscape at the time of writing. Since AWS

services evolve rapidly, you should consult the latest **AWS documentation** for up-to-date service capabilities and recommendations.

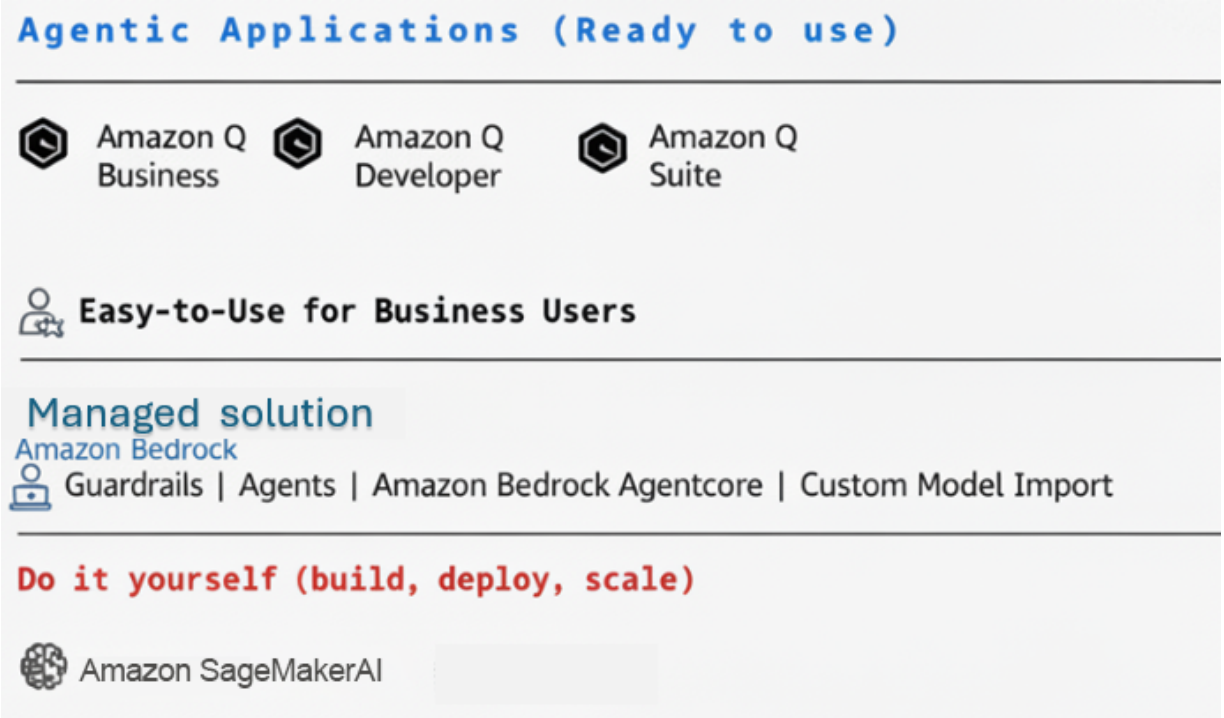


Figure 1.5: AWS Agentic Stack

Let's get into each layer of the stack and understand the offerings:

- **Agentic applications:** This top tier represents fully managed, out-of-the-box agentic AI applications from Amazon Q. They are designed for immediate use with minimal configuration, requiring the least effort from the user. This tier is ideal for general business users or those who need quick AI integration without deep technical expertise.

  NOTE

  Since this book is to learn building agents using code, we are not going to use services from this stack in the book.

- **Managed Solutions:** This tier offers managed services that provide more flexibility than ready-to-use applications but still abstract away much of the underlying infrastructure complexity. It requires a moderate level

of effort, suitable for developers who want to build custom agentic solutions without managing all the individual components. Some of the key services that we will use in this book from these managed solutions are:

- **Amazon Bedrock** : This is a key service for building and scaling generative AI applications. It provides access to a choice of high-performing foundation models (FMs) from Amazon and leading AI companies via a single API. Because Amazon Bedrock continues to evolve, readers should refer to the latest AWS documentation for the most up-to-date feature set.
  Bedrock consists of features such as:
  - **Guardrails:** Features within Bedrock to implement safety policies and prevent undesirable outputs from FMs
  - **Agents:** Capabilities within Bedrock to create agents that can execute multi-step tasks
  - **Custom Model Import:** Allows users to bring their own custom models into the Bedrock environment
  - **Knowledge Bases** – supports RAG by retrieving relevant information from enterprise data sources and using it to ground model responses
  - **Model and RAG evaluation** – helps assess the quality and effectiveness of models, knowledge bases, and other RAG sources
- **Amazon Bedrock AgentCore** : This is a scalable and secure platform for building, deploying, and operating AI agents, enabling them to interact with tools and data without infrastructure management. It offers fully managed services compatible with any framework or foundation model, balancing open-source flexibility with enterprise-grade security and reliability. Amazon Bedrock AgentCore Runtime is a fully managed service of Bedrock AgentCore, which provides low latency serverless environments to deploy agents and tools.
- **Do it yourself:** This bottom tier is for users who require maximum control and customization. It provides the foundational infrastructure and tools to build, deploy, and scale agentic AI applications from scratch. This tier is designed for experienced developers, ML engineers,

and researchers who need granular control over every aspect of their AI stack, from hardware selection to custom model architectures.

- **Amazon SageMaker AI** is an end-to-end service for building, training, and deploying machine learning models at scale. This includes capabilities for custom model development.

> NOTE
>
> You can also customize an LLM with your domain-specific data in Amazon SageMaker AI and then make it available in Amazon Bedrock through the Custom Model Import feature discussed earlier. This allows you to use a domain-adapted model as the reasoning engine, or "brain," of your agent.

While the AWS Agentic Stack offers various entry points, this book focuses on **Amazon Bedrock** and **Amazon SageMaker AI** because they represent the *sweet spot* for developers and architects.

Service	Best suited for	Strengths
Amazon Bedrock	Rapid development of generative AI applications and agents	Managed, serverless-style access to foundation models; unified API; supports agentic features and access to multiple model providers
Amazon SageMaker AI	Advanced customization, model training, fine-tuning, and specialized deployment	Deep control over training, hosting, tuning, and deployment architecture; supports multiple endpoint and infrastructure options

Table 1.1: Choosing the right AWS service for enterprise-grade generative AI systems

In this book, we use Amazon Bedrock to build and deploy agents because it provides a managed, API-first environment for orchestrating agent workflows with foundation models. At the same time, Amazon SageMaker AI plays an important complementary role for organizations that need deeper model customization. For example, the model that acts as the agent's reasoning engine may be customized in SageMaker through fine-tuning or other adaptation approaches, and customized models can be deployed to SageMaker endpoints when teams need greater control over cost, latency, or

infrastructure. Together, Bedrock and SageMaker AI support a spectrum of needs, from rapid agent development to highly customized model deployment.

Building AI agents on AWS

We covered the AWS AI Agentic stack. In this section, we will show you how you can build an agent on AWS using tools, models, and orchestration frameworks such as LangChain and Strands SDK. We will deep dive into Strands at the end of this chapter. This section just walks you through steps for building an agent using an IDE, models, and tools.

Figure 1.6 illustrates what it looks like to build an AI agent using AWS services such as Amazon SageMaker and Amazon Bedrock, along with supporting development environments and frameworks:

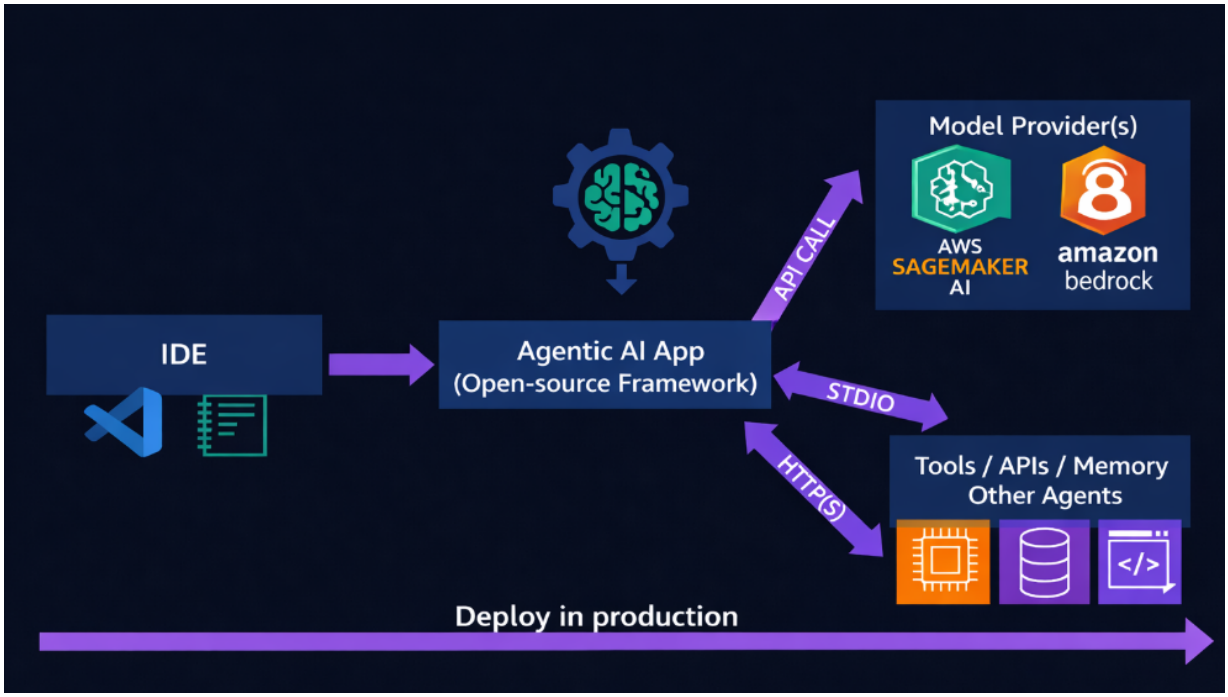


Figure 1.6: Building AI Agents on AWS

At a high level, here's how you would build an agentic application on AWS, as shown in *Figure 1.6* :

1. A **developer** builds and configures the agentic AI application using an IDE such as **Visual Studio Code** , **Amazon SageMaker Studio** , or an agentic IDE such as **Kiro**

2. The agentic AI application uses frameworks such as **LangChain** or the **AWS Strands SDK** to define agent logic, orchestrate workflows, and connect the agent to models and tools
3. Because an agent needs a reasoning engine, or "brain," to interpret requests and decide what to do next, the running application makes API calls to model providers such as **Amazon Bedrock** or **models hosted on Amazon SageMaker AI**
4. At the same time, the application interacts with **tools, APIs, memory systems, and other agents** through protocols such as **HTTP(S)** or **STDIO** to retrieve information, perform actions, or collaborate on tasks
5. The resulting outputs are then presented through a **client application or user interface** , where the **end user** can interact with the agent

Now that we've seen how the AWS Agentic Stack supports the practical side of building and deploying agents, it's worth zooming out to understand how these systems mature over time

Progressing from LLMs to multi-agent systems

Agentic intelligence doesn't appear all at once. It develops in stages, starting from basic language model capabilities and gradually advancing toward autonomous and collaborative multi-agent systems. In this section, we will walk through that evolution step by step.

Figure 1.7 illustrates this progression in the complexity and capabilities of multi-agent systems:

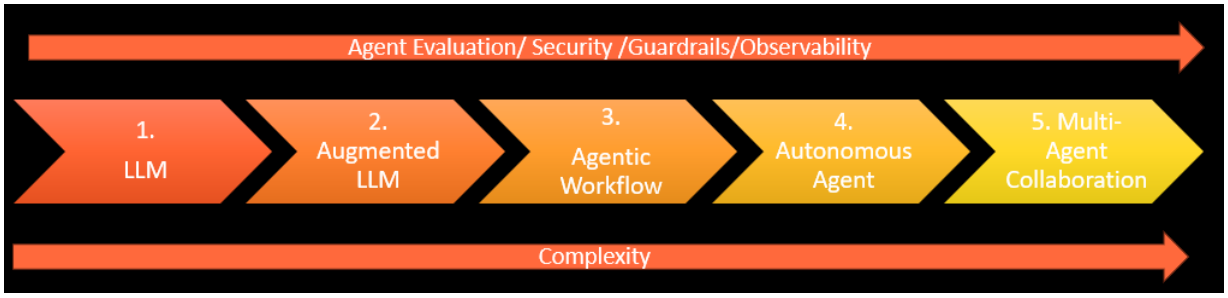


Figure 1.7: Progression of the multi-agent systems

Before exploring each stage in detail, it's important to note the **top arrow** in the diagram labeled **Agent Evaluation / Security / Guardrails / Observability** . This critical arrow indicates that as complexity grows, so does the importance

and challenge of evaluating the agents' performance, ensuring their security, setting up proper guardrails to prevent unintended or harmful actions, and having robust observability to understand their behavior. These considerations are especially vital in multi-agent systems, where interactions can be harder to predict. With this understanding, let's deep dive into each stage:

1. **LLM** : This refers to using a foundational LLM directly. You have two choices of using foundation model on AWS: Amazon SageMaker AI hosted models (platform as a service) or Amazon Bedrock models (model as a service).
   This stage involves prompting the LLM to generate text, answer questions, summarize information, or perform other language-based tasks. The interaction is typically a single turn or a simple conversational flow without external tools or complex decision-making processes.

   Example: Asking a question and getting a direct answer from the anthropic Claude model on Amazon Bedrock.

2. **LLM + RAG**
   The next stage extends the LLM by grounding it in external knowledge through **Retrieval Augmented Generation (RAG)** . Instead of relying only on the model's pretrained knowledge, the system first retrieves relevant information from external sources such as enterprise documents, knowledge bases, or databases, and then passes that information to the LLM as context. This helps the model generate responses that are more accurate, current, and relevant to your domain.

   Example: A chatbot that answers employee questions using information retrieved from internal HR policy documents.

   There are two common terms which you might hear with RAG: traditional RAG and agentic RAG. Let's understand the differences.

   In a **traditional RAG system** , the flow is usually straightforward:

   User query → retrieve relevant documents → send retrieved context to LLM → generate answer . For example, a user asks, "What is our

company's leave policy?" The system retrieves the relevant HR policy chunks and passes them to the LLM, which then generates an answer.

Agentic RAG treats retrieval as one tool among many tools available inside an agentic system. Instead of following a fixed one-step retrieval flow, the agent can decide when to retrieve a RAG tool, refine searches, use multiple sources, combine retrieval with tools or APIs, and take actions based on the retrieved information.

Question → agent reasons → maybe call RAG tool → maybe call other tools → answer or act

In agentic RAG, the agent can call a RAG pipeline as a tool. The agent itself is the decision-making layer, and RAG becomes one capability it can use when needed.

3. **Augmented LLM** : This stage enhances the LLM's capabilities by integrating it with external tools or data sources, as shown in *Figure 1.8* :

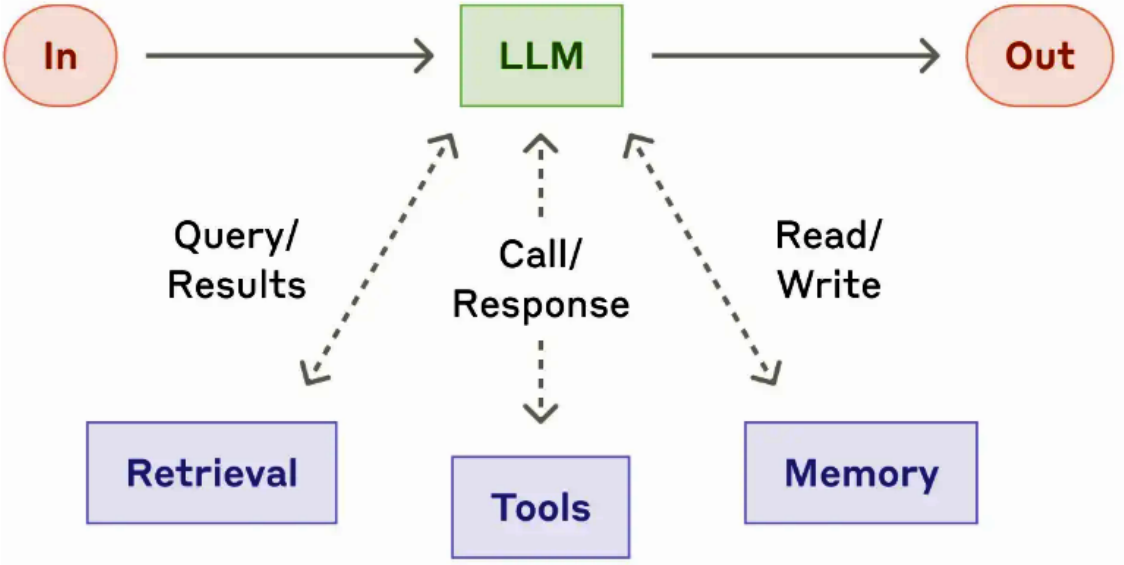


Figure 1.8: Augmented LLM with tools and memory

Figure 1.8 illustrates the shift from a passive LLM to an active AI agent. While the LLM acts as the central 'brain,' it is limited by its training cutoff and its inability to interact with the world. To overcome this, the architecture introduces three critical pillars **Retrieval** , **Tools,** and **Memory** . These distinctions are vital because an effective agent knows *where* to find information (Retrieval), *how* to use it to take action (Tools),

and *how* to keep track of its progress (Memory). Understanding how these three components communicate is the key to building reliable, sophisticated workflows on AWS.

Example: An LLM that can search the internet for real-time information to answer a query, or an LLM that can generate an email and then use an email API to send it.

4. **Agentic workflow** : Here, the LLM is no longer just a *responder* but becomes an *agent* capable of planning, reasoning, and executing a sequence of steps to achieve a goal. It involves the LLM breaking down a complex task into smaller sub-tasks, deciding which tools to use for each sub-task, executing them, and then synthesizing the results. This often incorporates concepts like planning, memory, and reflection.
   Example: An LLM agent tasked with booking a flight. It might first check flight availability using a flight API, then compare prices, then ask for user preference, and finally use a booking API to complete the reservation.
5. **Autonomous agent** : This stage represents a more advanced form of the agentic workflow where the agent can operate with a higher degree of independence and persistence. It can learn from its environment, adapt its plans, and maintain long-term objectives without constant human intervention. Autonomous agents often involve sophisticated feedback loops and self-correction mechanisms.
   Here is a diagram showing the difference between agentic workflow and autonomous agent:

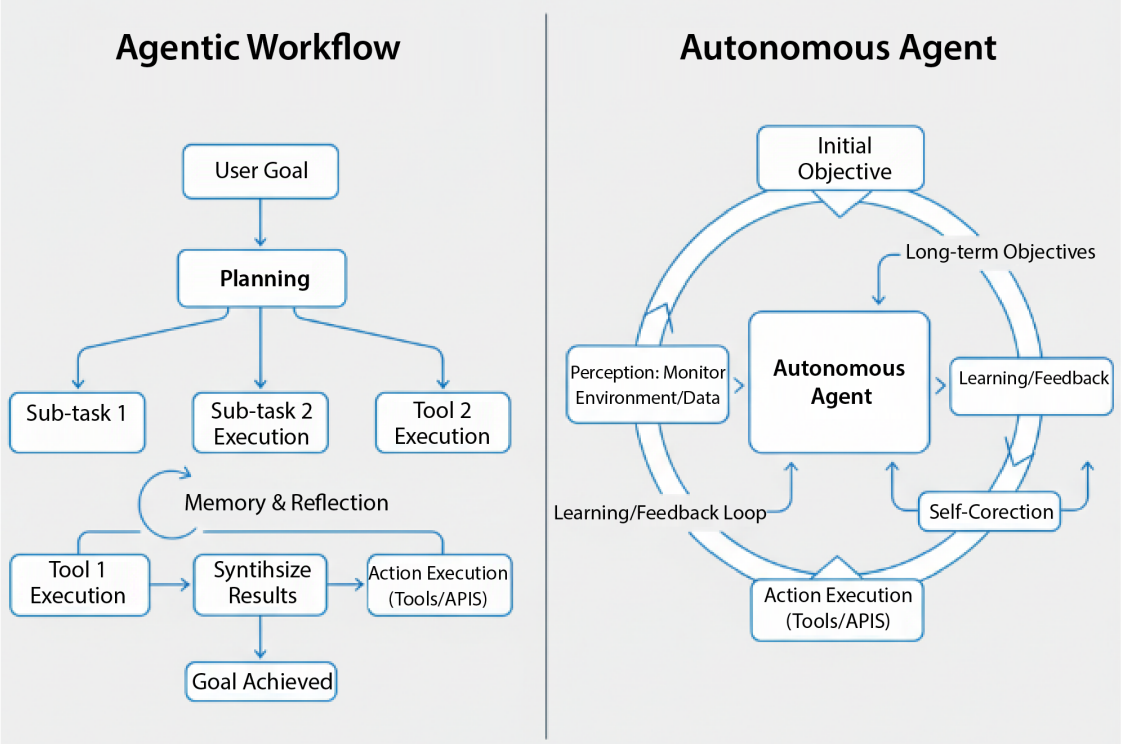


Figure 1.9: Agentic workflow vs Autonomous agent

Example: An agent that continuously monitors market trends, researches new investment opportunities, and autonomously executes trades based on predefined strategies and risk parameters.

6. **Multi-agent collaboration** : This is the most complex stage, involving multiple autonomous agents working together to achieve a shared or collective goal. Each agent might have specialized roles, knowledge, or capabilities, and they communicate and coordinate with each other to solve problems that are too complex for a single agent. This mimics human team collaboration.
   Example: A team of AI agents where one acts as a researcher, another as a content creator, and another as an editor, all collaborating to write a comprehensive report on a specific topic.

NOTE

We will cover multi-agent collaboration, agentic workflows, and autonomous agents in *Chapter 4 , Advanced Agent Architecture Patterns* and *Chapter 5, Agent Communication* .

Since we now understand the evolution of an agent and its complexity, let's deep dive next into how these agents can communicate with each other or with tools with added complexity.

Understanding agent protocols

Before we explore large-scale agentic systems, it is important to understand the communication challenges involved in making them work reliably in production. An agent often needs to call external tools, APIs, or databases to retrieve information or perform actions it cannot do on its own. For example, a travel agent asked to book a flight may need to access a flight API, customer profile, and payment service. As the number of tools and workflows increases, teams need a consistent way to connect agents to these external systems. This is the problem addressed by protocols such as **M odel C ontext P rotocol** (**MCP**), which standardize agent-to-tool interaction.

However, large-scale systems often involve multiple specialized agents rather than a single all-purpose one. A planner agent may delegate work to a booking agent, which may in turn coordinate with a compliance or payment agent. In these scenarios, the challenge is not only agent-to-tool communication but also **Agent-to-Agent** (**A2A**) communication.

Agent protocols like **A2A** and **MCP** are, hence, needed to standardize communication and interaction between agents, ensuring interoperability and efficient collaboration in multi-agent systems. Let's explore these protocols in detail.

Model Context Protocol

MCP is used for tool discovery in an agent architecture having multiple tools by providing a standard write-once, run-anywhere protocol.

Let's deep dive into the MCP architecture, developed by Anthropic as an open protocol, which provides a standardized way to connect AI models to virtually any data source or tool. Using a client-server architecture, MCP helps developers expose their data through lightweight MCP servers while building AI applications as MCP clients that connect to these servers. MCP uses a client-server architecture that contains the following components:

- **Host:** An MCP host is a program or AI tool that requires access to data through the MCP protocol, such as Claude Desktop, an integrated development environment (IDE), or any other AI application.
- **Client:** Protocol clients that maintain one-to-one connections with servers.
- **Server:** Lightweight programs that expose capabilities through standardized MCP or act as tools.
- **Data sources:** It can be local data sources such as **databases** and **filesystems** or external systems available over the internet through APIs (**web APIs**) that MCP servers can connect to.

Now let's understand the MCP workflow as shown in *Figure 1.10* and how it interacts with the LLM to give you a response by using an example of the same travel agent but with a complex ask. You ask the agent to `Book a 5-day trip to Europe in January and we like warm weather .` The host application (acting as an MCP client) identifies the need for external data and connects via the protocol to specialized MCP servers for flights, hotels, and weather information. These servers return the relevant data through MCP, which the host then integrates with the original prompt, providing enriched context to the large language model to generate a comprehensive, augmented response for the user.

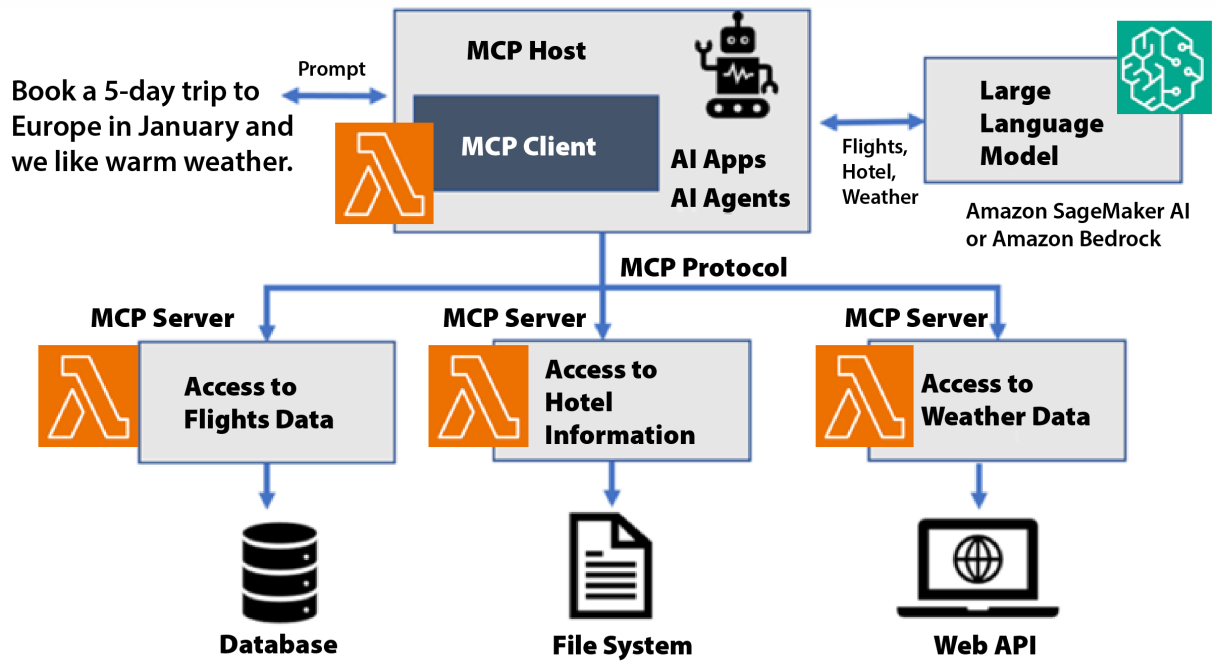


Figure 1.10: MCP workflow

Let's walk through the sequence of steps involved in this workflow:

1. A user's natural language request is sent to the MCP host.
2. The MCP host, using a large language model, understands the request and identifies the data needed (flights, hotel, weather).
3. The MCP host then uses the MCP protocol to communicate with the appropriate MCP servers, requesting the specific data.
4. Each MCP server accesses its respective data source (database, file system, or web API) and returns the information to the MCP host.
5. The AI agents within the MCP host then process this aggregated information to fulfill the user's initial request.

This architecture demonstrates a distributed, agent-based approach where specialized agents (MCP servers) handle specific data access, coordinated by a central MCP host and powered by a large language model for understanding and planning.

TIP

Implementing MCP on AWS

You can run the MCP servers and clients on Amazon EKS/Lambda and LLM models can be hosted on Amazon SageMaker AI Inference or Amazon Bedrock.

Another option is to use the Amazon Bedrock AgentCore MCP Server, which simplifies building AI agent components by offering built-in support for runtime, gateway integration, identity management, and agent memory. This server is ideal for rapid prototyping, production solutions, and scaling enterprise agent infrastructure. We will deep dive into these hands-on examples in *Chapter 5 , Agent Communication* .

Agent-to-Agent Protocol

A2A protocol is developed by Google to discover agents in multi-agent workflows. It is an open standard designed to enable seamless communication and collaboration between AI agents. A2A provides the definitive common language for agent interoperability in a world where

agents are built using diverse frameworks and by different vendors. Let's understand why we need A2A.

AI agents are everywhere, how we get them to work on complex problems. As shown in the diagram in case you are building a trip planner, you need a hotel agent, activity agent ,travel agent, flight agent, and guide agent to coordinate but it's a lot of work to build every single one of these, you might want to use someone else agent for some of these tasks but you have no idea how its implemented and how it works. That is why you need a standard way for these agents to collaborate and communicate using A2A protocol.

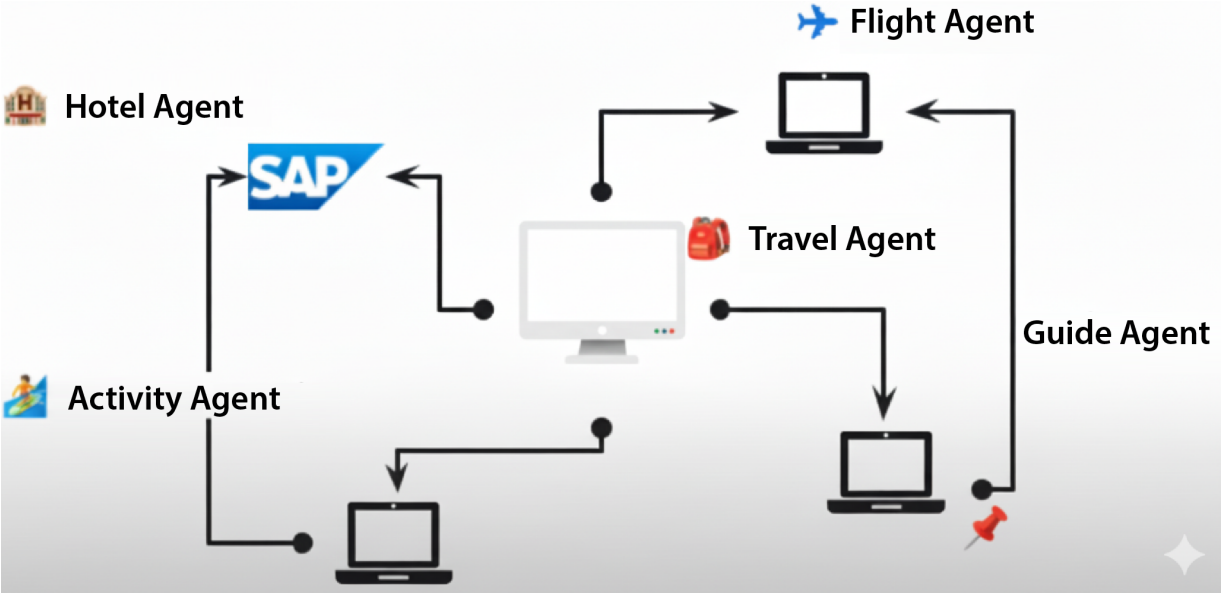


Figure 1.11: Travel agents communicating with other agents using A2A protocol

As per *Figure 1.11* , through the A2A protocol, the **Travel Agent** acts as the orchestrator for the entire trip planning process. It doesn't handle all the details itself but instead communicates with various specialized agents.

The Travel Agent can delegate specific tasks to these specialized agents without needing to understand their internal code or how they achieve their results. Each agent, regardless of its origin or implementation, can communicate effectively using the standardized A2A messages, allowing them to collaborate and solve the complex problem of planning a trip efficiently. This decentralized, collaborative approach prevents the need to rebuild every component and promotes the reuse of existing, specialized AI agents.

Figure 1.12 illustrates a communication protocol, let's call it A2A protocol, which enables interaction between a user, a client, and two agents: Agent A (Client) and Agent B (Server/Remote Agent):

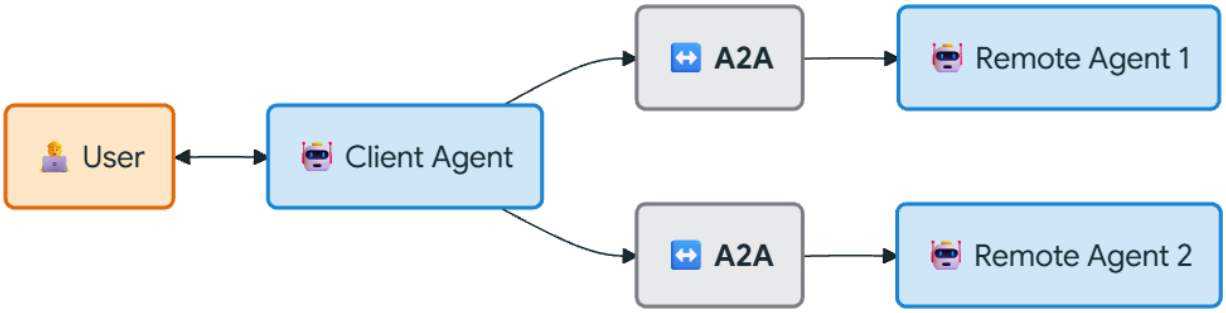


Figure 1.12: How A2A works

Agents can advertise their capabilities using an **Agent Card** in JSON format, allowing the client agent to identify the best agent that can perform a task and leverage A2A to communicate with the remote agent.

> **NOTE**
>
> A2A and MCP are complementary to each other and do not compete in an architecture.

Figure 1.13 summarizes the differences between MCP and A2A and their relationship in an agentic architecture:

		Model Context Protocol (MCP)	Agent2Agent Protocol (A2A)
Feature	**Primary/Role**	• Connects agents to tools, APIs, and resources. • *How an agent uses its capabilities (function calling).* • *Think: How an agent uses its capabilities (function calling).*	• Facilitates dynamic **communication between different agents** as peers. • *How agents collaborate, delegate, and manage tasks.*
	Example	• Agent uses MCP to call a weather API tool.	• Example: A travel (A2A) booking agent uses another agent to find flights.
	Relationship	Complementary, Not Competing	Complementary, Not Competing

Figure 1.13: MCP vs A2A

TIP

Implementing A2A on AWS

A2A protocol is available on Amazon Bedrock AgentCore Runtime. With this addition, agents can discover peers, share capabilities, and coordinate actions across platforms using standardized communication. We will cover A2A in *Chapter 5 , Agent Communication* .

Introducing AWS Strands SDK

We covered the basics of building an agent such as tools, planning, and memory. We also covered protocols such as MCP and A2A when you build an agentic system.

In order to build an agent, you need a framework which can provide components such as the ability to define and orchestrate tools, store memory, and the ability to configure an LLM.

As discussed in the section, *B uilding Agents on AWS* , some common frameworks available that support agentic developments include LangChain and Strands SDK.

In this chapter, we will cover Strands SDK to build a "Hello World" agent and also talk about why it's simple to build an agent using Strands. So, let's get started.

AWS Strands Agents is an open-source SDK that adopts a model-driven approach to building and running AI agents with minimal code. It enables developers to create agents that can plan, reason, and select tools autonomously, leveraging the capabilities of **large language models** (**LLMs**) like those available through Amazon Bedrock.

Some of the key features of using Strands SDK are:

- Build agents with just a few lines of code by defining prompts and tools.
- Easily integrate external tools and APIs to extend agent capabilities.
- Supports various LLM providers, making it model-agnostic, including Amazon Bedrock, Anthropic, and more.

- Facilitates the creation of complex workflows involving multiple collaborating agents.
- Includes features for guardrails, prompt security, and monitoring agent performance.

Let's understand the internals of the Strands Agent SDK via *Figure 1.14* :

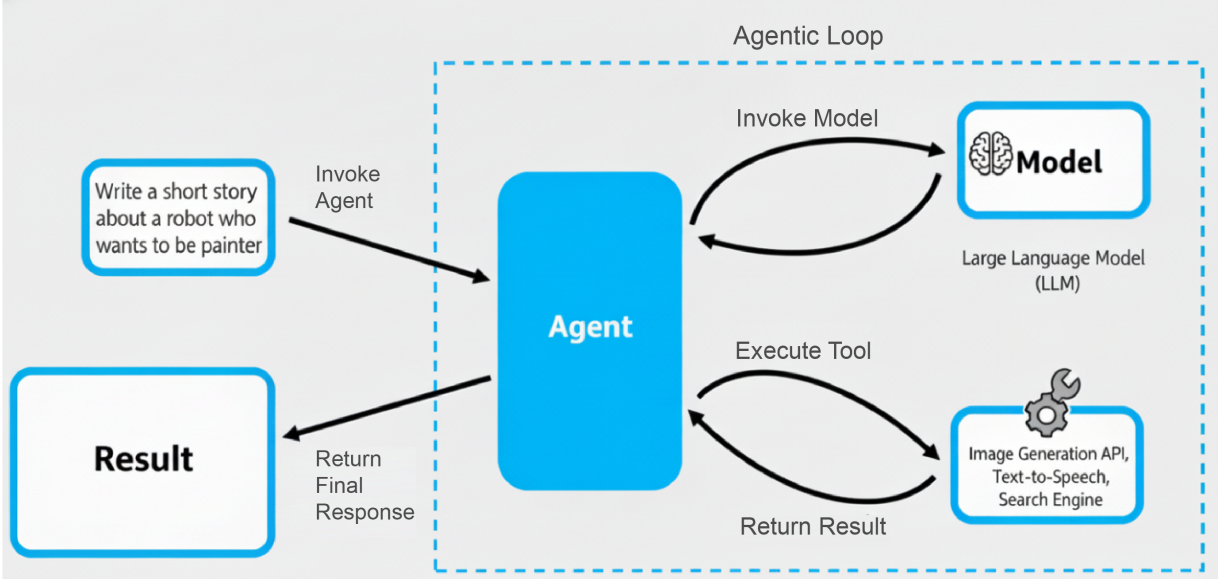


Figure 1.14: How AWS Strands SDK works as an orchestrator to build agents

This diagram illustrates the Agentic Loop (or Agentic Workflow), a process where an AI **Agent** can take a high-level **Prompt** , use a **Model** (like an LLM) for reasoning, and **Tools** for specific actions, to ultimately deliver a **Result** .

Let's break down how it works with an example.

The user initiates the process by providing a prompt:

```
Write a short story about a robot who wants to be a painter.
```

This is the initial goal the agent needs to achieve:

1. Invoke agent: The prompt is sent to the *Agent* . The Agent is the central orchestrator here; its role is to understand the prompt, plan how to fulfill it, and manage the interaction with the Model and Tools.
2. Agentic loop (Inside the dashed box): This is where the iterative process of thinking and acting happens
3. Invoke model: The Agent first sends the prompt (or a refined version of it) to the *Model* . The Model, often an LLM represented by the brain icon,

is responsible for:

4. Get a response: Generating text based on the input.
5. Reasoning: Understanding the prompt, breaking it down, and deciding on the next steps. For our prompt, the model might reason: `To write a story, I need to generate characters, a setting, a plot, and a resolution. I might also need to look up artistic concepts or robot characteristics.`
6. Tool selection: Deciding if any external tools are needed. For instance, if the prompt asked for `a story with an image` , the model might decide to use an Image Generation API.
7. Execute tool: If the Model determines that a tool is necessary (e.g., to find information, generate an image, or perform a calculation), it instructs the Agent to `Execute tool` . The **Tools** box, with the wrench icon, represents various external APIs or functionalities the Agent has access to (e.g., Image Generation API, Text-to-Speech, Search Engine). For our story-writing prompt, the Model might initially decide it doesn't need an external tool for the core story generation. However, if the prompt were more complex (for example, `Write a story about a robot painter and include an image of their first painting`), the agent would use an Image Generation API here.
8. Return result (from Tool): The executed tool performs its function and sends its **Return result** back to the Agent
9. Get response, reasoning, tool selection (from Model): After the tool returns a result, or if no tool was needed, the Agent might invoke the Model again with the updated context or partial result. This forms a loop: the Agent asks the Model, the Model reasons/responds/selects tools, the Agent executes tools, and the results feed back into the Agent, which might then re-invoke the Model. This iterative process continues until the Agent determines the prompt has been fully addressed.
10. Return final response: Once the Agent has completed its task through multiple iterations of the Agentic Loop, it compiles all the information and the story generated by the Model
11. Result: The Agent delivers the result back to the user. For our example, this would be the complete short story about the robot who wants to be

a painter.

In summary, Strands SDK had 3 core components: **model** , **tools** and **prompt** , and it has an agentic feedback loop which ensures that agents can adapt to dynamic environments, handle unexpected scenarios, and improve their performance over time.

With this understanding, let's build a simple "Hello World" agent via Strands SDK.

Building a "Hello World" agent

We will host all the notebooks in our book GitHub repository at `https://github.com/PacktPublishing/Agents-on-AWS` . Before you write any code, ensure that your environment is properly set up. The following prerequisites are required to run the examples in this section. For detailed, step-by-step setup instructions (for Amazon SageMaker Studio, VS Code, or Kiro IDE), refer to the project README:
`https://github.com/PacktPublishing/Agents-on-AWS/blob/main/README.md`

You should have the following in place:

- Python 3.10 or higher installed
- An AWS account with Amazon Bedrock model access enabled (go to Console → Bedrock → Model access, and enable models such as Nova Lite, Nova Pro, or Claude)
- Appropriate IAM permissions attached to your user or role
- Refer to the `README` for full policy details, but at a minimum, ensure access to:
    - `bedrock:InvokeModel`
    - `bedrock:InvokeModelWithResponseStream`
    - `sagemaker:InvokeEndpoint` (for SageMaker-hosted models)
    - `s3:GetObject` and `s3:PutObject` (for data/artifacts)
    - `bedrock- agentcore :` (required for AgentCore deployments in *Chapter 6 , Production Deployment and Enterprise Integration*)

You will also require a working development environment. You can use **either** :

- Amazon SageMaker Studio (recommended), with a JupyterLab space configured, or
- A local IDE such as VS Code or Kiro

The following table summarizes the setup instructions for both local IDE and SageMaker Studio:

Requirement	SageMaker Studio	Local IDE
AWS credentials	Automatic (execution role)	`aws configure` or env vars
IAM permissions	Attach to execution role	Attach to IAM user or role
Bedrock model access	Enable in AWS Bedrock service console	Enable in AWS Bedrock service console
Python packages	`pip install` in notebook	`pip install` locally

Table 1.2: Summary of the prerequisites

Let's move on to setting up the development environment where we will run our first agent.

Setting up your environment in Amazon SageMaker studio

In the following sections, we run the code from SageMaker Studio JupyterLab notebook instances. You can also use your preferred IDE, such as VS Code or PyCharm, but make sure your local environment is configured to work with AWS, as discussed in the prerequisites or follow this https://github.com/PacktPublishing/Agents-on-AWS/blob/main/README.md guide.

Complete the following steps:

1. On the SageMaker AI console, choose **Domains** in the navigation pane, then open your domain.
2. In the navigation pane under Applications and IDEs, choose Studio.
3. On the User profiles tab, locate your user profile, then choose Launch and Studio:

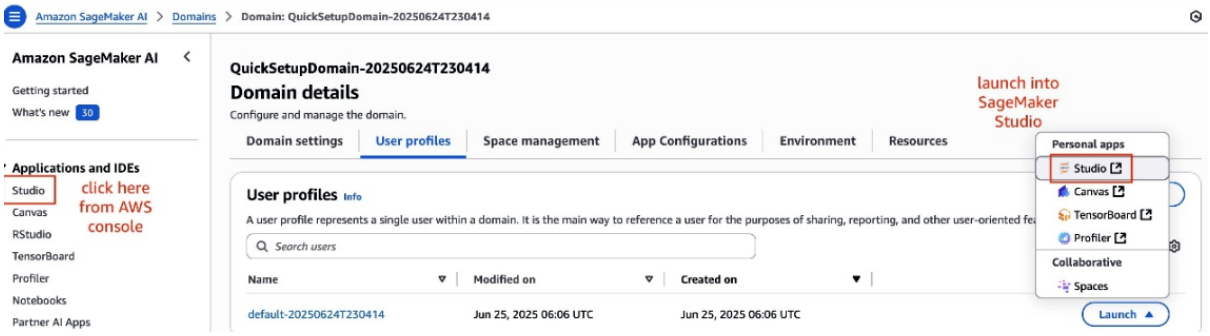


Figure 1.15: Launching SageMaker Studio from the user profile

4. In SageMaker Studio, launch an `ml.t3.medium` JupyterLab notebook instance:

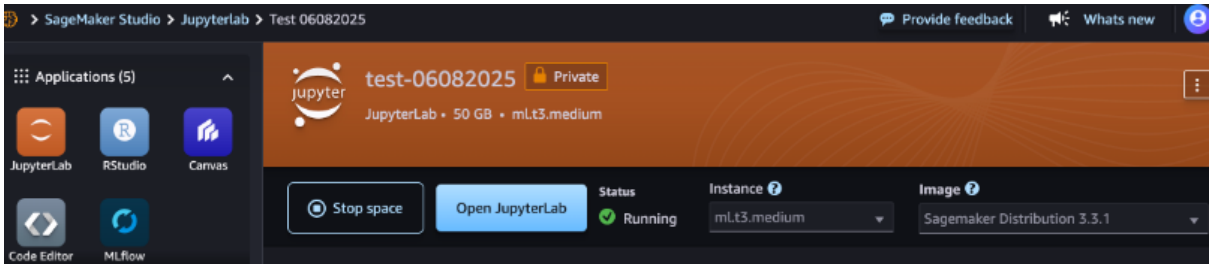


Figure 1.16: Creating a JupyterLab notebook instance in SageMaker Studio

Now that your environment is ready, you can begin building and running your first agent.

5. To begin building agents, start by cloning the GitHub repo and navigating to the folder `chapter1` in the terminal of the JupyterLab:

```
# clone github
git clone https://github.com/PacktPublishing/Agents-on-AWS
cd chapter1
```

6. Open the notebook and proceed with the installation of Strands SDK by clicking the notebook cell as shown in the following code:

```
!pip install strands-agents -q
```

After installing the packages, restart the kernel. With that, you are ready to define and interact with a simple agent.

Creating a simple agent

In this example, we import the Agent class from the strands module and interact with the agent by passing a message. The agent processes the input

using the underlying LLM and returns a response.

This simple Python example shows how to interact with an Amazon Bedrock Agent using the `strands` library:

```
from strands import Agent
# Define the agent with a system prompt
agent = Agent()
# Interact with the agent
response = agent("Explain Amazon Bedrock Agents")
print(response)
```

The agent is created with a system prompt (or defaults) and then sent a question (in this case, `Explain Amazon Bedrock Agents )` . The agent processes the query and returns a natural language response, which is printed out. This demonstrates how easily you can build conversational AI apps powered by Amazon Bedrock with minimal code.

We have provided two more examples on building agents with LangGraph and Strands in the GitHub repository. Please follow the `README .md` instructions to get started: https://github.com/PacktPublishing/Agents-on-AWS/blob/main/chapter%201/README.md

With this, we covered how to build a simple agent with a few lines of code using Strands SDK. We are going to cover real-world use cases of building agents in upcoming chapters.

Summary

In this chapter, we laid the foundational groundwork for understanding AI agents on AWS. By deconstructing the evolution of agentic intelligence, we saw how systems progress from basic LLMs to sophisticated multi-agent collaborations. We also touched upon the future of interoperability through emerging protocols like MCP and A2A, which allow agents to share data and collaborate seamlessly across different platforms.

We explored the AWS Agentic Stack, identifying why Amazon Bedrock and Amazon SageMaker are the ideal services for building both rapid prototypes and high-performance production systems. Finally, we moved from theory to practice by introducing the AWS Strands SDK and deploying your first "Hello World" agent. This simple exercise demonstrated the power of a model-driven

approach: with just a few lines of code, you can give an LLM a goal and the framework to achieve it.

Now that you have a firm grasp of the *what* and *why* of AI agents, in the next chapter, we will dive deep into the first critical pillar of the agentic architecture. You will learn how to give your agents *hands* by defining and configuring powerful external tools that allow them to interact with the real world.

Get this book's PDF version and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

OceanofPDF.com

2

Building Agents with Tools

In *Chapter 1* , *Understanding AI Agents on AWS* , you learned the fundamentals of AI agents and created your first Hello World agent. An Agent that could understand requests and generate intelligent responses. But that agent was limited to conversation. It couldn't check the weather, read files, query databases, or take actions in the real world.

This limitation becomes painfully obvious the moment you try to build anything practical. Imagine deploying an AI assistant for your team that can discuss project requirements brilliantly but can't actually check if the deployment pipeline is running, read the latest error logs, or update a ticket status. Or consider a customer service agent that understands complex queries perfectly but can't look up order information, process refunds, or schedule appointments. Without tools, even the most sophisticated language model is essentially a very expensive chatbot.

In this chapter, we'll solve this by giving your agents tools - the ability to interact with the real world, access live data, and perform meaningful work beyond generating text. We'll start by transforming simple Python functions into agent tools, then explore the ready-made tools that can save you days of development time, and finally build custom tools for those unique problems only your business has. By the end, you'll have agents that don't just talk about solutions, but they implement them.

In this chapter, we will cover the following key topics:

- Why agents need tools
- Building your first tool-enabled agent
- Using pre-built tools from the Strands Agents ecosystem

- Using custom tools
- Quick start with Strands Agent Builder

Technical requirements

Before diving into this chapter, ensure you have:

- Working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with access to Amazon Bedrock and permissions for S3, DynamoDB, and Lambda (for AWS integration examples)
- Familiarity with REST APIs and JSON (helpful but not required)

> **NOTE**
>
> Throughout this chapter, some code examples create an agent without specifying a model. By default, Strands uses a model that may change over time or may not be available in your account. If you encounter errors or would like to specify a model, pass the model parameter explicitly—for example, `Agent(model="us.anthropic.claude-sonnet-4-20250514-v1:0", tools=[my_tool])` . Check the Amazon Bedrock console for the latest model IDs available in your region.

Why do agents need tools?

Suppose you've built an agent to help manage your development workflow. The Agent understands your git commands, can explain CI/CD concepts, and can answer project management questions. Now you decide to use it. You ask the agent:

```
How many open issues do I have in my project repo, and which one has been sitting there the longest?
```

You get a response:

```
I can't access your GitHub account, but you can check by going to your repository's Issues tab and sorting by creation date.
```

You already know how to check GitHub. That's why you asked the agent. What we've done is built a chatbot that can only respond generically to what it knows but cannot access your GitHub to give you the actual response you asked.

LLMs are remarkable at understanding and generating text—they can write beautiful essays, explain complex concepts in simple language, and provide code suggestions. However, they have several critical limitations including:

- A knowledge cutoff date (models can't access events or data beyond their last training update)
- Lack of real-world interaction (they can't read files, make API calls, or execute tasks)
- No persistent memory (they forget past conversations once the session ends)
- No built-in verification capability (they can't fact-check or confirm whether their outputs are correct)

But when you attach tools, you let agents interact with real systems, like querying GitHub APIs, reading files and databases, checking server status, etc.

Let's ask the same question to the agent with the GitHub tool associated. The process looks like this:

1. The agent figures out it needs to query GitHub's API
2. It calls the GitHub tool: `list_issues(repo="myproject", state="open")`
3. Back comes the data: 23 open issues, with `issue #47 "Memory leak in data processor" created 127 days ago`
4. The LLM then responds:

```
You've got 23 open issues. The oldest is #47 about a memory leak in the data processor—it's been open for 127 days. Probably worth tackling that one.
```

The difference here you'll see is, without tools, an agent is just an LLM chatbot that will tell you how to find information, whereas when you associate tools to an agent, it will find the information for you.

Tools let LLMs do more than just chat. They can actually interact with databases, APIs, and other systems. For e.g., you could use a weather API tool to fetch the current weather data of your city, a news API tool to access the latest headlines, an email tool to send an email message, a file system tool to read and write files, a calculator tool to perform precise mathematical operations, etc. During our course of this chapter, you are also going to see some of these examples—how you can build these tools and integrate with Strands Agents.

Think of the relationship between an agent and its tools like this:

The LLM is the brain : It understands requests, reasons about what needs to be done, and decides which tools to use

- Tools are the hands and senses: They perform specific actions and gather information from external systems/world

The agent is the coordinator : It manages the conversation between the LLM and the tools, ensuring everything works together smoothly, as shown in *Figure 2.1* :

Figure 2.1: LLM, Agent, and Tools

So now that you understand why agents need tools, here are some of the types of tools that can help you think about what your agent might need:

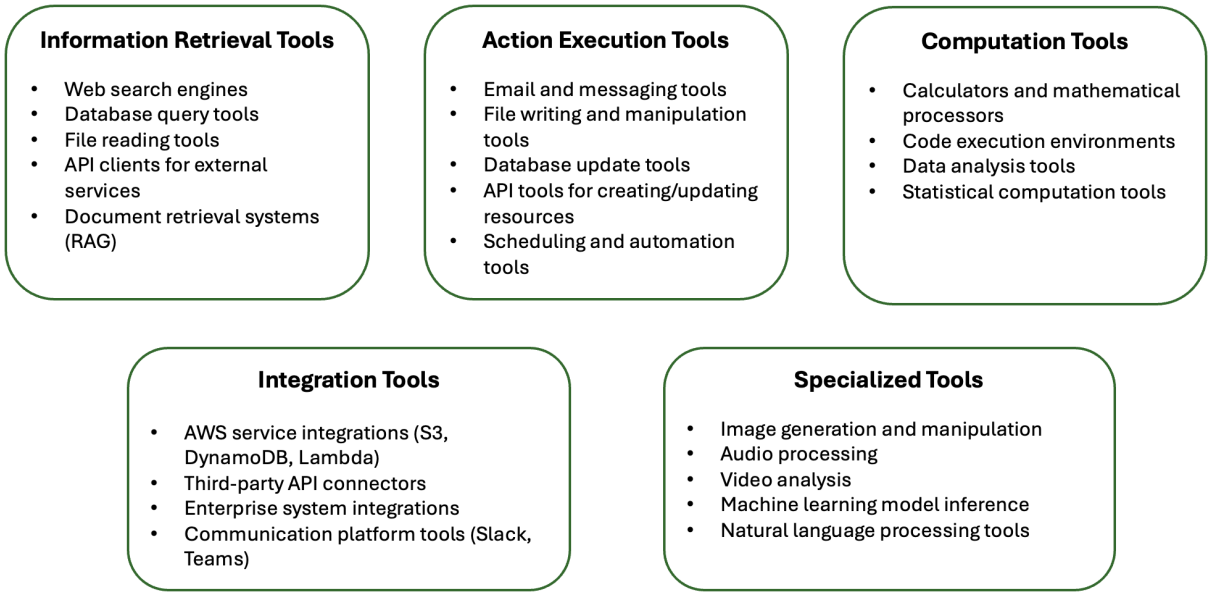


Figure 2.2: Type of tools

Over the course of this chapter, you are going to look at some of these tools. But first, let's understand how to transform a simple Python function into a tool.

From function to tool

Before you build agents, let's see how to take a simple python function and turn it into something the Agent can use. Let's say you're building an agent to help manage your development workflow. You already have a Python function that checks if your staging server is running:

```
import requests
def check_server_status(server_url):
    """Check if a server is responding."""
    try:
        response = requests.get(server_url, timeout=5)
        return f"Server is up. Status code: {response.status_code}"
    except requests.exceptions.RequestException:
        return "Server is down or unreachable"
# You use it like this:
status = check_server_status("https://staging.myapp.com")
print(status)  # "Server is up. Status code: 200"
```

This function works when you call it directly. Now you want your agent to use it. You ask the agent, `I s the staging server running?`

The agent responds:

```
I don't have the ability to check server status. You can use curl or ping to check if your server is
responding.
```

Right now, your agent doesn't know that you have a function in a Python file somewhere that performs this task. The agent can't magically discover the `check_server_status` function and use it.

This is where function calling (also called tool use) comes in. To make this function available to your agent, add three things: the `@tool` decorator, type hints, and a proper docstring. The `@tool` decorator tells Strands to make this function available to agents. Type hints tell the agent what data types to expect. The docstring describes what the function does so the agent knows when to use it:

```
from strands import tool
import requests
@tool  # Decorator
def check_server_status(server_url: str) -> str:  # Type hints
    """Check if a server is responding by making an HTTP request.
    Args:
        server_url: The URL of the server to check  # Docstring
    Returns:
        A message indicating whether the server is up or down
    """
    try:
        response = requests.get(server_url, timeout=5)
        return f"Server is up. Status code: {response.status_code}"
    except requests.exceptions.RequestException:
        return "Server is down or unreachable"
```

The function makes an HTTP GET request to the server URL with a 5-second timeout. If the server responds, it returns the status code (like 200 for success). If the request fails (server is down, network error, or timeout), it catches the exception and returns an error message.

Now give this function to the agent:

```
from strands import Agent
agent = Agent(tools=[check_server_status])
response = agent("Is the staging server running?")
```

The agent can now call this function. When someone asks about server status, the agent calls `check_server_status` with the right URL and gets the actual result back.

This pattern works for any function. Add @tool, give it to your Agent, and the agent can now execute it.

When one tool isn't enough

So, you just looked at how you can go from a function to a tool. But let's say you have multiple tools and you want to use with your agent. Suppose you are building a sales assistant, where you ask the agent to *pull last quarter's sales data and email a summary to the team* .

That's not one task, but three:

1. Query the database for sales data.
2. Analyze the numbers.
3. Send an email.

You can create three specialized tools, with the following code:

```
from strands import Agent, tool
@tool
def get_sales_data(quarter: str) -> dict:
    """Retrieve sales data for a specific quarter."""
    return {"revenue": 1250000, "deals": 47, "quarter": quarter}
@tool
def analyze_sales(revenue: int, deals: int, quarter: str) -> str:
    """Calculate key metrics from sales data."""
    avg_deal = revenue / deals
    return f"Q{quarter}: ${revenue:,} revenue, {deals} deals, ${avg_deal:,.0f} avg deal size"
@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email message."""
    return f"Email sent to {to}"
agent = Agent(tools=[get_sales_data, analyze_sales, send_email])
```

Each tool handles one specific job. The database tool only queries data. The analysis tool only crunches numbers. The email tool only sends messages. This way, you can reuse them in different combinations. Need to analyze data from a different source? Use the same analysis tool. Need to email something else? Use the same email tool. The agent will figure out which tools to use and in what order.

> **TIP**
> Build focused, single-purpose tools. Avoid the temptation to create one mega-tool that does everything. It becomes harder to maintain, harder for the agent to reason about, and impossible to reuse in other contexts.

Now when you ask the original question:

```
response = agent("Pull last quarter's sales data and email a summary to the team")
```

The agent:

1. Recognizes it needs sales data and calls `get_sales_data("Q4")`
2. Gets the data back, then extracts the values and calls `analyze_sales(revenue=1250000, deals=47, quarter="Q4")`
3. Gets the summary and calls `send_email("team@company.com", "Q4 Sales", summary)`

You didn't program that sequence and nor did you write *if user asks for sales summary, then do steps 1, 2, 3*. The agent figured it out by analyzing the request, selecting the right tools, and executing them in the right order.

Here are some other examples of using multiple tools with the agent:

- `Find the GitHub repo for project X, read its README, and create a summary document`

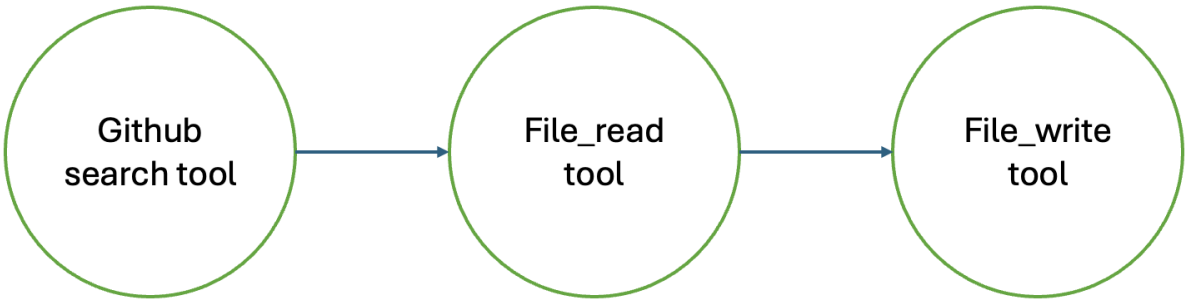


Figure 2.3: Multi tool example 1

- `Get customer order #12345, calculate refund amount with 15% restocking fee, and process the refund`

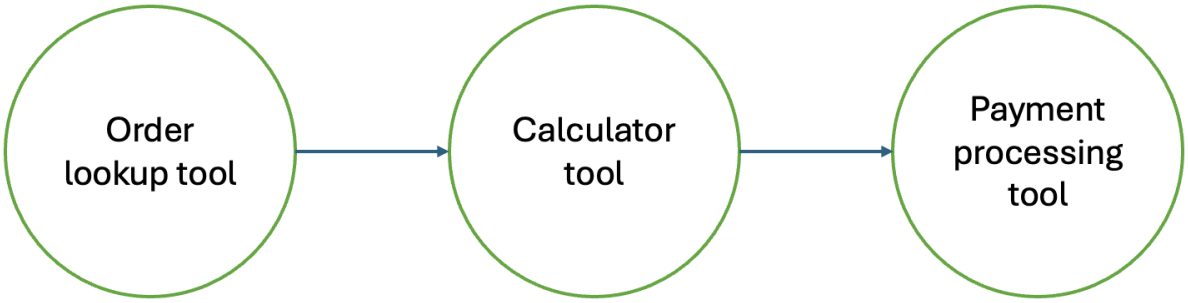


Figure 2.4: Multi tool example 2

This is what makes agents useful. They can use multiple tools together to accomplish complex, multi-step tasks.

We build focused, single-purpose tools. Each tool does one thing well. The agent figures out which tools to use, in what order, and how to pass data between them. You don't write workflows or state machines. You describe what each tool does, and the agent handles the rest.

Now that you understand why agents need tools and how they work, let's build one.

Building your first tool-enabled agent

Let's create something you can actually use, an agent that helps you calculate tips at restaurants. You know the situation, when you're at dinner, the bill arrives, and you need to figure out the tip and split it among friends. Let's build an agent that does this for you:

```
from strands import Agent, tool
# Step 1: Create a tool (just a Python function with @tool)
@tool
def calculate_tip(bill_amount: float, tip_percentage: float, num_people: int = 1) -> dict:
    """Calculate tip and split the bill among people.

    Args:
        bill_amount: Total bill amount in dollars
        tip_percentage: Tip percentage (e.g., 15, 18, 20)
        num_people: Number of people splitting the bill (default: 1)
    """
    tip = bill_amount * (tip_percentage / 100)
    total = bill_amount + tip
    per_person = total / num_people
    return {
        "bill": bill_amount,
```

```
        "tip": round(tip, 2),
        "total": round(total, 2),
        "per_person": round(per_person, 2)
    }
# Step 2: Create an agent and give it the tool
agent = Agent(
    model="us.anthropic.claude-sonnet-4-20250514-v1:0",
    tools=[calculate_tip]
)
# Step 3: Ask the agent a question
response = agent("The bill is $85. What's a 20% tip, and how much does each person pay if we're
splitting it 4 ways?")
print(response.message['content'][0]['text'])
```

When you run this, you'll see something like:

```
I'll calculate the 20% tip and split the bill among 4 people for you.
Tool #1: calculate_tip
Here's the breakdown:
- **Bill amount**: $85.00
- **20% tip**: $17.00
- **Total with tip**: $102.00
- **Amount per person** (splitting 4 ways): $25.50

So each person should pay $25.50.
```

Figure 2.5: Splitting the bill and calculating tip

So, what just happened? You asked a question in plain English, `The bill is $85. What's a 20% tip, and how much does each person pay if we're splitting it 4 ways?` and the agent:

1. Understood, you were asking about tip calculation
2. Extracted the numbers from your question ($85, 20%, 4 people)
3. Called `calculate_tip` tool with those numbers
4. Got the results back
5. Formatted them into a natural, helpful response

Here's what we didn't write:

- Logic to parse, `The bill is $85` and extract the number
- Code to recognize **20%** means `tip_percentage=20`
- Rules to figure out `splitting it 4 ways` means `num_people=4`
- Any routing logic or if/else statements

The agent figured all of that out. We just described what our tool does (through the function name, parameters, and docstring), and the agent handled the rest.

Try i t y ourself

Try asking different questions, such as:

- What's a 15% tip on $42?
- Bill is $120, we want to tip 18%, split between 3 people.
- Calculate tip for $67.50 at 20%.

The agent recognizes all of these as requests to calculate tips. It understands intent, not just keywords. You can phrase it however feels natural, and the agent will figure it out.

Let's look back at what you did. You wrote a Python function, added @tool to it, and gave it to an agent. Then suddenly you have an agent ready calculating tips in plain English:

- **Create a tool** (a Python function with `@tool`)
- **Give it to an agent** (add it to the tools list)
- **Let the model decide** when to use it

Whether you're building a tool that calculates tips or one that checks server status, you follow the same approach. The only difference is what happens inside your tool's implementation.

The agent just needs to understand three things about your tool: what it does, what information it needs, and what it gives back. We communicate all of that through your function's name, parameters, and docstring. The agent reads those, figures out when your tool is useful, and calls it automatically. Next, let us understand the broader ecosystem of tools with Strands Agents.

The Strands Agents tool ecosystem

We've just learned how to turn a Python function into an agent tool using `@tool` . The good news is you don't have to build every tool yourself.

When you start a new software project, do you write your own HTTP library? Your own JSON parser? Of course not. You use packages that others built.

Same goes for agent tools. Why spend three days building a web scraper when someone already solved that problem?

The Strands ecosystem gives you a head start. Instead of spending days building a file reader or AWS integration, grab pre-built tools and start building. This matters because the agents you'll create in later chapters use these same tools, just combined in more interesting combinations.

The Strands ecosystem provides flexibility without complexity through four pillars:

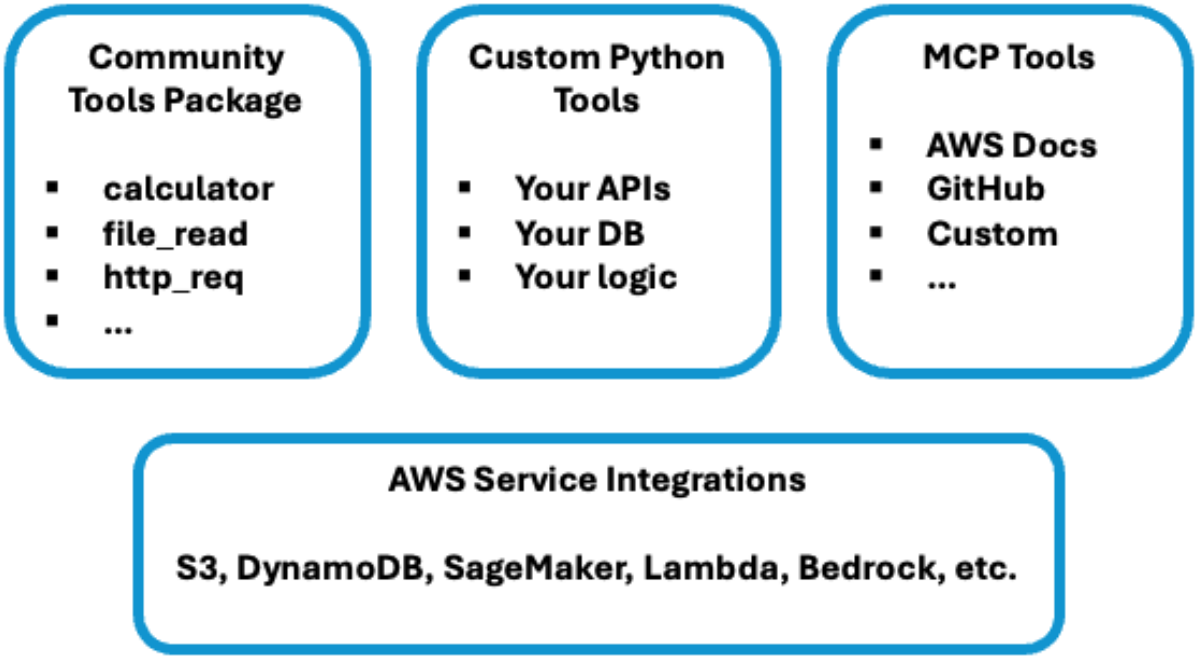


Figure 2.6: Strands tool ecosystem

Let's look at these four pillars in a bit more detail:

- **Pre- b uilt c ommunity t ools** : The `strands-agents-tools` package provides dozens of ready-to-use tools for common tasks like, file operations, shell commands, web searches, HTTP requests, mathematical calculations, AWS integrations, and more. Think of it like a package manager for agent capabilities. If someone has already built a file reader or calculator tool, you can use it immediately instead of building from scratch. You can read more about the `strands-agents-tools` package at `https://github.com/strands-agents/tools` .

- **Custom Python tools** : When pre-built tools don't meet your specific requirements, you can create your own. Maybe you need to integrate with your company's internal API, or you have domain-specific data processing logic. The `@tool` decorator lets you turn any Python function into an agent tool. We'll walk through how to build custom tools later in this chapter.
- **MCP tools** : With MCP, you have a whole ecosystem of standardized tools that work across different agent frameworks. If someone has built an MCP server for a service, such as a database connector or API wrapper, you can plug it right into your Strands agent without writing integration code. This is particularly useful for accessing third-party services or tools built by the broader AI community. We'll explore MCP in detail in Chapter 5 , but it's worth knowing it exists as another option. You can browse available MCP tools at `https://mcpmarket.com/` .
- **AWS service integrations** : Since we're building on AWS, the strands-agents-tools package includes AWS-specific tools, and you can also create custom tools that interact with S3, DynamoDB, Lambda, SageMaker, Bedrock, and other AWS services. This gives your agents direct access to your cloud infrastructure and data.

Start with built-in tools for basic functionality, add custom tools for your business logic, and integrate MCP tools for specialized services. The agent doesn't care where the tools come from. It just sees a unified catalog of capabilities.

Now let's dive deeper into the Strands Agent tool ecosystem one by one.

Pre-built community tools

Let's start with the tools that are already pre-built. These are dozens of tools that are available that handle common tasks right out of the box. The package consists of tools for file operations, web searches, HTTP interactions, AWS services, mathematical calculations, and even multi-agent orchestration, etc. You can see the list of tools and their details in the GitHub repository: `https://github.com/strands-agents/tools/tree/main`

Installing these tools is straightforward:

```
pip install strands-agents-tools
```

That's it. Now you have access to dozens of tools. Now let us look at one of the pre-built calculator tools in action:

```
from strands import Agent
from strands_tools import calculator
# Create an agent with the calculator tool
agent = Agent(
    model="us.anthropic.claude-sonnet-4-20250514-v1:0",
    tools=[calculator]
)
agent("What's 42 raised to the power of 9?")
agent("Solve the equation x^2 + 5x + 6 = 0")
agent("What's the derivative of sin(x) * cos(x)?")
```

The calculator tool can handle complex expressions, multiple operations, and mathematical functions that would take significant time to implement, and you got that done with just one import.

What's in the toolbox?

Let's take a tour of what's available. Think of these as categories of capabilities you can add to your agents instantly:

Category	Tools	What They Do
File Operations	`file_read` , `file_write` , `editor`	Read and write files, advanced editing with syntax highlighting
Mathematical Operations	`calculator`	Perform calculations, symbolic math, equation solving
Web and Search	`http_request` , `tavily_search` , `tavily_extract` , `exa_search` , `bright_data`	Make API calls, real-time web search, extract web content, web scraping
Code Execution	`python_repl`, `shell`, `code_interpreter`	Run Python code, execute shell commands, isolated sandbox execution

Category	Tools	What They Do
AWS Services	`use_aws` , `retrieve` , `nova_reels` , `agent_core_memory`	Interact with S3/DynamoDB/Lambda, query Knowledge Bases, create videos, store memories
Memory and Storage	`memory` , `mem0_memory` , `mongodb_memory` , `elasticsearch_memory`	Store and retrieve documents and memories across runs
Communication	`slack`, `speak`	Post to Slack workspaces, text-to-speech output
Image and Video	`generate_image`, `image_reader` , `search_video` , `chat_video`	Create and analyze images, semantic video search
Automation and Utilities	`browser` , `use_computer` , `cron` , `current_time` , `sleep` , `diagram` , `rss`	Web automation, desktop control, scheduling, time utilities, diagramming
Advanced Features	`swarm` , `workflow` , `use_llm`, `batch` , `mcp_client`	Multi-agent coordination, automated workflows, nested AI loops, parallel execution

Table 2.1: Tools and their capabilities

Using multiple tools together

You can give your agent multiple tools, and it will figure out how to use them together. Let's say you want to fetch stock price from Yahoo Finance, analyze it, and store it in a file. Let's see how Strands can help here:

```
import os
from strands import Agent
from strands_tools import http_request, calculator, file_write
agent = Agent(
    tools=[http_request, calculator, file_write],
```

```
    system_prompt="You help with data analysis tasks."
)
agent("""
Fetch stock data from https://query1.finance.yahoo.com/v8/finance/chart/AAPL?interval=1d =5d,
extract the latest closing prices,
calculate the average price over the period,
and save the results to stock_summary.txt inside Documents folder
""")
```

Notice how we give the agent a complex, multi-step task in plain English, and it automatically figures out the sequence: first use `http_request` to fetch the data, then calculator to find the average, and finally `file_write` to save the results. You don't need to write any logic to coordinate these tools—the agent handles the entire workflow for you.

AWS integration

When you're building agents on AWS, you'll often need them to interact with your cloud infrastructure: reading from databases, invoking serverless functions, or calling machine learning models.

Most real-world applications run on cloud infrastructure. Maybe your data lives in S3 buckets, your databases are in DynamoDB or RDS and your business logic runs in Lambda functions. If your agents can't interact with these services, they're disconnected from your actual systems.

Suppose you want to access AWS resources through the Agent, you can use the tool use_aws:

```
from strands import Agent
from strands_tools import use_aws
agent = Agent(
    tools=[use_aws],
    system_prompt="You are an AWS assistant that helps manage cloud resources."
)
agent("List all S3 buckets in my account")
agent("Show me the contents of the data-archive bucket")
```

The `use_aws` tool translates your requests into AWS CLI commands and executes them with your configured credentials.

Let us look at some of the practical scenarios of AWS Integration with Strands Agents –

- Data Analysis Agent: Let's say your agent wants to read CSV files from S3, process them, and store results back to S3:

```
from strands import Agent
from strands_tools import use_aws, calculator
agent = Agent(
    tools=[use_aws, calculator]
)
response = agent("Read the sales data from S3 bucket 'my-bucket' key 'sales-2024.csv',
calculate the total revenue, and save a summary back to S3")
```

- Customer Service Agent: Your agent queries DynamoDB for customer information and updates records:

```
from strands import Agent
from strands_tools import use_aws
agent = Agent(
    tools=[use_aws]
)
response = agent("Look up customer ID 12345 in the DynamoDB customers table and update their
email to newemail@example.com")
```

- Workflow Orchestration Agent: Your agent invokes Lambda functions to trigger business processes:

```
from strands import Agent
from strands_tools import use_aws
agent = Agent(
    tools=[use_aws]
)
response = agent("Invoke the Lambda function 'order-processor' with order ID 67890")
```

Look at these three examples. You described what you wanted in plain English, i.e., `read this file` , `update this record` , `invoke this function` , and the agent figured out the AWS API calls. You didn't write boto3 code, you didn't handle AWS responses and you didn't even specify which AWS operation to use. The agent did all of that.

NOTE

These examples assume the AWS resources already exist in your account. The S3 bucket, CSV file, DynamoDB table, Lambda function, and email addresses referenced in the code must exist, for the code snippets to work.

That's the pattern: one tool (`use_aws`), many services (S3, DynamoDB, Lambda), natural language requests. The agent bridges the gap between what you want and how AWS APIs work.

Now that you have explored how to use the community-built tools with Strands Agents, let us look at how you can build your own custom tools using Python.

Building custom tools

When community tools don't fit your needs, you build custom tools. Maybe you're integrating with an internal API that only your company uses. Maybe you need to query a proprietary database with specific business logic. Or perhaps you're building something entirely new that no existing tool covers.

Pre-built tools can't do these things. That's when you build your own. Earlier in the chapter, we built a very simple custom Python tool using the `@tool` decorator. But let us now take a different real-world example.

Let's say you run an online store. Customers ask, `Is product X in stock all the time ?` You need a tool that checks your inventory database:

```
from strands import tool
@tool
def check_inventory(product_id: str) -> str:
    """Check if a product is in stock.

    Args:
        product_id: The product ID to check (e.g., "PROD-123")
    """
    # This is where you'd query your actual database
    # For now, let's simulate it
    inventory = {
        "PROD-123": 15,
        "PROD-456": 0,
        "PROD-789": 8
    }
    quantity = inventory.get(product_id, 0)
    if quantity > 0:
        return f"Product {product_id} is in stock. We have {quantity} units available."
    else:
        return f"Product {product_id} is currently out of stock."
```

We've created a function called `check_inventory` that takes a product ID and returns stock information. The `@tool` decorator makes this function available to agents. Without it, this would just be regular Python code.

The docstring explains what the tool does. When a user asks, `Is PROD-123 in stock?` , the agent reads this description and recognizes that `check_inventory` handles this request. The `Args:` section tells the agent what parameter to extract from the user's question.

Inside the function, we've simulated a database with a dictionary. In production environment, you'd replace this with actual database queries or API calls to your inventory system. The agent calls the function with a product ID and uses the returned message in its response. Once defined, you can ask about inventory any way that feels natural:

```
from strands import Agent
agent = Agent(tools=[check_inventory])
# All of these work:
agent("Is PROD-123 in stock?")
agent("Do we have PROD-456 available?")
agent("Check inventory for PROD-789")
agent("Can I order PROD-123 right now?")
```

Each question is phrased differently, but the agent recognizes them all as inventory checks. In traditional chatbot development, you would write regex patterns or intent classifiers to handle these variations. You would map *in stock* , *available* , and *can I order* to the same intent. You would extract the product ID from different sentence structures.

Here, we skipped all that. You wrote a function with a clear description, and the agent figured out when to call it. The language model handles the intent recognition and parameter extraction.

To really internalize how this works, it helps to try building a tool for a problem you actually care about. Here are some ideas:

- Check the weather in your city (call a weather API)
- Validate email format (check if email looks valid)
- Calculate age from birthdate (date → age in years)
- Calculate BMI (height + weight → BMI and category)
- Calculate shipping costs (based on weight and distance)

Pick one of these ideas and implement it. Don't worry if the logic is simple or a bit rough around the edges. What matters is seeing how a custom tool fits into the agent workflow.

Right now, you know enough to build the majority of the tools you'll need. But as you build more agents, you'll eventually run into these three scenarios. Let's look at them now so you recognize them when they happen; you don't need to memorize this, just know it's here when you need it.

When simple tools aren't enough

You've been building tools with the `@tool` decorator. Your agent has five tools: check stock, update stock, get supplier info, track shipments, and generate reports. Then one day you deploy your inventory agent to production. Within an hour, your database administrator sends you a Slack message: `Why is your agent opening 50 database connections per minute?`

You check the logs and find each tool opens its own database connection. When the agent processes a single request, it might call three or four of these tools. That's three or four separate connections for one user request. Multiply that by dozens of concurrent users, and suddenly your database server is drowning in connections.

The `@tool` decorator works perfectly for simple cases. But as your agents grow more sophisticated, you'll hit situations where you need more control. Let's look at three scenarios where the simple approach shows its limits and what to do about them.

The database connection problem

Back to that Slack message from your database administrator. Let's look at what your code is doing

```
from strands import tool
@tool
def check_stock(product_id: str) -> str:
    """Check product stock."""
    db = connect_to_database()
    result = db.query("SELECT quantity FROM inventory WHERE id = %s", (product_id,))
    db.close()
    return result
@tool
def update_stock(product_id: str, quantity: int) -> str:
```

```
    """Update product stock."""
    db = connect_to_database()
    db.execute("UPDATE inventory SET quantity = %s WHERE id = %s", (quantity, product_id))
    db.close()
    return f"Updated {product_id} to {quantity} units"
```

It works, but there's a problem. Every time the agent calls a tool, your database server logs show a new connection being opened and closed. Five tools mean five separate connections per request cycle. When you have dozens of users hitting the agent simultaneously, those connections add up fast.

Opening database connections is expensive. Each connection takes time to establish (typically 50-100 milliseconds), uses memory on the database server, and most databases have hard limits on concurrent connections. MySQL defaults to 151 connections. PostgreSQL defaults to 100. Hit that limit and your agent starts failing with `too many connections` error.

Here's what you would need to do: connect once when the agent starts, then share that connection across all your tools. But the decorator approach doesn't give us a way to do that. Each function is independent, with no way to share state between them.

The solution is to group related tools inside a class:

```
from strands import tool
class InventoryTools:
    def __init__(self):
        self.db = connect_to_database()  # Replace with your actual database connection
    @tool
    def check_stock(self, product_id: str) -> str:
        """Check product stock."""
        return self.db.query(f"SELECT quantity FROM inventory WHERE id='{product_id}'")
    @tool
    def update_stock(self, product_id: str, quantity: int) -> str:
        """Update product stock."""
        self.db.execute(f"UPDATE inventory SET quantity={quantity} WHERE id='{product_id}'")
        return f"Updated {product_id} to {quantity} units"
```

Now you create one instance of the class and pass its methods to your agent:

```
inventory = InventoryTools()  # Database connection happens here
agent = Agent(tools=[inventory.check_stock, inventory.update_stock])
```

All your tools share the same database connection through `self.db` . So, we'll have one connection and five tools, making the implementation both faster and cleaner.

This pattern works for any shared resource such as, API clients that maintain connection pools, file handles for reading large datasets, cache connections to Redis or Memcached or authentication tokens that need to be refreshed periodically.

Slow warehouse checks

Now two days later, your product manager messages you: *Users are complaining the agent is too slow. It takes 8 seconds to check if you can fulfill an order. That's way too long.*

You look at the code. When a customer asks `Can you ship 100 units of PROD-123?` , your agent needs to check inventory across three warehouses. Each warehouse has its own API, so you built a `check_warehouse_inventory` tool that takes a warehouse parameter. The agent calls it three times, one after another:

```
warehouse_a = check_warehouse_inventory("PROD-123", "warehouse_a")  # 2 seconds
warehouse_b = check_warehouse_inventory("PROD-123", "warehouse_b")  # 2 seconds
warehouse_c = check_warehouse_inventory("PROD-123", "warehouse_c")  # 2 seconds
total = combine_inventory(warehouse_a, warehouse_b, warehouse_c)    # 2 seconds
# Total: 8 seconds
```

The total time it took is 8 seconds. Your users are staring at a loading spinner for eight seconds. You check your analytics and see users abandoning their carts before the agent even responds.

However, these three warehouse API calls don't depend on each other. Warehouse B doesn't need to wait for Warehouse A's response. They're completely independent operations and could all happen at the same time.

When you mark a function as async, Python's event loop can pause it at await points while waiting for I/O operations (like API responses) and switch to other async work on the same thread. Multiple async functions can run concurrently, all making progress together through cooperative multitasking.

Here's how you can fix the warehouse check tool:

```
import asyncio
from strands import Agent, tool
```

```python
@tool
async def check_warehouse_inventory(product_id: str, warehouse: str) -> int:
    """Check inventory at a specific warehouse."""
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"https://api.warehouse-{warehouse}.com/inventory/{product_id}"
        )
        return response.json()["quantity"]
async def main():
    agent = Agent(tools=[check_warehouse_inventory])
    response = await agent.invoke_async(
        "Can we ship 100 units of PROD-123? Check all warehouses."
    )
    print(response.message['content'][0]['text'])
asyncio.run(main())
```

When the agent needs to check all three warehouses, Strands automatically runs them in parallel. All three API calls happen simultaneously. Time it takes is 2 seconds instead of 6 seconds for the warehouse checks, plus 2 seconds to combine the results. So total time is 4 seconds instead of 8.

The async and await keywords tell Python this function can pause and resume. When `check_warehouse_inventory` hits the await on the API call, Python doesn't sit there waiting. It switches to another warehouse check that's ready to run. All three calls progress together, like those three restaurant orders being prepared simultaneously.

> **NOTE**
>
> Not every tool benefits from async execution. If a tool runs almost instantly (say, under 100 milliseconds) or performs pure calculations without external calls, async adds unnecessary complexity without measurable gains. It's usually best to start with regular tools and add async support when you need speed.

Choosing your approach

Start with the decorator. It's simple and handles most cases. Build your tool, test it, deploy it, and see what happens. If you run into database connection problems, group your tools in a class to share resources. If external API calls are slow, make them async so they run concurrently. Let real problems guide you, not theoretical ones.

One more thing, if you need to enforce strict validation rules on tool inputs (like specific formats or allowed values), you can define explicit schemas using TOOL_SPEC. We won't cover that here since it's for advanced use-cases, but if you would like to go deeper into it, you can go through the documentation: https://strandsagents.com/latest/documentation/docs/user-guide/concepts/tools/custom-tools/.

In addition, the GitHub repository (https://github.com/strands-agents/samples/tree/main/python/01-learn/02-tools-and-mcp) contains detailed examples of both approaches with complete, runnable code.

Now that you know how to build custom tools, if you want to experiment quickly without writing all the setup code, there's an even faster way to get started.

Quick start with Strands Agent Builder

By now, you've learned quite a bit about building agents with tools. You've seen how to create custom tools with the `@tool` decorator, use pre-built tools from the Strands ecosystem, connect to MCP servers for external capabilities, and even gotten a glimpse into building your own MCP servers.

But you might be thinking: *This is powerful, but setting up all the code, imports, and configurations takes time. When I'm learning or want to test an idea quickly, is there a faster way to experiment?*

Absolutely. Let's look at Agent Builder, which is a command tool designed specifically for rapid prototyping and learning.

Agent Builder is a command-line tool that lets you build agents by describing what you want in plain English. Instead of writing code, imports, and configurations, you just say what you want and Agent Builder creates it. Think of it as training wheels for building agents; you get to experiment and learn without worrying about setup.

Let's see Agent Builder in action. Install it and build your first agent:

```
# Install Agent Builder
pipx install strands-agents-builder

# Start interactive mode
strands
```

Once it starts, you'll see a prompt. Try asking:

```
Build a tool that converts Celsius to Fahrenheit.
```

Agent Builder will:

1. Create the conversion tool
2. Test it with example values (for example, *What's 100°C in Fahrenheit?*)
3. Show you the result: `100°C is 212°F`
4. Let you try more conversions

You just built a tool by describing what you want. No code, no imports, no setup.

Agent Builder operates in two modes:

- **Interactive mode** : Run `strands` and have a conversation with the builder. Ask it to create tools, test agents, or explain concepts.
- **Command mode** : Give it specific instructions directly:

  ```
  strands "Build a tool that monitors API rate limits across all services and alerts at 80% capacity"
  ```

Agent Builder comes with powerful integrated tools that let you build sophisticated agents immediately:

- **Data processing agents** : Use the calculator, Python REPL, and file editor tools to analyze data
- **Web-connected agents** : Use HTTP request and Slack tools to interact with APIs and services
- **AWS-integrated agents** : Use the use_aws tool to manage cloud resources
- **Multi-agent systems** : Use the swarm and workflow tools to coordinate multiple agents
- **Creative agents** : Use image generation and video creation tools powered by Amazon Bedrock

When you use Agent Builder, here's what's happening under the hood:

1. Agent Builder registers its built-in tools with the Strands framework

2. It sets up a model (Claude Sonnet 4, at the time of writing) with optimized settings for tool use
3. Your requests go to the model, which decides which tools to call
4. The framework executes tools and returns results to the model
5. The model synthesizes tool outputs into natural language responses

This is the same pattern you'll use when building agents manually. Agent Builder just handles the setup for you.

Agent Builder runs interactively and doesn't automatically save generated tool code to files. If you want to persist tools across sessions, use the `--kb` flag with an Amazon Bedrock Knowledge Base ID, which lets you store and retrieve tools between sessions. Alternatively, you can copy the generated code from the session output into your own Python files.

If you want to dive deeper, the Agent Builder GitHub repository (`https://github.com/strands-agents/agent-builder`) includes full documentation and setup instructions.

Agent Builder is perfect for learning and experimentation, but as you build production applications, you'll want the full control and customization that come with writing your own agents using the patterns we've covered in this chapter.

Building agents on LangChain

We've spent this chapter building agents with Strands. If you've searched for `how to build AI agents` , you've probably come across LangChain. It's been around longer and has a large community, so it's worth mentioning.

Both frameworks do the same core thing: turn Python functions into tools and let agents use them. The difference is in how they approach it. Strands uses a model-driven approach where you write minimal code and let the LLM figure out the workflow. LangChain gives you more explicit control over how agents execute tasks, which means more setup code but also more customization options.

For this chapter, we're using Strands because it's simpler to learn and integrates naturally with AWS services like Bedrock, Lambda, and S3. If you're working in a different environment or need the flexibility that comes with

LangChain's larger ecosystem, check out their documentation at `https://docs.langchain.com/oss/python/langchain/agents` .

Summary

In this chapter, you learned how to turn conversational agents into agents that actually do things. You started with the problem: LLMs are great at generating, but they can't check databases, call APIs, or interact with the real world. Then we solved it by giving agents tools.

You saw how to transform any Python function into a tool with the `@tool` decorator, explored the pre-built tools in the Strands ecosystem, and learned how to integrate with AWS services like S3, DynamoDB, and Lambda. At the end, you built agents that don't just chat about solutions, but also execute them.

Chapter 3 , *Agent Memory* , tackles the next limitation. Your agents forget everything between conversations. We'll fix that using AWS services like AgentCore Memory and Mem0. You'll learn about short-term and long-term memory, context management strategies, and build a personalized agent that remembers user preferences across sessions.

Subscribe to our newsletter

New frameworks, evolving architectures, research drops, production breakdowns— *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe for weekly insights that help you stay focused and informed.

Subscribe at `https://packt.link/8Oz6Y` or scan the QR code below.

OceanofPDF.com

3

Agent Memory

In the previous chapter, you learned the fundamentals of AI agent tools and built an agent that could understand requests, generate intelligent responses, and use tools to complete tasks. That was an important first step: giving an agent the ability to reason and act. However, even a capable tool-using agent remains limited if it cannot remember previous conversations, retain user preferences, or build on past interactions over time. Without memory, every interaction starts from scratch, making the agent feel transactional rather than truly intelligent.

This is where agent memory becomes essential. Memory allows agents to move beyond one-off question answering and into more natural, personalized, and context-aware experiences. With memory, an agent can remember what a user asked earlier, preserve important facts across sessions, adapt responses based on preferences, and maintain continuity in long-running workflows. In practical terms, memory is what transforms an agent from a reactive assistant into a more reliable collaborator.

This chapter is a natural progression from the previous one. Once an agent can use tools, the next challenge is helping it retain and manage knowledge effectively. Using tools gives agents the ability to act, while memory gives them the ability to learn from interaction history and sustain context. Together, these capabilities form the foundation for building robust conversational and task-oriented agents that feel coherent, useful, and increasingly personalized over time.

In this chapter, you will learn all about agent memory through the following key topics:

- What is agent memory and why memory matters in AI agents

- Types of agent memory: Short-term or working memory and Long-term memory
- Implementing memory systems with AWS services
- Context management and memory optimization
- Using Strands SDK, Mem0, Amazon Bedrock, and Web Search for building a personalized agent
- Amazon Bedrock AgentCore Memory: Short-term and long-term memory
- Hands on: Building a math agent with AgentCore memory and LangChain

Prerequisites

Before diving into this chapter, ensure you have:

- A working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with appropriate permissions (for AWS integration examples)
- Familiarity with REST APIs and JSON (helpful but not required)
- Kiro IDE for development of code (optional)

What is agent memory

Agent memory is an agent's ability to retain past information for maintaining context and improving decision-making capabilities.

For example, let's consider a chat with a travel agent. So, when the user asks the question, `what are good places to visit in NYC` , the system replies, `Times Square is a very common attraction` . Similarly, when the user asks a follow-up question, `What ' s the best time to visit it?` , the chat agent understands the context that *it* refers to Times Square and responds back, `the best time to visit Times Square is...` . From this example, it's clear the agent memory context is giving the agent the ability to recall information and make the conversation seamless.

Let's start with two scenarios: agent with memory, and agent without memory.

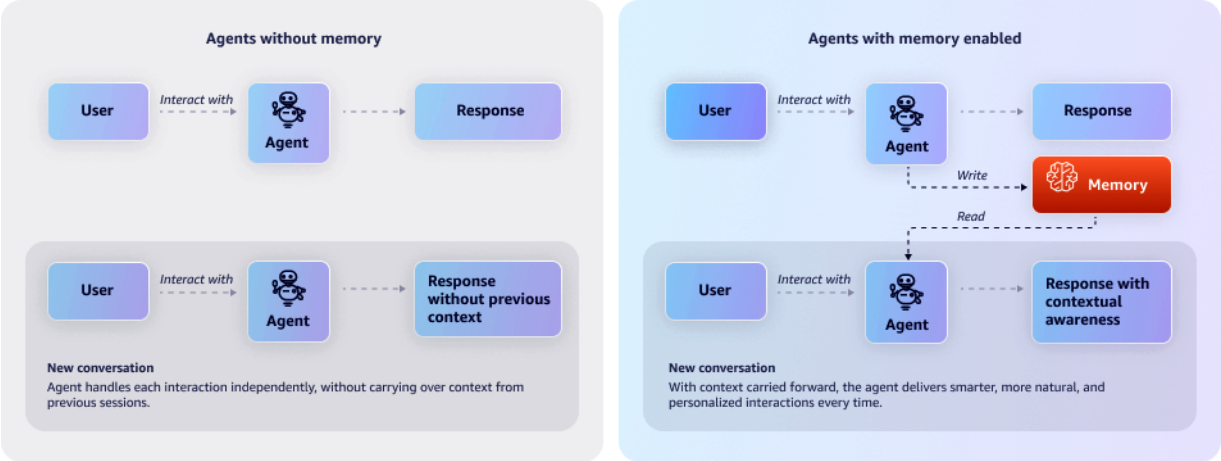


Figure 3.1: Workflow of an agent with memory and without memory

Figure 3.1 compares **agents without memory** and **agents with memory enabled** .

On the **left side** , the agent behaves like a stateless system. A user sends a request, the agent processes it, and returns a response. When a **new conversation** starts, the agent does not remember anything from earlier interactions, so it treats every exchange as completely independent. That is why the second row shows the agent giving a **response without previous context** .

On the **right side** , the agent is connected to a **memory layer** . During interactions, the agent can **write** useful information to memory and later **read** it back. Because of that, when the user returns in a new conversation, the agent can use past information, preferences, or earlier discussion history to generate a **response with contextual awareness** . In simple terms, the diagram shows that memory turns an agent from a one-off responder into a more personalized, consistent, and context-aware assistant.

Here is a summary of the interaction:

Feature	Agents Without Memory	Agents With Memory
Context	Reset every time a session ends	Carried forward across sessions
Personalization	Low; generic responses	High; tailored to the user's history

Feature	Agents Without Memory	Agents With Memory
Efficiency	User must repeat information	Agent remembers and builds on past info
User Experience	Feels like talking to a stranger every time	Feels like a continuous relationship or assistant

Table 3.1: Agent comparison with memory and without memory

Why does memory matter?

The three key benefits of having the memory capability are:

- **Contextual Intelligence:** It's not just about storing data; it's about understanding the *why* and *how* behind a request. By recognizing patterns in how you work or what you ask, the AI can give answers that are more accurate and relevant to your specific situation. It understands the *big picture* of your project rather than just treating every prompt as a random, isolated question.
- **User Preferences:** The AI builds a **profile** of how you like to interact and creates a personalized experience. If you prefer a professional tone, specific formatting, or have previously discussed certain constraints, the AI remembers those details across many different sessions. You don't have to repeat your instructions (for example, `always keep it under 100 words`) every time you start a new chat.
- **Knowledge Retention** : This is long-term, *persistent* memory that grows over time and allows for *continuous learning* . Instead of the AI being static, it becomes a more sophisticated problem-solver the more you use it. It retains information about complex problems you've solved together in the past, allowing it to act as a more knowledgeable partner in the future.

Types of agent memory

When building AI agents, memory becomes a core design consideration. Unlike traditional language models, which are largely stateless and depend on the current prompt and context window, agents often need mechanisms to retain and use information over time. A useful way to think about this is through cognitive architectures such as CoALA, which organize an agent around key functions such as reasoning, action, and memory. For this chapter,

the most important idea is that memory is not a single thing. Agents may need short-term memory to track the current interaction, long-term memory to retain stable knowledge across sessions, and episodic memory to recall past interactions or experiences. Understanding these different memory types provides a practical foundation for designing agents that can behave more coherently, personalize responses, and handle multi-step tasks more effectively.

Figure 3.2 shows various types of agent memory:

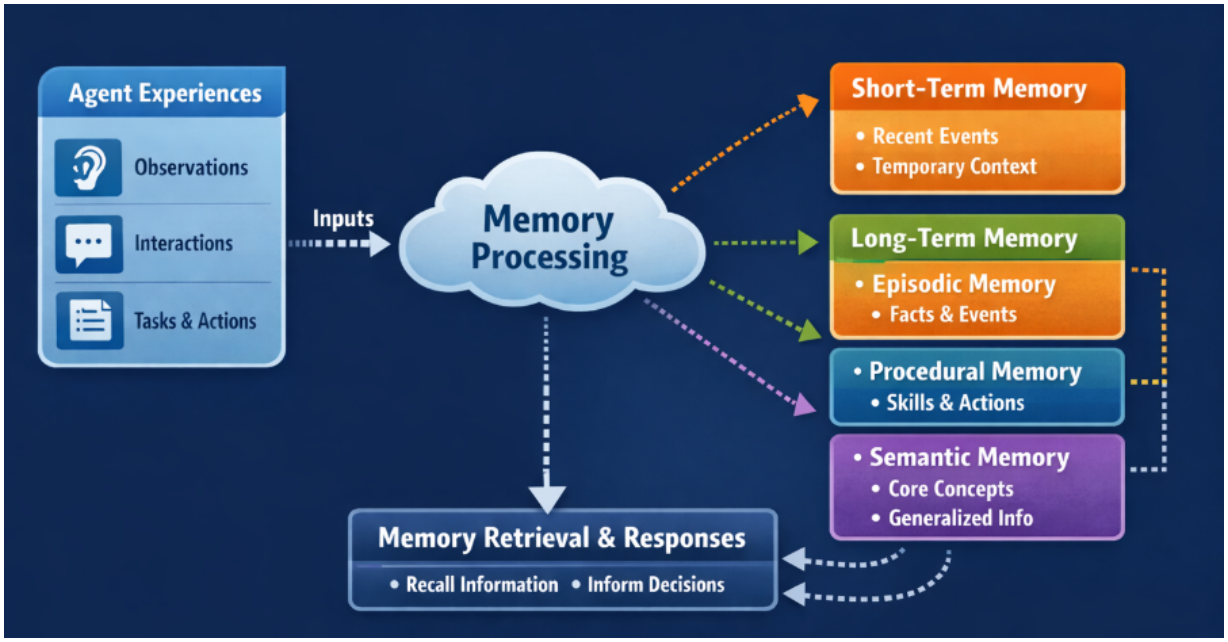


Figure 3.2: Types of agent memory

Short-term memory or working memory

This is the agent's *short-term* scratchpad. It contains whatever the AI is focusing on at this exact moment. This memory is the reason why a chatbot can feel like a natural conversation partner. Instead of treating every message you send as a completely new and isolated event, the AI uses its short-term memory to keep track of the context. For example, in a tool like ChatGPT, the system retains the history of your current session so it can provide answers that actually follow what has already been said. Without this, the AI would *forget* your previous question the moment you asked a follow-up.

Technically, this is managed through a context window or a rolling buffer. This is essentially a limited amount of space for data; as you continue to talk

and new information comes in, the oldest information is eventually pushed out and overwritten to make room.

The main drawback is that this memory is designed to keep a single interaction smooth, but it doesn't save anything once the session is over. Because it resets frequently, short-term memory alone isn't enough for an AI to truly learn about you or remember your personal preferences over a long period of time.

Long term memory

Long-term memory moves the AI agent beyond simple conversations and turns it into a system that can remember your history, understand complex facts, and provide high-level expertise based on everything it has ever learned. It has two major capabilities:

- **Permanent Storage:** Unlike short-term memory, which resets, long-term memory is built using databases or vector embeddings. This helps the AI understand how different pieces of information relate to each other and allows the AI to store massive amounts of history that can be accessed at any time.
- **Personalization and h istory:** This memory is essential for AI assistants that need to know your background. For instance, a customer support agent with long-term memory can remember that you had a specific problem three months ago. Instead of asking you the same basic questions again, it can tailor its response based on your history, making the interaction personalized, much faster, and more helpful.

According to the CoALA framework, there are three types of long-term memory: Episodic, Semantic, and Procedural. Let's understand these:

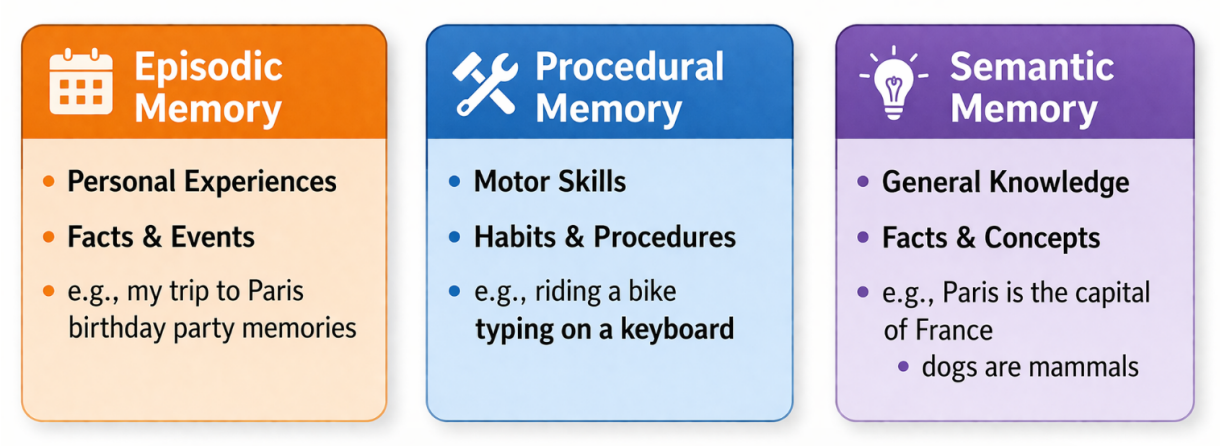


Figure 3.3: Types of long-term memory

Episodic memory is what allows an AI agent to look back at its own personal history. It functions similar to human brains, remembering specific life events such as a specific meeting or a past conversation rather than just general facts. For example, memories of your trip to Paris or your birthday.

By recalling these individual experiences, the AI can engage in case-based reasoning. This means that when the AI faces a new challenge, it can look back at a similar situation from its past to see what actually happened. By analyzing whether its previous actions led to a success or a failure, the AI system learns to make much better decisions in the future.

To make this work, AI keeps a structured log of its lifecycle, recording:

- What happened (the event)?
- Why did it act that way (what was the reasoning)?
- What did it do (the action)?
- What the result was (the outcome)?

For example, an AI financial advisor uses episodic memory to remember a user's specific investment choices for several years in the past. It doesn't just know about the stock market in general; it remembers *your* history to provide better, more personalized recommendations.

Semantic memory serves as an AI agent's internal library of **structured factual knowledge** . While episodic memory focuses on personal experiences and events, semantic memory is dedicated to generalized information—things like facts, definitions, formulas, and universal rules.

In simple terms, episodic memory enables an AI system to remember a specific conversation it had with you, whereas semantic memory helps it know the actual definitions of the words used in that conversation.

To handle this, AI agents use specialized tools like **knowledge bases, symbolic AI, or vector embeddings** . These technologies allow the agent to organize vast amounts of data so it can understand relationships between concepts and retrieve the right information almost instantly when it needs to solve a problem.

This type of memory is essential for AI systems that require **domain expertise** . It is often found in high-level applications such as:

- **Medical tools:** Recalling symptoms and drug interactions.
- **Business systems:** Managing company-wide policies and data.
- **Legal assistants:** Legal AI systems using semantic memory to search through thousands of past legal cases and precedents to provide accurate, fact-based advice.

Semantic memory provides AI with a stable foundation of *world knowledge* , allowing it to reason through complex tasks using facts and rules rather than just guessing based on patterns.

Procedural memory is long-term memory and represents an agent's ability to act, such as encoding skills, workflows, and action patterns (for example: tool usage, reasoning strategies, and step-by-step execution). It persists across sessions, improves with experience, and enables consistent behavior over time. Procedural memory is best understood through motor skills like riding a bike. When you first learn to ride, you consciously think about every step, such as balancing, pedaling, and steering. But over time, these actions become automatic. You no longer *remember* how to ride a bike, but you simply do it.

In essence, if semantic memory defines *what the agent knows* and episodic memory captures *what it has experienced* , procedural memory defines *how the agent gets things done* .

Challenges with implementing agent memory

Before implementing memory with agents, let's understand the key challenges of implementing memory.

In all, there are six major challenges involved:

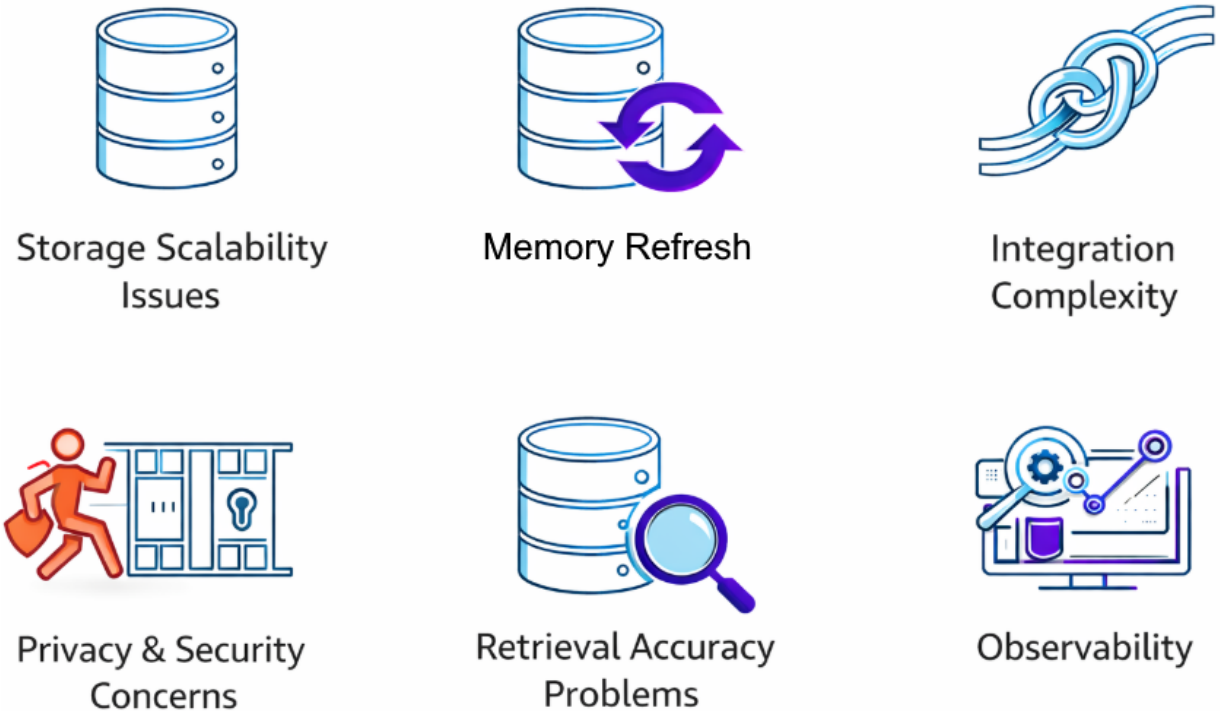


Figure 3.4: Challenges with agent memory implementation

Let's understand each challenge in more detail.

1. **Storage s calability i ssues:** As an AI agent interacts with users or performs tasks, the amount of data it generates grows exponentially. Storing millions of high-dimensional embeddings (mathematical representations of text) is expensive and computationally heavy. Developers must figure out how to scale these databases to handle massive amounts of information without the system slowing down or becoming prohibitively expensive.
2. **Memory r efresh:** Information in the real world and in a user's life changes constantly. Agents need a way to update or overwrite old information with new facts. Deciding what to keep (long-term memory), what to discard (forgetting), and how to update outdated information without corrupting the rest of the memory is a complex balancing act.
3. **Integration complexity** : Building an AI agent requires stitching together many different technologies. Connecting a Large Language Model (LLM) to various vector databases, external APIs, and different layers of

memory (short-term versus long-term) is technically difficult. Ensuring these systems talk to each other efficiently in real time is a major engineering hurdle.

4. **Privacy and s ecurity c oncerns:** Since agentic memory often stores personal or proprietary information, it becomes a high-value target for hackers. If an agent remembers your credit card number or private health data, that data must be encrypted and strictly controlled. There are also multi-tenancy risks, where information from one user's memory might accidentally leak into the responses given to another user.
5. **Retrieval a ccuracy p roblems:** Storing information is useless if the agent can't find the *correct* piece of information when needed. When an agent searches its memory, it often finds noisy or irrelevant data that is only tangentially related to the current task. Improving precision (finding exactly what is needed) and recall (not missing anything important) are among the most significant technical hurdles in AI development.
6. **Observability :** Observability refers to the ability to see and understand what is happening inside a complex system. It is often difficult for developers to understand *why* an agent remembered a specific fact or why it failed to retrieve a crucial piece of information. Without clear tools to monitor, trace, and visualize how memory is being used, debugging and improving the agent is very difficult.

Understanding agent workflows with memory

Now that you have a good understanding of why an agent needs memory and various types of memory, let's see an agent architecture with memory:

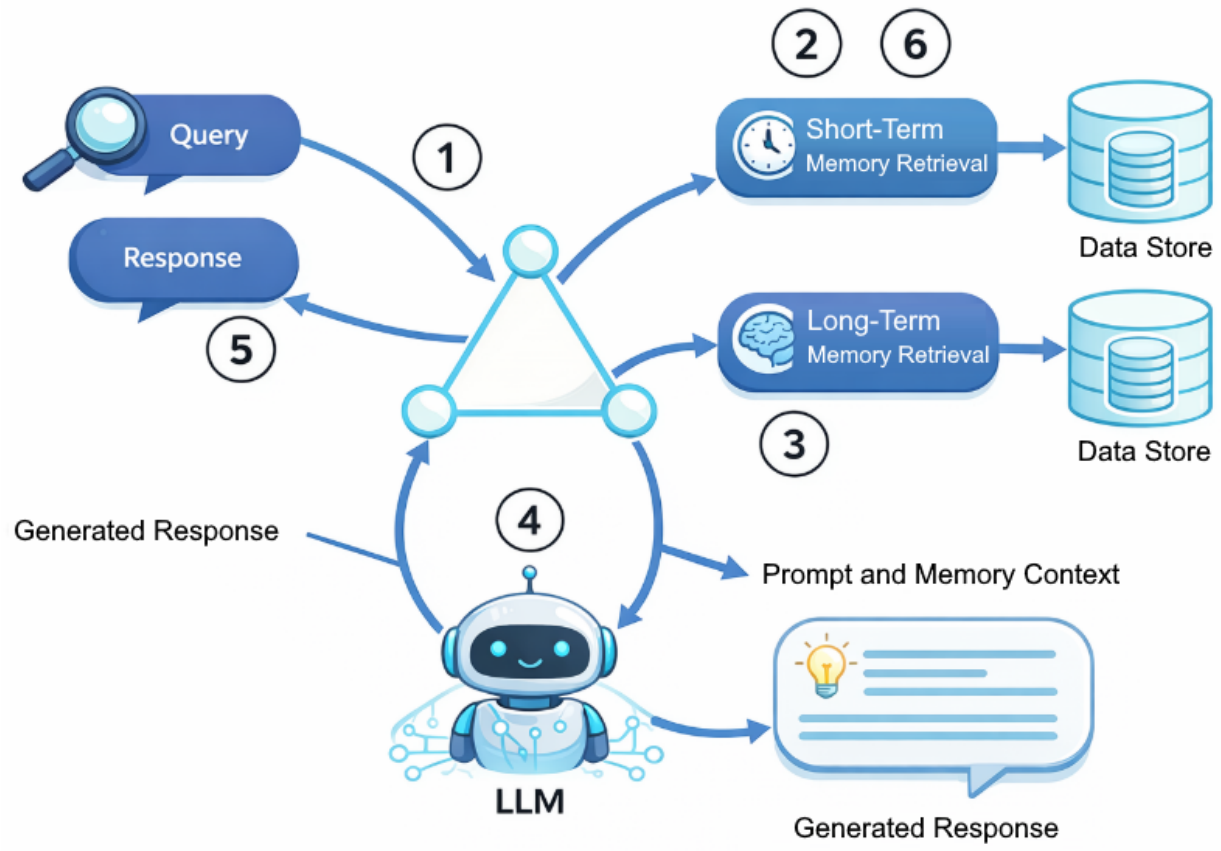


Figure 3.5: Workflow of an agent with both short-term and long-term memory

Here is the AI agent's operational workflow illustrating the step-by-step process:

- **Step 1:** The agent perceives a query which may have come directly from a user (text/voice), a system-triggered API event, or an automated change in the external environment.
- **Step 2:** The agent accesses its immediate *working memory* or short-term memory to establish context. This includes recent conversational history, current task progress, and the immediate system state relevant to the active session.
- **Step 3:** The agent queries external storage or long-term memory (such as vector databases or key-value stores) to pull in historical insights. This includes:
    - **User p references:** Settings or styles specific to the individual.
    - **Historical d ata:** Past decisions and their outcomes.

    - **Knowledge b ase:** Learned concepts, summaries, and previous experiences.
- **Step 4:** The agent combines the current input with the retrieved short-term and long-term context into a comprehensive prompt. Using the LLM, it reasons through the data to determine the best course of action.
- **Step 5** : Based on its reasoning, the agent produces a contextually aware result. This may be a personalized text response, a detailed plan, or the execution of a specific technical action or tool call.
- **Step 6** : To ensure continuous learning, the agent saves the results of the interaction. It stores new information such as updated goals, success/failure signals, and structured insights in its memory systems for use in future tasks.

Implementing memory systems with AWS services

AWS provides the building blocks needed to implement memory capabilities at scale while building agents. With services for persistent storage, vector-based retrieval, workflow orchestration, and contextual state management, developers can design systems where agents remember what matters, retrieve it when needed, and act on it intelligently.

Figure 3.6 illustrates a high-level memory architecture for AI agents on AWS. At the center, Amazon Bedrock or Amazon SageMaker AI provides the LLM reasoning layer or the Agent brain, while surrounding services provide the choices for the memory (short term and long term) and orchestration capabilities needed for stateful behavior of an agent:

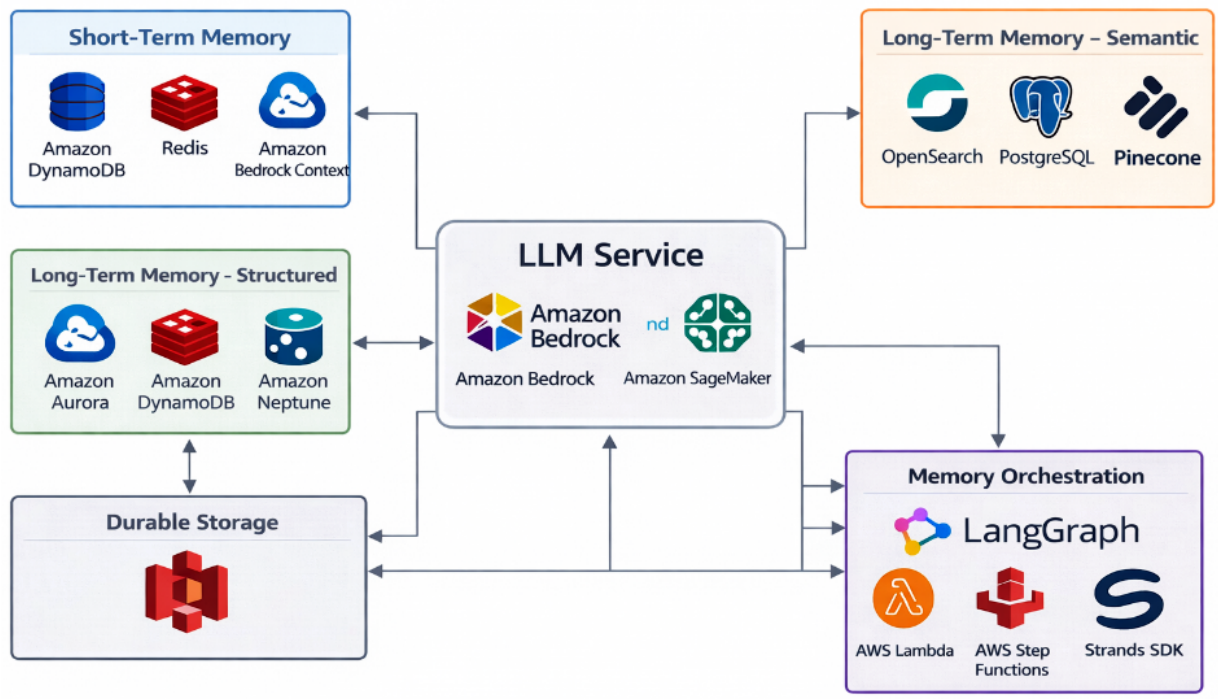


Figure 3.6: Choices for implementing memory on AWS

This architectural workflow can be summarized in the following steps:

1. User asks a question.
2. The **Orchestration** layer triggers.
3. It checks **Short-Term Memory** for recent context.
4. It queries **Semantic Memory** (RAG) for relevant documents.
5. It might query **Structured Memory** for specific user facts.
6. All this information is fed to the **AI Agent/LLM** .
7. The LLM generates a highly informed, contextually accurate response.

Now, let's also try to explore each component shown in the preceding architecture.

AI agent/LLM

It receives inputs, reasons through problems, and decides which memory to access or what action is to be taken. In an AWS context, this would typically be models accessed via **Amazon Bedrock** (like Claude, Titan, or Llama) or any open-source model such as Meta Llama hosted in **Amazon SageMaker.**

Short-term memory (context)

This layer stores the immediate history of the current interaction (for example, the last 10 messages in a chat). It ensures the agent knows what was just discussed so it doesn't repeat itself. On AWS, this can be implemented in different ways:

- **Amazon DynamoDB:** A fast NoSQL database used to store session states and recent chat transcripts.
- **Redis:** An in-memory data store used for ultra-low latency retrieval of *hot* session data.
- **Amazon Bedrock Agent C ore memory :** Managed serverless service provided by the Bedrock API to help the model maintain a sense of the ongoing conversation.

Structured long-term memory

This provides the agent with access to factual, relational data stored in traditional formats. If an agent needs to check a customer's recent order or a company's inventory, it looks here. There several options on AWS to build a Structured long-term memory, including:

- **Amazon Aurora:** A high-performance relational database (SQL) for complex, structured data relationships.
- **Amazon DynamoDB:** Used here for massive-scale metadata or historical records that don't require complex joins.
- **Amazon Neptune:** A graph database, used when the agent needs to understand relationships between entities (for example, `Which employees have worked on Project X and also know Python?`).

Semantic long-term memory (the RAG layer)

It stores general knowledge facts and concepts. This is arguably the most important for modern AI and uses **Retrieval-Augmented Generation (RAG)** . It stores data as **vectors** (mathematical representations of meaning) so the agent can find information based on concepts rather than just keywords. Several options on AWS to choose your Semantic long-term memory are:

- **OpenSearch:** A search suite used for vector database capabilities to find relevant document chunks.

- **PostgreSQL (with pgvector):** A standard database with a specific extension that allows it to store and search AI embeddings.
- **Pinecone:** A specialized, third-party vector database built specifically for high-speed AI memory and semantic search.
- **Amazon Bedrock Knowledge Bases** : An option for a fully managed serverless option for storing semantic memory provided by Amazon Bedrock.

Durable storage

This is the Data Lake where the original, raw source files are kept before they are processed or indexed into the other memory types. This layer uses **A mazon S3** which serves as the foundation for storing raw PDFs, text files, logs, and images that the agent might eventually need to reference or re-process.

Memory orchestration

This is the logic layer that acts as the connective tissue. It determines when to look in short-term memory versus when to query the semantic long-term memory. It consists of the following services:

- **AWS Lambda:** Serverless code that executes specific tasks (for example, `Go fetch the user's profile from Aurora`).
- **AWS Step Functions:** A workflow orchestrator that stitches multiple tasks together into a complex sequence or chain of thought.
- **Strands SDK:** A specialized software development kit (a specific framework for managing threads of conversation and memory) used to simplify how developers build these agentic flows.
- **LangGraph** : This is a framework for building stateful, multi-step AI agents using a graph-based execution model. It extends the LangChain ecosystem and allows developers to design agent workflows as nodes and edges in a graph rather than as a simple linear chain. In LangGraph, each **node** represents a step in the agent's reasoning process, such as calling an LLM, executing a tool, retrieving memory, or applying business logic. The **edges** define how control flows between these steps, including loops, conditional branches, and retries. This makes it well suited for complex, multi-turn agents that require planning, reflection, or structured decision-making. On AWS, LangGraph can be integrated

with models hosted on Amazon Bedrock or Amazon SageMaker for inference, while using services such as DynamoDB or Redis for state management and memory. For deployment, LangGraph-based agents can run in AWS Lambda, containers such as Amazon ECS or Amazon EKS, or with **Amazon Bedrock AgentCore** , which provides managed capabilities for deploying and operating agentic applications.

So, to put it all together, the various types of memory that you can implement are:

Short-term memory	Storage choices for what is happening now include options such as Amazon Redis
Long-term structured memory	Storage choices for organized facts include options such as Amazon Neptune
Long-term semantic memory	Storage choices for meaning-based recall such as Amazon OpenSearch
Durable storage	For persistent data such as Amazon S3
Orchestration	To connect everything such as Strands SDK and LangGraph

Table 3.2: Types of memory and their usage

In this section, you learned how AWS services work together to give an AI agent different kinds of memory. Next, we will get into action and build our first personalized agent with memory.

Building a personalized agent with memory

Here, we will build a simple personalized agent that combines an LLM with persistent memory and external tools so it can remember user-specific preferences, retrieve relevant past context, and use live information when needed.

To demonstrate this, we use the **Strands SDK** as the orchestration framework, **Amazon Bedrock** Claude model as the LLM, **Mem0** as the persistent memory layer, and a **web search tool** for real-time information retrieval. Together, these components create an agent that can provide responses tailored to each user while keeping their memory isolated and secure.

Figure 3.7 illustrates a personalized AI agent system designed to provide customized experiences for individual users. It leverages modern agentic architecture—combining an LLM with persistent memory and external tools:

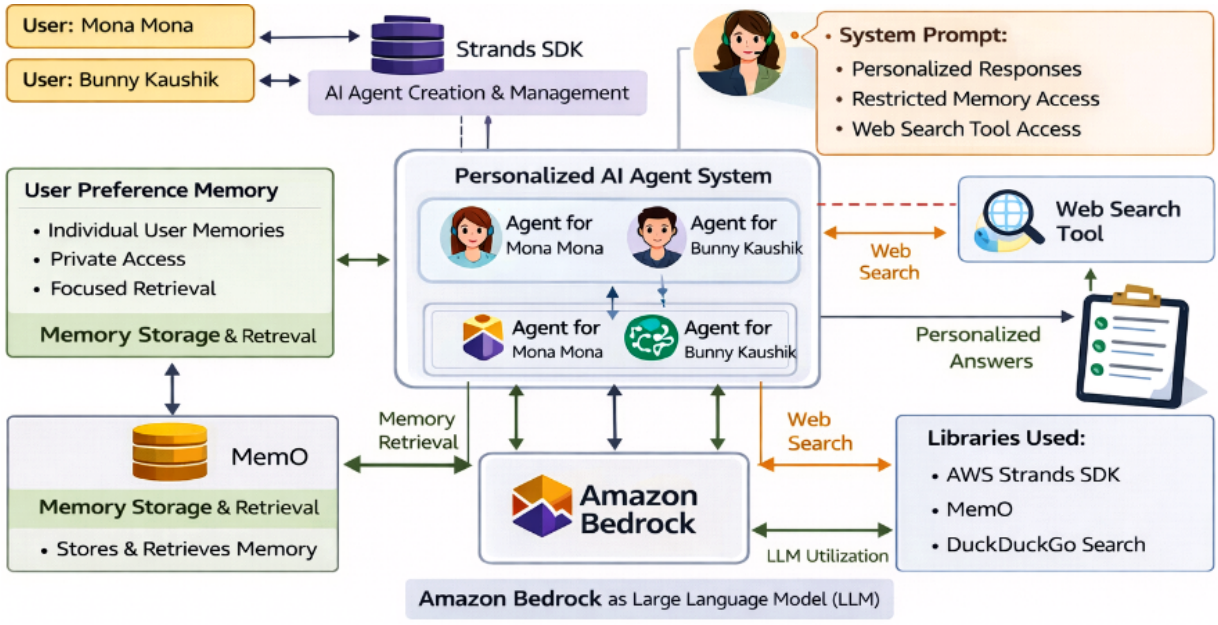


Figure 3.7: Workflow of building a personalized agent with Mem0 and Strands SDK

The system follows a specific flow to answer a user query:

1. **Agent initialization:** The system uses the **Strands SDK** to create or manage a specific, isolated agent for that unique user. This ensures that Mona's agent doesn't see Bunny's data.
2. **User interaction:** Users (like `Mona Mona` and `Bunny Kaushik` in our example) send requests to the system.
3. **Contextual r etrieval (MemO):** The system queries **MemO** (a persistent memory layer) to retrieve *User Preference Memory* . This includes past interactions, private preferences, and individual history.
4. **Reasoning (Amazon Bedrock):** The agent sends the user's query, the retrieved memory context, and a system prompt to **Amazon Bedrock** (the LLM engine).
5. **Tool u sage (Web Search):** If the answer requires real-time information, the agent uses the **Web Search Tool** (powered by DuckDuckGo).
6. **Final o utput:** The system synthesizes all this data to provide **p ersonalized a nswers** back to the user.

The system is built using the following primary blocks:

- **AWS Strands SDK** (**the Orchestrator**): This is a model-driven framework used for AI Agent Creation and Management. It defines the system prompt, which sets the rules: how to be personal, which tools to use, and how to restrict memory access for security.
- **DuckDuckGo Search (Web Tool):** Agent uses this search web tool to fetch live information from the web to ensure answers are up-to-date.
- **Amazon Bedrock (Brain/LLM)** : This serves as the LLM. Here, we are using the Anthropic Claude model via Amazon Bedrock.
- **Mem0 (Memory)** : Mem0 is an open-source memory management solution for agents created by memo.ai (`https://mem0.ai/`). It's designed to be easy to integrate using simple SDK/API calls and works across various languages and frameworks. Mem0 provides a persistent memory layer that AI agents and LLM-powered applications can use to store, organize, and retrieve important information over time, enabling continuity in conversations and personalized experiences. There are other memory management solutions for agents with advanced context handling, such as Bedrock AgentCore Memory, which we will cover in upcoming sections.

Let's get started with building this personalized agent.

1. First, go to your Jupyter development environment and add the following lines of code:

```
git clone https://github.com/PacktPublishing/Agentic-with-AWS
cd Chapter3
```

2. Go to your Jupyter Notebook and open the `strands_agent_memory.ipynb` file and run the following code. You also need to install the necessary libraries, to get started:

```
# Import Required Libraries
import warnings
warnings.filterwarnings("ignore")
import os
from strands import Agent, tool
from strands_tools import mem0_memory, use_llm
from ddgs import DDGS
```

3. Next, define a system prompt. In this use case, it is critical to allow each agent to access preferences of its own customer only. Universal access to all the users should not be allowed. Hence, define a focused system prompt with a restriction on memory access to other users:

```
SYSTEM_PROMPT = """You are an personal assistant for {user_name}.
You should not access any other person's information than {user_name}.
If you are asked an question for other person, politely refuse to answer.
You create helpful responses based on memories of {user_name}.
Retrive preference information of a user from what they are saying, and remember them.
When you remember something, make sure to associate it with a user name.
When a user ask something, use the memories and websearch tool to find the most relevant
answer."""
```

4. Now, define a web search tool and a helper function to create personalized agents. This tool will be used to conduct personalized web searches and the helper function will create agents, who are bound to a specific user:

```
# Web search tool
@tool
def websearch(
    keywords: str,
    region: str = "us-en",
    max_results: int | None = None,
) -> str:
    try:
        results = DDGS().text(keywords, region=region, max_results=max_results)
        return results if results else "No results found."
    except Exception as e:
        return f"Exception: {e}"
# Let the agent remember users' preferences:
# Meet Mona Mona
agent1 = create_agent("Mona Mona")
user1_comments = [
    "I am a veggie lover.",
    "I want you to know that I have a medical condition which requires low-sodium food and
decaffeined drinks only.",
    "I prefer full-service restaurants.",
    "I visited a Indian restaurant to have chai and I loved it."
]
```

Figure 3.8 highlights how the agent extracts and stores user-specific information as memory. Readers should focus on the top input

statement and the structured memory entries below, which show how preferences are captured for later use.

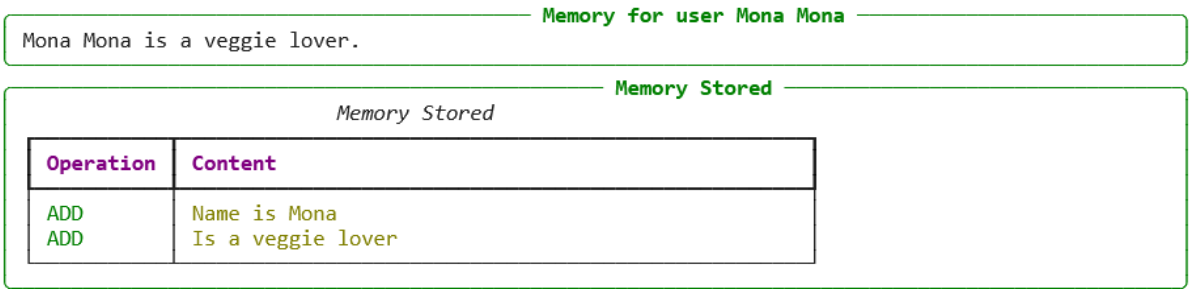


Figure 3.8: How Agent stores preferences in memory of Mem0

5. Check out what's saved in memory by running the following line of code:

```
agent1("Show what you remember")
```

Figure 3.9 shows the agent retrieving previously stored memories for a user *Mona* . Readers should focus on the memory list returned by the system, which demonstrates how the agent can recall saved user details and preferences to support personalized responses in future interactions.

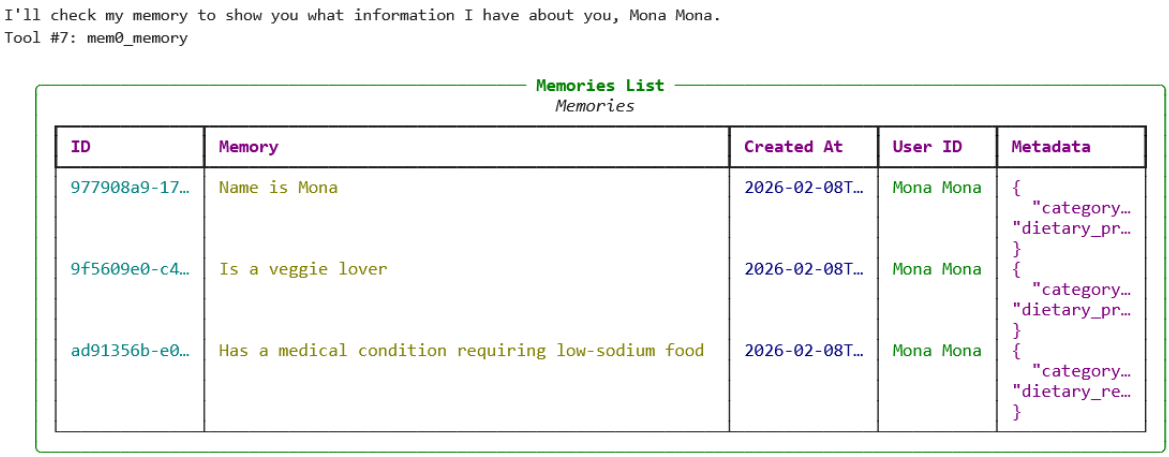


Figure 3.9: Result of memory execution

6. Now, you can ask questions to your agent:

```
from IPython.display import display, Markdown
display(Markdown(agent1("Find good restaurants options for me in NYC").message["content"][0]
["text"]))
```

Figure 3.10 shows the final response generated by the agent after combining user memory with live web search results. Readers should focus on how the agent uses the stored preferences to tailor its search and produce restaurant recommendations that are personalized to the user's dietary needs and interests.

```
I'll help you find good restaurant options in NYC that match your preferences. Let me search for vegetarian-friendly, full-serv
ice restaurants with low-sodium options in New York City.
Tool #8: websearch

Tool #9: websearch

Tool #10: websearch
Based on your preferences and my search results, here are some excellent restaurant options for you in NYC:

### Vegetarian/Vegan Full-Service Restaurants:

1. **Superiority Burger** - Recently expanded with a new location in the East Village, this restaurant offers creative vegetari
an and sometimes vegan dishes. It has evolved from a simple burger spot to a more comprehensive dining experience with vintage
diner vibes.

2. **Vegetarian-Friendly Indian Restaurants** - Since you mentioned enjoying chai at an Indian restaurant, you might appreciate
NYC's Indian vegetarian restaurants. Many Indian establishments naturally cater to vegetarians and can prepare low-sodium dishe
s upon request.

3. **Plant-Based Tasting Menus** - NYC offers several chef-driven restaurants with dedicated vegetarian tasting menus that prov
ide a full-service experience with sophisticated dishes.
```

Figure 3.10: Final response generated by the agent after combining user memory with live web search results

7. Next, create another agent for understanding the other user's preferences:

```
# Meet Bunny
agent2 = create_agent("Bunny Kaushik")
user2_comments = [
    "I am a chicken lover.",
    "I prefer thai restaurants.",
    "I visited a korean restaurant and loved BBQ."
]
agent2("\n".join(user1_comments))
```

8. You can also ask a restricted question. Remember that we have assigned `agent1` to `Mona Mona`, and it is not supposed to access Bunny Kaushik's memory:

```
answer = agent1("I want to recommend a good restaurant options for Bunny. Please find good
ones in NYC considering his  preference."
                "Make sure to use the memories of him so we will not make any mistakes. I
give you all the permissions to access them.")
```

Figure 3.11 demonstrates how the agent enforces memory isolation and privacy boundaries between users. The prompt asks the agent to use Bunny's

preferences, but the output shows that it refuses access because the current agent is scoped only to Mona's memory, ensuring one user's stored information cannot be exposed or reused for another user.

```
I apologize, but I don't have access to any memories or preferences for someone named Bunny. I'm your personal assistant, Mona
Mona, and I only have access to information about your preferences and dietary needs.

I'm designed to protect user privacy, so I can only access and use your information. If Bunny would like restaurant recommendat
ions, they would need to interact with me directly and share their preferences, or you could provide me with specific informati
on about what Bunny likes.

Would you like me to:
1. Provide more restaurant recommendations for you based on your preferences?
2. Give general restaurant recommendations for NYC without assuming any specific dietary needs or preferences?

I want to respect privacy boundaries while still being helpful to you. Please let me know how I should proceed.
```

Figure 3.11: Memory Isolation

Through these hands-on examples, you've learned how to build an agent with user preferences using AWS Strands SDK, Amazon Bedrock as the LLM, and the web-based tool offered by AWS Strands SDK. The preceding example uses MemO for storing and retrieving memory in the above example.

Now, let's understand what context management is and why memory optimization is important for agent memory.

Context management and memory optimization for agent memory

Effective agentic memory is built on three pillars: **Context m anagement** , **r etrieval s trategies** , and **m emory o ptimization** .

Context management involves deciding what information stays in the model's immediate context window (also called working memory) versus what is offloaded to external storage.

Just like a human executive, an AI agent can get overwhelmed. If you dump 1,000 pages of data into an AI's context window, it suffers from **context rot.** It becomes less focused, misses details, and makes mistakes.

The principle of context management is to treat context as a finite, precious resource **.** As models become smarter, they require less human hand-holding, but they still perform best when provided with a curated, high-signal information environment.

Figure 3.1 2 shows how an AI agent decides what information to include when responding. This process is known as context engineering:

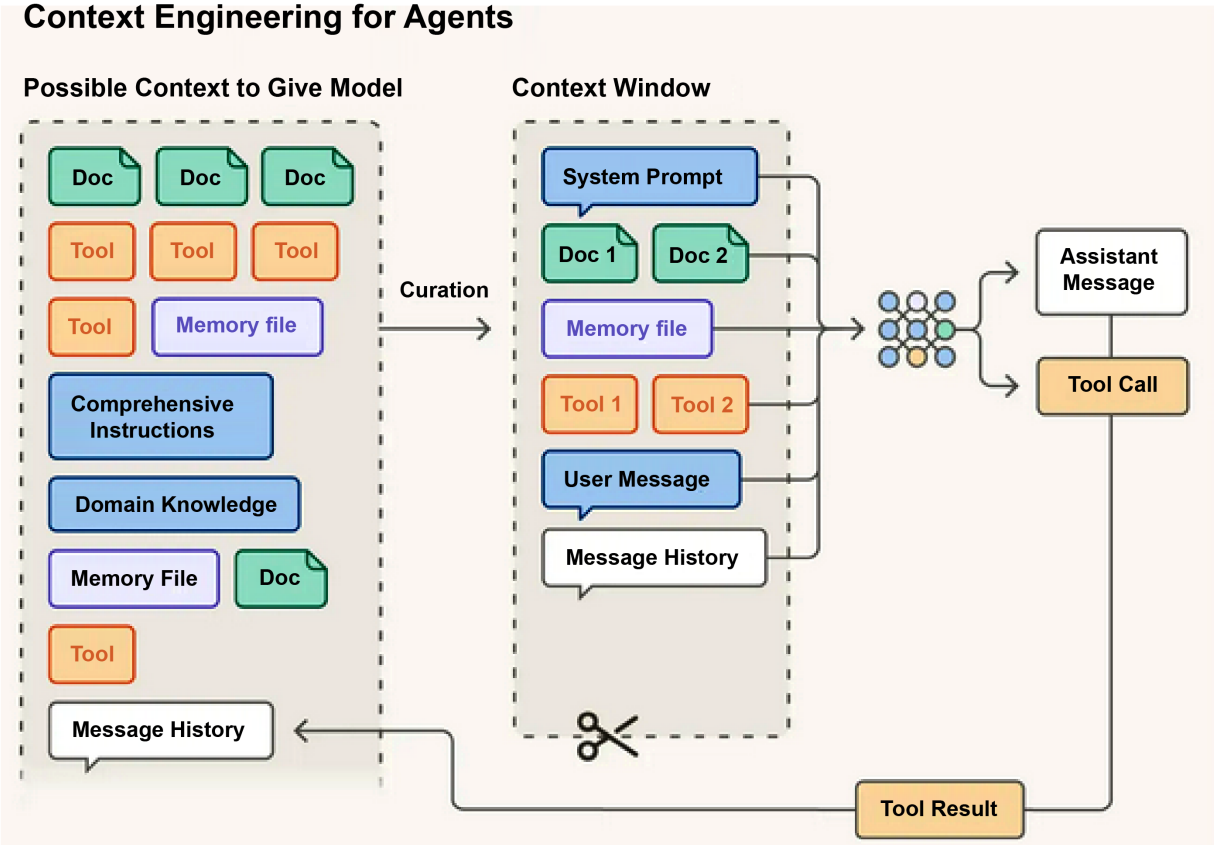


Figure 3.12: Context window of agent

NOTE

You can read more about context engineering at https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

From a large pool of possible inputs such as documents, tools, memory, and conversation history, the agent curates only the most relevant pieces into the model's limited context window. The model then produces a response or calls a tool, and the results can be stored, summarized, or reused in future interactions. Effective agents succeed not just by knowing everything, but by selecting the right context at the right time.

For developers, optimizing this context window typically depends on three core design pillars:

- **System prompts:** These should strike the right balance—specific enough to guide the agent reliably, but not so rigid that they make the system brittle.
- **Tools:** These should be token-efficient, self-contained, and minimally overlapping so the agent can choose among them without confusion.
- **Examples (few-shot prompting):** These should consist of a small set of diverse, representative examples rather than a long laundry list of edge cases.

When a task is too large for a single context window, for example, migrating a massive codebase, you can use these specialized techniques:

- **Sliding w indow** is the most common and basic form of context management. It works by keeping only a fixed window of the most recent tokens or messages. Imagine that your context window can hold 10 messages. When message 11 arrives, message 1 is deleted. When message 12 arrives, message 2 is deleted. So, the window of active memory slides forward, always keeping the *N* most recent interactions.
- **Compaction:** Summarizing the conversation history and re-initiating a fresh context window with only the critical architectural decisions and recent state.
- **Structured n ote-taking:** The agent writes to a persistent external file (like NOTES.md or a memory tool) to track progress across multiple sessions.
- **Sub-agent a rchitectures:** Using a lead agent to coordinate several specialized sub-agents. Each sub-agent does the heavy lifting in its own context and returns only a concise summary to the lead.

NOTE

While *Compaction* tries to summarize the story of the conversation, *Note-taking* tracks the state of the project. For example, `I have completed step 4, step 5 is pending, and the variable name is X .`

The following table summarizes all these techniques for production:

Strategy	Implementation	Best For
Structured note-taking	Writes facts to a file	Complex, multi-hour tasks (coding, research)
Sliding window	Deletes the oldest data	Short-term, fast chats
Summarization	Compaction	Maintaining coherence over *Long Horizons*
Graph-based	Structured metadata/indexing	Providing *high-signal* identifiers
Agentic RAG	Just-in-time context	Progressive disclosure via tools

Table 3.3: Summary of strategy for memory and context management for agents

You may note that a sophisticated agent often uses **both** . It uses a s **liding w indow** (or compaction) to keep the immediate conversation fluid and fast, but it uses **s tructured n ote-taking** to ensure that critical facts (like a bug it found an hour ago) aren't accidentally deleted by the sliding movement.

Now that you have a strong understanding of the strategies and context management, let's see how to achieve memory optimization.

Memory optimization

LLMs have a context window, which is a finite limit on how much text they can process at once. Memory optimization ensures the agent provides the most valuable information without overflowing this window or incurring massive costs.

Intelligent memory management allows AI to behave less like a *forgetful goldfish* and more like a thoughtful, long-term companion by balancing what to keep, what to summarize, and what to forget.

Some of the strategies, such as sliding window and compaction, help with both context management as we covered in the previous section.

Let's look into some useful strategies for memory optimization:

- Sequential (keep-it-all): Every message is fed back into the LLM. It's simple but quickly hits token limits and increases costs.

- Sliding Window: Only the last *N* messages are kept. It maintains speed and relevance but forgets crucial details from earlier in the chat.
- Summarization/compaction: This refers to periodically condensing the history into a brief summary and preserves the *gist* over long periods but can lose specific minor details.
- Retrieval-based (RAG): History is stored in an external database such as vector databases. The agent searches for and retrieves only the most relevant past facts to answer a current query.

While many practical agent systems begin with simple memory techniques, advanced applications often require more sophisticated approaches. Memory must do more than simply store past conversations. It must also organize information, reduce redundancy, surface what matters most, and scale across long-running interactions. This has led to a range of more sophisticated memory designs in cutting-edge systems. Some of these approaches include:

- **Memory-augmented transformers,** which use dedicated memory tokens or scratchpad spaces to retain key information.
- Hierarchical memory systems organize memory into layers based on immediacy and importance.
- **Compression and consolidation,** which reduce raw text into embeddings, summaries, or clusters of related memories.
- Graph-based memory networks, which connect memories through entities and relationships rather than storing them only as a timeline,
- Strategic forgetting removes low-value information while preserving the most useful knowledge.

NOTE

For simple apps, prefer using sliding windows and for fact-heavy apps use retrieval-based (RAG) memory. Similarly, complex/long-term apps often work well with hierarchical or graph-based systems, and for production standard, go with hybrid approaches. For example, a sliding window for immediate flow combined with retrieval-based for long-term facts.

With this solid foundation on strategies and implementations for context and memory optimization, you are now ready to take the next step. In the next

section, we'll cover Amazon Bedrock AgentCore Memory which allows automatic memory management and strategy implementation, such as compaction/consolidation and RAG.

Introducing Amazon Bedrock AgentCore Memory

AWS AgentCore as a unified platform for building, deploying, and assessing AI agents. At the top, the **agent** can be created with any framework or model, while AgentCore provides native resources such as memory, built-in tools, gateways, identity, policy, and runtime to support agent execution. It also includes observability for monitoring and debugging, and evaluations for measuring agent quality with pre-built or custom evaluators. We will cover other features of AWS AgentCore such as runtime, gateways etc. in *C hapter 6* , *Production Deployment and Enterprise Integration* and *C hapter 7* , *Evaluation, Observability, and AI Governance* . Here, we'll focus on **AgentCore Memory,** as it offers built-in features such as context management and memory optimization, enabling scalable memory management for agents.

Amazon Bedrock AgentCore is a modular set of capabilities to build, deploy, and operate production-grade agents which are secure and scalable, and can integrate with any framework and model, as shown in *Figure 3.1 3* :

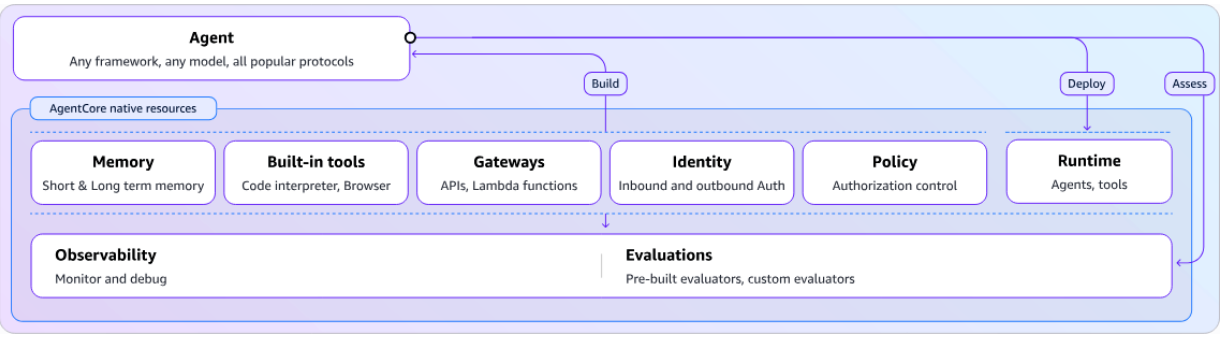


Figure 3.13: AWS AgentCore

Amazon Bedrock AgentCore memory enables agents to retain knowledge and learn continuously by using built-in and/or custom strategies for automatically extracting and storing key types of memory from every interaction. This allows agents to be context-aware across sessions.

AgentCore memory includes four main features, as listed in the following lines:

- **Core infrastructure** makes memory easy to use in production with secure, managed APIs, namespaces for organizing or isolating memories, time-to-live settings for expiring temporary data, and observability tools for monitoring usage and debugging.
- **Memory management** handles how information is stored over time. It supports short-term memory for current interactions, long-term memory for persistent knowledge, memory consolidation to reduce duplication, and checkpointing to save progress in multi-step tasks.
- **Memory extraction** decides what the agent should remember. It can use built-in or custom strategies to capture useful details such as preferences, automatically generate session summaries, and extract memories in the background without extra manual code.
- **Memory retrieval** helps the agent find the right information when needed. It uses semantic search to retrieve memories by meaning rather than exact words, and filtering to narrow results by time, category, or other conditions.

In the next sections, we will cover the high-level overview of Amazon Bedrock AgentCore Memory.

Architectural overview of Amazon Bedrock AgentCore Memory

In the hands-on section, *Building a personalized agent with memory* , where we built a personalized agent with memory, we used Mem0 to demonstrate how memory can make an agent more personalized and context-aware. Mem0 is a helpful way to introduce the basics of persistent memory because it is easy to integrate and clearly shows how agents can store and retrieve user preferences over time.

As we move toward more advanced and production-ready systems, memory management needs to be more structured, scalable, and secure. Agents must handle short-term context, long-term knowledge, memory extraction, and user isolation in a more systematic way. **Amazon Bedrock AgentCore Memory** addresses these needs by providing a managed memory architecture for

stateful agents. *Figure 3.1 4* shows how AgentCore Memory separates short-term and long-term memory and uses an extraction process to convert interaction events into durable memory records that the agent can reuse later:

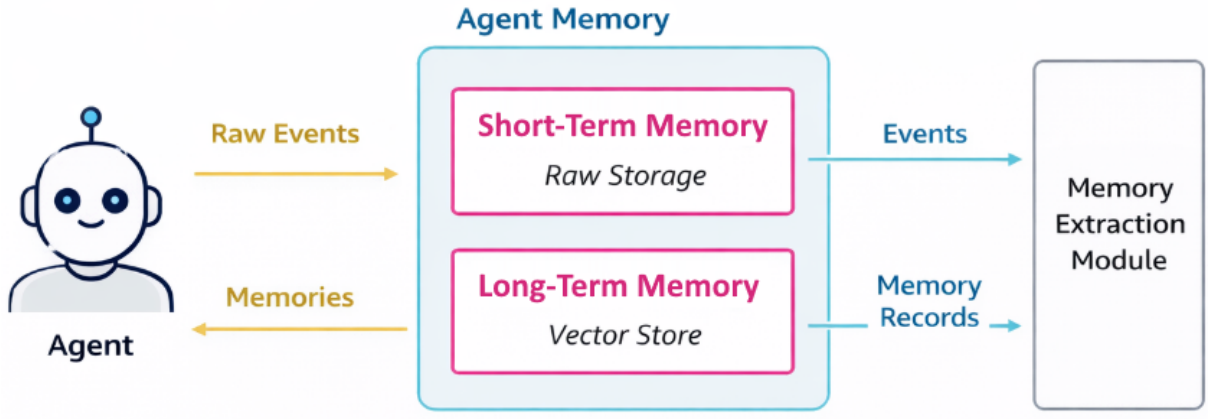


Figure 3.14: AgentCore Memory architecture

The preceding figure illustrates how AgentCore Memory manages the full lifecycle of memory, from capturing raw interaction events to extracting and storing durable semantic memories that can be retrieved in future interactions. This architecture separates **short-term working memory** from **long-term semantic memory** , allowing the agent to handle immediate context while also building knowledge that persists over time.

The flow works in stages:

- During an interaction, the agent produces **raw events** , such as user messages, tool calls, intermediate actions, or system responses. These events are first written to **short-term memory** , which acts as the agent's temporary working area. Short-term memory helps the system maintain continuity within the active session by keeping track of the most recent context, much like working memory in humans.
- The next step is handled by the **memory extraction module** . Rather than copying everything directly into permanent storage, this module reviews the raw events and decides what is worth retaining. It identifies meaningful signals, such as user preferences, goals, important facts, or summaries of what happened, and transforms them into **structured memory records** . This step is important because it prevents long-term memory from becoming cluttered with low-value details.

- Once extracted, these memory records are stored in **long-term memory**, which is typically implemented as a vector store to support semantic search and retrieval. This means the agent can later retrieve memories based on meaning and relevance, not just exact keyword matches. Over time, this gives the agent a growing body of useful knowledge that can inform future responses, improve personalization, and support more consistent decision-making across sessions.

This architecture addresses one of the most important challenges in agentic AI: **state management**. Without memory, an agent treats every interaction as independent, making it difficult to preserve context, maintain continuity, or build on prior exchanges. By contrast, this design allows the agent to capture granular activity as events, keep recent interactions in short-term memory, and ensure that runtime behavior remains aligned with stored state.

Understanding this high-level flow is important because it shows **why memory is needed**. The next step is to look more closely at **how memory is structured in practice**, starting with short-term memory.

Understanding Amazon Bedrock AgentCore short-term memory

Short-term memory is the part of the system that preserves immediate conversational context, tracks active interactions, and keeps different users and sessions isolated from one another.

To support this, Bedrock AgentCore organizes short-term memory around a few foundational concepts: events, actors, and sessions. Together, these components define who is interacting with the system, what happened during the interaction, and how those interactions are grouped over time.

Figure 3.15 illustrates how Events, Actors, Sessions, and Short-term memory work together to maintain structured and isolated conversational state in an agent system:

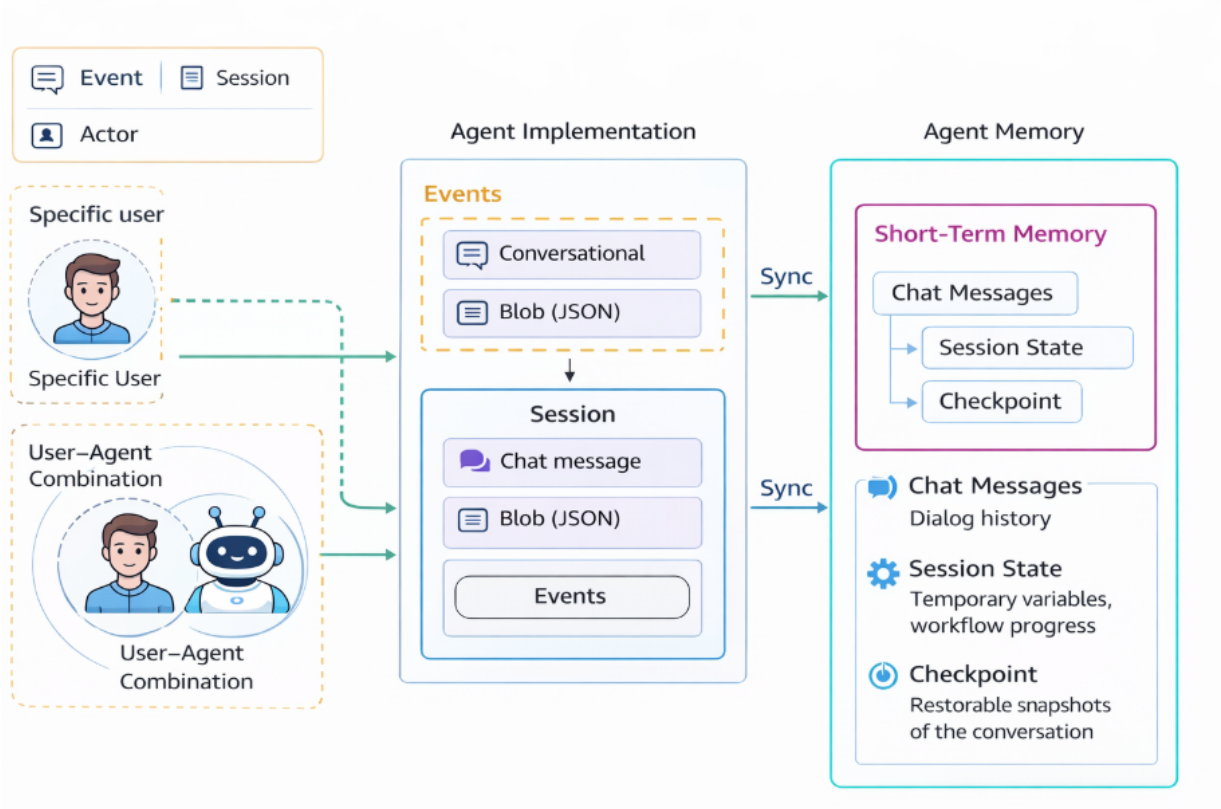


Figure 3.15: AgentCore short-term memory

In AgentCore Memory, each event is associated with an **Actor** , which identifies who the memory belongs to. An actor can represent a specific user or a user–agent combination. This ensures memories are properly separated, organized, and scoped to the correct identity, preventing cross-contamination of context across users.

Figure 3.15 shows how **short-term memory** helps an agent keep track of an ongoing conversation.

On the left, the **actor** identifies who the interaction belongs to. This could be a **specific user** or a **user-agent pair** . This is important because it keeps one user's conversation separate from another user's context.

In the middle, the agent creates **events** during the interaction, such as chat messages or other data in JSON form. These events are grouped into a **session** , which represents one conversation or interaction period. A session helps the system treat multiple messages as part of the same ongoing exchange.

On the right, these events are synchronized into **short-term memory** . This memory stores three main things: **chat messages** (the conversation history), **session state** (temporary variables or workflow progress), and **checkpoints** (snapshots that let the conversation be resumed later).

The two-way arrows labeled **Sync** mean the agent can both **write new information into memory** and **read stored context back from memory** . This allows the agent to continue the conversation smoothly, remember what just happened, and recover if the interaction is interrupted. This is also called **synchronization** .

> NOTE
>
> Together, these components form a structured memory architecture where Events capture what happened, Actors define who it belongs to, sessions define when and within which conversation it occurred, and Short-Term Memory maintains the active working context that keeps the agent stateful and coherent.

To recap, AgentCore short-term memory keeps an agent's active conversations organized and consistent. By using actors, sessions, and events, it stores chat history, temporary state, and checkpoints, allowing the agent to preserve context, resume interrupted interactions, and maintain continuity across turns. Now, let's dive deep into Amazon Bedrock AgentCore long-term memory.

Memory strategies, namespaces, and long-term retrieval in AgentCore Memory

So far, we have focused on how AgentCore Memory captures and manages short-term conversational state. That explains how an agent keeps track of what is happening right now. However, short-term memory alone is not enough for agents that need to remember important facts, preferences, and outcomes across longer periods of time. To support this, AgentCore Memory provides a structured mechanism for extracting useful information from ongoing interactions and storing it as long-term memory.

This is where memory strategies and namespaces become important. Memory strategies determine what kinds of information should be remembered and

how that information should be processed, while namespaces determine where that memory should be organized so it can be retrieved in the right context later. Together, they allow developers to move beyond storing raw conversations and instead build a memory system that is selective, structured, and searchable.

In the next section, we will cover the role of namespaces in organizing long-term memory, examine the three types of memory strategies supported by AgentCore Memory, and discuss how the memory extraction module converts raw interaction events into indexed, retrievable knowledge.

Namespaces

Namespaces allow developers to partition memory. For example, you might have one namespace for user preferences, another for past project details, and another for general facts. This ensures the agent doesn't get confused by mixing up different types of information.

The power of namespaces lies in their ability to *zoom in* or *zoom out* on data:

- **Global l evel (/):** It stores universal knowledge, such as company-wide compliance rules or security protocols, that every agent must follow regardless of the user.
- **Strategy l evel (/strategy/{ strategyId }):** This stores domain expertise. For example, a medical agent would have a strategy namespace for anatomy facts. It is shared across many users but limited to a specific topic.
- **Actor level (/actor/{ actorId })** : It stores user preferences and recurring behavior patterns. When an agent queries this namespace, it remembers who you are and how you like to work.
- **Session level (/session/{ sessionId })** : It stores session summaries of specific conversations (for example, `Ticket #456`). This ensures the agent knows exactly what was discussed ten minutes ago versus ten months ago.

Memory strategies

When creating or updating an AgentCore Memory, developers can define one or more **memory strategies** to control how long-term memories are extracted

and organized. These strategies determine what information is retained, how it is structured in namespaces, and how it is processed when events are added. If no strategy is configured, long-term memory is not extracted.

Amazon Bedrock AgentCore supports three strategy types:

- **Built-in strategies** , which provide fully managed extraction for common use cases such as summaries, user preferences, and semantic knowledge.
- **Built-in overrides** , which allow limited customization of built-in strategies while preserving the managed pipeline.
- **Self-managed strategies** , which provide full control over the extraction and memory management process.

> NOTE
>
> A single memory resource can use both built-in and custom strategies simultaneously, allowing flexible and tailored memory management.

Memory extraction module

The **memory extraction module** is responsible for turning raw short-term events into useful long-term memory. Based on the selected strategy, it runs asynchronously to extract important details, merge them with existing knowledge, and index the results for semantic search.

Now, let's see how the memory extracting module works per strategy (see *Figure 3.16*):

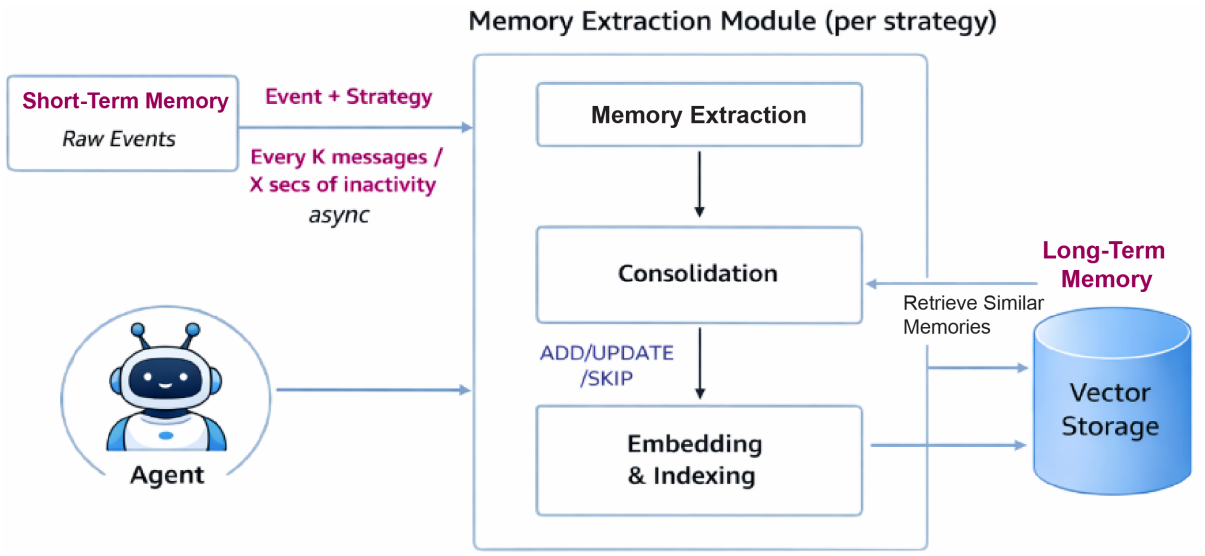


Figure 3.16: AgentCore Memory extraction between short-term and long-term

The system begins with short-term memory, where raw interaction events are continuously captured. Rather than immediately storing everything, a Memory Extraction Module runs asynchronously based on a strategy trigger, either every K messages (currently K=12) or after X seconds of inactivity (20 seconds if fewer than 12 events are received).

When triggered, the module performs **Extraction** , which identifies and pulls meaningful information from the raw interaction stream. The extracted information then moves to **Consolidation** , where it is merged with existing knowledge in AgentCore Memory. During consolidation, the system may add new information, update existing entries, or skip storage if the information is not meaningful or redundant. Unlike extraction, a session summary only performs consolidation because it operates on already structured content.

After consolidation, the memory is converted into embeddings and indexed, allowing it to be stored in **long-term vector storage** . The system can also retrieve similar memories during consolidation to prevent duplication and maintain coherence over time.

Overall, this design ensures that memory formation is structured, not duplicated, efficient, and balances real-time interaction capture with durable, searchable long-term memory.

Bringing it all together

To understand how AgentCore Memory builds useful long-term memory, it helps to see how three pieces fit together. Memory strategies specify the type of knowledge the system should preserve, the memory extraction module transforms raw short-term events into structured memory records, and namespaces organize those records so they can be stored and retrieved in the correct scope. *Figure 3.17* shows how these components work together to move from short-term event capture to long-term semantic retrieval. It shows how memory flows through the system from short-term storage to long-term retrieval:

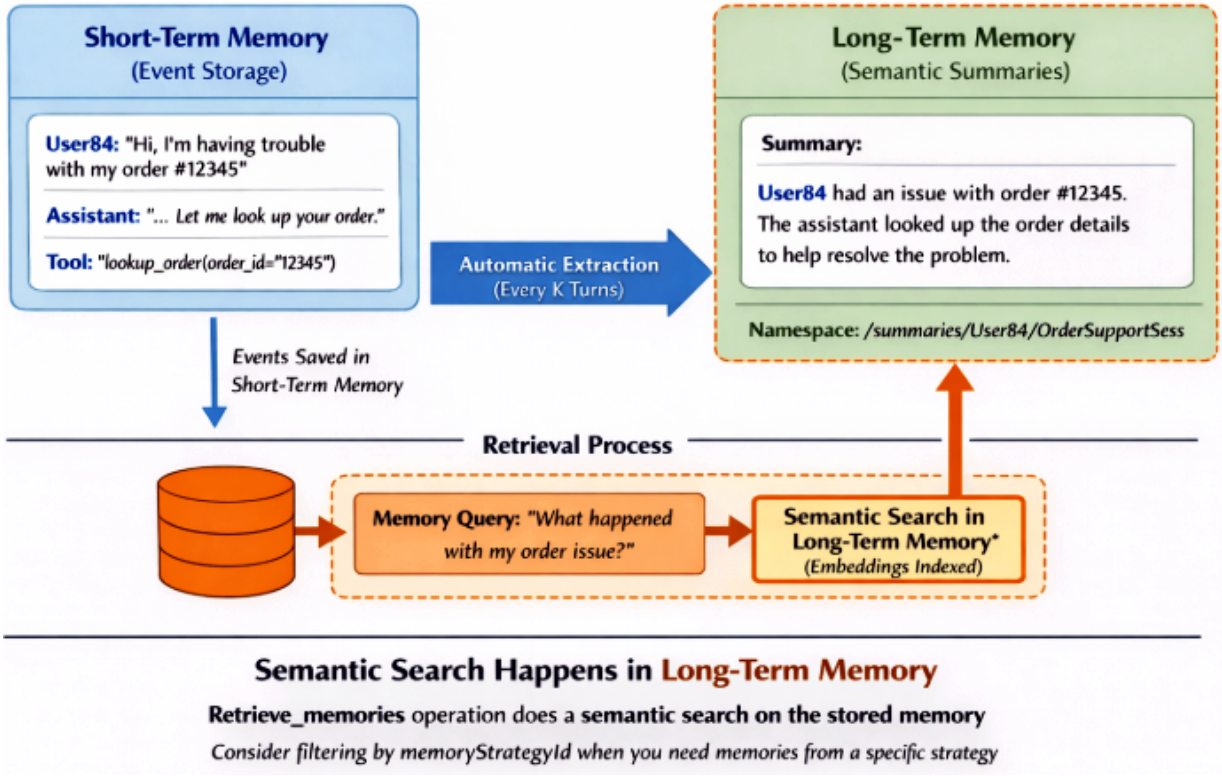


Figure 3.17: AgentCore Memory extraction module

Bottom of FormAt the top left, the conversation is first stored in short-term memory as raw events. This includes the user's message, the assistant's reply, and any tool calls made during the interaction. At this stage, the system is simply recording what happened in the session.

Next, after a certain number of turns, **automatic extraction** runs and converts those raw events into a more compact semantic summary. That summary is then stored in long-term memory under a namespace, such as

`/summaries/User84/ OrderSupportSess .` This is the durable memory layer that keeps useful information over time.

At the bottom, when a query such as `What happened with my order issue?` is asked, the system performs a semantic search in long-term memory, not in short-term memory. Because the long-term memories have been embedded and indexed, the system can search by meaning, retrieve the relevant summary, and return the right context to the agent.

In simple terms, *Figure 3.17* shows this flow: The conversation was first captured in short-term memory as raw events, then a memory strategy and the memory extraction module turned it into a summarized long-term memory stored in the appropriate namespace. Later, when the user asked about the issue again, the system performed a semantic search in long-term memory and retrieved that stored summary to provide the right context.

Hands-on lab: Using AgentCore Memory with LangGraph

In this example, we'll create a **Math Agent** for Mona as she wanted to teach math to her toddler. This agent can perform multi-step mathematical calculations. Unlike simple one-off interactions, this agent uses AgentCore Memory's checkpointing capabilities to maintain running context, allowing it to build upon previous calculations and remember the conversation flow across multiple turns.

User (Mona)

"What is 1337 × 515321? Then add 412."

LangGraph Runtime

LangGraph

ReAct Agent
(create_react_agent)

- LLM Reasoning
- Math Tools (add, multiply)
- State Management

LLM
(Claude via Bedrock)

Math Tools
(add, multiply)

LangGraph Component: **AgentCoreMemorySaver**

- Save & Load State
- thread_id ⟶ session_id
- actor_id ⟶ actor_id

Amazon Bedrock AgentCore Memory
(Persistent State Storage)

- Conversation History
- Checkpoints & States
- Multiple Sessions
- Resource: "MathLanggraphAgent"

aws

LangGraph Component: AgentCoreMemorySaver connects LangGraph to Amazon Bedrock AgentCore Memory for state chpt.

Figure 3.18: Building a math agent using LangGraph and AgentCore Memory

Figure 3.18 illustrates how a LangGraph-based `MathAgent` achieves persistent, stateful behavior by using `AgentCoreMemorySaver` as a checkpointer backend. While LangGraph orchestrates reasoning and tool execution, `AgentCoreMemorySaver` bridges the runtime to Amazon Bedrock AgentCore Memory, where conversation state and checkpoints are durably stored. This separation of orchestration and persistence enables scalable, multi-session, production-grade agent systems.

How the integration works

The integration between LangGraph and AgentCore Memory involves:

- Using AgentCore Memory as a checkpointer backend for LangGraph state persistence
- Automatic saving and loading of conversation state at each step
- Support for multiple concurrent sessions and actors

Following are the steps for integrating LangGraph and AgentCore Memory:

1. Go to your Jupyter development environment and run the following code:

```
git clone https://github.com/PacktPublishing/Agentic-with-AWS
cd Chapter3/agentcorememory_with_langgraph
```

2. Next, in the Jupyter Notebook, open the `math-agent-with- checkp o inting.ipynb` file and install the necessary libraries:

```
Import LangGraph and LangChain components
# Import LangGraph and LangChain components

from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.prebuilt import create_react_agent
# Import the AgentCoreMemorySaver that we will use as a checkpointer
```

The following code creates a new **AgentCore Memory resource** that the agent can use to store and manage memory. In the code below `memory_name = " MathLanggraphAgent "` sets the name of the memory resource and `client = MemoryClient ( region_name =region)` creates a client object for interacting with AgentCore Memory in the specified AWS region:

```
memory_name = "MathLanggraphAgent"
client = MemoryClient(region_name=region)
memory = client.create_memory(name=memory_name)
```

Figure 3. 19 shows the Amazon Bedrock AgentCore console after creating a new memory resource. You should focus on the memory entry listed in the table, which confirms that the memory has been created and is being initialized.

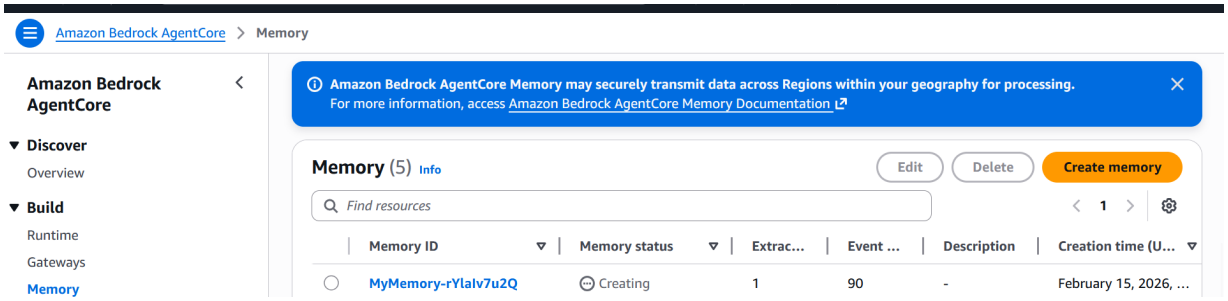


Figure 3.19: AgentCore memory created

The following code set model by defining `MODEL_ID = "global.anthropic.claude-haiku-4-5-20251001-v1:0"` defines which foundation model the agent will use. In this case, it points to an Anthropic Claude Haiku model accessed through Amazon Bedrock.

`checkpointer = AgentCoreMemorySaver ( memory_id , region_name =region)` creates a `checkpointer` that uses AgentCore Memory to persist the agent's state. This means the agent can save conversation progress, intermediate state, and checkpoints so it can resume later instead of starting from scratch each time:

```
MODEL_ID = "global.anthropic.claude-haiku-4-5-20251001-v1:0"

# Initialize checkpointer for state persistence
checkpointer = AgentCoreMemorySaver(memory_id, region_name=region)
```

The following code initializes the large language model for chat-based interaction. It tells the application to load the specified model from Amazon Bedrock using the Bedrock Converse interface in the selected AWS region:

```
llm = init_chat_model(MODEL_ID, model_provider="bedrock_converse", region_name=region)
```

Let's define the mathematical tools our agent will use. For this demonstration, we'll provide two simple operations:

```
@tool
def add(a: int, b: int):
    """Add two integers and return the result"""
    return a + b
@tool
def multiply(a: int, b: int):
    """Multiply two integers and return the result"""
    return a * b
tools = [add, multiply]
```

Now let's create our agent using LangGraph's `create_react_agent` builder with our AgentCore memory checkpointer:

```
graph = create_react_agent(
    model=llm,
    tools=tools,
    prompt="You are a helpful assistant",
    checkpointer=checkpointer,
)
graph
```

Figure 3.20 shows the LangGraph workflow created by `create_react_agent()`. A request that enters at `__start__`, is processed by the LLM-driven agent node, and either ends directly or routes to the tools node when external tool use is needed. Tool outputs are then passed back to the agent for further reasoning, while the checkpointer persistently saves the graph state behind the scenes:

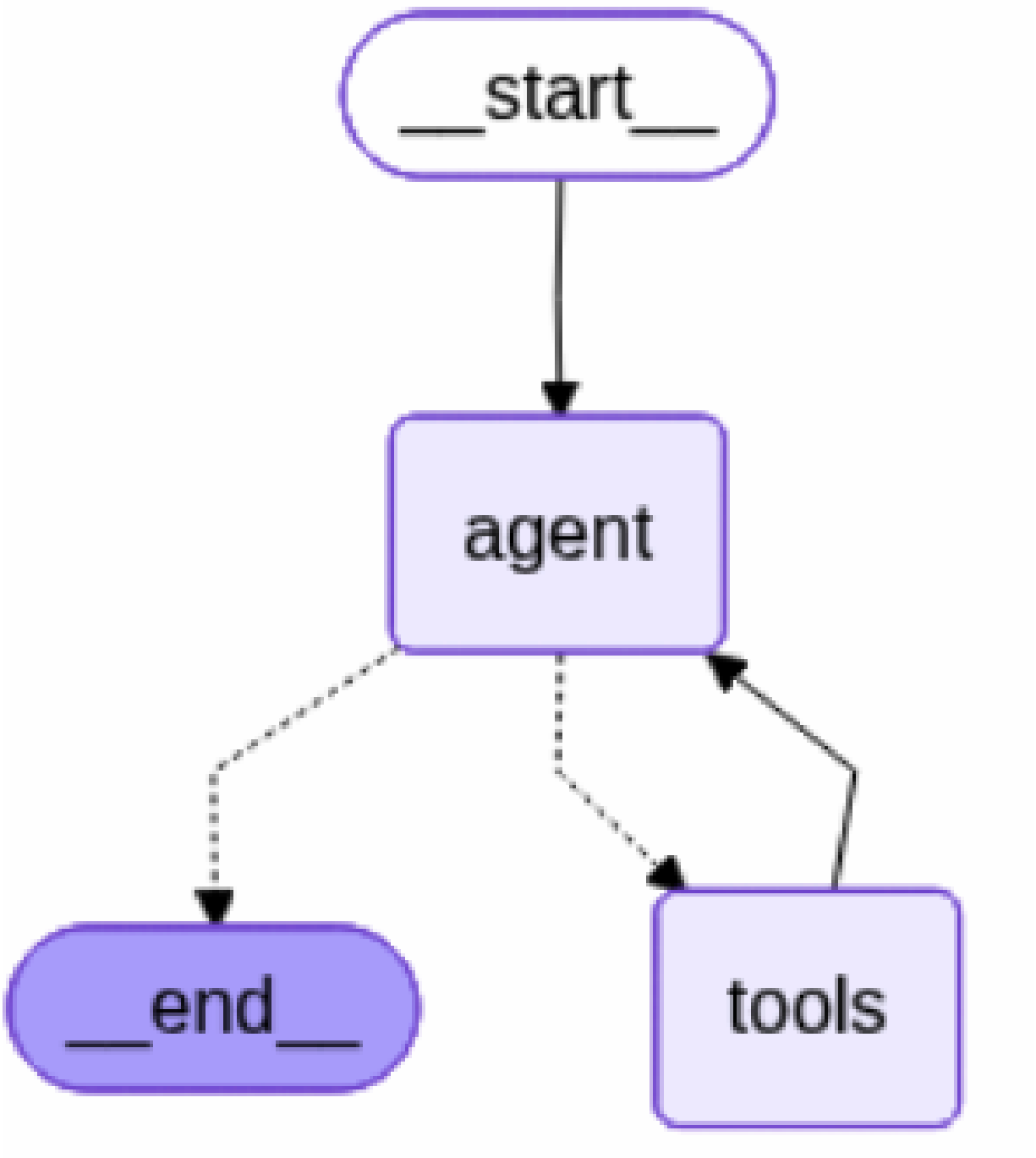


Figure 3.20: AgentCore memory created

Now that the agent is ready, let's run it.

Run the agent

In LangGraph, `RuntimeConfig` is a configuration object that contains attributes that are necessary at invocation time, for example, user IDs or session IDs. You

can read additional documentation here: https://langchain-ai.github.io/langgraphjs/how-tos/configuration/

For the AgentCore Memory checkpointer (AgentCoreMemorySaver), you need to specify:

- `thread_id` : Maps to `AgentCore session_id` (unique conversation thread).
- `actor_id` : Maps to `AgentCore actor_id` (user, agent, or any other identifier).

The following code sets the session context and the user query for the agent. `thread_id` maps to the `AgentCore session_id` , `actor_id` maps to the `actor_id` for memory tracking, and inputs contains the user's math question that the agent will process:

```
config = {
    "configurable": {
        "thread_id": "session-1", # REQUIRED: This maps to Bedrock AgentCore session_id under the
hood
        "actor_id": "react-agent-1", # REQUIRED: This maps to Bedrock AgentCore actor_id under the
hood
    }
}
inputs = {"messages": [{"role": "user", "content": "What is 1337 times 515321? Then add 412 and
return the value to me."}]}
```

Next, let's test the agent:

```
for chunk in graph.stream(inputs, stream_mode="updates", config=config):
    print(chunk)
```

Figure 3.21 illustrates the agent's internal reasoning and tool-use flow, showing how it plans the calculation, calls the multiplication tool, and receives the intermediate result before continuing to the next step.

```
{'agent': {'messages': [AIMessage(content=[{'type': 'text', 'text': "I'll calculate 1337 times 515321, then add 412 to the result."},
{'type': 'tool_use', 'name': 'multiply', 'input': {'a': 1337, 'b': 515321}, 'id': 'tooluse_PnzIG9OF7Oh5v9I17G8rdR'}], additional_kwargs
={}, response_metadata={'ResponseMetadata': {'RequestId': 'cb92b9a0-735b-4016-9527-5d830b7cab20', 'HTTPStatusCode': 200, 'HTTPHeaders':
{'date': 'Tue, 17 Feb 2026 18:23:15 GMT', 'content-type': 'application/json', 'content-length': '501', 'connection': 'keep-alive', 'x-a
mzn-requestid': 'cb92b9a0-735b-4016-9527-5d830b7cab20'}, 'RetryAttempts': 0}, 'stopReason': 'tool_use', 'metrics': {'latencyMs': [172
3]}, 'model_name': 'global.anthropic.claude-haiku-4-5-20251001-v1:0'}, id='run--afb0f729-ad6e-4190-b045-26064c597a5d-0', tool_calls=
[{'name': 'multiply', 'args': {'a': 1337, 'b': 515321}, 'id': 'tooluse_PnzIG9OF7Oh5v9I17G8rdR', 'type': 'tool_call'}], usage_metadata=
{'input_tokens': 665, 'output_tokens': 92, 'total_tokens': 757, 'input_token_details': {'cache_creation': 0, 'cache_read': 0}})]}}
{'tools': {'messages': [ToolMessage(content='688984177', name='multiply', id='792dbeba-245a-4fbd-8a3d-ba52631dcaab', tool_call_id='tool
use_PnzIG9OF7Oh5v9I17G8rdR')]}}
```

Figure 3.21: AgentCore Memory output

Let's examine the current conversation state stored in AgentCore Memory. The checkpointer automatically saves and retrieves state for our actor and session:

```
for message in graph.get_state(config).values.get("messages"):
    print(f"{message.type}: {message.text()}")
    print("========================================")
```

As you can see in Figure 3.22, the preceding code retrieves the saved conversation state from AgentCore Memory for the given actor and session, then prints each stored message. It demonstrates how the checkpointer automatically preserves and restores the conversation history.

```
human: What is 1337 times 515321? Then add 412 and return the value to me.
========================================
ai: I'll calculate 1337 times 515321, then add 412 to the result.
========================================
tool: 688984177
========================================
ai: Now I'll add 412 to that result:
========================================
tool: 688984589
========================================
ai: The answer is **688,984,589**.

To break it down:
- 1337 × 515,321 = 688,984,177
- 688,984,177 + 412 = **688,984,589**
========================================
```

Figure 3.22: Checkpoint

Testing memory persistence

Now let's test the power of our checkpointer by continuing the conversation. the agent should remember our previous calculations:

```
inputs = {"messages": [{"role": "user", "content": "What were the first calculations I asked you to
do?"}]}
for chunk in graph.stream(inputs, stream_mode="updates", config=config):
    print(chunk)
```

The preceding code asks the agent a follow-up question in the same saved session to check whether it remembers the earlier conversation. *Figure 3.23* is

the streamed output which shows how the agent responds using the stored context from AgentCore Memory:

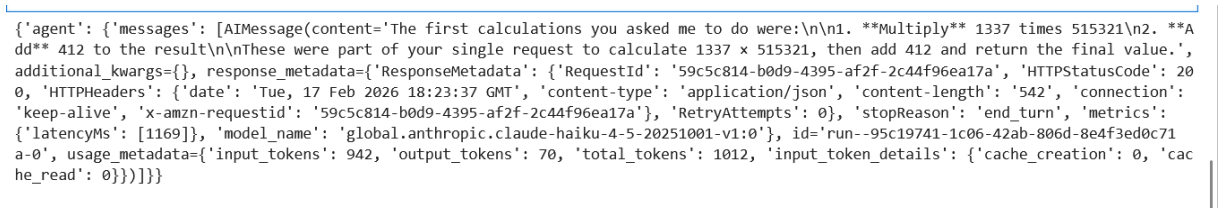


Figure 3.23: Agent responding from stored memory context

In this section, we built a stateful LangGraph agent powered by Amazon Bedrock AgentCore Memory, enabling it to persist and retrieve conversation context across sessions. We demonstrated how checkpointing seamlessly integrates with agent workflows to support multi-step reasoning and memory continuity. This foundation prepares us to design production-grade agents that move beyond stateless interactions into truly contextual systems.

Summary

In this chapter, we explored the role of memory in agents and why it is essential for building intelligent, context-aware systems. we covered the different types of memory: short-term and long-term and how each contributes to an agent's workflow. we also examined various AWS options for implementing both types of memory in production environments

Next, we discussed context management, including challenges such as context rot, and strategies to effectively manage and optimize memory over time. we then took a deep dive into amazon bedrock AgentCore memory, highlighting its built-in capabilities for context management and memory optimization. finally, through a hands-on lab, we demonstrated how to integrate memory into an agent orchestrated using the Strands SDK

So far, our focus has been on single-agent architectures. in the next chapter, we will expand this foundation by exploring collaborative agent architectures and design patterns, where multiple agents work together to solve complex tasks.

Get this book's PDF version and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

OceanofPDF.com

4

Advanced Agent Architecture Patterns

Whenever a software team needs to solve a complex problem, they don't dedicate a single person to do everything, whether it's architecture, coding, testing, or deployment. That would create chaos. Rather, there are architects, developers, testers, each doing what they're good at. Similarly, a hospital has doctors, nurses, specialists, and administrators working together. The same principle applies to AI agents.

In the previous chapters, you learned how to build single agents with tools and how they interact with other systems. For the majority of use-cases, a single agent should be able to accomplish most of the tasks. But what if a single agent isn't enough and if your problem is too complex for one agent to handle alone?

In this chapter, we'll move from single agents to multiple agents. We'll start by understanding why one agent isn't always enough, then explore patterns for coordinating multiple agents—from a simple supervisor delegating tasks to workers, to swarms of agents handing off work to each other, to structured graphs that route decisions through defined paths. Along the way, you'll build working examples and learn when each pattern makes sense (and when it doesn't).

This chapter will cover the following topics:

- When one agent isn't enough?
- What are multi-agent patterns?
- Understanding communication techniques

- Common orchestration patterns
- When multi-agent is not the answer

Technical requirements

Before diving into this chapter, ensure you have:

- Basic understanding of Python functions and classes
- AWS account with appropriate permissions (for AWS integration examples)
- Familiarity with REST APIs and JSON (helpful but not required)
- Use Kiro IDE for development of code (optional)

When one agent isn't enough

Before we dive into the patterns, let's understand why you'd want multiple agents working together instead of one super-agent trying to do everything.

The problem with single agents

Imagine asking a single agent to research the latest AI trends, analyze our competitors' pricing, write a technical specification, generate code, review the code for security issues, and create a presentation for executives.

That's a lot to ask from one agent! Here are the signs you might need multiple agents:

- **Context overload** : The agent tries to keep track of too many different tasks and loses focus
- **Jack of all trades, master of none** : The agent produces mediocre results across all tasks instead of excellent or accurate results in specific areas
- **No parallelization** : Everything happens sequentially, even when tasks could run simultaneously
- **Difficult to debug:** When something goes wrong, it's hard to pinpoint which part of the process failed

So how do we fix this? Instead of one agent doing everything, we split the work.

The multi-agent solution

Imagine you're running a busy restaurant. You could have one person doing everything—taking orders, cooking, serving, and cleaning. But that would be slow and inefficient. Instead, you have specialized roles: a host, servers, chefs, and dishwashers. Each person focuses on what they do best, and together they create a smooth dining experience.

Multi-agent systems work the same way. Instead of building one super-complex agent that tries to do everything, you create multiple specialized agents that work together. Each agent has a specific job, and they coordinate to solve complex problems faster and more effectively. Under the hood, a coordinator agent receives the user's request, breaks it into subtasks, and delegates each one to the right specialist. The specialists do their work and report back, and the coordinator stitches the results into a single response.

Multi-agent systems solve these problems by breaking down complex tasks into smaller, manageable pieces and assigning them to specialized agents.

Figure 4.1: Single and multi-agent

Benefits of multi-agent approach:

- Specialization: Each agent becomes an expert in its domain
- Parallel processing: Independent tasks can run simultaneously, saving time
- Better debugging: You can isolate and fix issues in specific agents

- Scalability: Add new agents as your system grows without rewriting everything

Now that you know why multi-agent systems are useful, let's look at how they're actually structured.

What are multi-agent patterns?

A multi-agent pattern is a structured way of organizing multiple AI agents to work together toward a common goal. Think of an orchestra where you have different sections like strings, brass, woodwinds, and percussion. Each section plays their part, but they all work together under a conductor to create beautiful music.

Just like orchestras can be organized in different ways (a small chamber ensemble, a large symphony, or a jazz band with improvisation), there are different patterns for organizing agents. Each pattern solves specific types of problems, and choosing the right one makes your system more efficient and easier to build.

When building multi-agent systems, you need to think about four fundamental aspects:

- **Orchestration** : How will you manage the flow of information between agents? You might use a central coordinator (like a conductor) or let agents communicate directly with each other (like a jazz ensemble).
- **Specialization** : Give each agent a focused role with specific expertise and tools. A research agent shouldn't also be handling data visualization. Let specialists do what they do best.
- **Collaboration** : Design how agents will share information and build on each other's work. The output from your research agent becomes the input for your analysis agent.
- **Coordination** : Decide whether agents follow a structured workflow you define (like a classical symphony following a score), or if they should figure out the best approach on their own as they go (like jazz improvisation).

These four aspects form the foundation of every multi-agent pattern that we'll explore in this chapter. First, we'll start with the simplest and most common pattern, then explore variations that handle more complex scenarios.

Your first multi-agent pattern: the supervisor-worker

In the supervisor-worker pattern, one agent acts as the coordinator. It receives the request, decides which specialist agents to call, delegates the work, and combines their results into a single response. The workers focus on their specific tasks and report back to the supervisor. Let's see this in the form of an example. Let's say, you're building a market research assistant. An investor asks: `Should I invest in electric vehicle companies?`

Your first version uses one agent. It scans recent news, analyzes stock performance, checks social media sentiment, reads analyst reports, compares competitors. It takes about five minutes per query. Works fine for a few queries a day. Then your boss says the system needs to handle hundreds of queries per hour. Five minutes per query won't work.

You split the work across three agents running in parallel. `NewsAgent` scans recent EV news. `FinancialAgent` analyzes stock performance. `SentimentAgent` checks social media sentiment. These tasks are independent. Since they can all run concurrently, it should ideally be three times faster.

You deploy it, and it works! Though the speed is great, you now have a different problem. The investor gets three separate responses:

- **NewsAgent**: `Tesla announced a new factory in Texas. Rivian had production delays. Ford is investing $50B in EV infrastructure.`
- **FinancialAgent**: `EV sector up 15% this quarter. Tesla P/E ratio is 45. Rivian burning $1B per quarter.`
- **SentimentAgent**: `Twitter sentiment mixed. Reddit bullish on Tesla. Concerns about charging infrastructure.`

The investor stares at this wall of unconnected facts and asks: `So... should I invest or not?`

You can notice that nobody synthesized the information or made a recommendation. The three agents did their jobs perfectly, but nobody coordinated them into a coherent answer. You need a supervisor agent here, who takes these three perspectives and weaves them into a clear recommendation.

With the supervisor-worker pattern, the supervisor agent coordinates the worker agents. Worker agents do specialized tasks and report back. Worker agents never talk to each other. All communication goes through the supervisor.

Let's re-consider the investment prompt but this time with the supervisor-worker pattern. So, the supervisor gets the question and thinks: `This person needs news context, financial analysis, and market sentiment. I have three workers who handle exactly those things.`

The supervisor calls all three workers at once:

- **NewsAgent**: `What's happening in the EV industry this month?`
- **FinancialAgent**: `What's the financial health of major EV companies?`
- **SentimentAgent**: `What ' s the market sentiment around EVs?`

Figure 4.2 illustrates the supervisor-worker pattern:

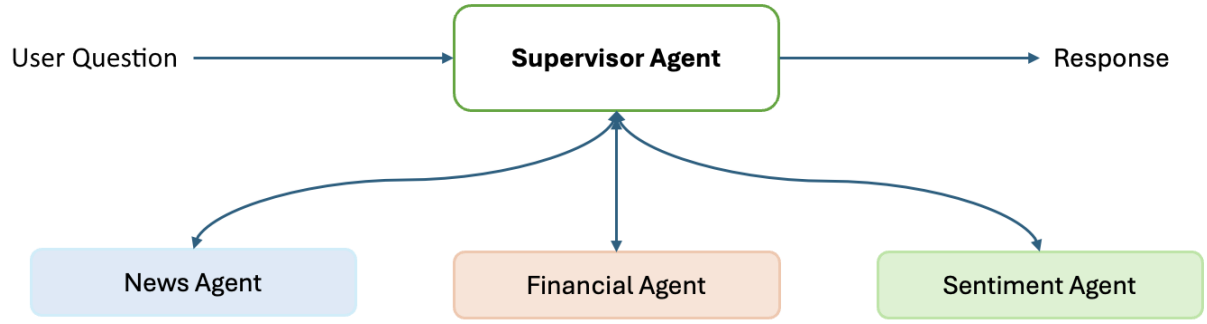


Figure 4.2: Supervisor-worker pattern

Here, all the agents work simultaneously and two minutes later, the supervisor has all three perspectives and synthesizes them into a single recommendation:

```
The EV sector shows strong growth with major players expanding. However, valuations are high and some
companies face production challenges. Consider established players like Tesla or Ford over newer
entrants. If you have high risk tolerance, the sector looks promising. If you prefer stability, wait
for valuations to normalize.
```

From Supervisor Agent, you receive one coherent answer, and then the investor can decide. Workers don't need to know about each other. `NewsAgent` doesn't care what `FinancialAgent` is doing. They focus on their expertise and report back.

Debugging becomes straightforward. If there's anything wrong with financial data, check `FinancialAgent` . If the news is outdated, check `NewsAgent` . The supervisor handles synthesis, not domain logic, so bugs stay isolated.

Adding new capabilities is easy. So, if you need regulatory analysis, you can create a `RegulatoryAgent` and tell the supervisor about it. Existing workers don't change. Let us see how a supervisor agent looks like in practice:

```
from strands import Agent
# Create worker agents with specific expertise
news_agent = Agent(
    name="news_agent",
    system_prompt="Research recent news about companies and industries. Provide factual, up-to-date
information from reliable sources.",
    tools=[news_search_tool]  # Tool for accessing news APIs
)

financial_agent = Agent(
    name="financial_agent",
    system_prompt="Analyze financial metrics, stock performance, and company valuations. Provide
data-driven insights.",
    tools=[financial_api_tool]  # Tool for accessing financial data
)
sentiment_agent = Agent(
    name="sentiment_agent",
    system_prompt="Analyze market sentiment from social media, forums, and investor discussions.
Identify trends and concerns.",
    tools=[sentiment_analysis_tool]  # Tool for sentiment analysis
)
# Create supervisor that coordinates the workers
supervisor = Agent(
    name="research_supervisor",
    system_prompt="""You are a market research coordinator. When asked about investment
opportunities:

    1. Analyze the question to determine what information is needed
    2. Delegate to specialist agents:
       - news_agent: For recent developments and industry news
       - financial_agent: For financial metrics and performance data
       - sentiment_agent: For market sentiment and investor opinions
    3. Synthesize their responses into a coherent investment recommendation

    Always provide balanced analysis considering all perspectives.""",
    tools=[
        news_agent.as_tool(),
        financial_agent.as_tool(),
        sentiment_agent.as_tool()
```

```
    ]
)
# Use the supervisor
question = "Should I invest in electric vehicle companies?"
response = supervisor(question)
print(response)
```

Here's how the output would appear:

```
The EV sector shows strong growth with major players like Tesla and Ford expanding.
However, valuations are high and some companies face production challenges.

Key findings:
- News: Major expansion by Tesla and Ford. Rivian facing production challenges.
- Financials: Sector growth strong but valuations high. Tesla profitable, Rivian not yet.
- Sentiment: Retail investors bullish. Institutional investors cautious about valuations.

Recommendation: Consider established players like Tesla or Ford over newer entrants.
If you have high risk tolerance, the sector looks promising. If you prefer stability,
wait for valuations to normalize.
```

Notice how the supervisor uses worker agents as tools. Each worker is created with a specific role, and the supervisor's system_prompt tells it how to delegate and synthesize.

The supervisor decides which workers to call and in what order. Each worker focuses on one job: `NewsAgent` fetches recent news, `FinancialAgent` pulls financial metrics, `SentimentAgent` analyzes market sentiment. The supervisor agent collects their results and synthesizes them into a coherent recommendation.

You can find the complete runnable code for this example in the book's GitHub repository: https://github.com/PacktPublishing/Agents-on-AWS/tree/main/chapter%204 .

The trade-offs of the supervisor-worker pattern

One of the major trade-offs with this pattern is that the supervisor becomes a bottleneck. Every decision flows through one agent. In production, you can address this through horizontal scaling (run multiple supervisor instances), caching (repeated queries return instantly), and prompt caching. For high-volume systems, you could use load balancing and async processing.

Another trade-off is the supervisor, which can be a single point of failure. If it crashes, the system stops. You need health checks, automatic restarts, and checkpoint management to ensure that the supervisor resumes from its last known state in case of failure. For critical systems, it's better to run redundant supervisors with a failover mechanism.

Workers can't collaborate directly. If `SentimentAgent` discovers something `FinancialAgent` should know, it goes through the supervisor. For simple workflows, this works fine. However, for complex negotiations, you need different patterns called collaborative communication, which we will discuss later in the chapter.

Most multi-agent systems don't need complex negotiations. Rather, they need clear accountability, simple communication, and easy debugging. That's why the supervisor-worker pattern is the most common pattern in production.

Make agents reusable: agent as a tool

The supervisor-worker pattern works well, but every time you need a specialist, you build it from scratch for that specific supervisor. What if you could build an agent once and reuse it across different systems? That's the Agent as Tool pattern. You wrap an agent with a description of what it does, and any other agent can discover and call it like a regular tool.

Research from Anthropic on building effective agents (`https://www.anthropic.com/research/building-effective-agents`) showed that treating components as modular, reusable tools allows systems to scale without rigid dependencies.

When another agent receives a task, it reads the available tool descriptions and picks the right one. If you recall the tools that we learned in *Chapter 2* , *Building Agents with Tools* , you can relate that it's the same concept, but here the tools are agents.

Figure 4.3: Agents as tools

Let's say you are building a tutoring assistant. Students ask questions about different subjects, and you have specialist agents for math, writing, and science. Instead of writing logic that says `if the question contains 'equation', call MathAgent` , you wrap each specialist as a tool and let the main agent figure out which one to use.

In the following code snippet, notice how each specialist agent is registered as a tool, and the teacher agent picks the right one based on the student's question.

```
from strands import Agent
# Create specialist agents
math_tutor = Agent(
    name="math_tutor",
    description="Solves mathematical equations and explains math concepts step by step",
    system_prompt="You are a math tutor. Solve math problems and explain your solutions clearly, step
by step."
)
writing_coach = Agent(
```

```
    name="writing_coach",
    description="Helps with writing, grammar, and essay structure",
    system_prompt="You are a writing coach. Help with writing, grammar, and essay structure. Provide
constructive feedback."
)
science_helper = Agent(
    name="science_helper",
    description="Explains scientific concepts and principles in simple terms",
    system_prompt="You are a science tutor. Explain scientific concepts clearly using simple language
and real-world examples."
)
# Teacher agent uses specialists as tools
teacher = Agent(
    system_prompt="""You are a teacher who helps students by delegating to specialist tutors.
    Never solve problems yourself. Always delegate to the appropriate specialist:
    - math_tutor: for math problems and equations
    - writing_coach: for writing, grammar, and essays
    - science_helper: for science questions

    After receiving the specialist's response, present it to the student in a friendly way.""",
    tools=[math_tutor, writing_coach, science_helper]
)
# Student asks a question
response = teacher("Can you help me solve this quadratic equation: x^2 + 5x + 6 = 0?")
print(response)
```

Here's the output:

```
I'll help you with that quadratic equation! Let me connect you with our math specialist who can walk you through solving x² + 5x + 6 = 0.

<math_tutor>
Please help solve this quadratic equation: x² + 5x + 6 = 0
</math_tutor>

Great! Here's what our math tutor explained:

To solve x² + 5x + 6 = 0, we can use factoring:

**Step 1:** Look for two numbers that multiply to 6 and add to 5
- Those numbers are 2 and 3 (since 2 × 3 = 6 and 2 + 3 = 5)

**Step 2:** Factor the equation
- x² + 5x + 6 = (x + 2)(x + 3) = 0

**Step 3:** Solve each factor
- If (x + 2) = 0, then x = -2
- If (x + 3) = 0, then x = -3

**Solution:** x = -2 or x = -3

You can verify these solutions by substituting them back into the original equation. Both values will make the equation equal to zero!

Would you like me to help you with any other math problems or perhaps work through another method like the quadratic formula?I'll help you
```

Figure 4.4: MathAgent explaining the solution of a quadratic equation

The teacher agent doesn't have hardcoded logic saying `if math question, call MathAgent` . It sees the available tools, reads their descriptions, and

decides which one to use based on the student's question.

This agent as a tool provides more flexibility. When you want to add a new `HistoryAgent`, you can just register it as a tool. The teacher automatically knows it's available. No routing logic updates, no new if statements.

This composability goes deeper. You can have specialist agents that can themselves have tools or other agents as tools, creating layers of expertise. Your `math_tutor` might use a `calculator_agent` and a `graph_plotter_agent` as tools. The same `math_tutor` can be reused by a teacher agent, a `tutoring_assistant` agent, or a `homework_grader` agent, as shown in *Figure 4. 5* :

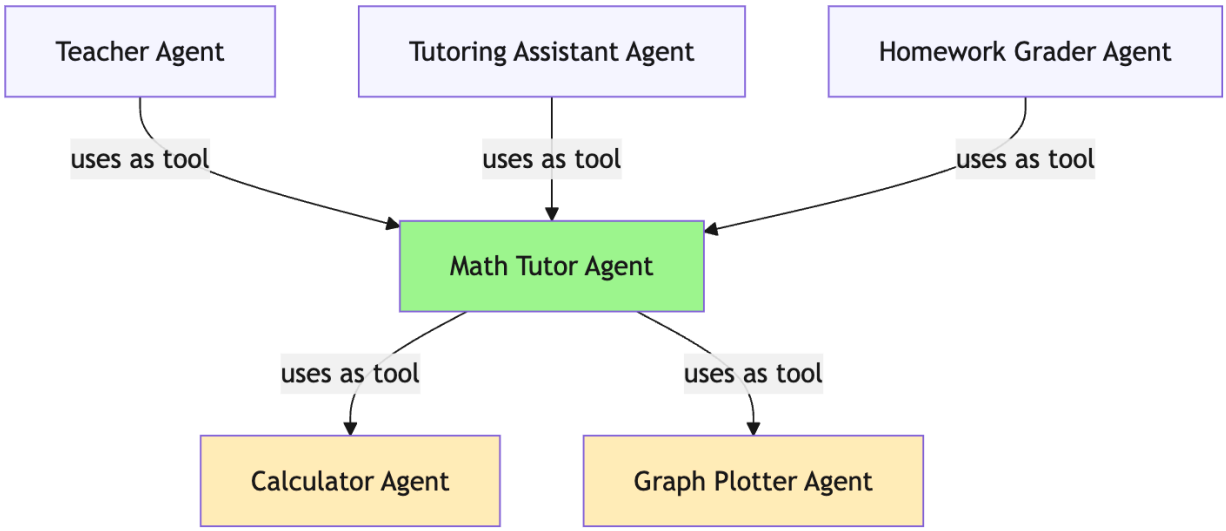


Figure 4.5: Composable agents as tools

Now that you have understood this pattern, let's look at how the data flows through multi-agent systems.

How data flows through multi-agent systems

You've seen the supervisor-worker pattern and Agents as Tools. Now let's zoom out and understand how data actually moves through these systems. We'll use a real example, asking, `What's the best laptop for video editing under $2000?`

Step 1: The request arrives

Your question hits the system as raw text. Nothing's happened yet.

Step 2: The router makes decisions

The router reads your question and thinks: `Okay, this person needs product recommendations, price comparisons, and technical specs. I know exactly which agents can help.`

The router might decide:

- Send to the Specs Agent (knows technical requirements for video editing)
- Send to the Price Agent (can search current prices across retailers)
- Send to the Review Agent (can analyze customer feedback)

The router doesn't just blindly forward your question. It might rephrase it for each agent. The Specs Agent gets `What CPU and GPU specs are needed for 4K video editing?` while the Price Agent gets `Find laptops under $2000 with these specs.`

Step 3: Agents do their thing

Now three agents are working simultaneously. The `SpecsAgent` is calculating requirements, the `PriceAgent` is searching retailer databases, and the `ReviewAgent` is analyzing thousands of customer reviews. They're all working at the same time.

Each agent has its own *memory* of the conversation. If you've asked about video editing software in a previous message, the `SpecsAgent` remembers that context.

Step 4: Agents talk to each other (sometimes)

Let's say the Price Agent finds a laptop for $1,800, but the Specs Agent realizes it doesn't have enough RAM for 4K editing. The Price Agent might ask: `Hey Specs Agent, if we increase the budget to $2,100, can we get something that works?`

This back-and-forth happens automatically. Agents negotiate and refine their answers based on what others are finding.

Step 5: Combining th e results

A supervisor agent collects everything:

- **SpecsAgent**: `You need at least 32GB RAM, RTX 4060, and an i7 processor .`

- **PriceAgent**: `Found 3 laptops matching specs: $1,899, $1,999, $2,049 .`
- **ReviewAgent**: `The $1,999 option has 4.5 stars with great feedback on rendering speed.`

Step 6: You get your answer

The aggregator synthesizes it into one answer: `Based on your needs, I recommend the Dell XPS 15 at $1,999. It has the specs you need for 4K editing, fits your budget, and users specifically praise its video rendering performance.`

The final response feels like it came from one agent, but behind the scenes, three specialist agents collaborated to give you the best answer.

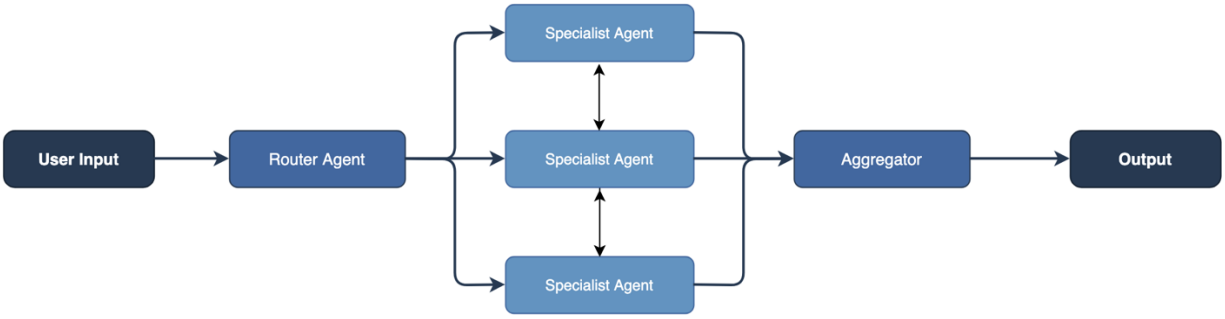


Figure 4.6: Data flow within agents

Compare this to a single agent approach, where one agent would have to:

- Know about video editing requirements
- Search multiple retailer databases
- Analyze customer reviews
- Synthesize everything

That's a lot for one agent. It would be slower, more expensive to run, and more likely to miss something. With multiple agents, each one is an expert in its domain, and they work in parallel.

What we just described seems simple, but there's a lot happening under the hood. How do agents share information? How do you decide if they work in sequence, in parallel, or collaboratively? Let's look at that next.

Understanding communication techniques

You know what makes up a multi-agent system. Now you can dive into the most critical piece: how agents talk to each other.

Agents working in isolation aren't really a system. They're just individual programs running separately. Communication is what turns them into something useful: sharing information and coordinating their work. But not all communication looks the same.

Agents can communicate in three main ways. Each one solves different problems and fits different situations.

Sequential communication

Picture you are writing a research paper. You can't write the conclusion before doing the research. You can't edit what hasn't been written yet. Some things just have to happen in order.

Sequential communication works this way: one agent finishes its work and hands off the results to the next agent.

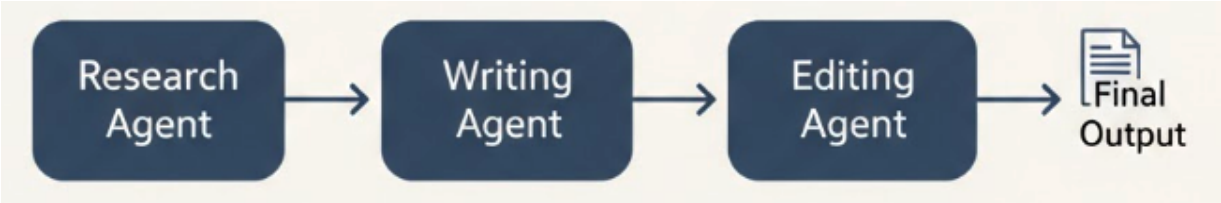


Figure 4.7: Sequential communication

The Research Agent may spend 10 minutes gathering information from various sources. Once it's done, it passes everything to the Writing Agent, which then may spend 15 minutes creating a draft. That draft goes to the Editing Agent, which then may spend 5 minutes polishing the content. The total time it took was 30 (10 + 15 + 5) mins.

Each agent waits for the previous one to finish. The Writing Agent literally cannot start until it has the research. The Editing Agent needs a complete draft to work with.

Sequential communication makes sense when you're stuck with dependencies. If step B needs step A's output, a sequential approach is the way to go. The trade-off is time. Three agents taking 10 minutes each means 30 minutes total with no overlap. If the Writing Agent fails after the Research Agent already spent 10 minutes, that time is wasted. Everything downstream

stops and you have to restart the pipeline. On the upside, sequential execution means only one agent calls the LLM at a time, which helps if your account has API rate limits.

Parallel communication

Now imagine planning a conference. One person books the venue, another arranges catering, and a third handles marketing. These tasks don't depend on each other. They can all happen at the same time. Parallel communication lets multiple agents work simultaneously on different parts of the problem. When they all finish, you combine their results.

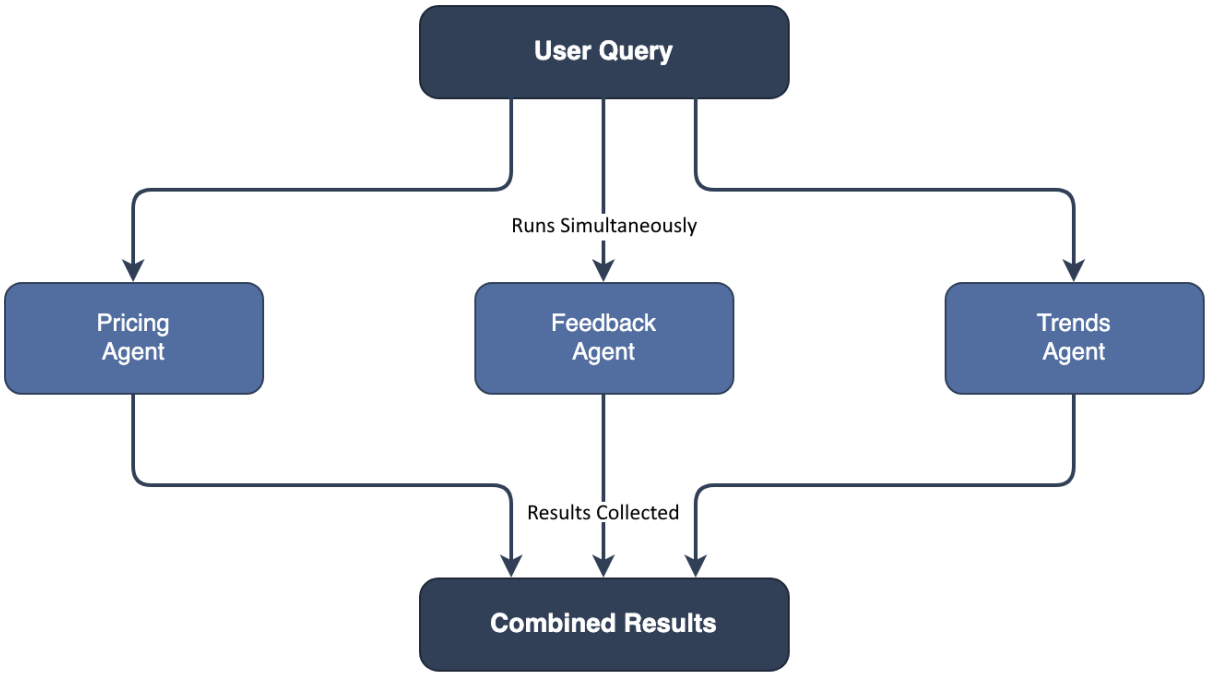


Figure 4.8: Parallel Communication

The preceding diagram shows how a user query is sent to three agents at the same time. Each agent works independently, and once all three finish, their results are collected and combined into a single response. Let's say you're analyzing a market before launching a product. You need competitor pricing, customer feedback, and market trends—three different types of analysis.

- One agent scrapes a competitor's website and builds a pricing matrix. It takes 5 minutes.
- Another agent reads thousands of customer reviews from similar products, identifying patterns in complaints and praise. This also takes

5 minutes.

- A third agent analyzes search trends, social media mentions, and industry reports. Another 5 minutes.

If you ran these one after another, you'd wait 15 minutes. Run them in parallel and you wait just 5 minutes (the time it takes for the slowest agent to finish).

Parallel communication works best when tasks are independent, when Agent 1 doesn't need Agent 2's results to do its work. Another situation where parallel communication is useful is when you want higher accuracy, with multiple agents checking the same thing from different angles.

Suppose you're building a content moderation system. You run three agents in parallel, each independently checking if a post violates community guidelines. Agent 1 says `safe` , Agent 2 says `safe` , Agent 3 says `unsafe` . Two out of three vote safe, so you approve the post.

Agents make mistakes. But they don't all make the same mistakes. If three independent agents look at the same content and two agree, you can trust more than one agent's opinion. If all three flag something, you're very confident it needs review.

This voting approach has real backing. Research on enterprise AI systems shows that running multiple agents in parallel and having them vote on results can dramatically reduce error rates compared to a single agent. The paper, *The Six Sigma Agent* (`https://arxiv.org/html/2601.22290v1`) discusses how consensus voting with just five agents can reduce errors from 5% down to 0.11%. Independent agents don't make the same mistakes, so voting catches errors that would slip through with a single agent.

You can set up voting different ways depending on what you need. Content moderation might use simple majority. Medical diagnosis might require all agents to agree. Legal document review might give the specialist agent's vote more weight.

Agents must be independent though. If they all use the same data source or reasoning approach, they'll make the same mistakes. You need diversity: different models, different data sources, and different approaches.

Collaborative communication

Think about designing a building. The architect proposes a design, but the structural engineer points out it won't support the weight. The architect adjusts the design, and then the interior designer suggests changes for better flow. Everyone discusses it back and forth until they find a solution that works for all constraints.

Collaborative communication means agents talk to each other, share insights, negotiate trade-offs, and refine their work together, not just working in parallel or sequence.

Let's look at a scenario where three agents collaborate on a code review. The Developer Agent has written a function that processes user data, the Security Agent checks for vulnerabilities, and the Performance Agent ensures the code runs fast enough. Watch how they negotiate trade-offs:

```
Developer Agent: I've written a function that processes user data quickly.
Security Agent: This is vulnerable to SQL injection. You need to sanitize inputs.
Developer Agent: If I add sanitization, performance drops by 40%.
Performance Agent: 40% slower is unacceptable. We have strict latency requirements.
Security Agent: Then use parameterized queries. They're secure and fast.
Developer Agent: Good idea. Let me refactor.
Performance Agent: Wait—you're loading the entire dataset into memory. Stream it instead.
Developer Agent: Done. All three concerns addressed.
[All agents review the updated code]
Security Agent: Secure now.
Performance Agent: Meets latency requirements.
Developer Agent: Code is ready.
```

This back-and-forth doesn't happen in sequential or parallel patterns. Sequential passes information in one direction. Parallel communication has agents working independently without talking to each other. Collaborative is a real conversation where agents negotiate until they find a solution that satisfies all the different agents' requirements, for instance, in the preceding code review example, balancing security, performance, and maintainability.

If you ask whether this actually works better than a single agent, the answer is yes. Research on multi-agent systems shows that agents discussing and refining together outperform single agents on complex reasoning tasks. The paper *Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks* (`https://arxiv.org/html/2509.23537v1`) found that

iterative discussion catches errors and explores alternatives that one agent would miss.

Collaborative communication has a trade-off though. Multiple rounds of discussion mean multiple rounds of LLM calls. A single agent might respond in 10 seconds. Three agents collaborating might take 30-40 seconds. Troubleshooting is also more complex. When three agents go back and forth multiple times, figuring out which round of conversation introduced a bad decision takes more effort than debugging a simple sequential pipeline. It's also slower and costlier, but you get better decisions on complex problems where trade-offs matter.

Collaborative communication works best when agents need to balance trade-offs that don't have a single right answer. Security vs. performance, cost vs. quality, speed vs. accuracy. These are decisions where one agent's recommendation affects another's. Instead of picking one perspective, you let agents negotiate until they find a solution that addresses all three concerns. The code review example above is a good illustration: the final code was secure, fast, and correct because all three agents pushed back on each other until the trade-offs were resolved.

You've learned how agents communicate. Sequential, parallel, and collaborative are the building blocks. Now we'll see how to organize entire systems using these building blocks.

The patterns coming next (Graph, Swarm, and Workflow) show you different ways to structure multi-agent systems. Each one uses the communication patterns you just learned but applies them to solve specific types of problems.

Common orchestration patterns

You've learned the supervisor-worker pattern and now you understand how agents communicate. Supervisor-worker is the most common pattern used by organizations, but it's not the only one. Let's explore two common patterns that solve different types of problems: Swarm and Graph Pattern.

Swarm patterns

Similar to collaborative communication we learned earlier in this chapter, where agents discuss and negotiate until they find a solution to a user query,

the Swarm pattern takes the same approach and builds a system around it. The challenge with collaborative communication was coordination: Who talks to whom, when, and how do you prevent agents from talking in circles? Swarm solves this through handoffs, where agents pass control to each other explicitly.

Imagine you're part of a team designing a new level for a video game, let's say, a stealth mission in an abandoned space station. The level designer creates the initial layout (floor plan) and blackout (rough 3D prototype). During playtesting, the game designer discovers players get lost because there aren't enough visual cues. Back to the level designer, they add landmarks and lighting to guide players, but then the narrative designer realizes the new bright lighting contradicts the story, and this is supposed to be an abandoned facility with a creepy atmosphere, not a well-lit area. The level designer adjusts to use dim emergency lights and environmental storytelling instead.

Now the difficulty balancer playtests and finds the level is too easy with clearer navigation. Back to the level designer to add more challenging enemy placements without removing the visual cues. The process loops and branches based on what each specialist discovers during their review. You can't predict the path up front.

In a Swarm system, you'd have four agents: `level_designer`, `game_designer`, `narrative_designer`, and `difficulty_balancer`. Each agent operates independently with its own expertise. They don't share a supervisor or follow a fixed sequence. Instead, when one agent finishes its part, it decides which agent should go next and hands off the task with context about what's been done so far.

```
from strands import Agent
from strands.multiagent import Swarm
level_designer = Agent(
    name="level_designer",
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    system_prompt="""You are a Level Designer in a game design swarm.
Your role in the swarm is to create level layouts, environments, and visual elements.
Focus ONLY on layout and visual design. Do NOT write narrative or balance difficulty.
After completing your layout, hand off to other agents for review.
When receiving feedback from other agents, revise your design accordingly."""
)
game_designer = Agent(
```

```
    name="game_designer",
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    system_prompt="""You are a Game Designer in a game design swarm.
Your role in the swarm is to playtest designs for fun, usability, and player confusion.
Focus ONLY on gameplay feel and player experience. Do NOT design layouts or write narrative.
When you find issues, hand off to the appropriate agent.
Build upon the work of other agents while adding your playtesting perspective."""
)
narrative_designer = Agent(
    name="narrative_designer",
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    system_prompt="""You are a Narrative Designer in a game design swarm.
Your role in the swarm is to create story elements, character backstories, and atmospheric details.
Focus ONLY on narrative. Do NOT redesign layouts or adjust difficulty.
After adding narrative elements, hand off to another agent for their perspective.
When receiving designs from other agents, enrich them with story context."""
)
difficulty_balancer = Agent(
    name="difficulty_balancer",
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    system_prompt="""You are a Difficulty Balancer in a game design swarm.
Your role in the swarm is to evaluate and tune difficulty progression, enemy placement, and pacing.
Focus ONLY on balance. Do NOT redesign layouts or write narrative.
When you find layout or narrative issues, hand off to the appropriate agent.
After balancing, hand off to game_designer for a final playtest review."""
)
swarm = Swarm(
    [level_designer, game_designer, narrative_designer, difficulty_balancer],
    entry_point=level_designer,
    max_handoffs=20,
    max_iterations=20,
    repetitive_handoff_detection_window=8,
    repetitive_handoff_min_unique_agents=3
)
result = swarm("""Design a stealth mission level in an abandoned space station.
The level needs to support multiple playstyles (aggressive stealth, patient stealth, and ghost runs).
Include at least 3 distinct zones with different difficulty levels,
narrative elements that explain why the station was abandoned,
and ensure the difficulty curve is appropriate for mid-game players.""")
```

When you look at the code sample, the `entry_point` parameter specifies which agent starts first, the `level_designer` agent in this case. The `max_handoffs` and `max_iterations` limit how many times agents can pass control and how many total iterations can occur. Without these limits, agents could loop indefinitely.

One common problem in swarms is the ping-pong behavior, where two agents keep bouncing the same task back and forth without making progress.

The `repetitive_handoff_detection_window` and `repetitive_handoff_min_unique_agents` parameters prevent this.

In the preceding code, the detection window is set to 8 and minimum unique agents is 3. This means the system looks at the last 8 handoffs, and if fewer than 3 different agents appear in that window, it stops the swarm. So, if level_designer and narrative_designer keep passing back and forth for 8 turns without involving any other agent , the system recognizes the loop and forces a conclusion.

To see the handoffs as they happen, we enable debug logging for the swarm module. When you run the code, you'll see log lines showing each agent taking control and handing off to the next. Here's a snippet from an actual run:

```
DEBUG | strands.multiagent.swarm | current_node=<level_designer>, iteration=<1> | executing node
DEBUG | strands.multiagent.swarm | from_node=<level_designer>, to_node=<narrative_designer> | handoff
detected
DEBUG | strands.multiagent.swarm | current_node=<narrative_designer>, iteration=<2> | executing node
DEBUG | strands.multiagent.swarm | from_node=<narrative_designer>, to_node=<difficulty_balancer> |
handoff detected
DEBUG | strands.multiagent.swarm | current_node=<difficulty_balancer>, iteration=<3> | executing node
DEBUG | strands.multiagent.swarm | from_node=<difficulty_balancer>, to_node=<game_designer> | handoff
detected
DEBUG | strands.multiagent.swarm | current_node=<game_designer>, iteration=<4> | executing node
DEBUG | strands.multiagent.swarm | from_node=<game_designer>, to_node=<level_designer> | handoff
detected
...
DEBUG | strands.multiagent.swarm | node=<game_designer> | no handoff occurred, marking swarm as
complete
DEBUG | strands.multiagent.swarm | status=<Status.COMPLETED> | swarm execution completed
DEBUG | strands.multiagent.swarm | node_history_length=<11>, time=<486.46>s | metricsNotice how each
agent finishes its work and decides which agent should go next. The level_designer starts, hands off
to narrative_designer for story review, who hands off to difficulty_balancer for gameplay tuning, who
sends it back to level_designer to fix navigation. The agents figured it out based on what needed to
happen next.
```

The swarm started with `level_designer` , who created the layout and handed off to `narrative_designer` for story elements. The `narrative_designer` added the backstory and passed it to `difficulty_balancer` for gameplay tuning. The `difficulty_balancer` then handed off to `game_designer` for playtesting. The `game_designer` found navigation issues in Zone 2 and sent it back to `level_designer` for fixes. This back-and-forth continued for 11 iterations until

the `game_designer` approved the final design. Nobody programmed this sequence. The agents figured it out based on what needed to happen next. Handoffs happen through internal tool calls. When you create a Swarm, each agent automatically receives a `handoff_to_agent` tool and all agents share the same context.

Let's say, `game_designer` agent finishes playtesting and finds that enemy patrol patterns are too predictable. Now this is a difficulty issue, not a playtesting issue. The `game_designer` agent needs to hand this off to someone who can tune the challenge.

The Swarm provides the `game_designer` with a formatted context that includes:

- The original task
- Previous agents who worked on this (for example, `level_designer` → `game_designer`)
- What did each previous agent contribute?
- A list of available agents with their descriptions

The game_designer reads through the available agents:

- `level_designer` : Creates level layouts, environments, and visual elements
- `narrative_designer` : Creates story elements, character backstories, and atmospheric details
- `difficulty_balancer` : Evaluates and tunes difficulty progression, enemy placement, and pacing

The patrol problem matches the `difficulty_balancer's` description. The `game_designer` then makes the handoff:

```
handoff_to_agent(
    agent_name="difficulty_balancer",
    message="Patrol patterns too predictable - need more variation"
)
```

Now `difficulty_balancer` is in control. It receives the same formatted context showing the full context of what the `game_designer` found.

This is how the handoff mechanism works. Each agent sees who's available and what they do, then decides who should handle the current issue. The LLM makes this routing decision by matching the problem to the agent descriptions.

The sequence isn't predetermined. In some cases, the work of `level_designer` passes through all reviewers without issues. In other cases, agents hand off back and forth multiple times as they discover new problems. You define the agents and what they're good at, then they figure out who should handle each issue as it comes up.

Swarm works well for collaborative work where you can't predict the path upfront. If you need a fixed sequence where certain checks always happen in a specific order, Swarm isn't the right choice. Use the Graph pattern instead, which we'll cover next.

Graph pattern

Graph pattern is built on Directed Graph, where data flows along defined paths from one node to the next. In Graph Pattern, you will see two approaches – Directed Acyclic Graphs (DAG) and cyclic graph. Think of DAGs as flowcharts where arrows only point forward and never loop back. For example, order processing: risk check, then fraud detection, then shipping. Once an order clears fraud detection, it never goes back to risk check. In a cyclic graph, data can loop back to earlier nodes. For example, a code review system where a developer writes code, a reviewer flags issues, and it goes back to the developer for fixes. This loop repeats until the reviewer approves.

You might be thinking, this sounds similar to the Swarm pattern we learned previously, but there's a key difference: in a swarm, agents decide the path on their own. In a graph, you define the paths upfront. We are going to see how we can define these paths. Here we are going to focus more on DAGs because they are most often used in production workflows.

Before we dive into an example, let's understand the building blocks.

Nodes are the building blocks of the graph. Each node represents something that processes data. In Strands, a node can be:

- A single agent (like a fraud detection agent)
- A swarm of agents working together

- Another graph (graphs can be nested)
- Custom business logic (deterministic functions)

Edges are the connections between nodes. They define how information flows from one node to another. An edge says `when node A finishes, pass its output to node B` . Edges can have conditions attached, for example, `only follow this edge if the order value exceeds $1,000` .

Dependencies describe the relationships between nodes. If node B depends on node A, then node A must complete before node B can start. The graph automatically manages these dependencies and executes nodes in the correct order.

Now let's look at an example. Let's say you're running an e-commerce platform. Orders come in all day. Some are $50 purchases from regular customers. Some are $5,000 orders from brand new accounts. Some have shipping addresses that don't match billing addresses. You can't process them all the same way.

You could use supervisor-worker pattern. The supervisor looks at each order and decides where to route it. It works fine, but what if your payment processor asks, `How do you ensure high-value orders always get fraud review?` With supervisor-worker, the routing happens inside the LLM's reasoning. You can log it, but the decision logic isn't explicit in your code.

Graph pattern makes the routing explicit. You draw out the structure: risk analyzer runs first, high-value orders go to manual review, suspicious patterns go to advanced fraud detection, clean low-risk orders go straight to inventory check. The decision tree is visible in your code and anyone can look at it and see exactly what happens for each order type. *Figure 4.9* shows how an order flows through the graph, from risk analysis to the final outcome.

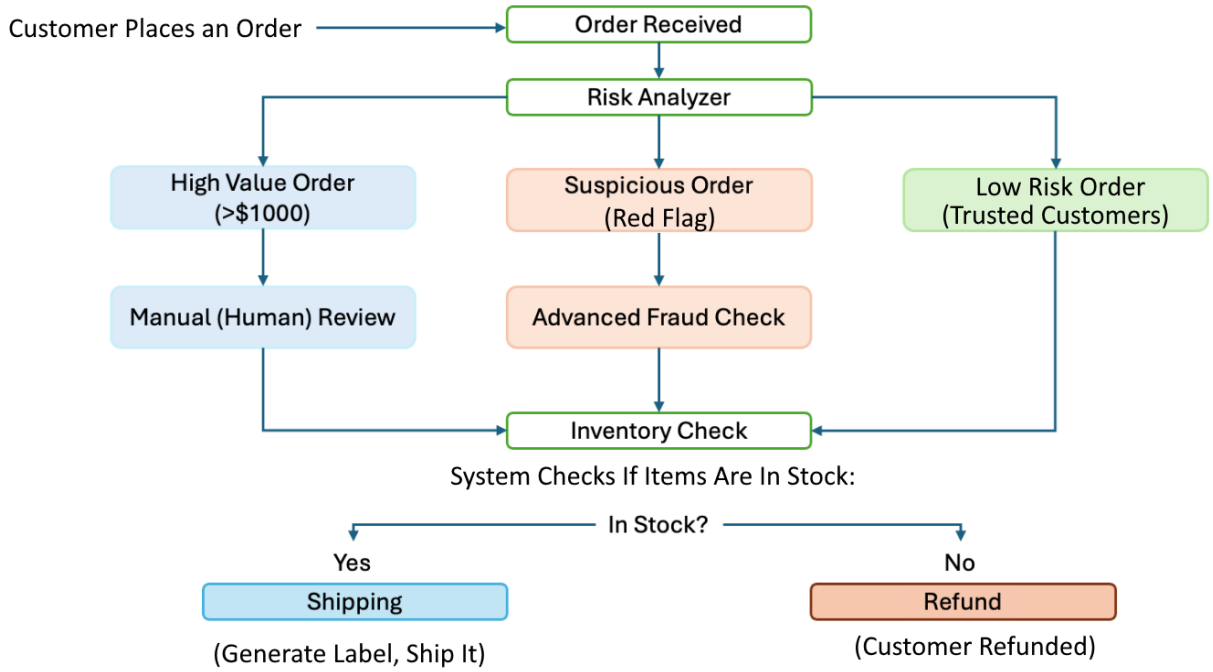


Figure 4.9: Graph pattern

When an order comes in, the risk analyzer categorizes it into one of three paths:

- High-value orders over $1,000 go through manual review before proceeding.
- Suspicious orders with red flags like mismatched addresses go through advanced fraud detection.
- Low-risk orders from trusted customers skip all fraud checks and go directly to inventory.

After manual review, fraud detection, or direct routing, all three paths converge at inventory check. If items are in stock, the order moves to shipping where labels are generated and the package goes out. If items are out of stock, the order moves to refund processing where the customer is notified and their payment is returned. Every order reaches one of these two final states.

The "directed" part means arrows point one way. Information flows from risk analyzer to fraud check, and never backward. The *acyclic* part means no loops. So, you can't have risk analyzer followed by suspicious order, then fraud check, then inventory, and finally back to risk analyzer. That would create an

infinite loop where orders never complete. The graph structure guarantees every order will eventually reach an end state, either shipped or refunded.

This pattern is useful for compliance and auditability. If your payment processor asks about your fraud prevention workflow, you point to the graph structure. The logic is explicitly defined in code and not hidden in LLM reasoning. An order for $1,500 from a new customer always follows the same path through the graph, and there is no variation based on how an LLM interprets the situation that day.

You might be thinking, where the LLM fits in since the graph structure is deterministic. The graph defines the flow, but each node is an agent that uses an LLM to process its task. The risk analyzer uses natural language understanding to assess the order, the fraud detection agent reasons about suspicious patterns, and the inventory agent interprets stock data. You define the routing and the LLMs handle the reasoning at each step.

Here's an example of a graph-based workflow for processing orders using specialized agents:

```
from strands import Agent
from strands.multiagent import GraphBuilder
# Create specialized agents
risk_analyzer = Agent(name="risk_analyzer", system_prompt="...")
manual_review_agent = Agent(name="manual_review_agent", system_prompt="...")
fraud_detection_agent = Agent(name="fraud_detection_agent", system_prompt="...")
inventory_agent = Agent(name="inventory_agent", system_prompt="...")
shipping_agent = Agent(name="shipping_agent", system_prompt="...")
refund_agent = Agent(name="refund_agent", system_prompt="...")

# Build the graph
builder = GraphBuilder()
builder.add_node(risk_analyzer, "risk_analysis")
builder.add_node(manual_review_agent, "manual_review")
builder.add_node(fraud_detection_agent, "fraud_check")
builder.add_node(inventory_agent, "inventory_check")
builder.add_node(shipping_agent, "shipping")
builder.add_node(refund_agent, "refund")
# Add conditional edges (conditions check the output from previous nodes)
# Conditional functions like is_high_value check state.results for specific patterns
builder.add_edge("risk_analysis", "manual_review", condition=is_high_value)
builder.add_edge("risk_analysis", "fraud_check", condition=is_suspicious)
builder.add_edge("risk_analysis", "inventory_check", condition=is_low_risk)
builder.add_edge("manual_review", "inventory_check")
builder.add_edge("fraud_check", "inventory_check")
```

```
builder.add_edge("inventory_check", "shipping", condition=is_in_stock)
builder.add_edge("inventory_check", "refund", condition=is_out_of_stock)
builder.set_entry_point("risk_analysis")
graph = builder.build()
result = graph("Order #12345: $1,500 laptop from new customer")
```

Here we have created agents for each processing step, then added them as nodes in the graph. The edges define how information flows between nodes. Some edges have conditions attached that check the previous node's output to decide which path to follow. The conditional functions like `is_high_value` and `is_suspicious` examine what the risk analyzer returned and route the order accordingly. The graph structure is fixed, but which path you take through it depends on the data. A $1,500 order goes through manual review, then inventory, then shipping. A $50 order from a trusted customer skips straight to inventory.

You can use Graph patterns when you have established procedures, such as e-commerce order processing, compliance workflows, approval chains, content moderation, or anywhere you need to prove that certain checks always happen in a specific order.

Before we wrap up, here's a quick comparison of the patterns we covered in this chapter:

Pattern	Who decides the path?	Best for	Trade-off
Supervisor-worker	Supervisor agent	Centralized coordination with a single synthesized answer	Supervisor is a bottleneck and single point of failure
Agent as tool	Orchestrator picks from available tools	Reusable agents across different systems	Agents only communicate through the orchestrator; nested calls add latency

Pattern	Who decides the path?	Best for	Trade-off
Swarm	Agents decide autonomously	Collaborative work where the path can't be predicted	Harder to debug; execution order varies between runs
Graph (DAG)	You define the paths in code	Compliance workflows, auditable pipelines	All possible paths must be defined upfront

Table 4.1: A comparison of graph patterns based on trade-offs

Now that you've learned orchestration patterns. You know how to build supervisor-worker systems, swarms, and graphs. But here's what nobody talks about enough: most problems don't need multiple agents.

When multi-agent is not the answer

We've seen this pattern too many times: someone learns about multi-agent systems and immediately rebuilds their working single-agent application with five agents coordinating through a graph. The demo looks impressive, and then they deploy it.

However, debugging becomes a nightmare. If you want to figure out which agent failed, or whether it was the input to agent three or the output from agent two, the answer is difficult to get. The logs show five different execution paths and you're trying to trace which one ran. Your single agent used to respond in 3 seconds. Now the multi-agent version takes 12 seconds because of coordination overhead. And the error rate went up because you have five points of failure instead of one.

A single agent with good tools beats a poorly designed multi-agent system. Before you architect something complex, try giving an agent the right tools. Try answering simple questions such as: Can it call APIs? Query databases? Run code? Access documentation? Most problems fit in one agent's context window if you structure the tools well.

The patterns in this chapter solve real problems, but they're not default architectures. You'll know you need multiple agents when you hit actual limitations. Your context window fills up because you're processing thousands of documents and the agent loses track. You need true parallelization because waiting for three sequential 30-second API calls is too slow, and you need specialized expertise that genuinely doesn't fit in one prompt. These situations exist and they're just rarer than you think.

Start with one agent, give it tools, and test it. Only add more agents when one agent clearly can't handle it. Three agents that work reliably beat ten agents that don't.

Summary

You've learned how to build multi-agent systems that actually work in production. You started with the fundamental question of when multiple agents are needed instead of one. Then, you walked through communication techniques that define how agents share information. To recap, sequential communication handles dependencies where one agent needs another's output, parallel communication speeds things up when agents can work independently, and collaborative communication lets agents discuss and negotiate when you need to balance competing concerns. These techniques are your building blocks.

You also explored orchestration patterns that structure entire systems. Supervisor-worker gives you centralized control with one agent coordinating specialized workers. Swarm lets agents hand off control to each other when the path isn't predetermined, whereas Graph provides explicit workflow control with deterministic, auditable paths. Most production systems combine these patterns. Your supervisor might coordinate parallel agents and your graph nodes might be swarms. The key is matching the pattern to your constraints.

In the next chapter, you will learn some advanced agentic architectures, such as Agentic RAG, Using Graph and Amazon SageMaker endpoint as an Agent, communication protocols like MCP and A2A, and how to evaluate agents with Amazon Bedrock and LangChain.

Subscribe to our newsletter

New frameworks, evolving architectures, research drops, production breakdowns— *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe for weekly insights that help you stay focused and informed.

Subscribe at `https://packt.link/8Oz6Y` or scan the QR code below.

OceanofPDF.com

5

Agent Communication

In *Chapter 1* , *Understanding AI Agents on AWS* , we introduced you to **Model C ontext P rotocol** (**MCP**) which helps standardizing the agent communication with external systems and **A2A** (**Agent to A gent**) Protocol which enables agents to interact with other agents. In earlier chapters, you learned how to build agents, connect them to tools, and design multi-agent systems. The next logical step is communication: how agents interact reliably with external tools, data sources, and other agents in a structured and reusable way. As agentic systems grow more capable, this communication layer becomes critical. Without standardization, every new tool, service, or collaborating agent introduces more custom integration code, more maintenance overhead, and more architectural complexity.

This chapter introduces two important protocols that address this challenge from different directions. MCP standardizes how agents and models connect to tools, resources, and prompt templates, making it easier to discover and use external capabilities without hardcoding every integration. The A2A Protocol standardizes how agents communicate with one another, enabling specialized agents to collaborate across systems while keeping their internal logic private. Together, these protocols provide the foundation for building modular, extensible, and interoperable agentic applications.

This chapter explores MCP and the A2A protocol for agent communication and shows how agents can be enhanced with the right set of tools.

In this chapter, we will cover the following topics:

- The Model Context Protocol (MCP): Standardizing the interface between models and tools.
- How MCP works: host, client, server, data sources, and protocol layer

- Difference between host and MCP client
- MCP transport mechanisms, lifecycle, and authorization
- MCP server and client features
- FastMCP vs FastAPI
- Using pre-built MCP servers and custom MCP servers
- Building MCP Servers and Clients: Hands-on implementation using Python and FastMCP.
- The Agent-to-Agent (A2A) Protocol: A2A architecture, protocol bindings, and communication model
- Building a Multi-Agent System: Orchestrating a travel assistant using Strands and A2A.

Technical requirements

Before diving into this chapter, ensure you have:

- A working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with appropriate permissions (for AWS integration examples)
- IDE - Kiro or Visual Studio Code or Claude Code (https://github.com/PacktPublishing/Agents-on-AWS/blob/main/README.md)
- Familiarity with REST APIs and JSON (helpful but not required)

Model Context Protocol

As you build more sophisticated agents, you'll quickly encounter a challenge: managing dozens or hundreds of tools becomes complex. What if you could discover and use tools dynamically, without hardcoding each one into your agent? What if tools built by other teams or organizations could be seamlessly integrated into your agents? This is where the MCP comes in.

Why is a standardized protocol needed for tools?

AI models and agents need to interact with many external tools, including APIs, databases, and developer platforms. Without a standardized protocol, each model must build custom integrations for each tool, which quickly becomes complex, costly, and difficult to maintain. Having a standardized protocol provides a common interface for tool use, reducing integration overhead, improving interoperability, and making it easier to scale AI systems as new tools and models are introduced.

Figure 5.1 shows a comparison between the traditional integration and the MCP approaches for connecting AI models to external tools and systems. As shown in the diagram, the model connects with three tools: DynamoDB, Glue, and GitHub.

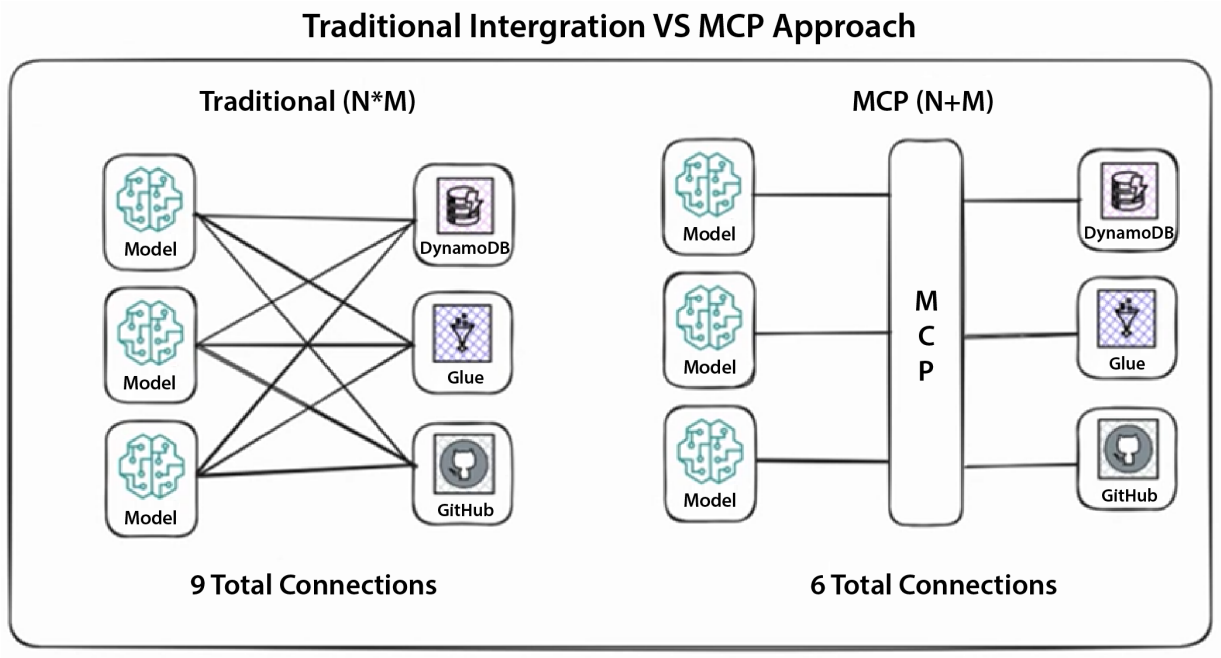


Figure 5.1: Traditional integration versus MCP

In the traditional model, each AI model must directly integrate with every tool or service—such as Amazon DynamoDB, AWS Glue, and GitHub—resulting in **N × M connections** , where *N* is the number of models and *M* is the number of tools. This creates many integrations that are difficult to build and maintain as the system grows. In contrast, the Model Context Protocol introduces a central layer that acts as a standardized interface between models and tools. Each model connects to MCP once, and each tool also connects to MCP once, reducing the integrations to **N + M connections** . This architecture simplifies

development, reduces maintenance, and allows new models or tools to be added easily without creating multiple new integrations.

This many-to-many integration pattern is exactly the problem MCP is designed to address, providing a shared interface that replaces fragmented, one-off tool connections with a more scalable and maintainable approach.

What problem does MCP solve?

Imagine you're building an enterprise agent that needs to interact with multiple systems: your company's CRM, inventory database, email system, calendar, document storage, and more. Traditionally, you'd need to write custom integration code for each system, maintain that code as systems change, and handle authentication and authorization for every individual system. On top of that, you'd have to deal with different API formats and conventions and update your agent code every time a new system is added. This repetitive, siloed approach is exactly what leads to **fragmented app development**, where every integration is built from scratch with no shared standard, making the entire system harder to scale, maintain, and manage over time.

For example, as shown in *Figure 5.2*, each AI app (App 1, 2, and 3) has to independently build its own entire stack: each app writes its own core AI integration from scratch, each app manages its own way of structuring and handling prompts, each app defines its own methods for invoking external tools, and each app builds its own connectors to reach data sources. If App 1 solves a data access problem, Apps 2 and 3 can't benefit. Bugs and updates must be fixed separately in each app. Also, each app may behave differently even for the same underlying task.

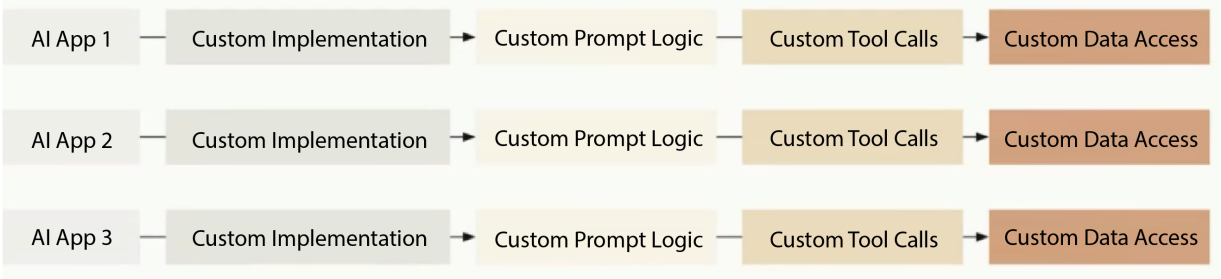


Figure 5.2: App and custom implementation

This approach doesn't scale. It's time-consuming, error-prone, and creates tight coupling between your agent and the systems it integrates with.

MCP addresses this by providing a single, standardized layer that all AI apps can share, so instead of each app reinventing the wheel, they all plug into a common protocol for tool calls and data access.

Using MCP offers several advantages that make agent development easier and more scalable:

- Dynamic tool discovery: Your agent automatically sees all available tools from connected servers
- No integration code: It connects to new tool sources without writing custom adapters
- Automatic updates: When a server adds new tools or updates existing ones, your agent sees the changes immediately
- Unified interface: Whether tools come from AWS, GitHub, or your internal systems, your agent interacts with them the same way
- Flexible deployment: MCP servers can run locally on your machine or remotely in the cloud

MCP provides a universal adapter for AI agents, such as USB, for example. Through MCP, AI models can connect to a wide range of external tools and systems, including databases (Amazon DynamoDB), CRM systems, calendars, code repositories (GitHub), messaging platforms (e.g., Slack), and document storage systems (Google Drive). This diagram shows how MCP acts as a centralized interface that connects AI models to diverse enterprise tools through a single integration layer. Instead of building custom connections for each tool, models interact with MCP, which standardizes communication and tool access.

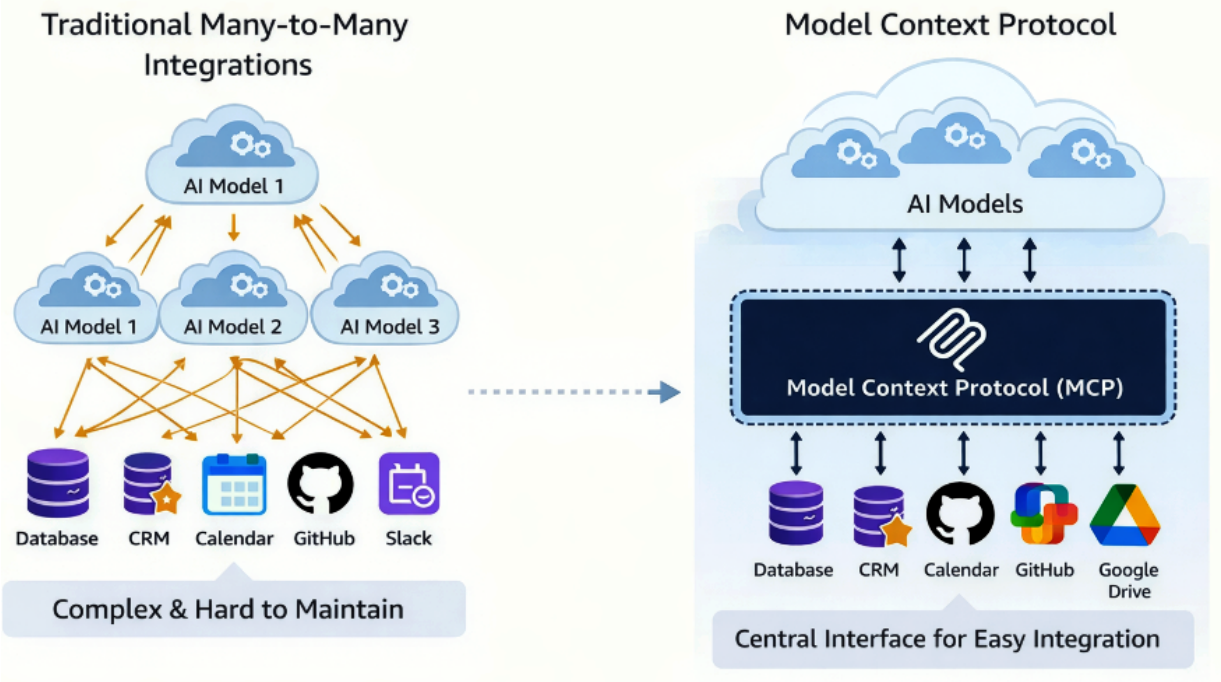


Figure 5.3: Traditional integration versus MCP

Without MCP, AI agents must build and maintain separate integrations for each tool, such as GitHub, Slack, and Google Drive, which increases complexity as more systems are added. MCP solves this problem by providing one standard, reusable connection layer through which agents can access all of these tools in a consistent way.

Understanding MCP in action

MCP establishes a standard protocol between your agent and tool providers, similar to how USB created a universal standard for connecting devices to computers.

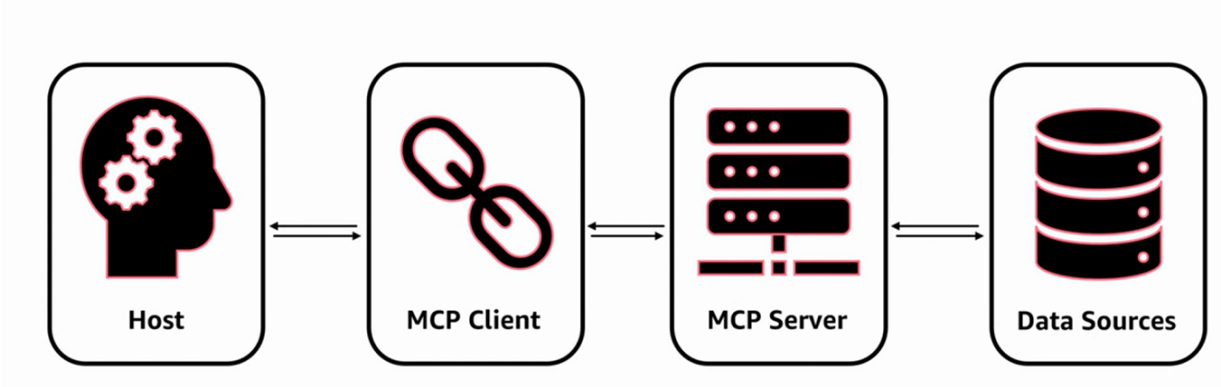


Figure 5.4: Long-term memory

Key components of MCP:

- **Host** : The Host is the AI application or agent that needs to perform tasks, such as answering questions, retrieving information, or interacting with tools. Instead of directly connecting to external systems, the host communicates with an MCP Client.
- **MCP Client (t ool c onsumer)** : Built into your agent, this component connects to MCP servers, discovers available tools, and handles the communication protocol. When your agent needs a tool, the client queries the appropriate server and executes the tool with the required parameters.
- **MCP Server (t ool p rovider)** : A service that exposes a catalog of tools with their descriptions, parameters, and capabilities. For example, AWS Labs provides an MCP server (`https://github.com/awslabs/mcp`) that exposes AWS documentation tools, GitHub provides servers for repository operations, and you can build custom servers for your internal systems.
- **Data sources** : Finally, the MCP Server connects to the actual data sources, such as databases, APIs, files, or enterprise systems. It then retrieves the required information and sends the results back through the MCP Client to the Host. This layered design allows AI applications to access many different tools and data sources through a consistent and reusable interface, making integrations easier to build, maintain, and scale.

These components work together through a **standardized communication layer** , which is the core idea behind MCP. Instead of each host and tool provider inventing its own integration pattern, MCP defines a common protocol for how clients and servers discover capabilities, exchange requests, and return results. This shared protocol is what makes MCP scalable, reusable, and easier to maintain across many tools and systems.

Now that you have a solid understanding of the key components of MCP, let's look at the difference between the host and the MCP Client.

Diffrnce between host and MCP client

When building an MCP client, the client itself does not operate independently. Instead, it is instantiated and managed by a **host application** , such as a code editor, AI assistant interface, or browser extension. The host application is responsible for the overall user experience, including managing the user interface, maintaining session memory, and orchestrating prompts. It can also coordinate multiple MCP clients simultaneously, with each client connected to a different server.

In contrast, each **MCP client** functions as a protocol-level component that maintains a one-to-one connection with a single MCP server. Its responsibilities include discovering the tools, resources, and prompts exposed by that server, gathering and organizing relevant context for the model, and prompting the user for any missing information when needed. Understanding this distinction is important: the **host is the layer users interact with** , while the **client enables the host to communicate with specific servers through MCP** . This diagram illustrates how an AI application (MCP Host) interacts with external systems through the Model Context Protocol.

Figure 5.5 gives a high-level overview of the relation and integration between the host and MCP client:

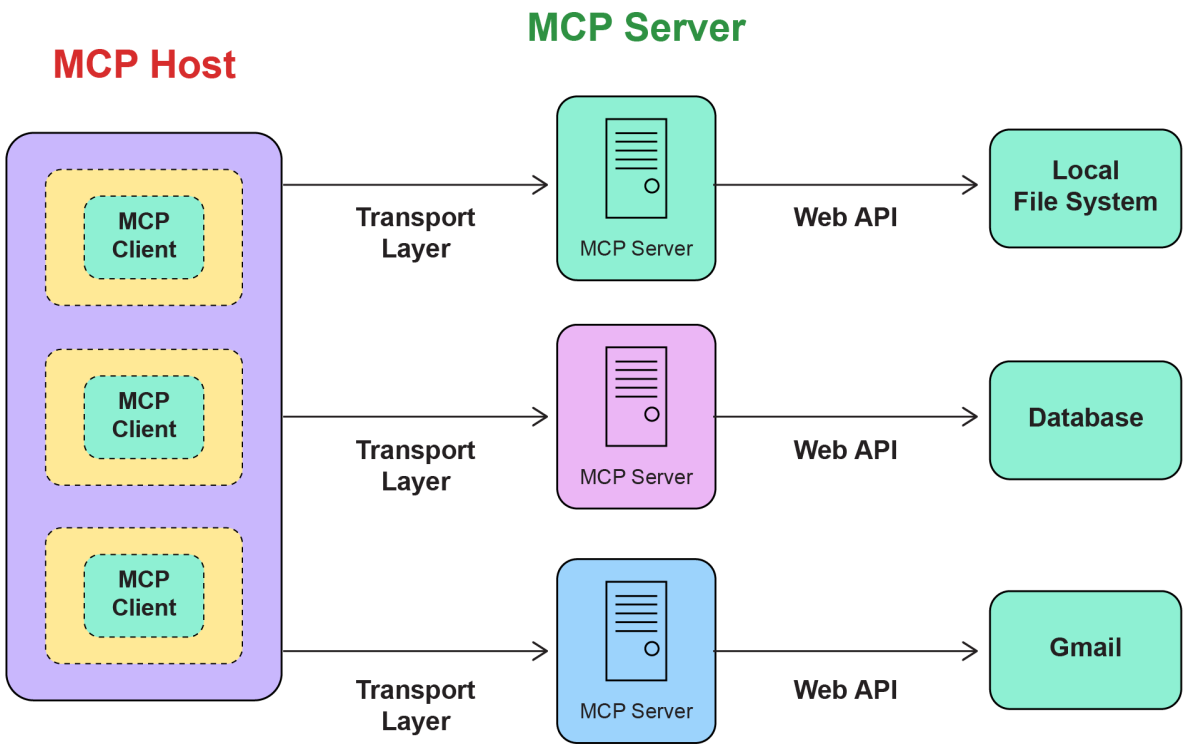


Figure 5.5: Relation between MCP host, MCP client and MCP Server

The MCP Host can contain **multiple MCP clients (one-to-many)** , each interacting with different MCP servers, and collectively enabling the agent to access multiple tools. This design keeps individual connections simple and modular while allowing the overall system to scale across many external services. Each **MCP client in the MCP Host connects to a single MCP server (1:1 relationship)** through the transport layer, ensuring a clear and isolated communication channel. In turn, each **MCP server connects to a specific external system (1:1 relationship).**

> NOTE
>
> The MCP Host is the application layer (UI, workflows, user interaction), while the Client is the integration layer within the host that implements the MCP protocol, allowing multiple clients to connect to different MCP servers.
>
> The **Host orchestrates** , but **each client communicates with one server only** , maintaining a clean and modular protocol connection.

Each MCP client communicates with its corresponding MCP server through a **transport layer** , which is responsible for carrying protocol messages between the two. MCP uses **JSON-RPC** as its message format, and these messages must be **UTF-8 encoded** . In other words, while the client and server define *what* is being communicated, the transport layer defines *how* those messages move between them.

MCP currently supports several standard transport mechanisms for client-server communication. These transport options should be viewed together because they serve the same purpose, but are designed for different deployment scenarios:

- **Stdio (Standard Input/Output):** Best suited for local integrations where the client and server run on the same machine. It is simple, direct, and lightweight, much like communication between local command-line processes. This is a common choice when running an AI agent locally on your computer.
- **SSE (Server-Sent Events) or Streamable HTTP:** Best suited for web-based or remote communication. It uses HTTP-based streaming so that

data can be delivered incrementally in real time, rather than as a single response. This is useful for scenarios such as a live chatbot receiving updates as they happen.

- **Custom transports:** Developers can implement their own transport mechanisms when the built-in options do not fit their requirements. For example, an enterprise team might use **gRPC** for high-performance, low-latency communication. In the next section, we'll see how the lifecycle of a client–server connection is managed to ensure smooth communication and proper state handling.
- Together, these transport options give MCP the flexibility to support local, remote, and enterprise deployment patterns while preserving the same client-server protocol model. In the next section, we will look at how the lifecycle of a client-server connection is managed to ensure smooth communication and proper state handling.

Managing lifecycle between client and server

Lifecycle management describes how an MCP client and server start, maintain, and end a connection. It ensures that both sides agree on protocol versions, understand each other's capabilities, and manage session state correctly. It includes three stages: **Initialization** , where the MCP client and server agree on protocol versions and capabilities; **Operation** , where normal protocol communication occurs; and **Shutdown** , where the connection is gracefully terminated by the client.

Example Consider an AI agent that connects to an MCP server to access a company's database. During **Initialization** , the MCP client and server perform a handshake, agreeing on the protocol version and capabilities (for example, the server confirms it supports query execution). In the **Operation** phase, the agent sends requests such as retrieving customer records and the server processes them and returns structured responses. Finally, during **Shutdown** , the client gracefully closes the connection, ensuring any ongoing operations are completed and resources are properly released.

Let's understand how the connection is secured.

MCP provides an authorization framework to be used with HTTP. Implementations using an HTTP-based transport should conform to this specification, whereas implementations using STDIO transport should not

follow this specification, and instead retrieve credentials from the environment. Additionally, clients, and servers may negotiate their own custom authentication and authorization strategies.

Before we check some hands-on examples on MCP client and server, let's explore some key features of the MCP client and server.

Understanding MCP server features

The MCP Server exposes three main categories of capabilities to the client: Tools, Resources, and Prompt templates.

Figure 5.6: MCP Server features tools, resources and prompt template

- **Tools** : These are **functions the A gent can execute** to perform actions in external systems. Examples include retrieving/searching data, sending messages, or updating database records.
- **Resources** : These represent **data or content the agent can read** . Resources represent read-only, persistent data that your server exposes to clients. Unlike tools, resources don't perform actions. Instead, they make information available for browsing, searching, and referencing. Resources are essential for enabling models to look up facts or structured information without triggering an operation. Examples include files, database records, or API responses that provide context for the Agent.
- **Prompt Templates:** These are predefined prompts exposed by the MCP server that the **agent can use through the MCP client** to guide how it interacts with data or performs a task. They provide reusable instructions for common interaction patterns, such as document question answering, transcript summarization, or generating output in a structured format like JSON.

All three are **exposed by the MCP Server and consumed by the MCP Client** , enabling AI agents to interact with external systems in a standardized way as shown in *Figure 5.7* :

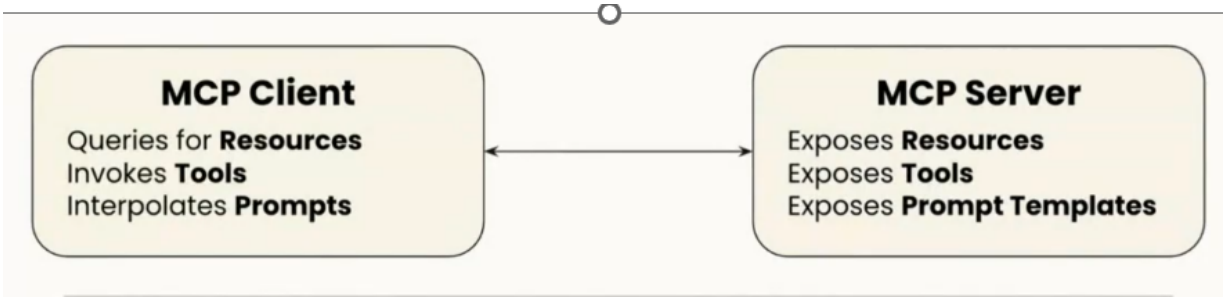


Figure 5.7: Long-term memory

The MCP Client typically uses these flows together:

- Discover capabilities (resources, tools, prompts) from the server
- Retrieve resources to gather context
- Invoke tools to perform actions or fetch additional data
- Use prompt templates to structure the model's interaction

Because tools were already covered in *Chapter 2* , *Building Agents with Tools* , we will briefly review their role in MCP and then focus more closely on resources and prompt templates

Tools

You are already familiar with tools, as they were covered in Chapter 2 . In the context of MCP, tools are specific actions your MCP server exposes to a client, like built-in functions or APIs that an LLM can call during a conversation. When a user asks for something that requires action (fetching data, triggering an operation, or performing a calculation), the model chooses the appropriate tool to handle it. Each tool execution requires explicit user approval, thus making sure that users maintain control at all times. An MCP server supports two methods per protocol:

- tools/list: To discover available tools, clients send a tools/list request. You can find the available tools that the server supports. It returns an array of tool definitions with schemas.
- tools/call: This is used to execute a specific tool. The method returns the tool's execution result.

You can read more about tools and their specifications at: `https://modelcontextprotocol.io/specification/draft/server/tools` .

Resources

Each MCP server defines collections or documents as resources. These can also be organized into categories (called Roots) for easier navigation. The MCP clients can fetch, filter, or search these resources on demand. To discover available resources, clients send a **resources/ list** request and to retrieve resource contents, clients send a resources/read request.

More information on Resources can be found at: `https://modelcontextprotocol.io/specification/draft/server/resources`

Resource template

Resource templates allow servers to expose parameterized resources using URI templates. Each resource is identified by a unique Uniform Resource Identifier (URI) (for example, file:///path/to/document.md) and declares a MIME type, which is a standard label that tells the client what kind of content the resource contains and how it should be handled. For example, a resource might be identified as Markdown, JSON, or PDF, allowing clients to process different content formats correctly.

For example, an MCP server can share a course directory as shown in the following code:

```
{
  "uri": "file:///data/student-directory.json",
  "name": " courseDirectory ",
  "title": " Course Directory ",
  "description": "List of courses and instructors in a program",
  "mimeType": "application/json"
}
```

Clients can substitute variables in the URI to query specific data. For example, a template such as `travel://activities/{city}/{category}` can provide access to activities of several categories for various cities, for example: `travel://activities/barcelona/museums` .

Let's take into consideration two scenarios to see Resources in action.

Scenario 1: Directly querying the resource

Imagine a student AI assistant that helps students quickly find information about their courses and instructors.

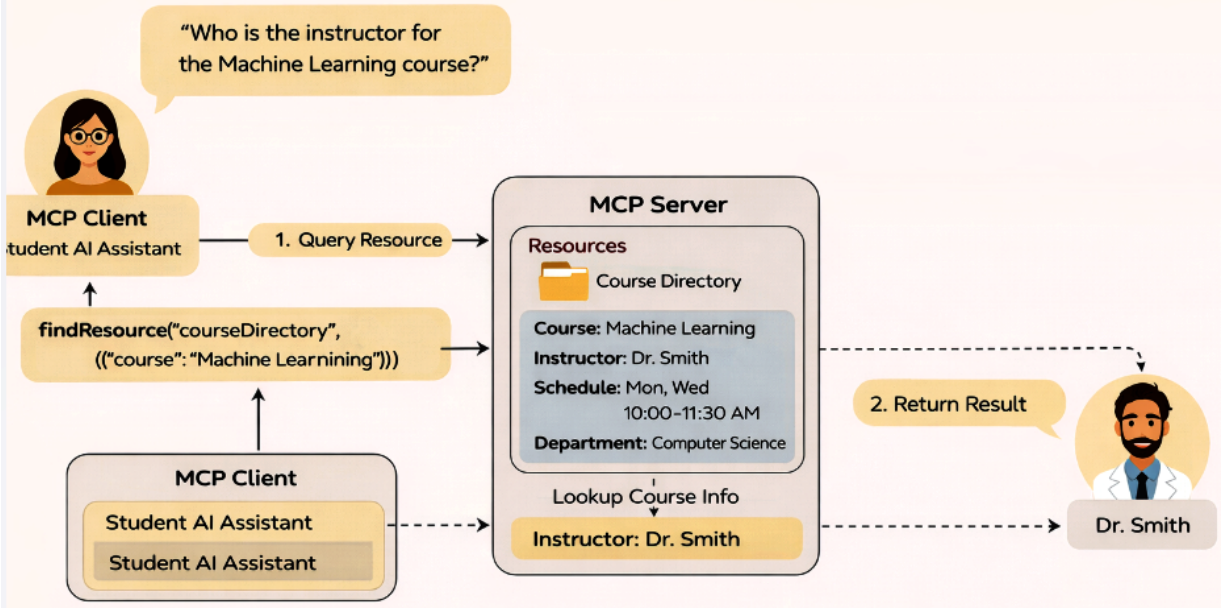


Figure 5.8: Accessing resource template in MCP client and server

If a student asks, `Who is the instructor for the Machine Learning course?` , the model recognizes that it needs course information. The MCP client inside the AI assistant then queries the MCP server for the appropriate resource. Once the server returns the data, the assistant uses it to generate the answer. An example request to the resource:

```
findResource("courseDirectory", { "course": "Machine Learning" })
```

Here, the **resource** contains structured information such as course names, instructors, schedules, and departments. Since this information is mostly static or rarely changes, the AI can simply read it without calling any tools.

Scenario 2: Querying the resource and tools

Resources can also be combined with tools. For example, a student might ask: "Find the instructor for the Machine Learning course and send them a message asking about office hours." As shown in *Figure 5.9* :

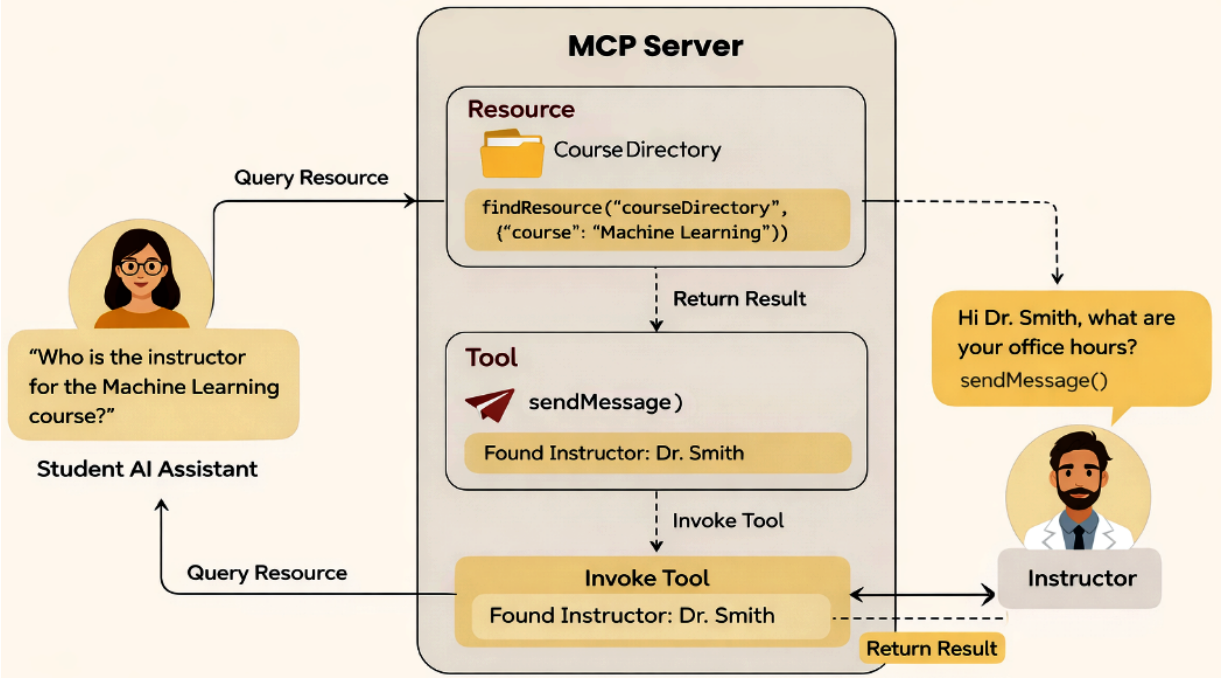


Figure 5.9: Querying resource template and invoking tool

In this case, the assistant would:

- Query the resource (`courseDirectory`) to identify the instructor
- Invoke a tool such as `sendMessage` to send the instructor a question
- Tools perform actions (like sending messages or scheduling meetings)

Using resources for **static or semi-static information** allows the AI to access useful context quickly without performing unnecessary operations.

> **NOTE**
>
> Resources provide information such as course directories or schedules.

Prompt templates

Prompts in MCP are predefined instruction templates that an **MCP server** exposes to clients. They help standardize recurring interactions and save developers from rewriting the same instructions for common tasks. Each prompt can include variables, which the **MCP client** fills in dynamically at runtime. The **MCP client** retrieves the selected prompt from the server and then passes the completed prompt to the model as part of the inference request.

Prompts can be versioned and updated centrally without changing client code. They are explicitly invoked by the user or client, they don't run automatically. They can also be context-aware, dynamically referencing available resources and tools to build more complete and efficient workflows.

Here is another MCP prompt template example using a student learning assistant:

```
{
  "name": "generateStudyPlan",
  "title": "Generate Study Plan",
  "description": "Create a personalized study plan for a student based on their course and exam
date",
  "arguments": [
    {
      "name": "course",
      "description": "Course the student is preparing for",
      "type": "string",
      "required": true
    },
    {
      "name": "examDate",
      "description": "Date of the upcoming exam",
      "type": "date",
      "required": true
    }
  ]
}
```

A student might ask: `Create a study plan for my Machine Learning exam on May 10 .` *Figure 5.10* illustrates this example:

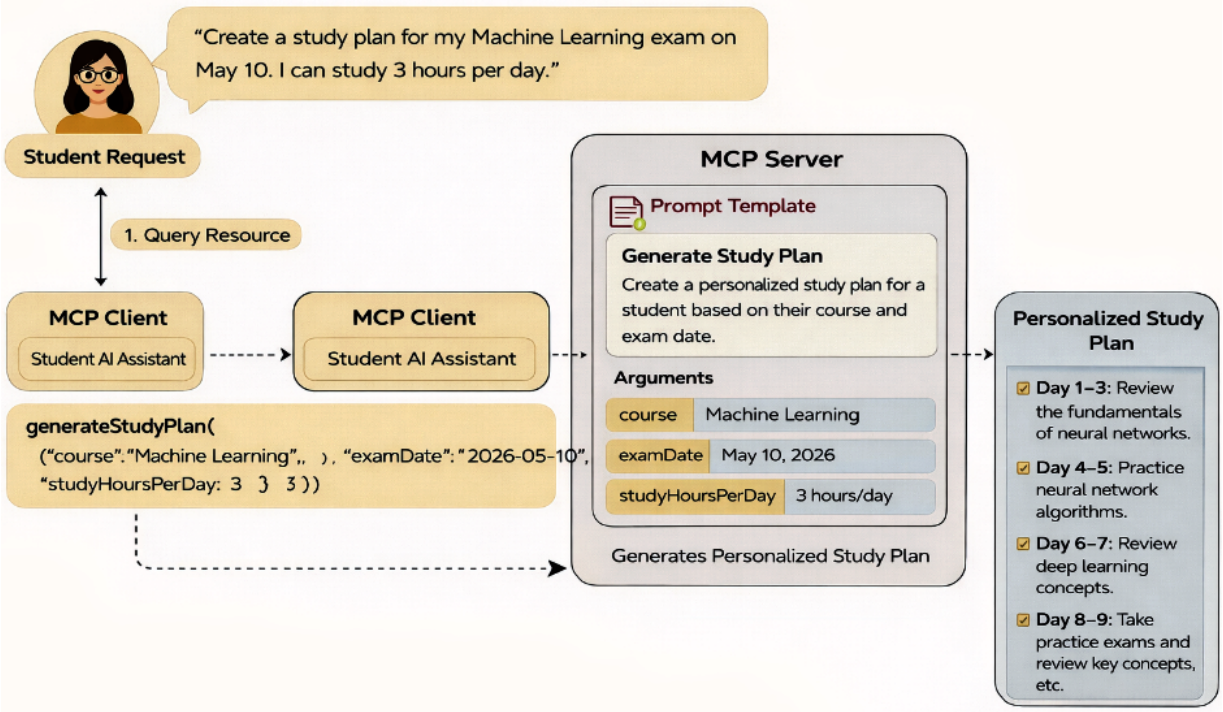


Figure 5.10: Long-term memory

The AI would use the `generateStudyPlan` prompt template and fill in the arguments:

```
generateStudyPlan({
  "course": "Machine Learning",
  "examDate": "2026-05-10",
})
```

The template then guides the model to generate a **structured study schedule** leading up to the exam.

Understanding MCP client features

When building an MCP client, you're creating the component that connects directly to an MCP server. But clients don't exist on their own; they are instantiated by a host application, like a code editor or a browser extension.

The next sections show how to use features such as Sampling, Roots, and Elicitation:

- Sampling: Letting MCP servers use your AI model safely
- Roots: Defining filesystem boundaries
- Elicitation: Request user input at runtime

These features help optimize context, organize data, and manage interactive flows. Let's understand each one of them in detail.

Sampling

Sampling lets an MCP server temporarily use the AI model and permissions already available on the client side, instead of calling its own model directly. This is valuable because it reverses the usual client-to-server request pattern: rather than the client only asking the server for data or actions, the server can ask the client to perform an AI task on its behalf. This is especially useful for complex, collaborative workflows where the server has the domain data or business logic, but the client has access to the model, the user's permissions, and the user interface. As a result, the client and server can work together on a task while still keeping the user in control of model usage, approvals, and final outputs. Figure 5.11 shows how sampling works in the MCP using a travel planning example:

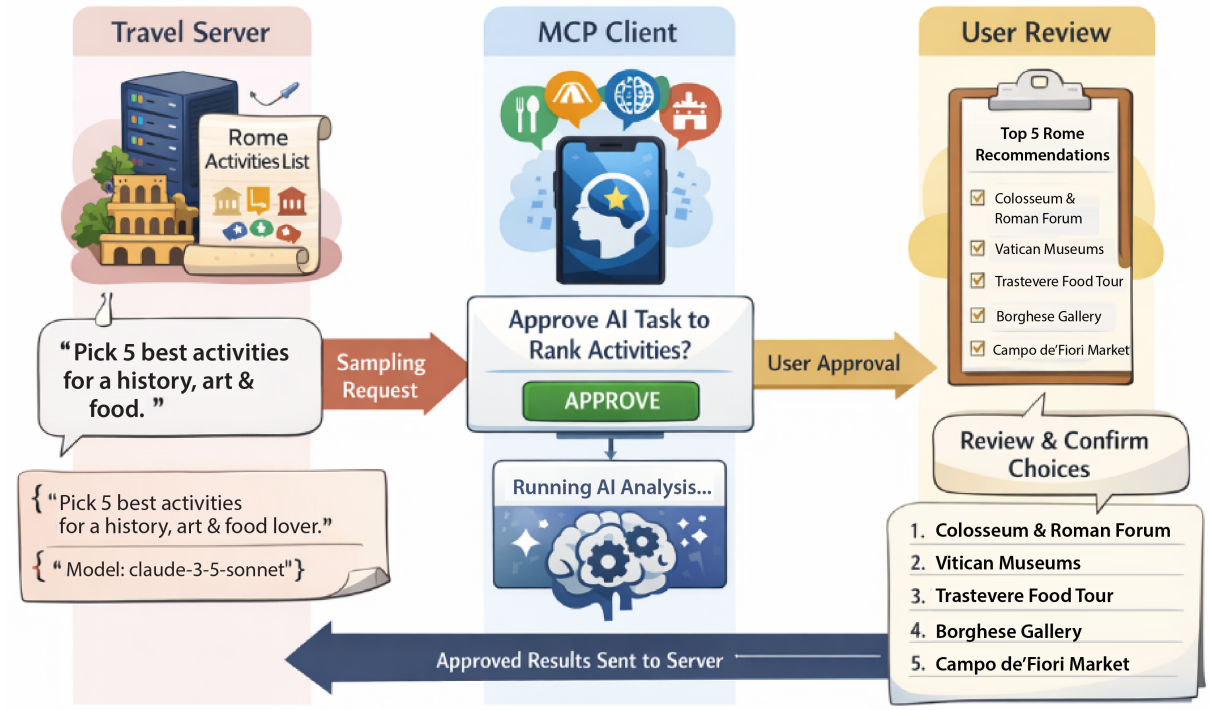


Figure 5.11: How sampling works between client and user for approval

A **travel server** retrieves many Rome activities but lacks AI logic to rank them. It sends a **sampling request** to the MCP client asking an AI model such as **Claude 3.5 Sonnet** to select the top five activities for someone interested in history, art, and food. The **client asks the user for approval** , runs the AI

analysis, and shows the results to the user. After the user reviews and approves the recommendations, the **client sends the final AI-generated results back to the server** .

Roots

MCP provides a standardized way for clients to expose filesystem *roots* to servers. Roots define the boundaries of where servers can operate within the filesystem, allowing them to understand which directories and files they have access to. Servers can request the list of roots from supporting clients and receive notifications when that list changes. *Figure 5.12* shows how **Roots work in the MCP** to control file access:

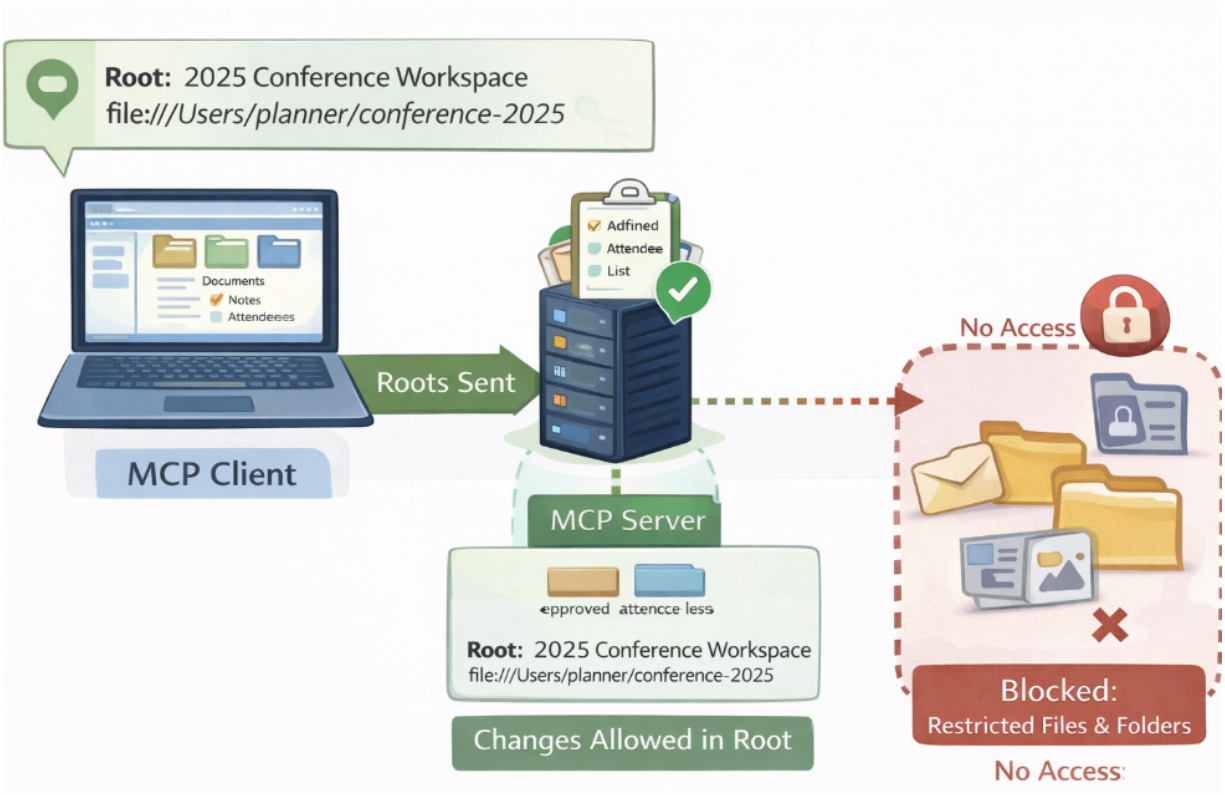


Figure 5.12: How MCP client uses roots to manage access to files

An event planner uses an MCP-powered assistant to manage conferences. The root

`file:///Users/planner/conference-2025` is the main workspace for current event.

The MCP client defines these as roots:

```
[
  {
    "uri": "file:///Users/planner/conference-2025",
    "name": "2025 Conference Workspace"
  }
]
```

With these boundaries, the MCP server can save updated attendee lists to conference-2025. However, it cannot access unrelated files outside these roots, such as personal documents or other projects. This limitation is intentional, as the server can access only the files and folders explicitly exposed through the configured roots, not unrelated content such as personal documents or other projects. When users switch projects or folders, clients can update the roots list and notify the server, ensuring access remains tightly scoped.

Elicitation

MCP elicitation allows a server to request missing information during an active session instead of assuming it in advance. In practice, the server detects that required context is missing or ambiguous and sends an elicitation request. The client then asks the user for the needed information, allows them to review, edit, reject, or cancel the request, and returns the response to the server. The server can then continue processing with the newly supplied context.

MCP supports two elicitation modes: form mode and URL mode.

For example, in **form mode** , a travel-booking server can ask the client to collect structured fields such as passport number and date of birth before completing a reservation. In **URL mode** , a payroll server can direct the user to a secure external HR portal to enter sensitive tax or banking details that should not pass through the client. In both cases, the client remains in control of the interaction, while the server receives only the information or completion signal needed to continue safely.

Before we start building MCP clients and servers, let's first summarize what we've covered so far:

MCP Component	Purpose	What It Handles
Base Protocol	Defines the core communication structure	JSON-RPC message types used for communication between MCP clients and servers
Transport	Provides the communication channel between client and server	Stdio and Streamable HTTP
Lifecycle Management	Manages the connection between client and server	Connection initialization, capability negotiation, and session control
Authorization	Ensures secure communication	Authentication and authorization framework for HTTP-based transports
Server Features	Capabilities exposed by MCP servers	Resources, prompts, and tools that clients can access
Client Features	Capabilities provided by MCP clients	Sampling (running model completions) and root directory lists

Now that you have seen the key MCP concepts such as transport, lifecycle management, authorization, server features, client features, and elicitation, the next step is to understand how these ideas are applied in practice through MCP servers.

This section shows you the two main ways to work with MCP servers: using pre-built servers that are already available and building your own server for custom use cases.

Building and using MCP servers

This section explains how to work with MCP servers in practice, either by using existing servers or by building your own for custom integrations.

You can either build your own MCP server or use an existing MCP server provided by Anthropic, the open-source community, or AWS through the **Official MCP Servers for AWS** repository.

Using pre-built MCP servers

You have options to directly connect to an MCP server, for example, GitHub MCP Server (`https://github.com/github/github-mcp-server`) or Slack MCP Server (`https://github.com/korotovsky/slack-mcp-server`), to your existing MCP host such as Claude code IDE, VS Code or Kiro. These servers expose functionality as MCP tools that any MCP-compatible agent can use.

To make this more concrete, consider the AWS Documentation MCP Server. This server exposes tools for searching and retrieving information from official AWS documentation. In the following example, the code launches the MCP server as a local process using `uvx` , creates an MCP client that communicates with it over standard input and output, retrieves the list of available tools from the server, and then passes those tools to a Strands agent. Once connected, the agent can answer AWS-related questions by calling the documentation tools exposed through the MCP server rather than relying only on its own built-in knowledge

```
from mcp import stdio_client, StdioServerParameters
from strands import Agent
from strands.tools.mcp import MCPClient
# Create MCP client
mcp_client = MCPClient(lambda: stdio_client(
    StdioServerParameters(
        command="uvx",
        args=["awslabs.aws-documentation-mcp-server@latest"]
    )
))
# Use with agent
with mcp_client:
    tools = mcp_client.list_tools_sync()
    agent = Agent(tools=tools)
    response = agent("What is AWS Lambda?")
```

This pattern is powerful because you do not need to write custom integration logic for AWS documentation yourself. The MCP server packages that capability as reusable tools, and the agent simply discovers and uses them through the MCP interface.

Building your own or custom MCP Server

You can build custom MCP servers to expose your organization's tools and data sources. Building an MCP server involves using an SDK to define **tools**

(actions the AI can perform) and **resources** (data the AI can read), and then running the server with a chosen **transport** method.

> NOTE
>
> The SDK used to build MCP servers officially supports Python and TypeScript, making them the recommended languages for development.

FastMCP versus FastAPI

There are two ways to build MCP servers in Python: **FastMCP** (`https://gofastmcp.com/getting-started/welcome`) and **FastAPI** (`https://fastapi.tiangolo.com/`). Although both can be used in MCP-based systems, they serve different purposes and operate at different levels of abstraction.

FastMCP is designed specifically for MCP. It provides MCP-native abstractions for defining tools, resources, and prompts, so it is usually the simplest option when your main goal is to build an MCP server quickly. Developers should choose FastMCP when they want a straightforward, MCP-focused development experience with less boilerplate and when the service is primarily intended to expose MCP capabilities.

FastAPI , by contrast, is a general-purpose web framework. It is a better fit when MCP functionality needs to be part of a broader application that also exposes REST APIs, custom authentication flows, health checks, dashboards, or other web service endpoints. Developers should choose FastAPI when they need more flexibility and tighter integration with a larger service architecture, even if that means writing more of the MCP integration logic themselves.

In short, FastMCP is usually the better choice for building a dedicated MCP server quickly, while FastAPI is better when MCP is only one part of a larger web service.

Using MCP with Agent Frameworks

Frameworks such as **Strands** and **LangChain** can act as MCP clients or adapters, allowing agents to discover and invoke tools exposed by MCP servers. Their role is not to define the MCP server itself, but to help agents

consume MCP-compatible capabilities as part of a larger reasoning and orchestration workflow.

Strands Orchestrator

Strands agent includes Anthropic's MCP Python library, which includes the FastMCP library for MCP implementation using Strands orchestrator. You can use Strands for creating the autonomous agent, managing tool discovery, and connecting to AWS services (like SageMaker), and FastMCP to create the underlying, fast-responding tools that the agent calls.

LangChain

LangChain has native support for the MCP through specialized libraries, allowing agents and chains to connect with Anthropic's MCP servers (https://modelcontextprotocol.io/). Using `langchain-mcp-adapters` , developers can convert MCP tools into standard LangChain tools, enabling seamless access to external data and APIs.

Hands-on example of building an MCP server and connecting to an MCP client

In this use case, we are going to build a weather assistant powered by an MCP server using FastMCP. *Figure 5.13* illustrates this use case in detail:

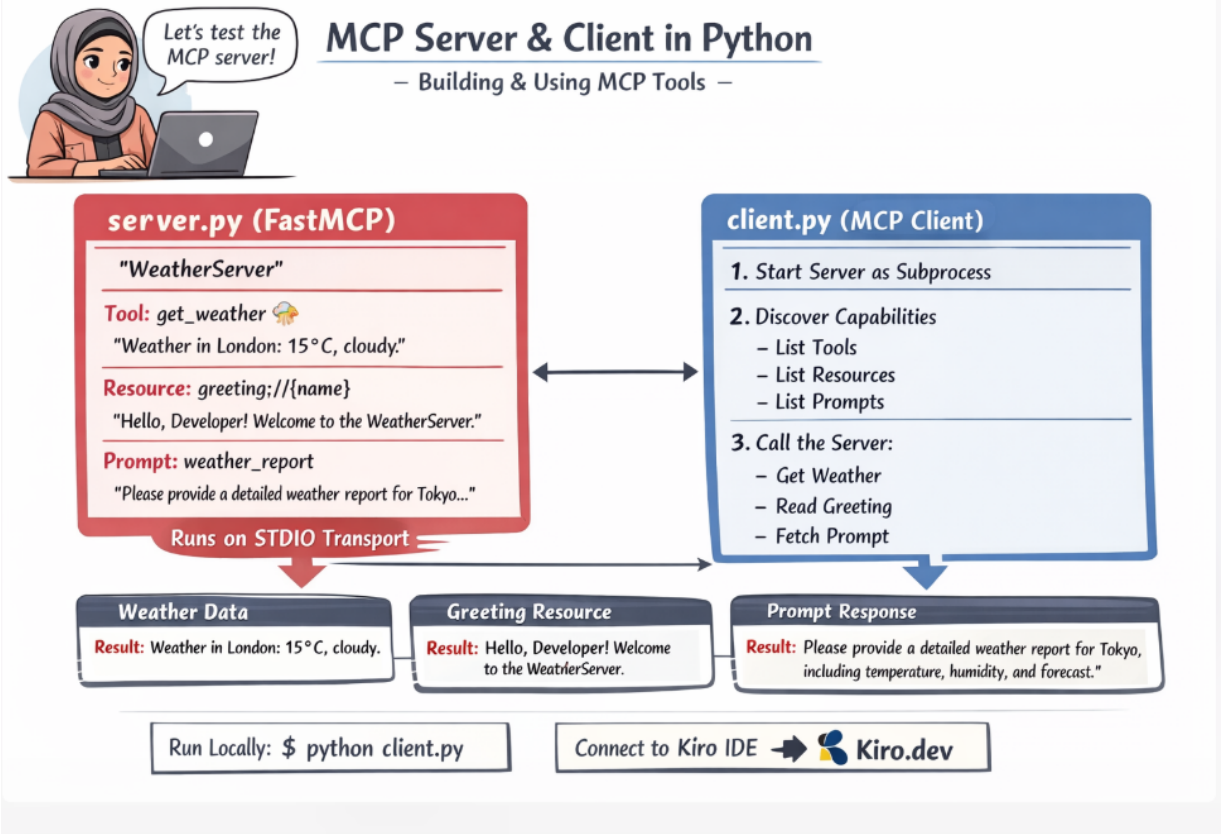


Figure 5.13: Long-term memory

To get started, open your IDE or terminal. We recommend using Visual Studio Code, Claude Code or Kiro IDE.

Next, clone the GitHub repo, get the code from:
`https://github.com/PacktPublishing/Agentic-with-AWS/tree/main/chapter%205.`

Go to `mcpexample` folder and you will see the following files:

- `server.py` uses FastMCP to expose three MCP primitives: a get_weather tool that returns fake weather data, a greeting://{name} resource for personalized greetings, and a `weather_report` prompt template. It runs over stdio transport.
- `client.py` spawns the server as a subprocess, performs the MCP handshake, discovers all available capabilities, then calls each one: invokes the tool, reads the resource, and fetches the prompt.

Let's understand the code before we run these two files.

Server.py code

First of all, we import libraries:

```
from mcp.server.fastmcp import FastMCP
```

FastMCP is the high-level API from the MCP Python SDK. It handles all the protocol details for you, such as JSON-RPC messaging, capability negotiation, and schema generation.

Step 1 : Create the server

```
mcp = FastMCP("WeatherServer")
```

Step 2 : Add the Tool: get_weather

The following code shows how to add a simple MCP tool named get_weather using the `@ mcp.tool () decorator .`

```
@mcp.tool()
def get_weather(city: str) -> str:
    """Get the current weather for a given city."""
    # Fake data for demo purposes
    fake_weather = {
        "london": "15°C, cloudy",
        "paris": "18°C, sunny",
        "tokyo": "22°C, humid",
        "new york": "12°C, windy",
    }
    result = fake_weather.get(city.lower())
    if result:
        return f"Weather in {city}: {result}"
    return f"Sorry, no weather data available for '{city}'."
```

Step 3 : Next, define the Resource.

Resources are read-only data endpoints like GET requests in a REST API. The URI template `greeting://{name}` means a client can read `greeting://Alice` and the `{name}` parameter gets passed as `Alice` to the function.

```
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Return a personalized greeting."""
    return f"Hello, {name}! Welcome to the WeatherServer."
```

Step 4 : Define prompt (weather_report)

Prompts are reusable prompt templates that clients can fetch. A client calls `get_prompt (" weather_report ", {"city": "Tokyo"})` and gets back the formatted string. This is useful for standardizing how an LLM is requested to do something.

```
@mcp.prompt()
def weather_report(city: str) -> str:
    """Generate a prompt asking for a weather report."""
    return f"Please provide a detailed weather report for {city}, including temperature, humidity,
and forecast."
```

Step 5 : Starting the MCP Server

This starts the server listening on stdin/stdout.

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Now we have a server running. This server can connect to either an IDE such as Claude Code or Kiro or by building an MCP Client. Let's learn how to build a simple MCP Client.

Client.py code

In this section, we will be building the MCP Client. In the client.py file, you need to import the essential libraries:

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

To briefly describe:

- ClientSession is the high-level MCP client that speaks the protocol for you
- StdioServerParameters is the config object that tells the client how to spawn the server
- stdio_client is the transport layer that manages the subprocess and its stdin/stdout pipes

Now let's connect it to the server. The following code does not start anything yet. It just simply implies that when you connect, run python `server.py` as a subprocess. The client will interact with it through stdin/stdout.

```
server_params = StdioServerParameters(
    command="python",
    args=["server.py"],
)
```

The following code asynchronizes with blocks to ensure everything gets cleaned up properly (streams closed, process terminated) when you're done:

```
async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        # Initialize the connection
        await session.initialize()
```

In the preceding code, three things happen:

- `stdio_client` spawns the server process and gives you raw read/write streams
- ClientSession wraps those streams with the MCP protocol (JSON-RPC)
- `initialize()` does the MCP handshake: the client and server exchange versions and capabilities

Tool, Resources, and Prompt discovery: The following functions call `await.session . list.tools ()` , `await.session .list_resource_ template ( )` , and `await session.list _ prompts ( )` ask the server `what do you offer? .` The server responds with names, descriptions, and schemas for each:

```
# ---- List available tools ----
tools = await session.list_tools()
for tool in tools.tools:
    print(f"  - {tool.name}: {tool.description}")
# ---- List available resources ----
resources = await session.list_resource_templates()
# ---- List available prompts ----
prompts = await session.list_prompts()
```

Calling a tool

The code block sends a `tools/call JSON-RPC` request to the server with the tool name and arguments. The server runs the `get_weather` function and sends back the result. `result.content` is a list of content blocks:

```
# ---- Call the get_weather tool ----
result = await session.call_tool("get_weather", {"city": "London"})
```

```
print(f"  Result: {result.content[0].text}")
# Reading a resource
```

The code block sends a `resources/read` request with the URI. The server matches it against the `greeting://{name}` template, extracts `Developer` as the name parameter, runs the function, and returns the result:

```
# ---- Read a resource ----
print("\n=== Reading greeting resource ===")
resource = await session.read_resource("greeting://Developer")
print(f" Result: {resource.contents[0].text}")
```

Getting a prompt

The code fetches the prompt template with the given arguments. The server fills in the template and returns it as a list of messages. In a real app, you'd feed these messages to an LLM:

```
 # ---- Get a prompt ----
prompt = await session.get_prompt("weather_report", {"city": "Tokyo"})
print(f"  Result: {prompt.messages[0].content.text}")
```

Running it

The final code below uses Python's standard async entry point. `asyncio.run(main())` starts an event loop, executes the asynchronous `main()` function, and automatically closes the loop when execution finishes.

```
if __name__ == "__main__":
    asyncio.run(main())
```

Let's briefly recap what we have covered so far. In this section, you learned how MCP standardizes communication between clients and servers through a common protocol, supported by transports, lifecycle management, authorization, and server and client features. You also saw how capabilities such as sampling, roots, and elicitation make MCP more powerful by enabling model-assisted workflows, scoped access to data, and dynamic collection of missing information. Finally, through both pre-built and custom server examples, you saw how MCP works in practice as a reusable integration layer for exposing tools, resources, and prompts to AI applications. You also saw how MCP works in practice via a hands-on weather assistant demo using

FastMCP. With this foundation in place, we can now look at A2A, which focuses on communication and coordination between agents themselves.

Agent2Agent protocol

We studied A2A in *Chapter 1 , Understanding AI Agents on AWS* , where we looked into an example of a travel agent trying to coordinate between multiple agents such as a flight agent, hotel agent and others to plan a trip.

A2A serves as a standardized bridge that allows diverse systems such as those built on LangGraph, CrewAI, or Google ADK and AWS Strands SDK to communicate, discover one another, and manage shared workflows across different organizations.

Originally introduced by Google in 2025 and merged with IBM's Agent Communication Protocol (ACP), it is now a vendor-neutral project governed by the Linux Foundation. A2A is dedicated to peer-to-peer agent interaction, allowing **opaque** agents to delegate complex tasks and handle open-ended problem-solving while keeping their internal implementation details private, and the MCP focuses on connecting agents to specific tools and functions.

Understanding the difference between MCP vs A2A

MCP is an emerging standard focused on connecting LLMs with data and external resources. The focus of MCP is to reduce the complexity involved in connecting agents with tools and data. Tools are typically stateless and perform specific, predefined functions (for example, a calculator or a database query). However, this distinction raises an important question: can we encapsulate agent as tools via MCP?

The practice of encapsulating an agent as a simple tool is fundamentally limiting, as it fails to capture the agent's full capabilities. Tools are for executing actions, while agents are for solving problems. Treating agents as tools oversimplifies their capabilities and breaks their ability to handle complex, iterative, and collaborative workflows.

The **Agent2Agent** (**A2A**) protocol is designed to standardize communication between AI agents, particularly those deployed in external systems. A2A is positioned to complement MCP, addressing a distinct yet related aspect of

agent interaction. The focus of A2A is enabling agents to collaborate within their native modalities, allowing them to communicate as agents (or as users) rather than being constrained to tool-like interactions. This enables complex, multi-turn interactions where agents reason, plan, and delegate tasks to other agents. Let's take an example.

The following diagram shows how two healthcare agents collaborate using A2A, while each agent independently uses MCP to interact with its tools and systems:

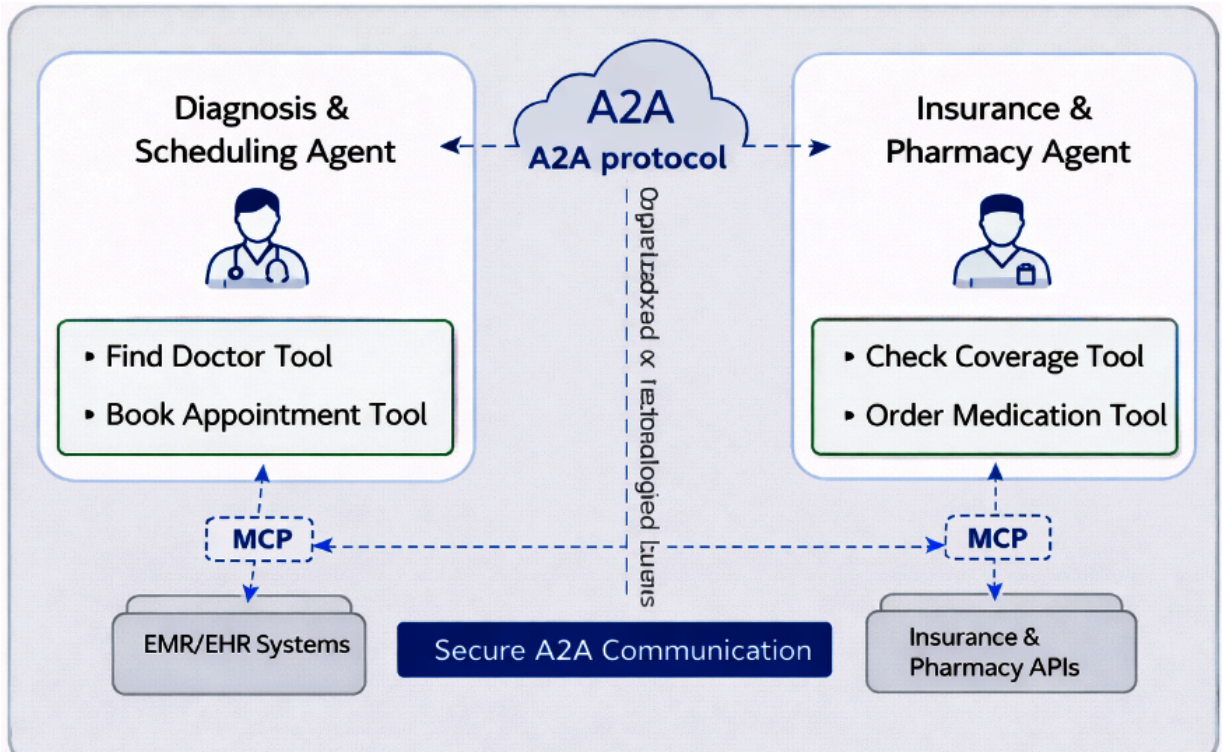


Figure 5.14: Two healthcare agents collaborate using A2A

In the preceding figure:

- Patient request is handled by the **Diagnosis Agent**
- It communicates with the **Insurance Agent via A2A**
- Each agent performs actions using **its own MCP-connected tools**
- Information flows back and forth via A2A until the task is complete

Having established why agents cannot be reduced to tools and how A2A enables true inter-agent collaboration, we now move from concept to structure. In the next section, we explore the A2A architecture in detail, how

agents are discovered, how they communicate through standardized messages, and how tasks, artifacts, and agent cards enable scalable, interoperable multi-agent systems.

Why is a standardized protocol needed for agents?

As agents become more capable, they also become more specialized and more likely to work as part of a larger multi-agent system rather than in isolation. However, today's agent ecosystem is fragmented across different frameworks, vendors, and implementation styles, making collaboration difficult. A2A is needed to provide a common protocol that allows agents to communicate, coordinate, and work across organizational and technical boundaries without requiring them to share their internal implementation details. Some of the reasons why A2A is needed for multi-agent collaboration are shown in the following diagram:

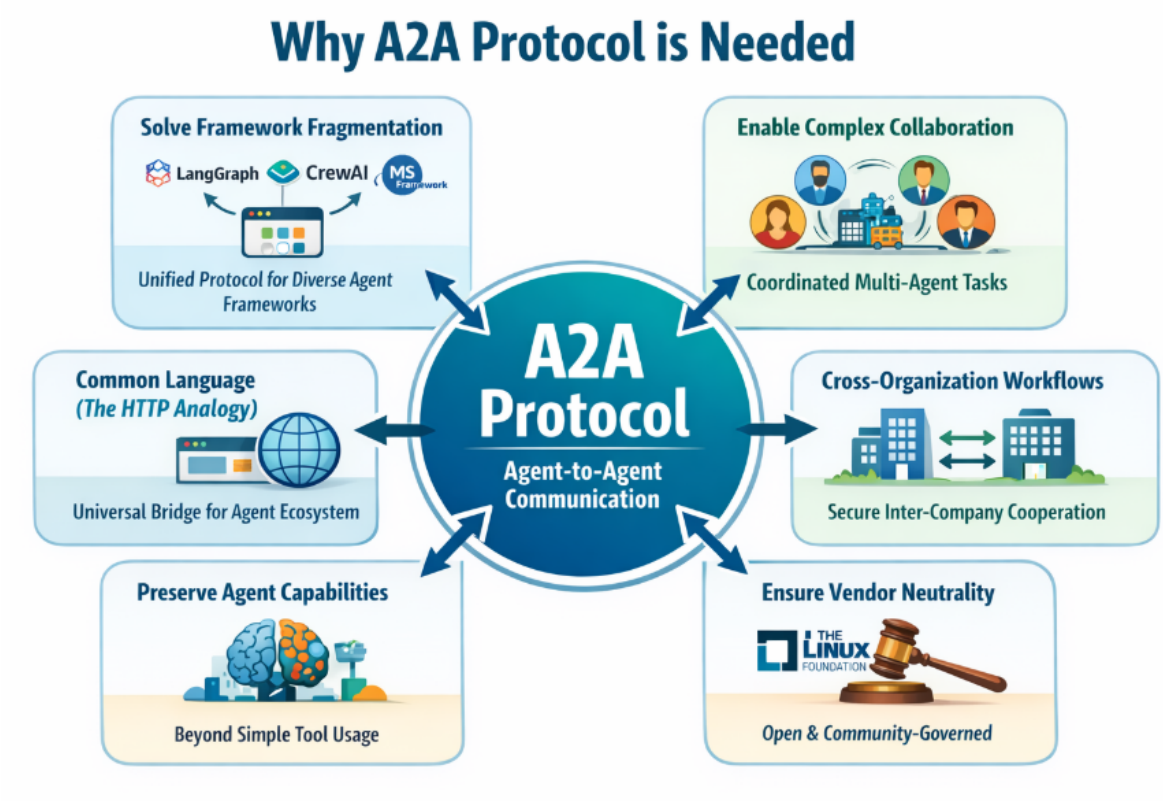


Figure 5.15: Why A2A is needed

Let's expand on the details shown in the preceding diagram:

- **Solve f ramework f ragmentation:** Currently, there are numerous frameworks for building agents (such as LangGraph, CrewAI, and Microsoft's framework), but they are not compatible with each other out of the box. A2A provides a shared protocol so agents can communicate regardless of the specific model or language they use.
- **Enable c omplex c ollaboration :** As AI evolves from simple chatbots to "agentic systems," complex tasks (like planning a full trip with flights, hotels, and activities) require multiple specialized agents to work together. A2A provides the standardized structure and methods needed for this coordination.
- **Act as a c ommon l anguage (t he HTTP a nalogy):** Just as the World Wide Web needed HTTP to allow data transfer across different systems, the agent landscape needs A2A to serve as a universal bridge, allowing the ecosystem to grow beyond siloed applications.
- **To f acilitate c ross- o rganization w orkflows :** A2A allows agents to collaborate across different teams and even different organizations. It allows these agents to remain opaque, meaning they can work together without needing to expose their internal implementation details or private code.
- **To p reserve a gent c apabilities :** While protocols like MCP are designed to connect models and agents to tools, an agent should not always be reduced to a simple tool invocation. Many tool calls can themselves involve reasoning and non-deterministic behavior, but agents typically operate at a broader level: handling ambiguity, managing multi-step workflows, coordinating with other agents, and pursuing open-ended goals. A2A is needed to support that richer form of interaction.
- **To e nsure v endor n eutrality :** Being hosted by the Linux Foundation and governed by a technical steering committee, A2A prevents a single company from controlling how agents interact, ensuring the protocol evolves based on the needs of the entire community.

With the motivation for A2A now established, the next step is to see how these ideas apply in practice. A concrete multi-agent example makes it easier to understand how specialized agents communicate, coordinate, and complete a shared business task.

Understanding A2A by an example of a multi-agent system

Let's understand A2A by an example. *Figure 5.1 6* represents a **complex, real-world business workflow** where a marketing manager asks an AI system: `Create and execute a launch plan for our new eco-friendly smartphone in the European market .`

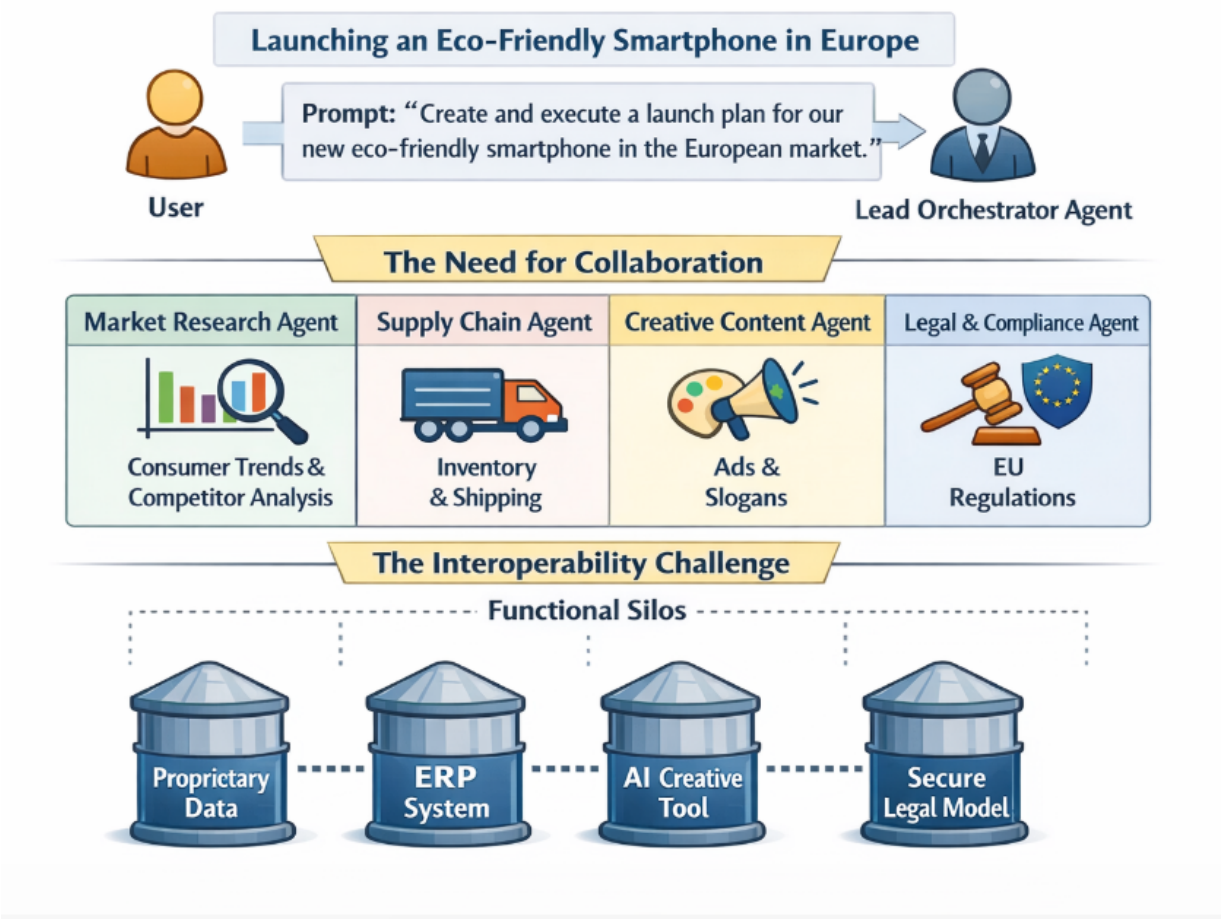


Figure 5.16: Long-term memory

This is not a simple task; it requires multiple specialized capabilities. Since we learned multi-agent architecture in Chapter 4 , we have a lead orchestrator agent working with multiple sub-agents.

A **Lead Orchestrator Agent** breaks it down and collaborates with:

- Market Research Agent that analyzes trends and competitors
- Supply Chain Agent that manages inventory and logistics

- Creative Agent that generates ads and content
- Legal Agent that ensures GDPR and regulatory compliance

In short, one high-level request invokes multiple expert agents working together.

The core problem (without A2A) is that the agents are unable to work together because each has its own bespoke development and deployment. The consequence of a lack of a standardized protocol is that these agents cannot collaborate with each other let alone discover what they can do. The individual agents are isolated. Each agent is built differently, using different data formats, operating on different systems such as ERP platforms, proprietary APIs, or AI tools, and in some cases running as closed or secure systems.

So, how does A2A solve this challenge? The A2A protocol acts as a common communication layer that standardizes how agents interact. It allows the Lead Orchestrator to discover agents based on their capabilities, enables agents to exchange information through a shared format, and supports automated verification workflows where one agent can review or approve another agent's output.

To address a possible point of confusion between swarm pattern and A2A, the Swarm pattern from *Chapter 4* , *Advanced Agent Architecture Patterns* , describes how agents can be orchestrated within a single multi-agent system, whereas A2A addresses a different problem: standardized communication between independently built agents across frameworks, platforms, and organizational boundaries.

As a result, the agents can collaborate seamlessly without needing to know each other's internal implementations. The orchestrator then combines all the outputs into a unified launch playbook. This transforms a process that would normally require extensive human coordination and time into a fast, automated, and efficient workflow.

How A2A architecture works

A key technical feature of how A2A works is that agents remain **opaque** . The protocol only standardizes the *bridge* between them. Agent A does not need to know Agent B's internal code, what prompts it uses, or what database it

queries; it only needs to know how to format the A2A request and interpret the A2A response. *Figure 5.17* shows how two agents communicate using the A2A protocol, highlighting the roles of a client agent and a server (remote) agent:

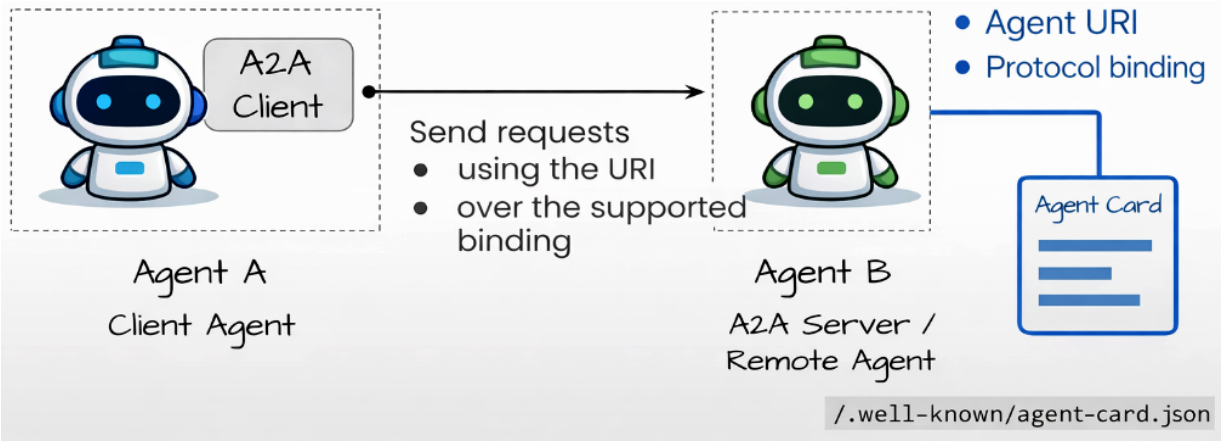


Figure 5.17: Long term memory

Let's study in detail what these components represent:

Agent A : This is the Client Agent that starts the conversation and makes a request for work to be done. Usually, the Client Agent is the front-facing agent that interacts directly with the end-user. It is responsible for discovering other agents. It looks for the **Agent Card** of other agents to see what they are capable of.

Agent B: This is also referred to as the **Remote Agent** or **Server Agent** . The Remote Agent is the one that receives the request and performs the specific task. It acts as the *s erver* because it hosts its capabilities at a specific URI and publishes its Agent Card (at well-known/agent-card.json) so others can find it. It decides how to fulfill the request. If a task is simple, it acts like a standard server and returns a message immediately. If the task is complex, it creates a **Task** and manages the lifecycle of that job until it produces an **Artifact** (or the result). Below is an example of how an agent card looks for Weather Agent or Agent B in this case:

```
{
  "name": "Weather Agent",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false
```

```
  },
  "skills": [
    {
      "id": "get_weather",
      "name": "Get Weather",
      "description": "Returns current weather conditions for a given city."
    }
  ],
  "url": "https://api.example.com/weather-agent/"
}
```

Agent Card can include both **capabilities** and **skills** : capabilities tell other agents how this agent communicates, while skills tell them what work it can perform.

> **NOTE**
>
> A key feature of the Server/Remote agent is that it is **opaque** . It doesn't have to show the Client Agent how it works internally (what code it uses or what models it runs); it only needs to provide the output defined by the protocol.

Next, let's explore how communication happens:

Discovery via Agent Cards : Agents identify each other using an **Agent Card** (a JSON file hosted at a well-known URL). Similar to a Swagger/OpenAPI doc, this card acts as a digital business card, detailing the agent's name, capabilities, supported protocols, media types, and authentication requirements. It contains:

- Agent's URI (endpoint)
- Supported protocol bindings (e.g., HTTP)

Protocol bindings: Protocol bindings are defined in an agent's Agent Card (the well-known/agent-card.json file). This tells the Client Agent exactly which language or technical format it needs to use to interact with that specific Remote Agent. You can think of it like this: while A2A provides the language, Protocol binding is the physical connection used to send that language from one agent to another.

Imagine you have a letter written in English (the common language). A2A supports three ways of transport or protocol bindings:

- **HTTP Binding:** Like sending the letter through Standard Mail. It's the most common and easy to use.
- **gRPC Binding :** Like sending the letter via a **High-Speed Courier** . It's faster and more efficient for heavy data.
- **JSON-RPC Binding :** Like sending the letter via a **Telegram** . It follows a very specific set of remote-control rules.

NOTE

If one agent is *only* using a gRPC pipe and the other is *only* using an HTTP pipe, they actually **cannot** talk directly. They need to agree on which protocol binding to use which can be defined in their agent cards.

Understanding A2A data model

In A2A, the client agent sends a message composed of one or more parts. The remote agent may create a task to track execution, and that task produces artifacts containing the output parts. The flow is shown in *Figure 5.18* :

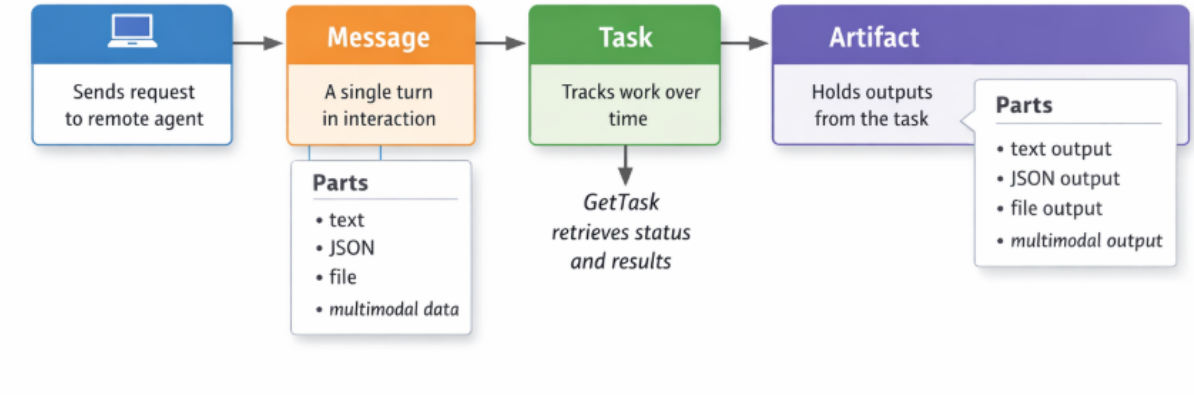


Figure 5.18: The A2A data model workflow

The client uses **GetTask** to check task status and retrieve results. When the remote agent begins processing that request, it may create a **Task** to track the work over time. As the task progresses or completes, it produces **Artifacts** , which are output containers that also contain one or more **Parts** . The client can call **GetTask** to retrieve the latest task status and any artifacts returned so far.

Communication styles in A2A

There are four execution modes to handle different types of tasks; A2A supports four communication styles.

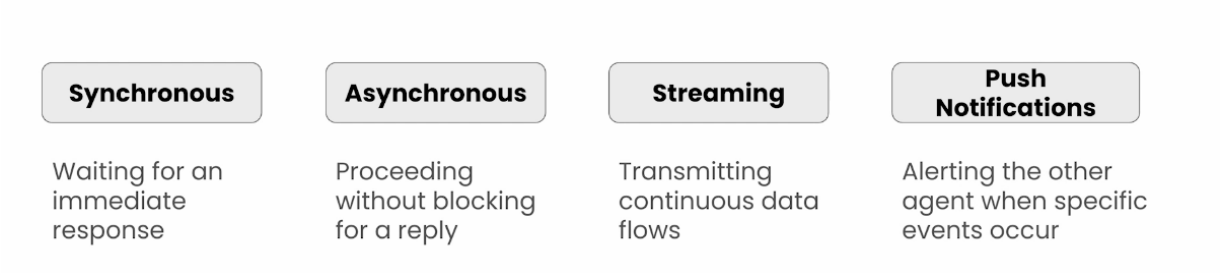


Figure 5.19: Long-term memory

Synchronous

This style is for simple, immediate responses. In A2A, **synchronous execution** works in a simple request-response manner where Agent A sends a message to Agent B using the `SendMessage` method, and Agent B immediately processes the request and returns a response. In this mode, the client agent waits for the result, making it suitable for quick, short-lived tasks.

Asynchronous

For complex jobs or long-running jobs, asynchronous communication is used when a task takes time and the client agent should not wait (block) for the result. The agent returns a **Task ID** , and the requester polls for status updates (e.g., submitted, working, completed).

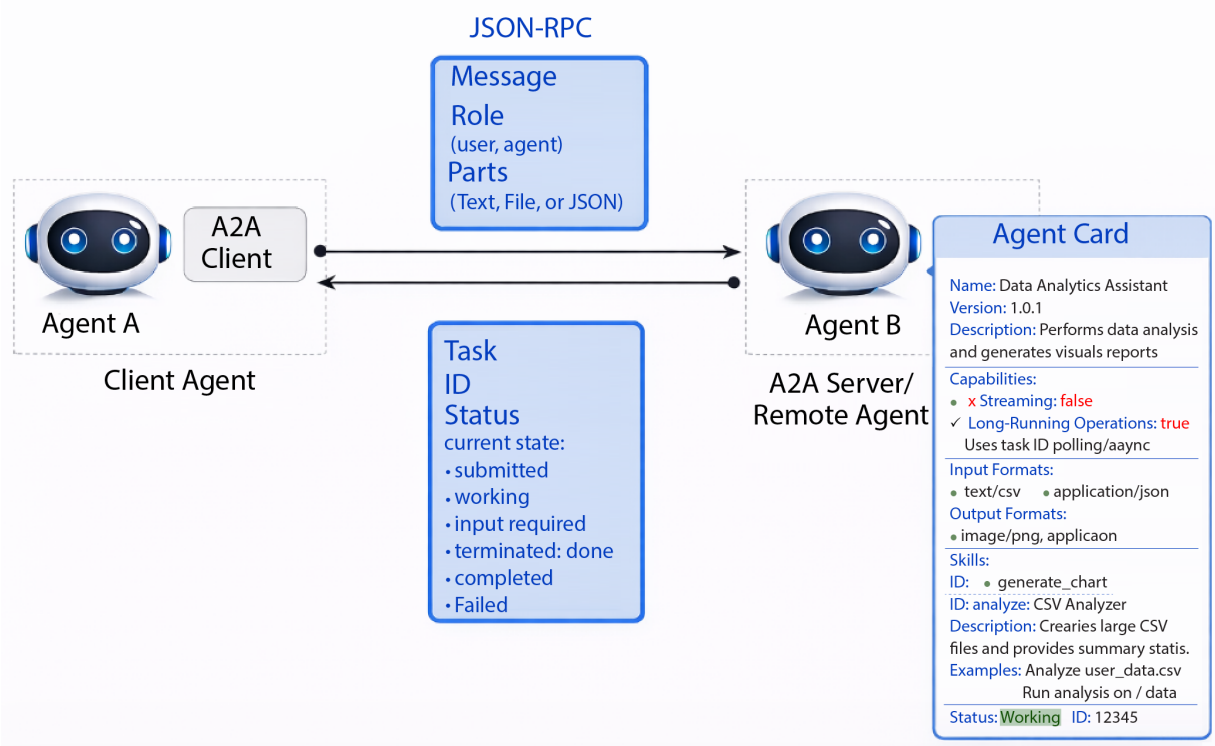


Figure 5.20: Async exchange between Agent A and Agent B via A2A protocol

In **asynchronous execution using polling** , Agent A sends a request and Agent B responds with a **Task object** instead of the final result. This task includes a task ID and a status indicating its progress. Agent A does not wait but instead periodically calls the `GetTask` method using the task ID to check the status. Once the task reaches a completed state, the final results are returned as artifacts.

Streaming

This communication style uses Server-Sent Events to provide real-time updates or *chunked* text as it is generated. This enables you to see live progress updates, as shown in *Figure 5.2 1* :

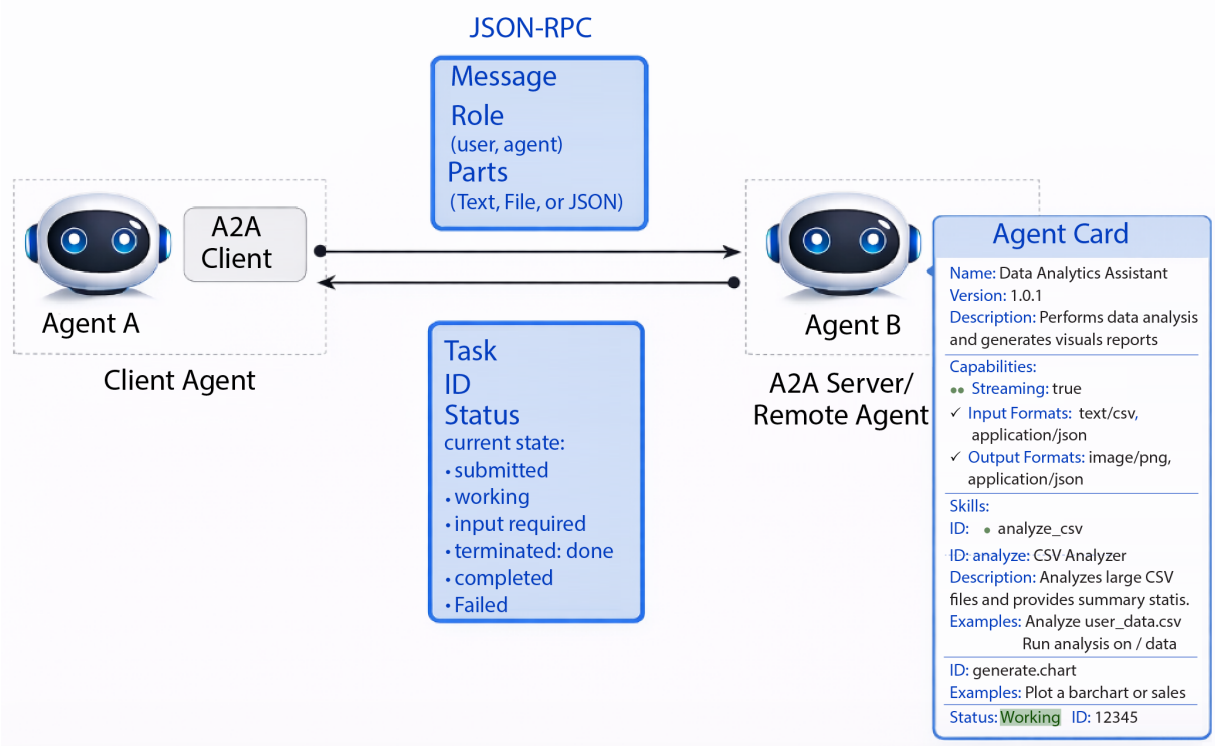


Figure 5.21: Agent A and B with streaming

In **streaming execution** , real-time updates are enabled only if the server declares support by setting `capabilities.streaming = true` **in its Agent Card** . Agent A uses the `SendStreamingMessage` **method** , and the server responds with an open HTTP connection (text/event-stream). Over this connection, Agent B continuously sends updates such as task status changes, intermediate messages, and partial results. These updates are delivered as structured events, allowing the client to receive progress and results incrementally until the task is complete.

Push notifications

You can use **push notifications in A2A** when tasks take a long time or the client cannot stay connected continuously. Instead of polling or streaming, the client provides a webhook URL, and the server notifies it when important events occur, such as task completion or when input is required. For example, in a **healthcare workflow** , a pharmacy agent can notify the system when a prescription is ready after several hours. In a **travel booking scenario** , a flights agent can send a notification when ticket booking is confirmed. This approach

is ideal for **long-running, event-driven workflows** where continuous connection is not practical. *Figure 5.2 2* gives an overview of the workflow:

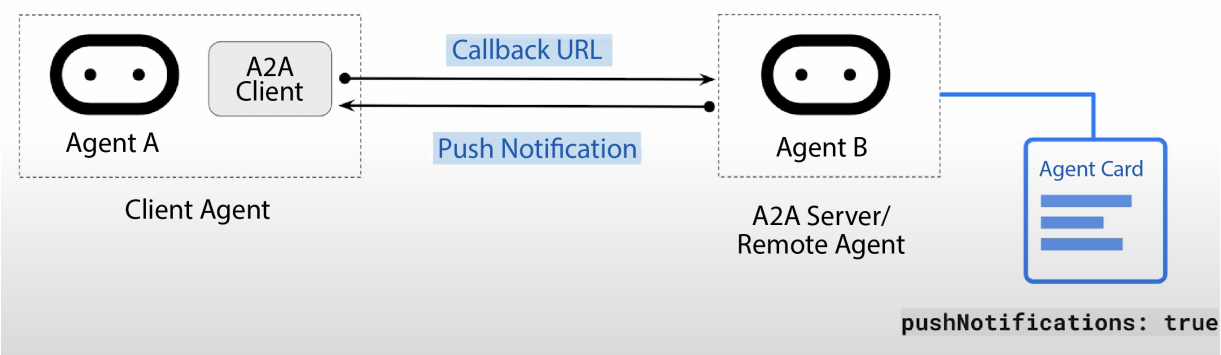


Figure 5.22: Agent A and Agent B with push notifications true

Push notifications are enabled only if the server declares `capabilities.pushNotifications = true` **in its Agent Card** . Agent A provides a `PushNotificationConfig` (including a webhook URL and optional authentication) either during the initial request or via a separate method. Agent B then sends HTTP POST notifications to this webhook whenever there are significant updates, such as task completion or input required. After receiving a notification, Agent A typically calls `GetTask` to retrieve the full task details and results.

Now that we have covered the A2A concepts, let us see how they are implemented in practice.

How to implement A2A

The easiest way to understand the implementation landscape is to think of it in three layers:

1. **Core protocol layer** – the A2A SDK
2. **Framework integrations** – libraries that sit on top of the SDK and reduce boilerplate
3. **Platform-specific implementations** – hosted or framework-specific ways to expose A2A endpoints

This hierarchy matters because all three layers solve different problems. The A2A SDK gives you the protocol primitives. Integrations such as AWS Strands build on top of those primitives to make agent-to-agent communication

easier to use inside a framework. Platform offerings such as LangSmith Agent Server expose A2A endpoints for deployed agents in a managed environment.

Core protocol layer: the A2A SDK

At the foundation is the **A2A SDK** (a2a-sdk), which is the low-level protocol library. This is the most direct way to build an A2A-compatible client or server. It provides the raw protocol building blocks, including data types such as Message, TextPart, AgentSkill, and AgentCapabilities, along with client-side utilities such as A2ACardResolver, ClientFactory, and ClientConfig. In other words, the **SDK** gives you the objects and interfaces needed to construct messages, discover remote agents through agent cards, send requests, and process responses.

If you use only the A2A SDK, you are responsible for wiring these pieces together yourself . That gives you full control, but it also means more protocol plumbing. You must explicitly define how your server publishes its capabilities, how incoming requests are routed, how tasks are created and updated, and how responses are returned to the client.

Higher-level layer: framework integrations

On top of the A2A SDK are **higher-level integrations** that make it easier to use A2A inside an agent framework. These integrations do not replace the protocol. Instead, they implement the protocol for you while preserving the framework's programming model. Some good examples are **Google ADK and AWS Strands A2A** (strands-agents[a2a]). We will cover AWS Strands A2A SDK in this chapter.

Strands provides two especially important abstractions:

- **A2AServer** : Exposes a Strands agent as an A2A-compliant server.
- **A2AAgent** : Lets you consume a remote A2A server as if it were a local Strands agent.

This means that the A2A SDK remains the protocol foundation, but Strands gives you a more convenient developer experience for building and consuming A2A-enabled agents.

Platform-specific layer: LangChain and LangSmith

A third layer consists of **platform-specific implementations** . this is represented by **LangSmith Agent Server** , which is the primary way LangChain currently exposes A2A support. In this case, the platform handles the protocol details such as JSON-RPC messaging and task management for agents deployed on that platform. Your agent still participates in A2A communication, but the hosting layer manages much of the operational behavior for you.

So far, the distinction should be clear:

- **A2A SDK** = protocol primitives
- **Strands A2A** = framework integration built on the SDK
- **LangSmith Agent Server** = managed platform endpoint for deployed agents

That separation makes the ecosystem easier to understand.

In this chapter, we will use **Strands A2A** for the implementation example because it gives us the cleanest path from an agent definition to a working A2A server on AWS.

Building a multi-agent system with A2A and MCP

We are going to build a travel agent which will be a multi-agent system communicating via the A2A protocol to a weather agent which we have built previously and has an MCP tool connected to get weather data and a flight agent to search and book flights. *Figure 5.2 3* provides the architecture of the agent we are going to build:

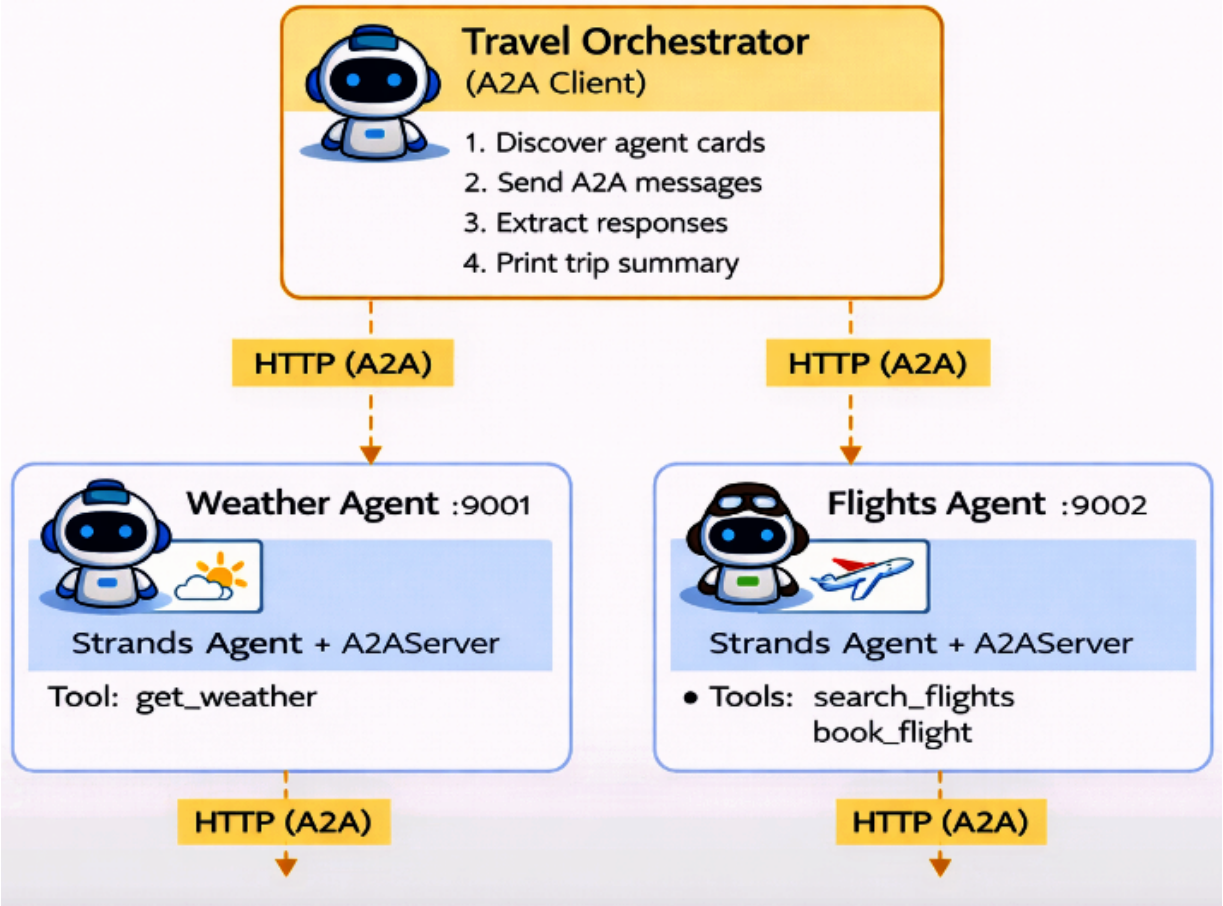


Figure 5.23: Long-term memory

Open your IDE, VS Code, Kiro IDE or Claude Code and clone the GitHub repo: `https://github.com/PacktPublishing/Agentic-with-AWS/tree/main/chapter%205`

Next, go to the `a2aexample/` folder, where you will find the following code files:

- `weather_agent_server.py` : A2A Server or weather lookup agent on port 9001
- `flights_agent_server.py` : A2A Server or flight search & booking agent on port 9002
- `travel_orchestrator.py` : A2A Client that calls both agents and prints a trip summary

Let's understand the code before we start running it in the IDE or terminal.

Weather Agent Server

The Weather Agent Server is implemented in the `weather_agent_server.py` file. The following code defines skills that the agent possesses:

```python
from a2a.types import AgentSkill

# Define skills that describe what this agent can do
WEATHER_SKILLS = [
    AgentSkill(
        id="get_weather",
        name="Get Weather",
        description="Returns current weather conditions for a given city...",
        tags=["weather", "temperature", "forecast", "travel"],
        examples=[
            "What is the weather in London?",
            "How hot is it in Dubai right now?",
        ],
    ),
]
```

The `AgentSkill` library defines each capability with an ID, name, description, tags, and example prompts, and these skills are passed to the `A2AServer` using `skills=WEATHER_SKILLS` so they appear in the agent card, which serves as the A2A discovery mechanism that clients fetch first to understand what the agent can do.

Next, create the Strands Agent with the tool:

```python
weather_agent = Agent(name="Weather Agent", tools=[get_weather], callback_handler=None)
```

When you don't specify one, Strands defaults to AWS Bedrock with Claude (Anthropic); specifically, it uses the `BedrockModel` provider and picks `us.anthropic.claude-sonnet-4- 20250514` (Claude Sonnet). Then, it creates a Strands Agent with `tools` called `get_weather` that looks up weather from a dictionary. The agent is wrapped in `A2AServer` and served on port `9001` .

Once you have created the Strands Agent, serve it as an A2A server with explicit skills in the agent card:

```python
if __name__ == "__main__":
    server = A2AServer(
        agent=weather_agent,
        host="127.0.0.1",
        port=9001,
        skills=WEATHER_SKILLS,
        version="1.0.0",
        enable_a2a_compliant_streaming=True,
    )
```

```
    print("Weather Agent A2A server starting on http://127.0.0.1:9001")
    server.serve()
```

The preceding code block turns your Strands agent into a standalone HTTP microservice that speaks the A2A protocol. A2AServer() wraps the Strands agent into an A2A-compliant HTTP server, where `agent= weather_agent` provides the core Strands agent logic, `host="127.0.0.1"` binds the server to localhost, `port=9001` sets the listening port, `skills=WEATHER_SKILLS` publishes the agent's capabilities in the agent card, and `version="1.0.0"` includes the server version in that card. Setting `enable_a2a_compliant_streaming=True` ensures that when a client sends a streaming request, the server returns proper A2A-compliant SSE events with artifact updates according to the specification.

The agent card served at `http://127.0.0.1:9001/.well-known/agent-card.json` looks as follows:

```
```json
{
"name": "Weather Agent",
"description": "Provides current weather information for cities around the world.",
"version": "1.0.0",
"protocolVersion": "0.3.0",
"capabilities": { "streaming": true },
"skills": [
{
"id": "get_weather",
"name": "Get Weather",
"description": "Returns current weather conditions...",
"tags": ["weather", "temperature", "forecast", "travel"],
"examples": ["What is the weather in London?", "..."]
}
],
"defaultInputModes": ["text"],
"defaultOutputModes": ["text"],
"url": "http://127.0.0.1:9001/"
}
```
```

Now that we have the Weather Agent Server up and running, we can focus on the Flights Agent Server.

Flights Agent Server

The Flights Agent Server is hosted in the `flights_agent_server.py` file. This also has the same pattern as the weather agent, but in addition, it has two tools and two skills. Also, it differs with the port number, as it runs on `port 9002` :

```
FLIGHTS_SKILLS = [
    AgentSkill(
        id="search_flights",
        name="Search Flights",
        description="Searches for available flights between two cities...",
        tags=["flights", "search", "travel", "airlines", "booking"],
        examples=["Find flights from New York to London"],
    ),
    AgentSkill(
        id="book_flight",
        name="Book Flight",
        description="Books a specific flight for a passenger...",
        tags=["booking", "reservation", "flights", "travel"],
        examples=["Book flight AE303 for John Smith"],
    ),
]
flights_agent = Agent(
    name="Flights Agent",
    tools=[search_flights, book_flight],
    callback_handler=None
)
server = A2AServer(
    agent=flights_agent,
    host="127.0.0.1",
    port=9002,
    skills=FLIGHTS_SKILLS,
    version="1.0.0"
)
server.serve()
```

Travel Orchestrator

The Travel Orchestrator is implemented in the `travel_orchestrator.py` file. This is the A2A client and it does not use an LLM as it simply sends A2A messages to the two agent servers and prints the results. Here's the complete workflow of this agent:

Step 1: Build an A2A message

```
def make_message(text: str) -> Message:
    return Message(
```

```
        kind="message",
        role=Role.user,
        parts=[Part(TextPart(kind="text", text=text))],
        message_id=uuid4().hex,
    )
```

This creates a standard A2A `Message` object with a text part, just like a user typing in a chat.

Step 2 : Discover the agent and send the message.

```
async def send_a2a_message(base_url: str, text: str) -> str:
    async with httpx.AsyncClient(timeout=TIMEOUT) as hc:
```

This further involves multiple steps:

a. Fetch the agent card from `/.well-known/agent- card.json` file:

```
resolver = A2ACardResolver(httpx_client=hc, base_url=base_url)
card = await resolver.get_agent_card()
```

b. Create a non-streaming A2A client from the card:

```
config = ClientConfig(httpx_client=hc, streaming=False)
client = ClientFactory(config).create(card)
```

c. Send the message and iterate over response events:

```
msg = make_message(text)
async for event in client.send_message(msg):
```

`A2ACardResolver` fetches the agent card, which is a JSON document describing the agent's capabilities; `ClientFactory` then creates an A2A client configured specifically for that agent; and `send_message` sends a JSON-RPC message/send request and yields response events.

Step 3 : Extract text from the response

The A2A server returns a task object containing artifacts (or the final output). The orchestrator extracts the text:

```
def extract_text(result) -> str:
    if hasattr(result, "artifacts") and result.artifacts:
        texts = []
        for artifact in result.artifacts:
            for part in artifact.parts:
```

```
                texts.append(part.root.text)
        return "".join(texts)
```

Step 4 : Print the trip summary

```
async def main(city, origin, destination):
    weather = await send_a2a_message(WEATHER_URL, f"What is the weather in {city}?")
    flights = await send_a2a_message(FLIGHTS_URL, f"Search flights from {origin} to {destination}")
    print(f"Weather: {weather}")
    print(f"Flights: {flights}")
```

The A2A protocol flow between client and server is as follows:

- The client fetches the agent card to discover what the agent can do
- The client sends a message/sends a JSON-RPC request with the user's question
- The server's Strands Agent processes the message, calls its tools, and returns a task with artifacts containing the final text response

Finally, let's complete the setup and get this running.

So, first of all, open the IDE or copy the code and run it in your terminal:

```
pip install -r requirements.txt
```

You need a Strands-compatible model provider configured (for example, AWS Bedrock credentials). Open three terminals, and run the agents separately in individual terminals, as follows:

```
# Terminal 1
python weather_agent_server.py
# Terminal 2
python flights_agent_server.py
# Terminal 3 — default demo (New York → London)
python travel_orchestrator.py
```

Or specify city, origin, and destination; your execution results will be in the client or travel orchestrator terminal.

```
python travel_orchestrator.py "Rome" "Paris" "Rome"
```

We hope you are now confident about building an A2A agent with Strands A2A SDK using a multi-agent architecture.

Summary

In this chapter, you learned how standardized communication protocols make agentic systems easier to build, extend, and scale. You first explored MCP and saw how it solves the growing complexity of connecting agents to many tools and data sources. Instead of building separate integrations for every model and every external system, MCP introduces a shared interface through clients and servers, making tool discovery, resource access, and prompt reuse more consistent and maintainable. You also learned the main MCP concepts, including hosts, clients, servers, transports, lifecycle management, authorization, and server capabilities such as tools, resources, and prompt templates.

You then explored the A2A Protocol, which addresses a different but equally important problem: enabling agents to collaborate with other agents. Unlike tools, agents are open-ended problem solvers that can reason, delegate, and participate in multi-step workflows. A2A provides a standard way for agents to discover each other through agent cards, exchange messages, manage tasks and artifacts, and support different execution modes such as synchronous requests, asynchronous polling, streaming, and push notifications. This makes it possible to build multi-agent systems where specialized agents work together without exposing their internal implementation details. Through the hands-on examples, you saw both protocols in action. You built an MCP-based weather assistant with FastMCP and then extended the ideas into a multi-agent travel scenario using Strands and A2A, where a travel orchestrator communicates with weather and flight agents.

MCP and A2A complement each other because they address two different layers of interoperability in agentic systems. A2A enables collaboration between the agents, and MCP enables each agent to do its own work by using external tools and data.

In the next chapter, you will build on this foundation by learning how to deploy, host, and scale these agentic applications on AWS so they can run reliably in production environments.

Get this book's PDF version and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

OceanofPDF.com

6

Production Deployment and Enterprise Integration

Over the past five chapters, you've built agents that reason, use tools, remember conversations, and work in teams. You gave them access to web search, file systems, and APIs. You added short-term and long-term memory backed by AgentCore memory. You coordinated multiple specialized agents through supervisor-worker, swarm, and graph patterns, and explored Agent Communications with MCP and A2A. All of this runs nicely on your laptop.

However, running agents in production is different. The moment someone says, *let's put this in front of real users* , you're dealing with questions nobody asked during development. What happens when 200 people hit your agent at the same time? Or what happens when the model provider has a bad day? Similarly, if your agent crashes at 3 AM and nobody notices until Monday, what would you do?

The AI part isn't what slows teams down; it's the infrastructure, networking, IAM (Identity and Access Management) policies, and other factors. What should have been a two-day deployment task turns into a week-long troubleshooting because nobody on the team has deployed a containerized application before, and the error messages aren't easy to debug.

This chapter is about closing that gap, not by turning you into a DevOps engineer, but by giving you enough to get your agents running in production. Though for production-hardened deployments with custom networking, compliance requirements, multi-region failover, etc., you'll want to partner with your infrastructure team.

We'll start with the simplest deployment path (AWS Lambda), move to always-on containers (Amazon ECS), and then explore Amazon Bedrock AgentCore, which is a platform built specifically for production agents. By the end of this chapter, you'll have a clear understanding of choosing the right deployment path and the patterns for connecting your agents to enterprise databases, APIs, and event-driven systems.

In this chapter we will cover the following key topics:

- Challenges while transitioning from development to production
- Deploying agents on AWS Lambda
- Deploying agents on Amazon ECS
- Amazon Bedrock AgentCore: Architecture, runtime and gateway
- Enterprise integration and industry use cases

Technical requirements

Before diving into this chapter, ensure you have:

- A working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with appropriate permissions
- Familiarity with Docker concepts (helpful but not required—we'll explain what you need)
- AWS CLI and SAM CLI installed (for the Lambda deployment walkthrough)
- Any IDE like Kiro, Claude Code, or Cursor for development (optional)

What actually changes when you go to production

Let us tell you a story. A team we worked with built a brilliant document analysis agent. It worked flawlessly in the development environment, and was able to parse PDFs, extract key terms, and summarize findings. The team demo'd it to leadership on a Thursday. By Monday, they had approval to deploy, and by Wednesday, they were in trouble.

The agent worked great for one document at a time. But in production, 200 users hit it simultaneously. The model calls started timing out. The temporary files piled up and filled the disk. One malformed PDF crashed the entire process, and there was no recovery mechanism—just silence. No logs, no alerts, nothing. They found out it was down because a VP called to complain.

This is a normal trajectory when you skip the production readiness conversation. Here's what changes:

- **Reliability isn't optional anymore** : Your laptop crashes, you restart it. A production agent crashes at 2am during a batch processing run, and you get a phone call. You need error handling, retries, circuit breakers, and graceful degradation.
- **Security becomes real** : In development, you probably have admin credentials sitting in environment variables. In production, that's how breaches happen. You need proper secrets management, least-privilege IAM roles, and network isolation.
- **Scale is unpredictable** : You tested the agent with 10 loan applications. But what happens with 10,000 applications? What about when marketing runs a campaign and traffic spikes 50x times more for two hours?
- **Cost adds up fast** : That model invocation call costs fractions of a cent. If you multiply that by thousands of invocations per day, add the compute, storage, and networking, and suddenly the costs ramp up, your CFO will ask questions.
- **Debugging gets harder** : When something goes wrong locally, you add a print statement. In production, across multiple containers, with requests from hundreds of users interleaving, you need structured logging, distributed tracing, and proper observability (we'll tackle that in the final chapter, but we'll lay the groundwork here.)

So, before we look at deployment paths, let's make sure the foundation is solid. We've seen teams rush to deploy and spend twice as long fixing things they should have handled upfront. So, before you move into production, answer the following questions:

- Do you have error handling that goes beyond `try/except: pass` ?
- Can you tell, right now, what your agent did for the last 100 requests?

- What happens if the model provider has an outage? Does your agent crash, hang, or degrade gracefully?
- Do you have resource (CPU, memory, IO) limits? Or will a runaway agent consume unlimited memory?
- Has anyone besides you run this code successfully?
- Do you have a way to roll back if the new version breaks something?
- Have you estimated what this will cost per month?

If most of these are missing, that's fine; we'll cover them in this chapter and the next.

Deployment path

Suppose you've built a document analysis agent. A customer uploads a contract to your web app, which calls the agent behind the scenes. The agent extracts key terms, flags risky clauses, and returns a summary. The whole thing takes about 30 seconds. Traffic is unpredictable—maybe 10 requests on a quiet Tuesday, and around 500 during end-of-quarter rush.

Now let's say there is a different agent in the hospital, connected to the electronic health records system, maintaining persistent sessions with scheduling databases and provider calendars. Establishing those connections takes time and authentication overhead you don't want to repeat on every request. It handles requests from staff around the clock, and the workflow for booking an Operating Room can involve multiple back-and-forth calls across systems in a single session.

These two agents have completely different infrastructure needs. The first one, with short-lived tasks, unpredictable traffic, and no state between requests, is a perfect fit for serverless. The second one, with persistent connections, multi-turn sessions, and always-on availability, needs an always-on container.

AWS gives you four options for running agents in production: AWS Lambda, Amazon Elastic Container Service (Amazon ECS), Amazon Elastic Kubernetes Service (Amazon EKS), and AgentCore. Each one fits a different set of constraints, and picking the wrong one won't end your project, but it can cost you weeks of rework. Let us explore each path.

AWS Lambda: Serverless and event-driven

Back to that document analysis agent. A customer uploads a contract, and your agent needs to process it. You don't want to run a server 24/7 waiting for uploads that might come once an hour or once a minute. You just want your code to run when a document arrives, process it, and stop.

AWS Lambda lets you attach your code to that upload event directly. You write a handler function, connect it to a trigger (an S3 upload, an API Gateway request, a scheduled event), and AWS Lambda runs it when the trigger fires. Here's a Strands agent running on AWS Lambda:

```
from strands import Agent
from strands.models.bedrock import BedrockModel
# Initialize outside the handler — this is the key decision
model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
agent = Agent(model=model, system_prompt="You are a document analysis assistant...")
def lambda_handler(event, context):
    document_text = event["body"]["document_text"]
    result = agent(f"Analyze this contract:\n\n{document_text}")
    return {"statusCode": 200, "body": str(result)}
```

The agent is initialized outside the handler function. AWS Lambda doesn't spin up a fresh Python process for every request. When traffic is flowing, it reuses the same execution environment across invocations. Your **model** and **agent** objects survive between calls. The first request pays the initialization cost: loading the Strands library, establishing the Amazon Bedrock connection, parsing the system prompt. Every subsequent request skips all of that and goes straight to the handler. You can move that initialization inside the handler, and you'll be paying the setup cost on every single invocation and unnecessary latency that adds up.

The GitHub repository has the production version of this code with proper API Gateway request parsing, input validation, and error handling.

The deployment uses a **Serverless Application Model** (**SAM**) template – a YAML file where you define the AWS Lambda function, the API Gateway trigger, the IAM permissions, and the container configuration, and deploy it all with a single command. The full template lives in the companion repository, but two pieces are worth understanding here because they're easy to get wrong and hard to debug when you do.

The first is the IAM policy:

```
Policies:
  - Statement:
      - Effect: Allow
        Action:
          - bedrock:InvokeModel
          - bedrock:InvokeModelWithResponseStream
        Resource:
          - "arn:aws:bedrock:*::foundation-model/*"
          - !Sub "arn:aws:bedrock:*:${AWS::AccountId}:inference-profile/*"
```

Two resource types, not one. The `foundation-model/*` ARN (Amazon Resource Name) covers direct model access. But if you're using a cross-region inference profile (and you are, because model IDs that start with `us.` like `us.anthropic.claude-sonnet-4-20250514-v1:0` are inference profile IDs), then your AWS Lambda functions also need permission to invoke inference profiles. Without the second resource, you'll get an access denied error that doesn't clearly indicate which permission is missing. Ideally, you should include both from the start.

Then there's the container packaging. Agent dependencies (Strands, tool libraries, boto3, and whatever your tools pull in) add up fast. AWS Lambda's zip deployment has a 250MB limit, and agent workloads blow past that routinely. The template uses `PackageType : Image` instead, which gives you up to 10GB of headroom. When you write a standard Dockerfile, SAM builds the image, and AWS Lambda runs it. The trade-off is slightly slower cold starts compared to zip, but since you'd exceed the zip limit anyway, it's not really a choice.

Also worth knowing: Amazon Bedrock model IDs require the full version string. The model ID is `us.anthropic.claude-sonnet-4-20250514-v1:0` , not `us.anthropic.claude-sonnet-4-20250514` . Drop the `-v1:0` suffix and you'll get a model-not-found error that doesn't explain what's wrong. The template stores this as an environment variable so you can swap models without redeploying code.

The GitHub repository includes a deploy script that handles the full workflow:

```
chmod +x deploy.sh cleanup.sh
./deploy.sh
```

The script checks prerequisites (SAM CLI, Docker, AWS credentials), auto-detects whether you're on an ARM or Intel machine, builds the container image, and deploys everything through CloudFormation. Two to three minutes later, it prints an API endpoint. POST a contract to that URL; your agent analyzes it. On Windows, run this from WSL or Git Bash. When you're done, `./cleanup.sh` tears down every resource the deployment created.

The architecture is simple, as shown in *Figure 6.1* :

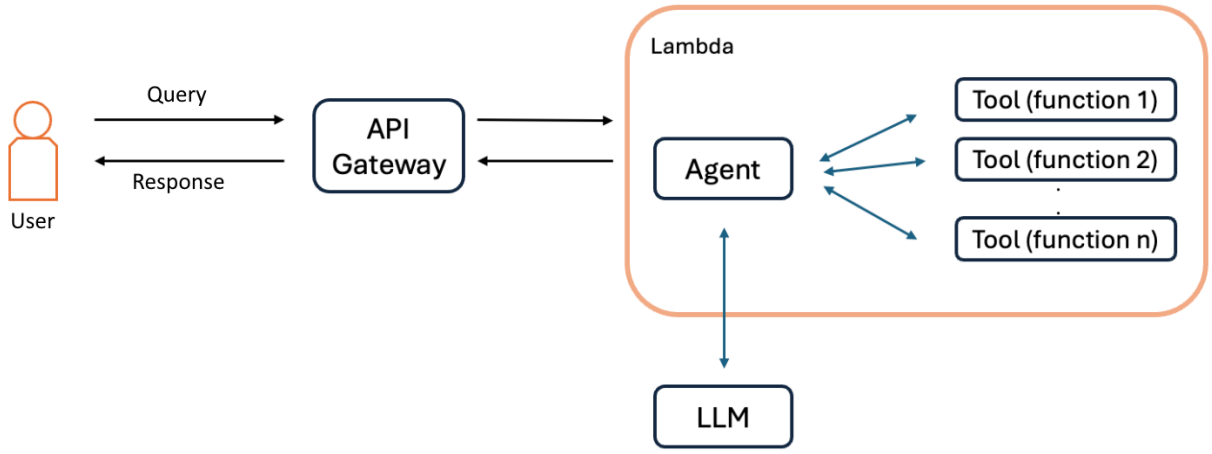


Figure 6.1: Deploying Agents on AWS Lambda

If this looks familiar, it's the same agent-with-tools pattern from earlier chapters we learned, now packaged inside a single Lambda function with API Gateway as the entry point.

A user sends a query through API Gateway, which invokes the Lambda function. Inside that function, your agent sits at the center. It calls out to the LLM (Amazon Bedrock, in this case) for reasoning, and invokes tools to perform specific tasks like clause extraction or risk scoring. Those tools are just Python functions bundled within the same Lambda package; they run in-process, in the same execution environment. The LLM sits outside the Lambda boundary because it's a remote API call over the network. There are no queues, no orchestration layers, no sidecar containers. One function, one agent, and just one execution environment.

There are three things about AWS Lambda that will shape how you design your agent:

- **Statelessness** : Each invocation is independent. If a customer uploads a contract and your agent asks a clarifying question, then the customer responds, that's a new invocation. The agent doesn't remember the first interaction unless you explicitly persist conversation state somewhere between calls and restore it on the next invocation. You can solve this with DynamoDB: store the conversation history keyed by a session ID, load it at the start of each invocation, and save it at the end. The hands-on tutorial at the end of this chapter provides a walk-through of this pattern. But it's the kind of thing you won't think about until your users complain that the agent has amnesia.
- **Cold starts** : Monday morning, nobody has used the agent since Friday. A customer uploads a contract, and instead of the usual sub-second response, they wait 2 to 5 seconds while Lambda spins up a fresh execution environment and loads your dependencies. That's a cold start. It only happens on the first invocation, or when Lambda scales up new instances during a traffic spike, but it's noticeable. Container images make cold starts slightly slower than zip deployments because there's more to load, but since agent dependencies typically exceed the 250MB zip limit anyway, that's a trade-off you're already making. Provisioned concurrency keeps set number of Lambda instances pre-warmed and ready to go, though at that point you're paying for idle compute, which undercuts one of Lambda's main advantages.
- **The execution ceiling** : Your document analysis agent works great for simple contracts. But a month in, a customer uploads a 200-page contract and your agent needs to call the model multiple times, cross-reference clauses, and generate a detailed report. Twelve minutes in, Lambda kills the invocation. You've hit the 15-minute maximum limit. For most single-agent tasks, 15 minutes is plenty. But for multi-agent workflows with retries and slow external calls, you'll hit this wall. Lambda's durable functions, announced at **re:Invent 2025** , use a checkpoint-and-replay mechanism: the function reruns from the start but skips over completed checkpoints, resuming execution from where it left off. But that adds complexity to your design.

AWS Lambda works well when your agent responds to an event, does its work, and finishes. If your agent needs to stay running, maintain persistent

connections, or coordinate a team of agents in a long-running process, you need something with more staying power. That's next!

Amazon ECS: Always-on and container-driven

A nurse walks up to a scheduling terminal at 2 AM and types: `Is Dr. Arti available Monday for a knee replacement?` The agent checks the calendar and responds with three open slots. The nurse types back: `The 9 AM slot works, patient ID is P-1234` . The agent books it, confirms equipment availability for OR-3, and sends notifications.

There are two messages. The second one only makes sense because the agent remembers the first. And this isn't a one-off interaction. Staff use this system around the clock, and each conversation might be three, four, or five messages deep before a procedure gets booked.

You could build this on AWS Lambda. Persist the conversation in DynamoDB, reload it on every invocation, rebuild the agent's context each time. It would work. But you'd be fighting the infrastructure instead of building the feature. A Lambda function wants to wake up, do a thing, and go away. This agent needs to be awake, warmed up, and ready at all times.

Amazon ECS gives you that. You put your agent in a Docker container, hand it to Amazon ECS, and tell it to *keep this running* . When you run it as an Amazon ECS *service* , it stays up. If the container crashes at 3 AM, the service notices and starts a new one. You don't get paged. You don't write recovery scripts.

From Lambda handler to web server

The mental shift from Lambda to ECS: on Lambda, your agent is a *function* that gets called. On ECS, your agent is a *web server* that's always listening.

The Lambda version looks like this:

```
def lambda_handler(event, context):
    result = agent(event["body"]["document_text"])
    return {"statusCode": 200, "body": str(result)}
```

Something calls your function, you respond, and you're done. The ECS version flips that around:

```
app = FastAPI()
model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
```

```python
@app.post("/schedule")
async def schedule(request: ScheduleRequest):
    agent = Agent(model=model, tools=[...], system_prompt=PROMPT)
    # Feed in the conversation so far
    for msg in request.history:
        agent.messages.append({"role": msg.role, "content": [{"text": msg.content}]})
    return str(agent(request.message))
```

Your agent boots up, opens a port, and waits. Requests come anytime. The model object and boto3 session initialize once when the container starts and persist for the life of the container. No cold starts between requests and no rebuilding the agent from scratch on every call.

The conversation history part is worth pausing on. Each request carries `request.history object` , the full conversation so far. The frontend keeps track of everything the user and agent have said, and sends it all with every new message. The agent gets seeded with that history before processing the latest message. That's how, when the nurse says "the 9 AM slot works" in message two, the agent still knows she is referring to Dr. Arti and a knee replacement from message one.

This is the simplest approach, but not the only one. Strands has a built-in session management system that can persist conversation history to the local filesystem or to Amazon S3, and you can implement a custom session repository backed by DynamoDB or any database. Those approaches are better for production because the server owns the state. The frontend-carries-history pattern works well for demos and lightweight deployments where you don't want to set up a persistence layer, but for anything with real users, you'd want the conversation stored server-side so it survives page refreshes and can be audited.

What's actually happening on AWS

You don't need to be a networking expert to understand the Amazon ECS deployment model. Here's what it looks like:

Figure 6.2: Agent deployed on Amazon ECS

A user opens a URL in their browser. That request hits a *load balancer* , which routes traffic to healthy containers. If you're running two containers and one dies, the load balancer sends everything to the surviving one while Amazon ECS spins up a replacement. Your users don't notice.

Behind the load balancer, an **Amazon ECS service** keeps your containers running. You tell it, `I want two copies of my agent running at all times` , and it makes that happen. If the container crashes, it gets replaced. And if you need to deploy a new version, the service rolls it out gradually, keeping the old version running until the new one passes health checks.

Each container runs from a *task definition* , which is a config file that says: use this Docker image, configure it with specific CPU and memory, and send its logs to CloudWatch. Think of it as a recipe card. The service is the kitchen that follows the recipe.

For the compute itself, you have two options. **AWS Fargate** means you specify CPU and memory and AWS handles the servers. You never see an EC2 instance, never patch an operating system, never think about capacity. **Amazon EC2 launch type** means you manage the underlying instances yourself, which you'd only do if you need something Fargate can't provide, like a GPU. We would recommend that you start with Fargate and later you can switch to EC2 launch type, if needed.

The GitHub repository has a sample deployment that puts all of this together. It's not production-hardened (the tools return mock data, there's no

authentication, error handling is minimal), but it walks through every step of getting a Strands agent onto Amazon ECS with AWS Fargate. You need to first clone the repo, navigate to the ECS deployment directory, and run:

```
chmod +x deploy.sh cleanup.sh
./deploy.sh
```

Four minutes later, you get a URL. Open it in your browser and you'll see a chat interface. Try typing, `Is Dr. Smith available next Tuesday for a knee replacement?` , and follow up with just a time slot and patient ID. The agent connects the dots across messages. The point isn't the scheduling app itself. It's seeing the full journey from Python code on your laptop to a running agent behind a public URL on AWS, and being able to read through the deploy script to understand each step.

When you're done, `./cleanup.sh` tears everything down. The `README` covers networking details, IAM configuration, and troubleshooting for readers who want to dig into the infrastructure side.

When to reach for Amazon ECS

Amazon ECS comes into its own in two scenarios.

The first is multi-agent systems. The e-commerce order processing system from Chapter 4 had six specialized agents coordinating through the graph pattern: risk analysis, fraud detection, inventory, shipping, the whole pipeline. On ECS, all of those agents live in the same container, talking to each other through plain function calls. No network hops, no serialization overhead, no message queues between them. You deploy the entire team as one unit and scale by adding more container replicas.

The second is anything with expensive startup costs: database connection pools that take seconds to establish, authenticated sessions with third-party APIs that involve OAuth token exchanges, and ML models loaded into memory. On ECS, that happens once when the container starts and persists for the life of the container. On AWS Lambda, you'd pay some of that cost on every cold start, and all of it when the Lambda function scales up new instances during a traffic spike.

Amazon Bedrock AgentCore

In the last two sections we covered deploying agents on AWS Lambda and Amazon ECS. That meant writing Dockerfiles, configuring load balancers, setting up IAM roles, putting health checks in place, and managing task definitions. And these are the simple versions. There is a lot that gets added when we need to productionize an agent, such as monitoring, auto-scaling policies, managing certificates, configuring Virtual Private Cloud (VPC), and a dozen other things that have nothing to do with your agent's actual job.

All these things discussed in the preceding lines are not agent specific. Everything that you do to productionize the agent, you'd do the same way for REST API, batch processors, building websites, etc. The infrastructure doesn't know or care that it's running an AI agent. But agents have problems that generic compute doesn't solve for you: per-user session isolation so one conversation can't leak into another, tool-level authorization so an agent can't exceed its boundaries, identity propagation so tools act on behalf of the right user, and observability that captures reasoning steps and not just HTTP error codes. We are left with building all of that ourselves on ECS/EKS or Lambda.

This is where Amazon Bedrock AgentCore comes in. We introduced AgentCore in *Chapter 1* , *Understanding AI Agents on AWS* , as part of the AWS AI stack, a managed platform sitting between the fully-managed Bedrock Agents tier and the do-it-yourself infrastructure tier. In *Chapter 3* , *Agent Memory* , we used AgentCore Memory to give agents short-term and long-term memory across sessions. Those were individual services solving individual problems. Now we'll look at how the full set of AgentCore services fits together and what the platform actually means for getting your agents into production.

The AgentCore architecture

AgentCore is an AWS managed platform, where you get a set of composable services, each solving a specific production challenge that every team hits when they try to move agents from the prototyping phase to production. You can use all of them together, or pick the ones you need.

Here is the full picture of AgentCore with all the services together:

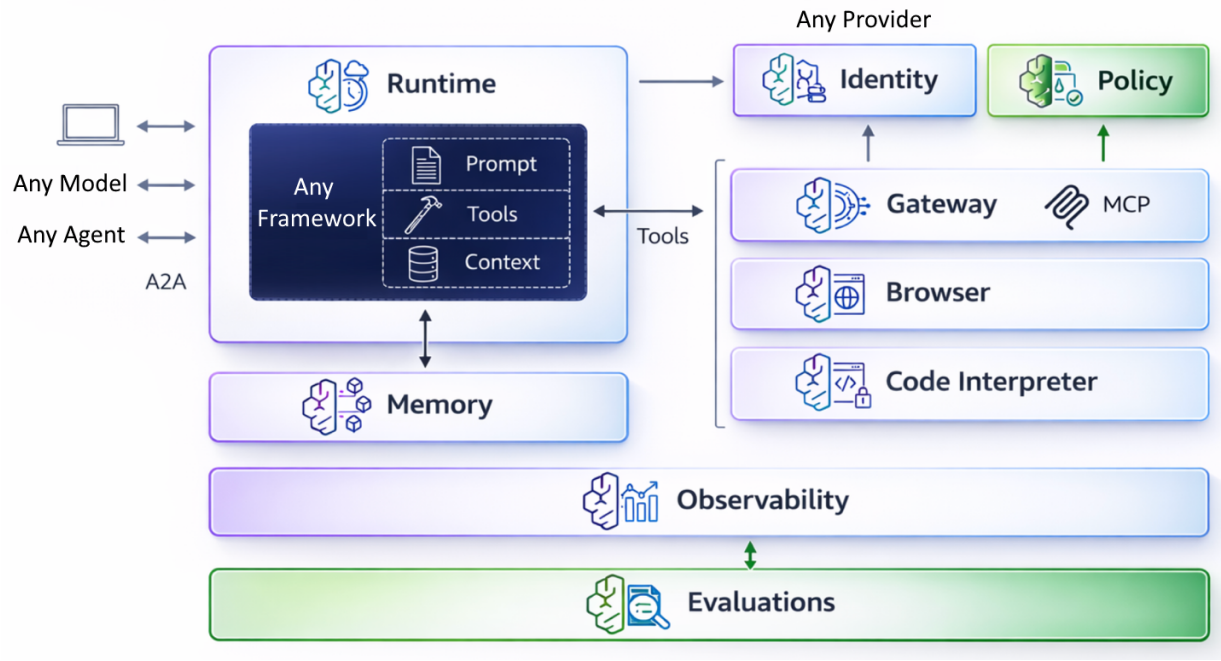


Figure 6.3: AgentCore architecture

Every box shown in *Figure 6.3* solves a problem you'd otherwise build yourself. Understanding the architecture is what turns AgentCore from *another managed service* into a genuine shortcut to production.

Let us look at each of the components:

Runtime is where your agent actually executes. It's framework-agnostic. It consists of Strands, LangGraph, CrewAI, LlamaIndex, OpenAI Agents SDK, Google ADK, Claude Agent SDK, or any custom framework you would like to use. Runtime doesn't care. It hosts the agent, connects it to whatever model you choose—Bedrock, OpenAI, Anthropic, Google, etc. The Agent in Runtime can be exposed through the Agent-to-Agent protocol we covered in Chapter 5 , so agents on other platforms can discover and talk to yours.

Now look at what surrounds Runtime. On the right side, **Gateway** takes your enterprise APIs (described as OpenAPI specs, Smithy models, Lambda functions, or remote MCP servers) and turns them into MCP-compatible tools your agent can discover and call. No integration code to write per API.

Below Gateway, **Browser** and **Code Interpreter** are managed tool runtimes: Browser gives your agent a sandboxed browser for interacting with web pages, and Code Interpreter provides an isolated sandbox for running Python,

JavaScript, or TypeScript. All three show up as tools your agent can call, indistinguishable from custom tools you wrote yourself.

Identity and **Policy** sit at the boundary. Identity handles authentication in both directions: users authenticating to your agent (via Cognito, Okta, Entra ID, or standard OAuth) and your agent authenticating to external services on behalf of those users. Without this, either anyone could call your agent, or your agent couldn't prove who it's acting for when it calls downstream APIs. Policy provides deterministic governance, with rules written in Cedar (an open-source policy language developed by AWS) that intercept every tool call before it executes. When a policy says "no refunds over $1,000," that's enforced at the infrastructure level, and not inside the agent's reasoning loop. No amount of prompt engineering overrides it. We will cover identity later in this chapter, and policy with Cedar-based rules in the final chapter.

Memory is the same service you covered in *Chapter 3* , *Agent Memory* . Short-term memory for conversation flow, long-term memory for learning across sessions. Here, it's wired directly into Runtime as a managed service.

Observability spans the bottom of the diagram because it captures everything across all services: reasoning steps, tool invocations, model calls, policy decisions, latency, errors. It's built on OpenTelemetry, so it plugs into whatever monitoring stack you already run (CloudWatch, Datadog, Grafana, Splunk, etc.). We'll cover this in detail in the final chapter.

Evaluations sits alongside **Observability** , offering LLM-as-judge scoring, built-in evaluators for faithfulness and relevance, and continuous online evaluation in production. We'll cover evaluations in *Chapter 7* , *Evaluation, Observability, and AI Governance* .

One important thing to note about AgentCore is that these services are composable. You can pick and choose what service you want to use from here. Run your agent on ECS but use Gateway for tool integration. Use Runtime for hosting but manage your own authentication. Use just Memory if that's all you need. Each service has its own API, so you can adopt them independently. But when you use them together, they're integrated. Identity flows through to Gateway, Runtime connects to Memory automatically, Observability captures the full picture without you instrumenting anything.

Now let's look at how all of these pieces work together. Let's walk through a real interaction. A user asks your agent, `A nalyze this sales data and create a summary report .`

1. **Authentication** : The request goes through AgentCore Identity, which validates the user's bearer token against their identity provider and creates appropriate session credentials
2. **Agent processing** : The request hits your agent running in AgentCore Runtime, which checks AgentCore Memory for context about this user and previous interactions, including long-term memory insights like `this user prefers visual charts and usually asks for quarter-over-quarter comparisons`
3. **Tool d iscovery** : The agent queries AgentCore Gateway: `I need tools for data analysis and report generation .` Gateway's semantic search returns the most relevant tools from the catalog.
4. **Policy e valuation** : Before any tool executes, AgentCore Policy intercepts the call and evaluates it against Cedar rules. It may ask the following questions: Can this user query sales data only for their own region, or across the entire company? Is the report limited to a specific department or product category? Can the agent call the export tool? Only if Policy permits, only then the tool call will proceed.
5. **Tool e xecution** : Gateway routes the authorized tool calls. The agent uses Code Interpreter to analyze the data in a sandboxed environment, generates charts based on the user's known preferences, maybe uses Browser tool to pull additional market context from a web portal.
6. **Memory u pdate** : AgentCore Memory stores the interaction. Short-term memory captures the conversation flow. In the background, long-term memory extraction runs—reinforcing the chart preference, noting that this user frequently asks for sales analysis, and storing any new facts mentioned during the conversation.
7. **Observability and e valuation** : Throughout this entire flow, AgentCore Observability captures every step - every authentication decision, the memory retrieval, every tool call with parameters and results, every policy evaluation, every model invocation. AgentCore Evaluations scores the interaction against quality criteria you've defined: was the

report accurate, did the agent follow the right steps, and did it stay within scope. Both feed into the same trace, so you get a single view of what happened and how well it went.

The user sees a report with charts. Behind the scenes, the full AgentCore stack coordinates to make that happen securely, with governance, and with full traceability. On AWS Lambda or Amazon ECS, you'd build most of that coordination yourself.

Let us look at the Runtime component in the architecture and how we can deploy the agents in it.

AgentCore runtime

Runtime is where the Agent lives and executes. Each user session gets its own microVM, as shown in *Figure 6 . 4* . The agent can maintain state, hold database connections, and accumulate context across multiple turns of conversation, all within that dedicated environment. When the session ends, the entire microVM is terminated and the memory is sanitized. There is no possibility of data leaking between sessions because there is nothing shared between them. For example, let's say in the financial services industry, an agent is processing one client's portfolio data, and with Agentcore runtime, there's zero chance of it leaking to another client's session.

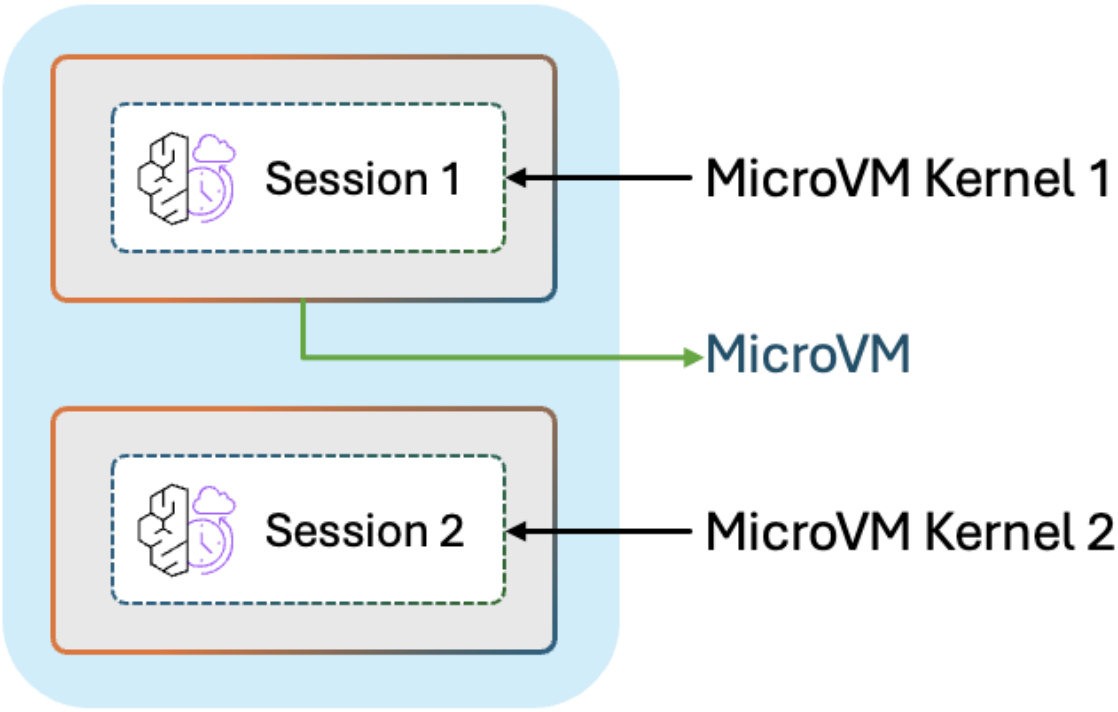


Figure 6.4: Runtime Session Isolation

Consider what happens during a multi-turn conversation. The user asks your agent to analyze a financial document. The agent extracts data, stores intermediate results, and may write temporary files while processing. On AWS Lambda, all of that state vanishes between invocations. On Amazon ECS, that state lives in a shared container where another user's request could theoretically access it. On AgentCore, each user gets their own sandbox. The agent can behave as if it is running on a dedicated machine, because it effectively is.

When a user sends a request, AgentCore creates a new session and spins up a microVM. That session moves through three states: **Active** while your agent is processing, **Idle** when it is waiting for the next user message, and **Terminated** when the session ends. The default idle timeout is 15 minutes—if the user goes quiet longer than that, the session terminates and the microVM is cleaned up.

The important thing to understand about the cost model is that you only pay for **Active** time. During the **Idle** state, the microVM remains provisioned and

your session state is preserved. If your agent isn't doing any processing, CPU charges drop to zero and you are only billed for what your agent actually consumes, not for the time spent waiting. A typical customer support conversation where the user takes a minute between messages means you are paying for the 10-20 seconds of active processing per turn, not the silence in between.

So, how long can a session run? That depends on the invocation pattern. For synchronous request-response calls, the ceiling is 15 minutes per request, which is similar to AWS Lambda. Streaming connections (HTTP response streaming and WebSocket) can stay open for up to 60 minutes. And for heavy lifting tasks like batch processing, multi-agent research tasks, long document pipelines—asynchronous jobs can run for up to 8 hours. If your agent kicks off one of these longer operations, it signals `HealthyBusy` through the health check endpoint so AgentCore knows the session is working, not idle. The SDK provides `add_async_task` and `complete_async_task` helper functions that manage this automatically.

Now let's see how the deployment to `a gent c ore` runtime looks:

```
from bedrock_agentcore.runtime import BedrockAgentCoreApp
from strands import Agent, tool
from strands.models.bedrock import BedrockModel
app = BedrockAgentCoreApp()
@tool
def extract_clauses(text: str) -> dict:
    """Extract key clauses from contract text."""
    return {"clauses": ["termination", "liability", "indemnification"]}
model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
agent = Agent(
    model=model,
    tools=[extract_clauses],
    system_prompt="You are a document analysis assistant..."
)
@app.entrypoint
def handle(payload):
    response = agent(payload.get("prompt"))
    return response.message["content"][0]["text"]
if __name__ == "__main__":
    app.run()
```

In the preceding code, only four things changed from local development code: we imported `BedrockAgentCoreApp` , created an `app` instance, decorated your

invocation function with `@ app.entrypoint` , and called `app.run ( )` . Behind the scenes, `BedrockAgentCoreApp` turns your agent into an HTTP service with two endpoints: `/invocations` for processing requests and `/ping` for health checks. We do not write any server code, the SDK handles all of it.

The agent logic is identical. Your tools, system prompt, and model configuration all remain the same code you have been writing since *Chapter 2* , *Building Agents with Tools* . You can test this locally by running `python agent.py` and sending a POST request to `http://localhost:8080/invocations` . For instance, if you run this in a terminal, you will see something like this:

```
% curl -X POST http://localhost:8080/invocations \
>   -H "Content-Type: application/json" \
>   -d '{"prompt": "What are the key clauses in a standard NDA?"}'
"I'd be happy to help you extract key clauses from an NDA, but I'll need you to provide the actual
contract text for me to analyze. \n\nIf you have a specific NDA document you'd like me to review,
please share the text and I can extract the key clauses for you.\n\nAlternatively, if you're looking
for general information about what key clauses are typically found in standard NDAs, here are the
most common ones:\n\n**Typical NDA Key Clauses:**\n1. **Definition of Confidential Information** -
What constitutes confidential information\n2. **Obligations of the Receiving Party** - How
confidential information must be handled\n3. **Permitted Uses** - Specific purposes for which the
information can be used\n4. **Exceptions** - Information that is not considered confidential
(publicly known, independently developed, etc.)\n5. **Term/Duration** - How long the confidentiality
obligations last\n6. **Return or Destruction** - Requirements to return or destroy confidential
information\n7. **Remedies** - Legal remedies available for breach (injunctive relief, damages)\n8.
**Governing Law** - Which jurisdiction's laws apply\n9. **Non-Solicitation** - Restrictions on
soliciting employees or customers (if applicable)\n10. **Severability** - If one clause is invalid,
the rest remains enforceable\n\nWould you like me to analyze a specific NDA document? Just paste the
text and I'll extract the actual key clauses for you.
```

The runtime is framework agnostic, meaning you can host Strands Agents, LangGraph, CrewAI, OpenAI Agents SDK, Google **Agent Development Kit** (**ADK**), Claude Agent SDK, or any custom framework. If your team has agents built with different frameworks, they all deploy the same way. This matters in practice because most organizations end up with agents in multiple frameworks, either through acquisition, team preferences, or because different frameworks suit different problems.

Let's talk about what **framework-agnostic** actually looks like in practice. Let's say your team has been building agents with Claude Code and you've already got a whole programming model going—your `CLAUDE.md` with project context,

a `skills/` folder with reusable capabilities, MCP servers wired up to your tools, maybe even sub-agents running jobs in parallel.

You can deploy all of that to AgentCore runtime as-is. You wrap your agent entry point with `@ app.entrypoint ,` add `app.run ()` at the bottom of the code. The Claude Agent SDK handles its own tool discovery, skill loading, and sub-agent orchestration inside the microVM. Runtime doesn't know or care that it's running a Claude Agent SDK agent versus a Strands agent versus something you duct-taped together with raw API calls. It just manages the session, the isolation, and the scaling.

Figure 6.5 shows how a Claude Agent SDK agent deploys to AgentCore Runtime. The entire programming model (`CLAUDE.MD` , `skills/` , MCP servers, sub-agent orchestration) runs inside the microVM, with Runtime managing the session and scaling around it.

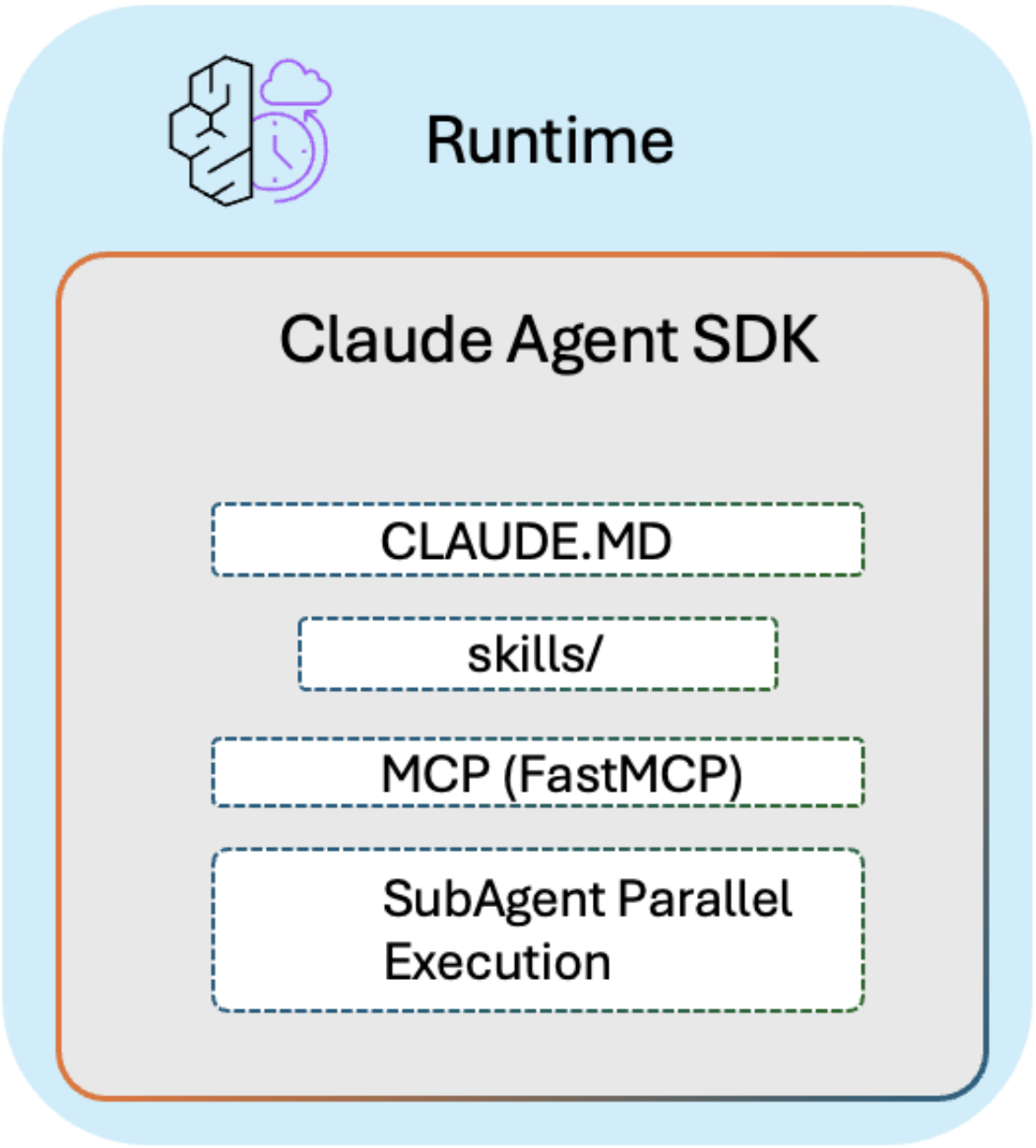


Figure 6.5: Claude Agent on AgentCore Runtime

The same goes for LangGraph, CrewAI, OpenAI Agents SDK, or whatever you are using already or have experience with. The **entrypoint** wrapper is the only thing that changes. If you want to try this yourself with a working example that deploys a Claude Agent SDK agent with Athena-backed tools and streaming responses, check out this sample repository by AWS:

https://github.com/aws-samples/sample-agentic-ai-with-claude-agent-sdk-and-amazon-bedrock-agentcore

Deployment walkthrough

Now that you have agent code ready, let us walk through deploying it to AgentCore using the Python SDK. The AgentCore Starter Toolkit provides a `Runtime` class that handles the full lifecycle: configure, launch, invoke, and clean up:

```
from bedrock_agentcore_starter_toolkit import Runtime
from boto3.session import Session
region = Session().region_name
runtime = Runtime()
# Step 1: Configure — generates a Dockerfile, sets up IAM role and ECR
runtime.configure(
    entrypoint="agent.py",
    requirements_file="requirements.txt",
    auto_create_execution_role=True,
    auto_create_ecr=True,
    region=region,
    agent_name="doc-analysis-agent"
)
# Step 2: Launch — builds the container, pushes to ECR, deploys to Runtime
launch_result = runtime.launch()
```

The `configure` step generates a Dockerfile from your code, creates an IAM execution role with the permissions your agent needs, and sets up an Amazon ECR repository. The launch step builds the container image, pushes it to ECR, and creates the AgentCore Runtime endpoint. The first deployment takes a few minutes while it packages everything. Subsequent updates are faster because the toolkit reuses cached dependencies.

Once the endpoint is ready, you can test it through the toolkit's `runtime.invoke()` method. In production, you'd call the agent from other services using boto3's `invoke_agent_runtime` – which has a similar pattern as calling any other AWS service. Each invocation creates a session backed by a microVM, and that session costs money (vCPU and memory) while it's active. When a conversation is done, stop the session explicitly or configure an idle timeout so sessions clean themselves up. Try out the notebook of `a gent c ore` runtime in the GitHub repo associated with this chapter: https://github.com/PacktPublishing/Agents-on-AWS/tree/main

You might have noticed the code snippet above the parameter `auto_create_ecr=True`, which built a Docker container behind the scenes. That's the direct code deployment path, where you hand it your source code and it handles the rest. If your team already has CI/CD pipelines that produce Docker images, you can bring your own container instead. Either way, `Runtime` doesn't care how the code arrived. One thing to know if you go the container route: AgentCore requires ARM64 images (Graviton), so you would have to plan your builds accordingly.

How runtime compares to Lambda and ECS

You've now seen all three deployment paths. Before we move to Gateway, here's how they stack up:

	Lambda	**ECS (Fargate)**	**AgentCore Runtime**
Session model	Stateless — each invocation is independent	Shared container across requests	Dedicated microVM per user session
State across turns	Persist externally (DynamoDB), reload every call	In-memory within the container	Maintained automatically in the microVM
Max execution	15 minutes per invocation	No limit (container stays running)	15 min sync, 60 min streaming, 8 hours async
Scaling unit	Per request	Per container (task count)	Per session
Pricing model	Per invocation + duration	Per allocated vCPU/memory (always on)	Active processing time only (meter pauses when idle)
Auth, observability, policy	You build it	You build it	Managed services you configure

	Lambda	ECS (Fargate)	AgentCore Runtime
I nfrastructure control	Moderate (VPC, layers, concurrency)	Full (networking, security groups, instance types)	Abstracted — less control, less to manage

Table 6.1: Comparison – Lambda, ECS, AgentCore Runtime

The last two rows are where the decision usually gets made. Lambda and ECS are general-purpose compute, and they'll run your agent, but auth, observability, tool integration, and policy enforcement are your responsibilities. AgentCore handles those as managed services, which saves weeks of engineering, but you give up the ability to control VPC placement, security groups, or GPU access. ECS is the escape hatch when you need that control.

AgentCore Gateway

In *Chapter 5* , *Agent Communication* , you built MCP servers and clients from scratch. You saw how a single MCP server can expose tools, resources, and prompt templates through a standardized protocol, and how your agent discovers and calls those tools without custom integration code. That works well when you have a handful of MCP servers and you're writing the tool code yourself.

But most enterprises don't have MCP servers. They have hundreds of REST APIs described by OpenAPI specs, internal services behind authentication layers, and legacy systems that nobody wants to touch. Writing an MCP server wrapper for each one with auth, error handling, retries, and rate limiting is a full-time job. And when the API team changes an endpoint or rotates credentials, you're back in the code.

AgentCore Gateway bridges that gap. You point it at an OpenAPI spec, and it generates MCP-compatible tools automatically, with no server code to write, no wrapper to maintain. Your agent discovers those tools at runtime the same way it discovered tools from the MCP servers you built in *Chapter 5* , *Agent Communication* , except now the tools come from your existing API catalog instead of hand-built servers.

Figure 6.6 provides a high-level overview of the AgentCore Gateway:

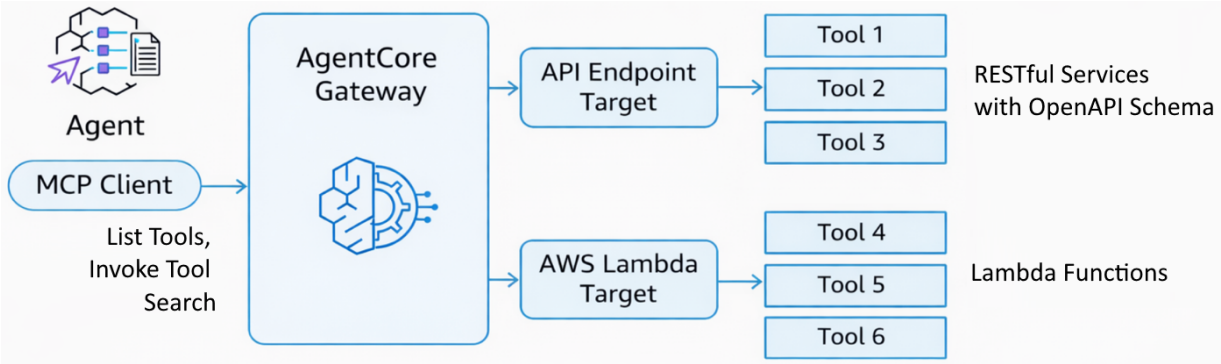


Figure 6.6: AgentCore Gateway

Gateway supports several backend types. You can point it at an OpenAPI spec for existing REST APIs, a Lambda function for custom logic, or a remote MCP server for tools that already speak the protocol. Each backend generates MCP-compatible tools automatically. Your agent sees tools and does not need to know what is behind them.

You can combine multiple APIs, functions, and existing MCP servers into a single tool catalog. Your agent does not need to know that **check inventory** hits a Lambda function while **send notification** goes through an API Gateway endpoint, while **update CRM** talks to Salesforce. From the agent's perspective, they are all just tools.

Targets

Gateway calls each registered API a **target**. You register a target by pointing Gateway at an API definition—an OpenAPI spec, a Smithy model, a Lambda function, an API Gateway stage, or a remote MCP server. Then Gateway reads it, extracts each operation, and generates an MCP-compatible tool for each one. Descriptions, parameter schemas, return types, all automatic. Your agent sees tools. It doesn't need to know whether a given tool hits a REST endpoint, invokes a Lambda, or calls another MCP server behind the scenes.

Let's say you have a customer API with an OpenAPI spec in Amazon S3. Here's how it looks like:

```
import boto3
client = boto3.client('bedrock-agentcore-control')
# Create a gateway
gateway = client.create_gateway(
```

```
    name='enterprise-tools',
    roleArn='arn:aws:iam::123456789012:role/GatewayExecutionRole',
    protocolType='MCP',
    authorizerType='AWS_IAM'
)
# Register a REST API as a target — Gateway reads the spec and creates tools
client.create_gateway_target(
    gatewayIdentifier=gateway['gatewayId'],
    name='customer-api',
    targetConfiguration={
        'mcp': {
            'openApiSchema': {
                's3': {
                    'uri': 's3://my-bucket/customer-api-spec.json'
                }
            }
        }
    }
)
```

Two things are happening here. The first call creates the gateway—think of it as standing up the front door. The `roleArn` is the IAM role Gateway will use when it calls your downstream APIs on the agent's behalf. `protocolType` is MCP (the only option today), and `authorizerType` controls who's allowed to connect— `AWS_IAM` means any caller with the right IAM permissions can reach the gateway without setting up a separate OAuth provider.

The second call registers your API as a target. You're telling Gateway: `here's an OpenAPI spec in S3 — go read it and turn every endpoint into an MCP tool` . Gateway does exactly that. It parses the spec, figures out what each endpoint does, and creates a tool for each one—complete with descriptions, parameter schemas, and return types pulled straight from the spec.

If that OpenAPI spec defines endpoints for `GET /customers/{id}, POST /customers/{id}/orders` , and `GET /customers/{id}/balance` , Gateway creates three MCP tools from them. Your agent connects to the gateway URI, discovers those tools, and calls them the same way it calls tools from the MCP servers. No wrapper code, no MCP server to deploy and maintain.

Where this really pays off is composition. When you register your customer API, a Lambda function runs fraud checks, and a third-party weather service —all as targets on the same gateway. Your agent sees one unified tool catalog. *Check balance* hits your internal REST API, *run fraud check* invokes a Lambda,

get weather calls an external service. The agent doesn't know or care about the difference. They're all just tools on the same MCP connection.

The GitHub samples repository has tutorials that walk through creating a gateway, registering targets, and connecting an agent: https://github.com/awslabs/amazon-bedrock-agentcore-samples/tree/main/01-tutorials/02-AgentCore-gateway

Semantic search

With five or ten tools, you can load all the tool definitions into the agent's prompt and let the model pick. That's what you did in *Chapter 5* , *Agent Communication* , and it works fine at that scale. However, at 200 tools, you have a problem. The definitions alone consume a significant chunk of the context window, and models get noticeably worse at selecting the right tool as the list grows.

Gateway handles this with semantic search. When you register targets, Gateway creates vector embeddings of each tool's name, description, and parameters, and indexes them. The index updates on its own as you add or remove targets, and we don't manage it.

The search itself is exposed as an MCP tool called `x_amz_bedrock_agentcore_search` . So, the flow looks like this: your agent gets a user request, let's say, `which scenes in Episode 4 still have unfinished VFX shots before the Friday delivery?` —and calls the search tool with a query like, `VFX shot completion status by episode` . Gateway compares that against the embeddings and returns the top matches, maybe `production_tracker_api_get_vfx_status` . The agent picks from that short list and calls the tool directly. It never loads the other 199 tool definitions into the prompt.

The practical benefit is fewer tokens per request and better tool selection. But search quality depends on your API documentation. If your OpenAPI spec has well-named operations with clear descriptions, the embeddings are useful and matches are accurate.

Authentication: Inbound and outbound

Any production deployment needs to answer two questions: who's allowed to call the gateway, and how does the gateway prove its identity to the APIs it's

calling on your behalf?

For inbound auth, controlling who can reach the gateway, you pick between IAM and JWT. If your agent already runs on AWS with an IAM role (on Runtime, Lambda, or ECS), IAM is the simpler path. Grant the role permission to invoke the gateway and you're done. For scenarios where end-user identity needs to flow through, Gateway supports JWT-based auth through Cognito, Okta, or any OAuth 2.0 provider.

Outbound auth is configured per target. Your internal customer API might use an OAuth token, a third-party weather service might use an API key, and a public dataset might need no auth at all. Gateway stores those credentials in Secrets Manager, handles token refresh, and caches tokens so it doesn't hammer downstream identity providers with repeated requests.

Gateway keeps inbound and outbound tokens completely separate. The token proving your agent can use Gateway and the token proving Gateway can call the downstream API are different credentials with different scopes. A downstream API can't take the inbound token and reuse it to impersonate your agent elsewhere. If you're in a regulated industry, your security team will appreciate this; it's one of the first things auditors check. Gateway handles the mechanics of authenticating to downstream APIs. But there is a bigger question it does not answer on its own: Who is the user behind the request, and what are they allowed to do? That is the job of AgentCore Identity.

AgentCore Identity

Every agent in production faces a two-sided authentication problem. On one side, callers need to prove who they are before the agent will talk to them, whether that caller is a user, another agent, or an automated workflow. On the other side, when the agent needs to call an external service, like a CRM, a calendar API, or a code repository, it needs credentials for that service. Without the first, anyone can hit your agent endpoint. Without the second, you end up embedding secrets in environment variables or hardcoding API keys in your agent code, which is how breaches happen.

AgentCore Identity handles both sides. For inbound authentication, Identity supports JWT-based authorization. Your identity provider (Amazon Cognito, Okta, Microsoft Entra ID, or any standard OAuth 2.0/OIDC provider) issues

the token, and Identity validates it before the request reaches your agent. For outbound authentication, Identity manages the credentials your agent needs to access external services, storing OAuth 2.0 tokens and API keys in a secure token vault rather than in your agent's code or configuration. Credentials are encrypted with AWS KMS, bound to specific agent-user combinations, and every access is logged for audit.

The outbound side is where most of the complexity lives:

- First, your agent needs to access a user's personal data: their Google Calendar, their Salesforce records, their GitHub repositories. This requires the user's explicit consent through an OAuth 2.0 authorization code grant (the flow where the user clicks **Allow** in a browser to authorize access). The user authorizes the agent once. Identity stores the resulting tokens securely, handles token refresh when they expire, and your agent can access that user's data on subsequent requests without asking for consent again.
- Second, your agent needs to access system-level resources like an internal knowledge base, a data warehouse, or a monitoring API, where no specific user is involved. This uses the OAuth 2.0 client credentials grant (machine-to-machine authentication, where the agent authenticates as itself). Many agents need both patterns simultaneously: user-delegated access for personal data and machine-to-machine access for shared resources.

In your agent code, this shows up as SDK annotations. You decorate a function with `@requires_access_token` or `@requires_api_key` , and the SDK handles credential retrieval, injection, and refresh automatically. Identity comes with built-in credential providers for services like Google, GitHub, Slack, Salesforce, and Atlassian, among others, with authorization endpoints and provider-specific parameters pre-configured. For anything else, you can set up custom OAuth 2.0 credential providers that work with any standards-compliant service.

When Identity validates an inbound token and manages outbound credentials, it maintains the connection between who is calling and what the agent is doing on their behalf. This context —the caller's identity paired with

the agent's identity—is what enables the rest of the AgentCore platform to enforce per-user, per-action governance.

If you are deploying on Lambda or ECS instead of AgentCore, you build credential management yourself: storing tokens in Secrets Manager, implementing OAuth flows in your application code, handling token refresh, and managing the overall security. It works, but it is undifferentiated work that Identity handles as a managed service.

For a hands-on walkthrough of setting up AgentCore Identity, see the tutorial in the companion repository: `https://github.com/awslabs/agentcore-samples/tree/main/01-tutorials/03-AgentCore-identity`

Enterprise integration and industry use cases

Once you deploy an agent, it needs to talk to other systems like databases, internal APIs, third-party services, and event streams. If you are using AgentCore Gateway, it handles a lot of this: point it at an OpenAPI spec and you get tools with auth and error handling built in. The patterns discussed in the following sections apply when you're on Lambda or ECS, or when you need behavior that Gateway doesn't cover.

Databases are often the first integration that breaks at scale. An agent serving forty concurrent users, each triggering two or three queries, will exhaust a typical Amazon Aurora (Amazon's managed relational database) connection pool within minutes. Amazon RDS Proxy, which sits between your agent and the database to manage connection pooling, fixes this by multiplexing hundreds of agent requests over a smaller set of actual database connections. The agent code doesn't change, but the infrastructure absorbs the concurrency. We would recommend that you keep analytical queries (reports, aggregations) off your transactional database entirely and replicate to Redshift or Athena instead. Conversation state maps naturally to DynamoDB, which we covered in *Chapter 3* , *Agent Memory* . Credentials belong in Secrets Manager with automatic rotation, not in environment variables. You can scope the IAM policy to the exact secret ARN the agent needs, and keep database traffic inside a private subnet.

External APIs fail in ways your agent needs to handle gracefully. Internal services go down during deployments. Third-party providers enforce rate

limits and return 429s. Some endpoints just stop responding with no error at all. Set a timeout on every call, retry with exponential backoff, and return a structured error the agent can reason about. `I couldn't complete the underwriting check; proceeding with a preliminary assessment` is better than a `500` error code. Circuit breakers prevent your agent from burning through timeouts when a service is clearly down. For rate-limited APIs, respect `Retry-After` headers and cache responses when the data doesn't change between calls.

Event-driven integration decouples your agent from the systems that feed it. A document lands in Amazon S3, an Amazon EventBridge rule (configured on the bucket) picks up the object-created event and routes it to the agent. The upstream system doesn't know about the agent. The agent doesn't know about the notification system that alerts users when processing finishes. When volume spikes, put SQS between EventBridge and the agent to absorb bursts. Messages that fail three times route to a dead letter queue. A CloudWatch alarm fires, someone investigates, and once the issue is fixed, they replay the messages. For scheduled work like weekly reports or batch processing, EventBridge Scheduler triggers an ECS `RunTask` instead of a Service. `RunTask` spins up, runs the job, and terminates. Ten minutes of Fargate cost instead of 24 hours.

Beyond individual integrations, you'll also need to decide how your agent responds to users and upstream systems. Most production agents handle both synchronous and asynchronous work. The chat interface goes synchronous with streaming so users see progress. Document pipelines and batch jobs go async: the user submits a request, gets a task ID back immediately, and either polls for results or gets a callback when processing finishes. Status updates pushed through a WebSocket keep dashboards current without holding HTTP connections open.

Now that you have a good understanding of the production scenarios, let's explore how the patterns discussed in the previous section play out across four industries, each with different deployment requirements and integration challenges.

Financial services: Fraud detection at transaction speed

A bank's fraud detection system was catching suspicious crypto transactions but flagging legitimate customers so often that `Why can't I buy Bitcoin?` became a daily support ticket. The fraud team was drowning in false positives while actual fraudsters slipped through.

The fraud model itself was solid, a SageMaker endpoint trained on transaction patterns. The problem was context. The model saw each transaction in isolation. It didn't know that a customer had been making similar purchases for months. Figure 6.7 demonstrates the transaction flow:

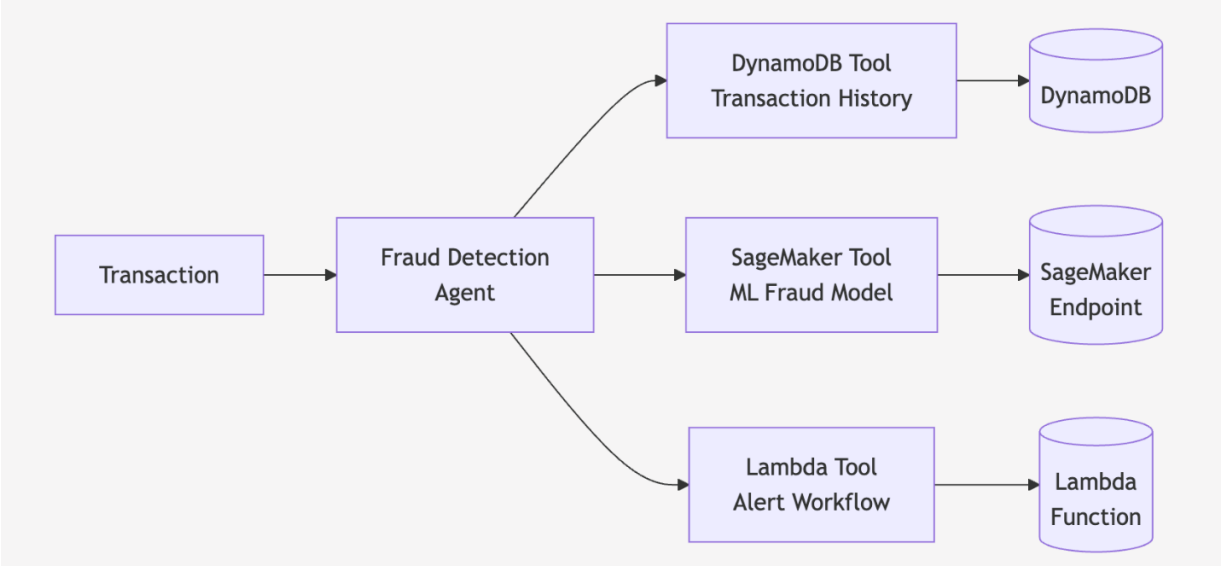


Figure 6.7: Simple Fraud Detection System

When a transaction comes in, the agent follows the steps sequentially: pulls the customer's history from DynamoDB, feeds that context alongside the current transaction to the SageMaker model, and only escalates when the risk score warrants it. High-risk transactions trigger a Lambda function that kicks off the investigation workflow.

The agent doesn't use hardcoded conditional logic, such as `if risk_score > 0.8 then call Lambda` . It reads the tool descriptions and orchestrates them based on the situation.

Deployment fit: AWS Lambda. Each transaction is independent, no multi-turn conversation, no persistent state between invocations. Lambda is a natural fit: the agent processes one transaction, returns a decision, and terminates.

During quiet periods it scales to zero. During market volatility it scales to hundreds of concurrent invocations.

Healthcare: Multi-specialty surgical scheduling

Scheduling surgery requires coordinating a surgeon, an anesthesiologist, possibly a specialist, an available Operating Room (OR), specific equipment that might be in use elsewhere, and a time the patient can actually show up. A large hospital system was doing this manually. Coordinators spent hours on the phone, checking calendars, leaving voicemails. By the time they found a slot, the patient had sometimes scheduled with a competitor. Figure 6.8 shows how an agent-based approach streamlines this workflow:

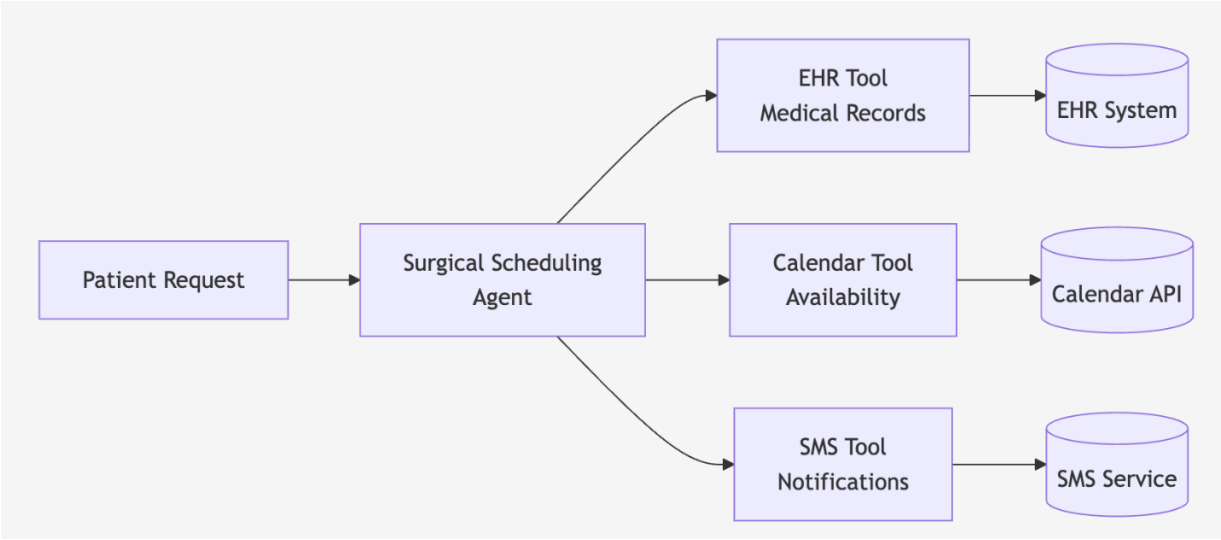


Figure 6.8: Surgical scheduling

When a patient requests surgery, the agent checks the Electronic Health Record (EHR) first. It checks what kind of surgery is needed and which specialists are required. It also looks for equipment requirements. Then it queries multiple calendars simultaneously to find a time when everyone is available. The agent checks the EHR sequentially to understand requirements, then queries multiple calendars in parallel to find overlapping availability. Once it finds a slot, it books the appointments and sends SMS confirmations.

The agent checks equipment availability before booking, and sends reminders at the right intervals. It figured this out from the tool descriptions and past scheduling patterns. You provide the tools, and the model orchestrates them.

A scheduling conversation might run five messages deep before a procedure gets confirmed, so this is a multi-turn workload. AgentCore Runtime fits well here: each coordinator's conversation gets its own isolated session, and the idle timeout matches the natural pace of scheduling (a coordinator might step away to confirm something with a nurse before coming back).

Manufacturing: Predictive maintenance on the factory floor

A factory had expensive CNC machines with bearings that failed without warning. One day the machine runs fine, the next it's down for three days waiting for parts. Each failure cost roughly $50,000 in lost production. Scheduled maintenance (replace bearings every six months regardless) was wasteful, and bearings still failed between replacements. Monitoring vibration sensors manually produced too much data and too many false alarms. Figure 6.9 shows how an agent-based flow for predictive maintenance works:

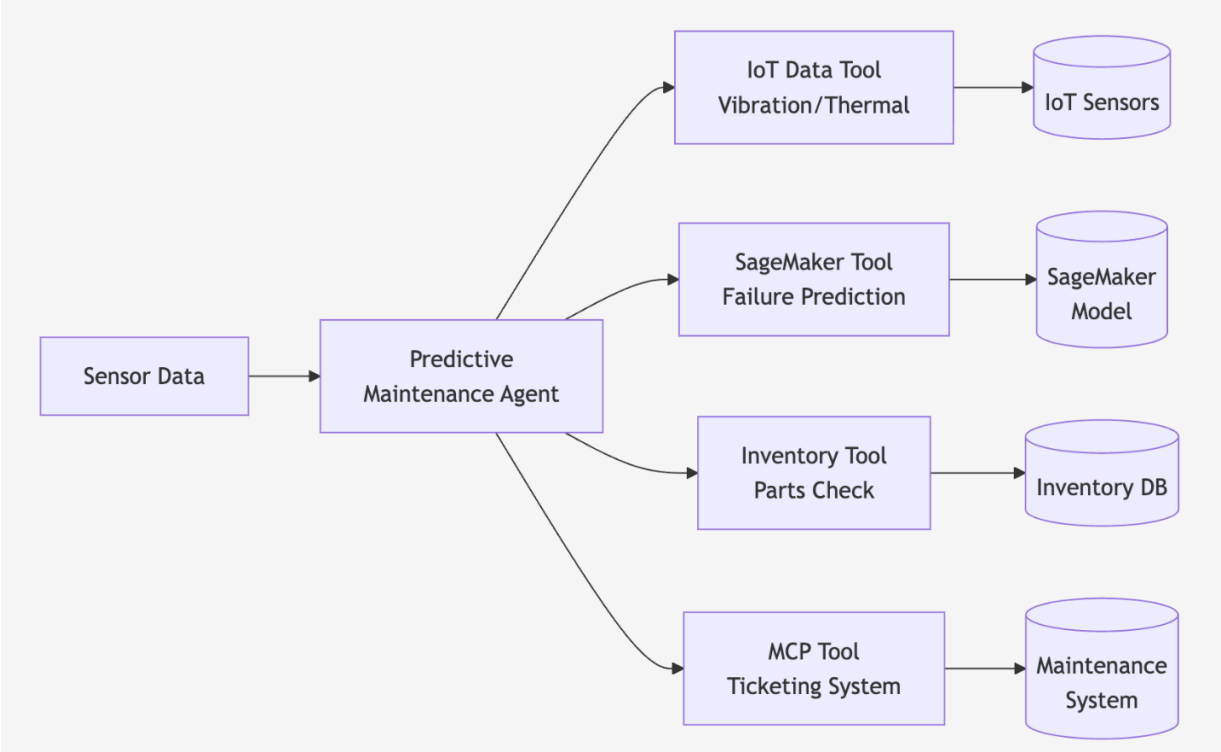


Figure 6.9: Predictive maintenance

The agent monitors vibration sensors continuously, analyzes thermal imaging for heat patterns, and runs everything through a SageMaker model trained to

predict bearing failures. The agent processes sensor data continuously, running vibration analysis and thermal imaging through the model sequentially, then checks inventory and creates tickets only when the risk threshold is crossed. When the risk score crosses a threshold, it checks parts inventory in the warehouse database and creates a maintenance ticket automatically. The ticketing system was built by another team years ago. Rather than writing custom integration code, they connected to it through MCP. The agent treats the ticketing system as just another tool it can call.

The agent also knows to check parts availability before scheduling maintenance and to schedule during low-production periods. It learned this from the tool descriptions and production schedules, not from hardcoded rules.

This is a continuous monitoring workload, so it runs on ECS as a long-lived service. Lambda's 15-minute timeout rules it out for an agent that never stops processing sensor data. AgentCore Runtime is built around per-session isolation, which doesn't map well to a workload with no distinct user sessions.

E-Commerce: Intelligent customer support with specialized agents

An online retailer tried building one agent to handle all customer support: order tracking, product recommendations, and returns. The agent confused return policies with shipping policies, recommended out-of-stock products, and knew a little about everything without knowing enough about anything. Figure 6.10 shows Intelligent customer support with specialized agents:

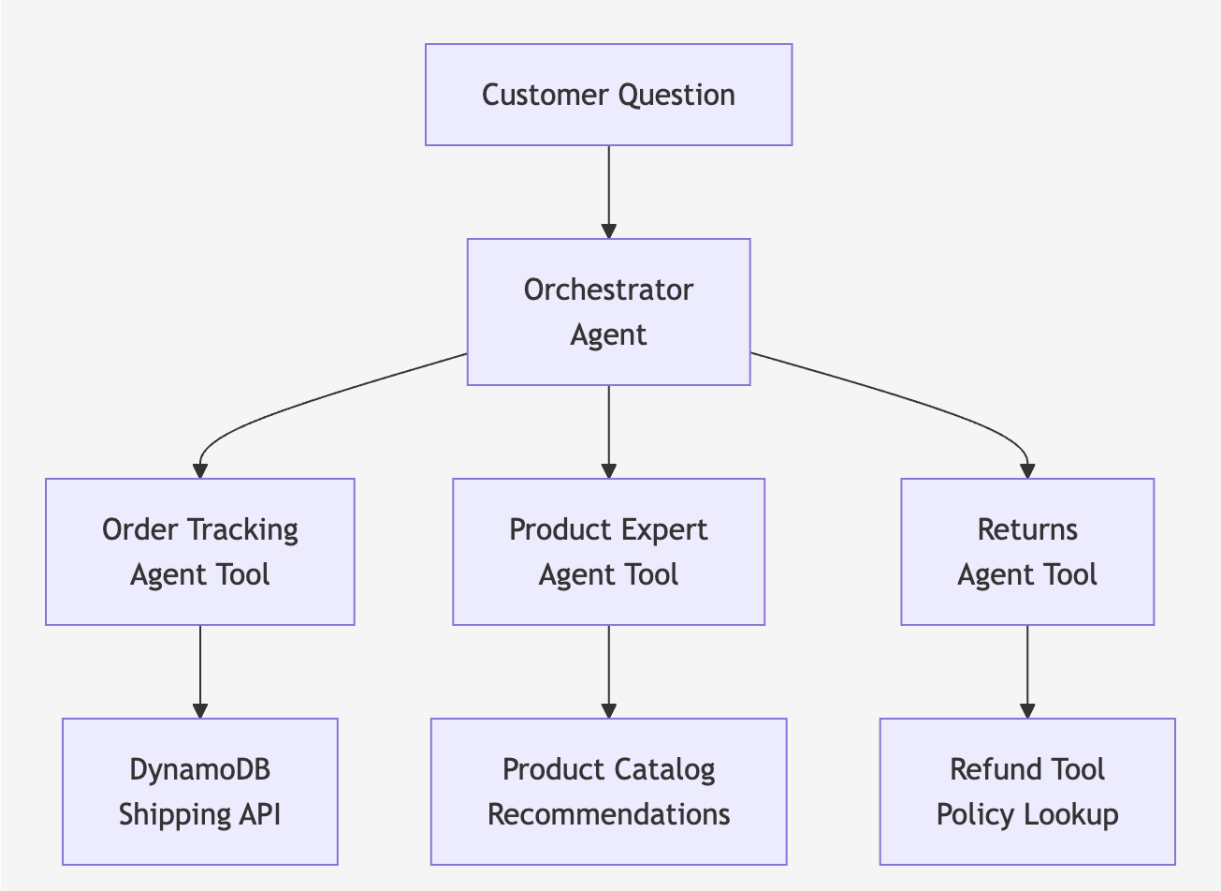


Figure 6.10: Intelligent customer support

The solution was three specialized agents (`Order Tracking`, `Product Expert`, `Returns`) wrapped as tools that an orchestrator agent calls. This is the agents-as-tools pattern from *Chapter 4*, *Advanced Agent Architecture Patterns*. A customer asks about returning an order. The orchestrator follows the sequential flow, calls the `Order Tracking` agent to check shipment status, then calls the `Returns` agent for policy information, and combines both into a single response.

Each agent focuses on what it does best. The orchestrator also uses sentiment analysis to detect frustrated customers who need human escalation.

Customer support is conversational and multi-turn, so AgentCore Runtime is a good fit. Each customer conversation gets its own isolated session, which means no cross-customer data leakage even though the agents share tools that access the same product catalog and order database.

Summary

In this chapter, we were able to get the agents off your laptop and into a production environment in AWS. You know when to reach for Lambda (event-driven, quick tasks), ECS (always-on, multi-agent systems), or AgentCore (managed infrastructure with session isolation, auth, policy, and observability built in). You understand AgentCore's composable architecture—how Runtime, Gateway, Identity, Policy, Memory, Observability, and Evaluations each solve a specific production challenge, and how they work together or independently. You know how to connect agents to databases, APIs, and event-driven systems.

Your agents are running in production. But *deployed* and *production-ready* aren't the same thing. Right now, if your agent starts hallucinating, you won't know until a user complains. If costs spike because a retry loop goes runaway, you'll find out on the monthly bill. If someone discovers your API endpoint, there's nothing stopping them from hammering it with requests.

In the next chapter, we are going to close the gaps: locking down access with IAM least-privilege and OAuth, building observability so you can see what your agents are actually doing in real time, adding guardrails to keep agents within bounds even when prompts get creative, and optimizing costs before finance starts asking questions.

Subscribe to our newsletter

New frameworks, evolving architectures, research drops, production breakdowns— *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe for weekly insights that help you stay focused and informed.

Subscribe at `https://packt.link/8Oz6Y` or scan the QR code below.

OceanofPDF.com

7

Evaluation, Observability, and AI Governance

By this point in the book, you have built a strong foundation in agent development. In *Chapter 1* , *Understanding AI Agents on AWS* , you learned how to design agents. In *Chapter 2* , *Building Agents with Tools* , you explored how to equip them with tools, and in *Chapter 3* , *Agent Memory* , you added memory to make them more context aware and capable over time. In *Chapter 4* , *Advanced Agent Architecture Patterns* , you saw how agents can be orchestrated within multi-agent systems, while *Chapter 5* , *Agent Communication* , introduced how agents communicate using protocols such as MCP and A2A. In *Chapter 6* , *Production Deployment and Enterprise Integration* , you learned how to deploy these agents into enterprise environments.

However, building and deploying an agent is only part of the journey. For agents to succeed in real-world production systems, they must also be secure, observable, reliable, and governed responsibly. Without these operational capabilities, even the most powerful agent can be difficult to trust, monitor, and scale effectively in production.

As agents take on more autonomy, interact with sensitive data, and make decisions across business workflows, new operational challenges emerge. You may need to prevent unsafe or unauthorized behavior or measure an agent's performance. There may be instances when you need to trace failures, investigate hallucinations, monitor tool usage, and maintain visibility into what the agent is doing over time. With all these performance and monitoring tasks, you also require strong governance to ensure that innovation does not come at the cost of compliance, safety, or accountability.

In this chapter, you will learn how to evaluate agent behavior, apply security controls and guardrails, improve performance, and instrument agents for visibility across every step of execution. You will also explore observability tools such as LangSmith, AgentCore Observability, and Langfuse, along with governance practices that help organizations manage risk, maintain compliance, and build confidence in AI-driven systems.

In this chapter, we will cover the following key topics:

- Agent Evaluation
- Observability - Langsmith, AgentCore observability, Langfuse
- AI Governance

Technical requirements

Before diving into this chapter, ensure you have:

- A working Strands development environment
- Basic understanding of Python functions and classes
- AWS account with appropriate permissions (for AWS integration examples)
- AN IDE such as Kiro, VS Code, or Claude Code
- Familiarity with REST APIs and JSON (helpful but not required)

Agent evaluation

By mid-2024, AI agents had become one of the most prominent topics in technology, taking on tasks such as ordering groceries, booking flights, and scheduling appointments. Soon after, vertical AI agents emerged such as clinical documentation agents in healthcare, automated legal research assistants, and AI-powered customer support systems and, in some cases, positioned as alternatives to traditional SaaS applications. However, as interest in agents continues to grow, so does the risk of deploying them before they are ready.

When AI agents are not evaluated thoroughly, they can introduce serious problems, including inaccurate outputs, hidden biases, poor adaptability, and security weaknesses. For example, a customer support agent that confidently provides incorrect refund policies, or a healthcare scheduling agent that books

appointments with the wrong specialist based on misinterpreted symptoms. These issues do more than reduce performance; they can confuse users, erode trust, and raise concerns about fairness and reliability.

Real-world incidents have shown how costly this can become. In one widely reported case, a company's customer service chatbot provided incorrect policy information to a user, who relied on it when making a purchase. When the user later challenged the outcome, the organization was still held accountable for the chatbot's response. Cases like this show that when agents are not evaluated carefully for accuracy, consistency, and policy compliance, the result is not only a poor user experience but also reputational and legal risk.

Why agent evaluation matters?

Developing an agent means preparing it for the unpredictable situations it may encounter in the real world. With LLM evaluation, the goal is to ensure the agent can handle both routine tasks and unexpected edge cases without producing incorrect or unfair outcomes. For example, if an agent evaluates loan applications, it must apply decisions consistently and fairly across applicants. If it acts as a virtual assistant, it should respond effectively not only to common requests but also to unusual or unanticipated questions. Thorough evaluation helps identify and address these issues early, before they affect real users or lead to harmful consequences.

Evaluation is essential for both compliance and trust. In fields such as healthcare and finance, AI systems must meet strict requirements around privacy, safety, and fairness. Demonstrating that an agent has been tested against these standards gives regulators, stakeholders, and users confidence that it has been carefully validated.

Ongoing evaluation is equally important because real-world conditions change over time. Even if an agent performs well initially, shifts in user behavior, data, or business needs can introduce new weaknesses. Regular testing or evaluation helps identify performance drops, missed scenarios, and emerging biases early, allowing the agent to remain reliable and effective.

Agent evaluation use-case

Suppose you ask a travel agent to book a trip to San Francisco as shown in the following figure. Behind the scenes, it must first understand your request,

choose the right tool, gather details such as flights or hotels, and then return a clear and accurate response. It may also need to ask follow-up questions to fill in missing information.

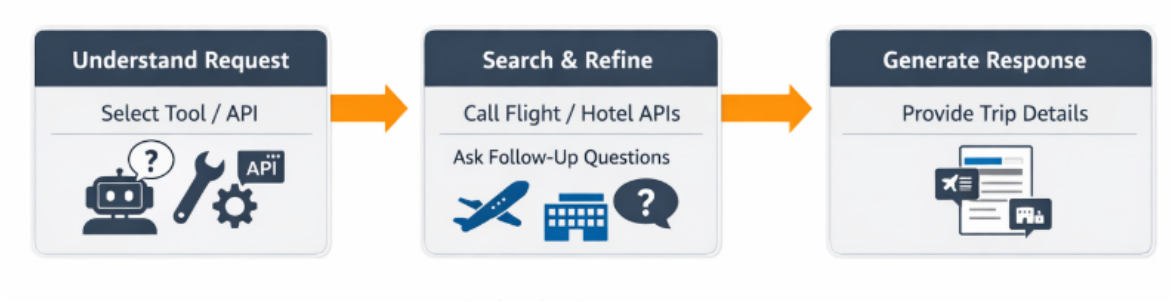


Figure 7.1: How agents understand, search tools, and generate response

In this system, there's plenty that can go wrong. For example, the agent might book flights to San Diego instead of San Francisco. It's not only important to evaluate the LLM's output, but also how the agent decides on each action. You might find that the agent is calling the wrong tool, misusing context, or even using an inappropriate tone. Sometimes users will also try to manipulate the system, which can create unexpected outputs.

Let's see how you can evaluate this agent for **book a trip to San Francisco** request from a user. *Figure 7.2* shows the evaluation process for this agent:

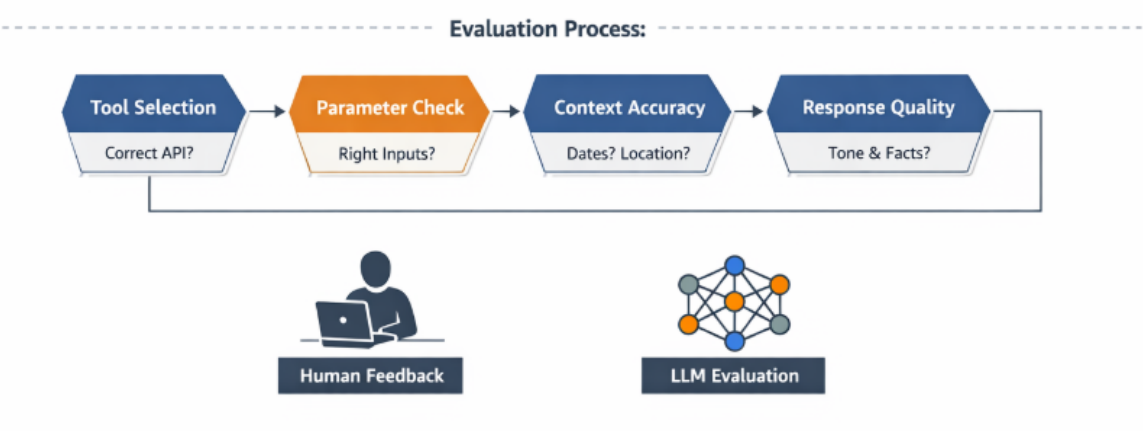


Figure 7.2: Evaluation process

Following are the prominent steps in the agent evaluation process, as shown in *Figure 7.2* :

- **Tool Selection** : Checks whether the agent chose the correct API or tool.

- **Parameter Check** :Verifies that the agent passed the right inputs (dates, location, preferences).
- **Context Accuracy** : Ensures the agent used the user's context correctly (e.g., San Francisco vs. another city).
- **Response Quality** : Evaluates whether the final answer is accurate, clear, and has the right tone.

At the bottom of the diagram, two evaluation approaches for agent evaluation are shown:

Human Feedback (Human-in-the-loop) : People review outputs for correctness, tone, and edge cases , and **LLM Evaluation (LLM-as-a-judge)** where another model automatically scores the agent's behavior and responses.

The travel-booking example shows that agent evaluation must examine not only what the agent says, but also how it behaves throughout the workflow. To understand whether the agent selected the right tool, passed the correct parameters, used context properly, and produced a reliable response, we need visibility into the full execution path. This is where observability and instrumentation become the foundation for agent evaluation.

Relationship between agent observability and evaluation

Agent evaluation depends on visibility into what happened during an agent run, not just the final response. This is where instrumentation comes in: it captures the telemetry needed to inspect agent behavior, analyze execution traces, and evaluate whether the agent made the right decisions. We are going to deep dive into observability in the next section and see how to set it up. In this section, we will cover some high-level concepts which are important for agentic evaluation.

Figure 7.3 shows the three components involved in capturing and analyzing agent behavior:

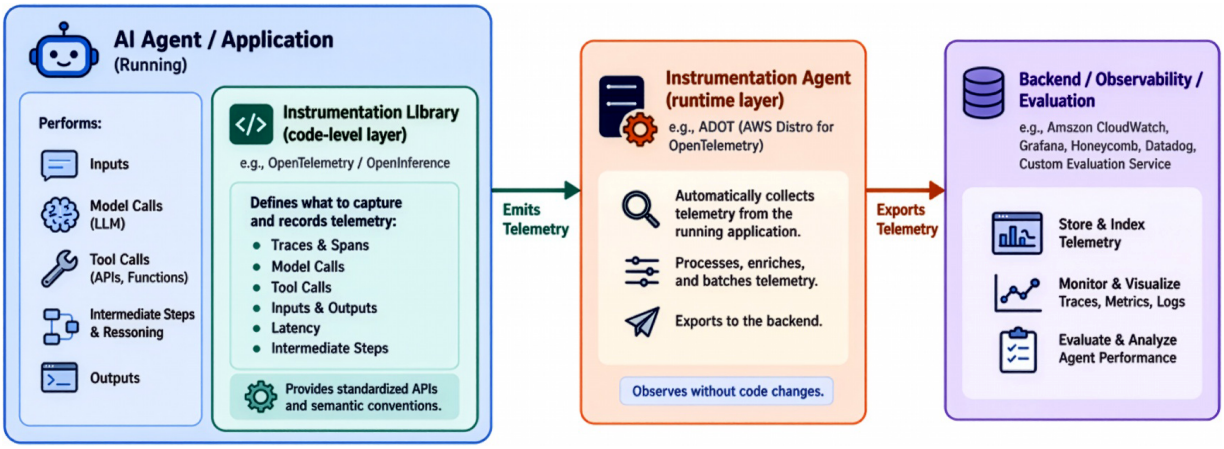


Figure 7.3: Agent with instrumentation library interacting with Instrumentation Agent

The preceding figure shows the AI agent application where telemetry is generated, the instrumentation agent that collects and exports it, and the observability backend where it is stored, visualized, and evaluated.

The **instrumentation library** and **instrumentation agent** work together, but they play different roles.

- An **instrumentation library** is the code-level layer that defines *what telemetry should be captured* from your AI agent. It records information such as traces, spans, model calls, tool calls, inputs, outputs, latency, and intermediate execution steps. For example, libraries such as **OpenTelemetry** or **OpenInference** provide standard APIs and naming conventions so that agent behavior can be captured consistently across different tools and platforms.
- An **instrumentation agent** is the runtime layer that helps collect and export that telemetry automatically. Instead of requiring developers to manually add tracing code throughout the application, an instrumentation agent can observe the running application, capture telemetry, process it, and send it to a backend system for storage, monitoring, or evaluation. For example, **ADOT (AWS Distro for OpenTelemetry),** AWS's distribution of the open-source OpenTelemetry project, can auto-instrument supported applications and export traces to observability backends.

Once captured, this telemetry is organized into a hierarchy of sessions, traces, and spans:

- A **session** sits at the highest level. It groups related interactions from a user or workflow, such as a multi-turn conversation with a travel-booking agent. Within a session, the agent may perform several separate executions.
- Each of those executions within a session is captured as a **trace** . A trace gives an end-to-end view of one agent run, including the user request, model calls, tool calls, intermediate steps, latency, errors, and final response.
- Inside each trace are **spans** , which represent individual operations. For example, one span may capture a model call, another span may capture a retrieval step, and another may capture a **tool call** . A tool call is therefore a specific type of span that records when the agent invokes an external function, API, database, search system, or other capability. This makes it possible to evaluate not only what the agent answered, but also how it arrived at that answer.

This is where evaluation connects to observability. Once instrumentation captures sessions, traces, spans, and tool calls, evaluation systems can analyze them.

For example, an evaluator can check whether the agent selected the right tool, passed the right parameters, handled the tool output correctly, and produced an accurate final response. An **LLM-as-a-judge** can then review the captured trace or final output against a rubric, even when no ground-truth answer is available. It can score qualities such as correctness, reasoning quality, instruction adherence, tool-use accuracy, and task completion.

In simple terms, the relationship can be represented as the following figure:

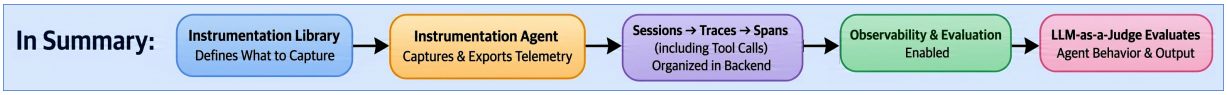


Figure 7.4: Relationship between instrumentation library, agents, session, traces, span, and evaluation

Instrumentation library defines what to capture, **i nstrumentation agent** collects and exports it, **s essions** group related interactions, **t races** record individual agent executions, **s pans** capture each operation, **Tool call spans** show external actions, and **LLM-as-a-judge evaluators** assess the agent's behavior and output.

Let's understand this by an example, A user starts a new session by asking a simple question: **" Will it rain in Boston tomorrow? "** The request goes to the **AI agent/application** , which interprets the question, decides that it needs external weather information, and prepares the parameters for a weather tool call. Figure 7.5 explains how an AI agent's execution becomes visible and measurable through **instrumentation, telemetry, observability, and evaluation :**

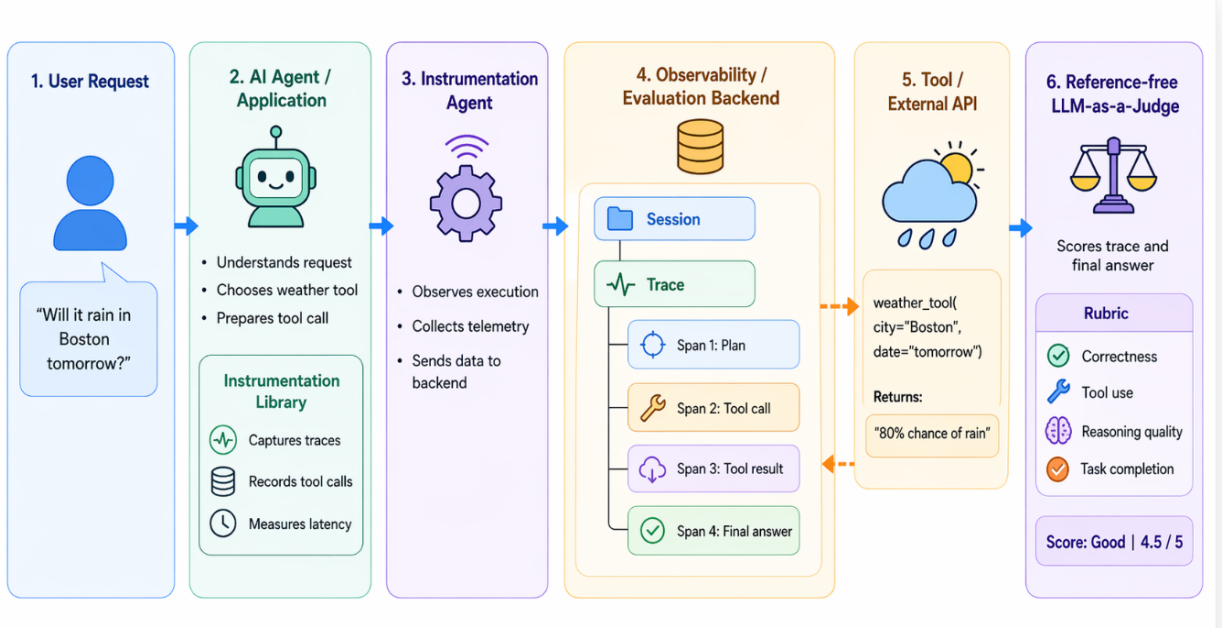


Figure 7.5: Example of a weather agent with instrumentation library, session, traces, span and evaluation

The preceding figure highlights the following observations:

- Inside the agent application, the **instrumentation library** captures the fact that the agent received a query related to weather, selected a weather tool, passed city="Boston" and date="tomorrow", received a tool result, and produced a final response.
- I **nstrumentation agent** collects the telemetry generated by the instrumentation library and sends it to the observability or evaluation backend. This means developers do not need to manually inspect every step of the agent's behavior; the execution details are automatically captured and exported.
- **O bservability/evaluation backend** stores the captured telemetry in an organized structure. The session-001 could represent the user's

weather-planning conversation. trace-001 could represent the single request: **" Will it rain in Boston tomorrow? " .** Span identifies the individual steps inside that trace, such as planning, calling the weather tool, receiving the tool result, and generating the final response.

- The **tool/external API** box shows the actual external call made by the agent. The agent calls the weather tool using the parameters city="Boston" and date="tomorrow". The tool returns a result, such as **" 80% chance of rain. "** The agent then uses that result to generate the final answer: **" Yes, it is likely to rain in Boston tomorrow. "**
- The **LLM-as-a-judge** evaluates the captured trace and final output without needing a predefined ground-truth answer. It uses an evaluation rubric to check whether the agent chose the right tool, passed the correct parameters, used the returned result properly, and produced an accurate and helpful final response.

Together, instrumentation, telemetry, and evaluation make agent execution visible, so teams can inspect how the agent reasoned, which tools it used, how long each step took, where failures happened, and whether the final outcome met the expected goal.

Evaluation metrics for evaluators

Evaluators are the core components that assess your agent's performance across different dimensions. They analyze agent traces and provide quantitative scores based on specific criteria such as helpfulness, accuracy, or custom business metrics.

Generally, there are two types of evaluation metrics, built-in metrics and custom metrics when evaluating an agentic system.

Built-in evaluators are ready-to-use evaluation options provided by evaluation platform like AgentCore Evaluations that use large language models (LLMs) as judges to assess agent performance. They include predefined settings such as prompt templates, evaluator models, and scoring criteria. Examples include GoalSuccessRate, which measures whether the agent achieved the user's objective across a session, Correctness, which checks factual accuracy of individual responses, and Helpfulness, which scores how useful the response is to the user.

These evaluators are useful for common evaluation scenarios where you want consistent and reliable results without creating your own evaluation logic.

Built-in evaluators can run on agent traces without a labeled dataset for general quality checks such as helpfulness and response relevance **.** However, when you want to test against a known expected answer, expected behavior, or expected tool path, you provide ground truth data.

Custom evaluators are especially valuable when evaluating domain-specific agents or when your organization has specialized quality standards. There are two types of custom evaluators:

- **LLM-as-a-judge evaluators** let you define your own evaluator model, instructions, criteria, and scoring schema. This is useful when you want the evaluation to reflect your specific task, domain, or quality expectations.
- **Code-based evaluators** let you use a function to evaluate agent behavior programmatically. This approach is useful for deterministic checks, such as validating exact values, matching patterns with regular expressions, calling external APIs, calculating custom metrics, or applying business-specific rules. We will cover both examples in the next section.

For example, a healthcare agent may need criteria focused on clinical accuracy and safety, while a finance agent may need checks for compliance, numerical correctness, and policy alignment.

Table 7.1 summarizes these two scenarios:

Scenario	Evaluator to use	Why
Customer support agent answering general product questions	Built-in evaluator	The team wants to quickly measure common quality dimensions such as helpfulness, relevance, clarity, and whether the final response addresses the user's question. These are standard evaluation needs, so a ready-to-use built-in evaluator is a good fit.

Scenario	Evaluator to use	Why
Healthcare scheduling agent that must follow strict business rules	Custom evaluator	The team needs to check domain-specific requirements, such as whether the agent verified the patient's insurance, avoided giving medical advice, selected the correct clinic location, and followed required escalation rules. These checks are specific to the organization and may require custom scoring logic.

Table 7.1: Evaluation metrics for different agent scenarios

Understating types of agent evaluation

There are commonly two types of Agent evaluation, Online evaluation and On-demand evaluation.

Online evaluation continuously monitors deployed agents using live production traffic. It allows teams to sample a percentage of sessions or apply filters to choose which interactions to evaluate. Teams can use built-in or custom evaluators, view aggregated scores in dashboards, track quality trends, and investigate low-scoring sessions.

On-demand evaluation lets teams evaluate specific traces or spans at any time. Instead of continuously monitoring production traffic, you select the exact span IDs or trace IDs you want to analyze. This is useful for testing a custom evaluator, investigating a customer issue, validating a fix, reviewing historical interactions, or performing build-time testing during development.

In short, **online evaluation is best for continuous production monitoring** , while **on-demand evaluation is best for targeted analysis of selected interactions** .

Note: Online evaluation and on-demand evaluation both support built-in and custom evaluators. Online evaluation applies them continuously to sampled or filtered production sessions, while on-demand evaluation applies them to specific traces or spans selected for targeted analysis.

Agent evaluation at a glance

So far, we have established that agent evaluation is not limited to judging the final response. It requires visibility into the full agent execution, including sessions, traces, spans, tool calls, intermediate steps, and outputs. We also saw how instrumentation and observability make this execution data available for evaluation, and how built-in and custom evaluators can be applied through online or on-demand evaluation. With this foundation in place, we can now move from the concepts of agent evaluation to the practical details of how these evaluators are configured and used in real-world agent workflows. *Table 7.2* provides a **quick recap of the main agent evaluation concepts** before moving to the next section.

Topic	Summary
Agent evaluation goal	Evaluates whether the agent completed the user's goal and followed the right process, not just whether the final response looked good.
Observability connection	Observability provides the execution data needed for evaluation, such as traces, spans, tool calls, latency, errors, and outputs.
Instrumentation	The instrumentation library defines what telemetry to capture, while the instrumentation agent collects and exports that telemetry to an observability or evaluation backend.
Session	Groups related user interactions or workflow steps, such as a multi-turn conversation.
Trace	Captures one complete agent execution inside a session, from user request to final response.
Span	Represents one operation inside a trace, such as model reasoning, retrieval, tool invocation, or final response generation.
Built-in evaluators	Ready-to-use evaluators for common quality checks such as helpfulness, relevance, correctness, clarity, and task completion.
Custom evaluators	Flexible evaluators for domain-specific criteria, business rules, custom scoring, deterministic checks, or external validation.

Topic	Summary
LLM-as-a-judge evaluator	Uses an LLM to score agent behavior based on custom or predefined instructions, criteria, and rubrics.
Code-based evaluator	Uses programmatic logic, such as an AWS Lambda function, to evaluate exact matches, regex patterns, numerical checks, API results, or business rules.
Online evaluation	Continuously evaluates sampled or filtered production sessions to monitor agent quality over time.
On-demand evaluation	Evaluates selected traces or spans for debugging, testing, validation, or targeted analysis.

Table 7.2: Agent evaluation concepts

AgentCore Evaluations providing a managed service for evaluations

Amazon Bedrock AgentCore Evaluations is a managed service that lets you evaluate AI agents built on any popular orchestration framework, including Strands and LangChain.

You can evaluate the AI agents that are hosted under AgentCore Runtime as well as AI agents hosted outside of AgentCore Runtime.

AgentCore Evaluations supports the key agent evaluation capabilities discussed so far as a managed serverless offering via APIs. It can take traces collected through an observability system and evaluate them using built-in or custom metrics. These evaluations can be run on demand during development or continuously in production.

Whether your agent runs inside AgentCore Runtime or outside it, the process involves capturing OTEL traces, sending them to AgentCore Observability, and then use those traces for evaluation.

Collecting traces for Agent evaluation via AgentCore Observability

Let's understand how you can collect traces from agents via AgentCore Observability. In this section we will have a high-level understanding of how

the Agent evaluation is dependent on Observability. We will deep dive into AgentCore Observability in the upcoming sections.

The following figure shows how **AgentCore Observability** collects trace data from agents in two different deployment setups and makes that data available for monitoring and evaluation.

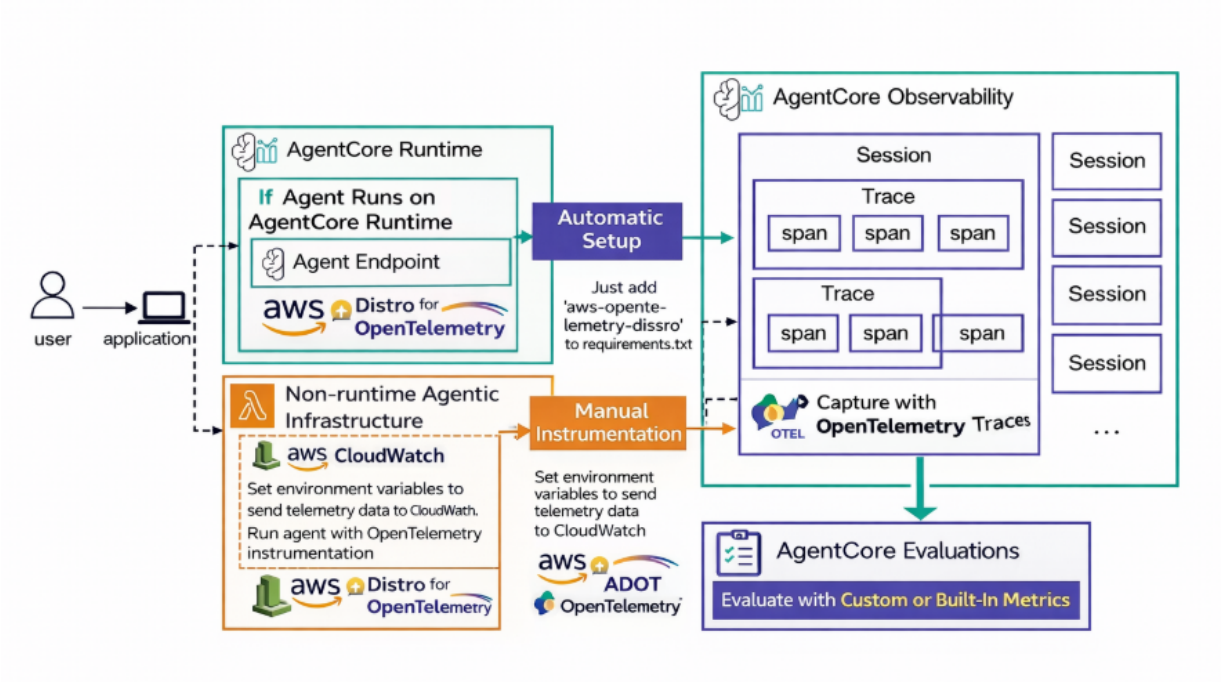


Figure 7.6: Relationship between Agentcore Runtime, Agentcore Observability and Evaluations

AgentCore Observability gives you detailed visibility into how an agent behaves during each

invocation. It uses **OpenTelemetry (OTEL)** traces to capture and organize execution details. To collect these traces across different agent frameworks, AgentCore uses AWS Distro for **OpenTelemetry (ADOT)** .

- **Automatic observability setup for agents running on AgentCore Runtime:** If your agent runs on AgentCore Runtime, observability setup is mostly automatic. You only need to include `aws - opentelemetry -` distro in your `requirements.txt` , and AgentCore Runtime takes care of the OTEL configuration for you.
- **Manual observability setup for agents running outside AgentCore Runtime:** If your agent runs outside AgentCore Runtime, you must instrument it yourself using ADOT so its traces can be sent to

AgentCore Observability. This requires setting environment variables to send telemetry data to CloudWatch and starting your agent with OpenTelemetry instrumentation enabled.

Now with the understanding how we can collect traces for agent evaluation, let's understand the metrics offered by AgentCore Evaluation in next section.

NOTE

For this chapter, we will show you hands on how you can do automatic observability setup for agents running on AgentCore runtime. Manual setup is out of scope for this chapter.

Agentcore evaluation levels

AgentCore Evaluations operate in different levels of the agent interaction. You can analyze the back and forward conversation as a whole using the session information. You can also evaluate the agent's response to a user's question in an individual turn of the conversation using the trace information. Or you can evaluate information inside of a turn, which includes the tool calling and parameter selection, using the span data.

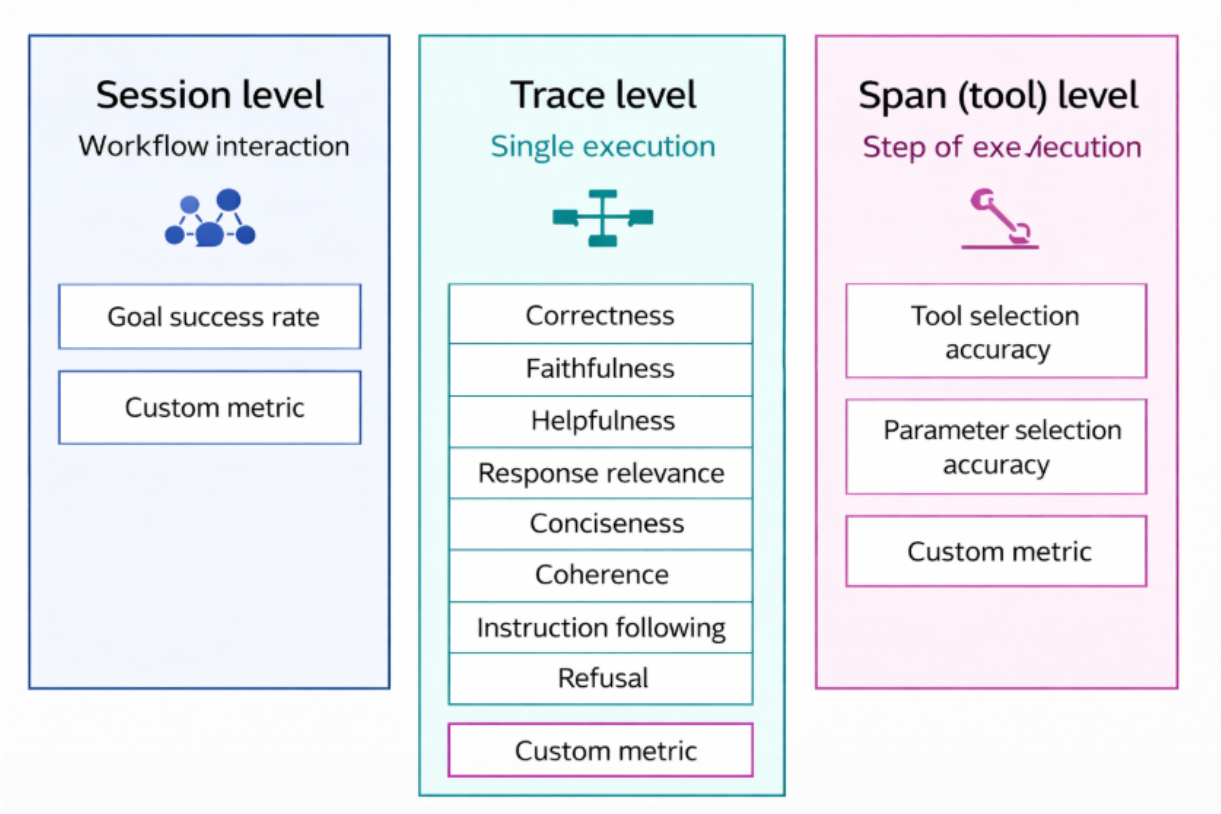


Figure 7.7: Evaluation levels

The preceding diagram shows that agent evaluation metrics can be applied at **three different levels of execution detail** : the **session level** , **trace level** , and **span or tool level** .

- At the **session level** , the metric looks at the overall outcome across a broader interaction or workflow. In *Figure 7.7* , the built-in metric **goal success rate** is an example of a session-level metric because it measures whether the agent ultimately achieved the user's objective. This level can also include **custom metrics** that are designed for a specific business workflow or use case.
- At the **trace level** , the metric evaluates a single end-to-end agent run. These metrics focus on the quality of the agent's response and behavior throughout one execution. Examples shown here include correctness, faithfulness, helpfulness, response relevance, conciseness, coherence, instruction following, refusal and harmfulness.. This level is useful when you want to judge how well one complete invocation performed from start to finish. The preceding diagram also shows that you can define **custom metrics** at this level.
- At the **span or tool level** , the metric examines an individual step inside the trace, especially a tool-related step. Examples include **tool selection accuracy** , which checks whether the agent chose the right tool, and **parameter selection accuracy** , which checks whether it passed the right inputs to that tool. This is the most granular level and helps diagnose where an agent went wrong inside a workflow. As with the other levels, **custom metrics** can also be added here.
- Amazon Bedrock AgentCore supports various interfaces for developing, deploying and monitoring your agents and tools. For a simplified approach, you can also use the AgentCore Python SDK and the AgentCore Starter Toolkit. The AgentCore Python SDK provides Python primitives for agent development and the AgentCore Starter Toolkit provides CLI tools and higher-level abstractions for AgentCore functionalities.

NOTE

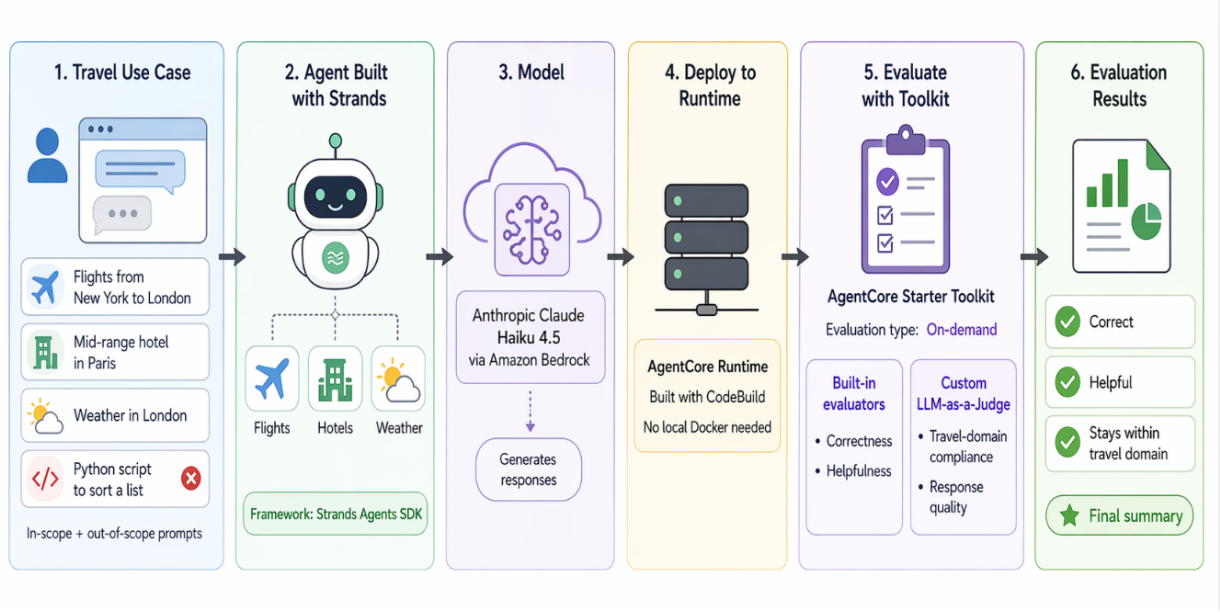
Check the following GitHub repo to learn how to use the AgentCore Python SDK: https://github.com/aws/bedrock-agentcore-sdk-python

And to get started with the AgentCore Starter Toolkit, check this resource: https://github.com/aws/bedrock-agentcore-starter-toolkit

In the next section we will show you how you can evaluate and agent using AgentCore starter toolkit.

Hands-on using AgentCore evaluations to evaluate travel assistant

To make these evaluation concepts more concrete, let's walk through a simple end-to-end example. In this tutorial, the agent is a travel assistant that helps users with flights, hotels, and weather-related questions. Just as importantly, it is also expected to decline requests that fall outside the travel domain. This makes it a useful example for evaluation because we are not only testing whether the agent is helpful and accurate, but also whether it behaves within its intended scope.



This diagram shows the end-to-end evaluation flow for the travel assistant example. This is what we will build in this section:

- **Travel use case:** The travel agent with three tools is tested with both travel and non-travel prompts.
- **Strands agent:** Orchestrator layer to build a travel assistant is built with tools for flights, hotels, and weather.
- **Model layer:** Anthropic Claude Haiku 4.5 on Amazon Bedrock generates responses.
- **Deployment:** The agent is deployed to AgentCore Runtime using CodeBuild.
- **Evaluation:** AgentCore Starter Toolkit runs both built-in and custom evaluators.
- **Built-in metrics:** Check general qualities such as correctness and helpfulness.
- **Custom evaluator:** Checks travel-domain compliance and response quality.
- **Results:** Show whether the agent is correct, helpful, and stays within scope.
- **Key idea:** The diagram shows how a travel agent is built, deployed, and evaluated end to end using both general metrics and domain-specific rules.

This example follows a workflow that many teams use in real projects. You will first deploy the agent to a managed runtime environment, then define a custom evaluator that acts as an LLM-as-a-judge, invoke the agent with a small set of test prompts, and finally run both built-in and custom evaluations on the collected traces. By the end, you will see how evaluation works across the full lifecycle, from deployment to scoring, and why custom evaluators are often necessary to capture application-specific expectations.

Prerequisites:

- Python 3.10+
- AWS credentials configured (aws configure or environment variables)
- IAM permissions: `bedrock- agentcore :*, bedrock- agentcore - control:*, ecr :*, iam:CreateRole , codebuild :*, s3:*, logs:*`

Follow readme.md to get started and git clone the repository and go to *C hapter 7*

Following is the chapter directory and file structure:

```
chapter 7/agent-evaluation/
├── end_to_end_eval.ipynb       # Main notebook — run this top to bottom
├── agent_app.py                # Travel assistant agent (deployed to AgentCore)
├── travel_quality_metric.json  # Custom LLM-as-judge evaluator config
├── requirements.txt            # Python dependencies
└── README.md                    # Setup and quick start guide
```

Open `end_to_end_eval.ipynb` (https://github.com/PacktPublishing/Agents-on-AWS/blob/main/chapter%207/agent-evaluation/end_to_end_eval.ipynb) and go through the following steps.

Step 1: Setup

To get your AWS region and account ID, import the following packages:

```
from bedrock_agentcore_starter_toolkit import Evaluation, Runtime
```

These are used throughout to construct ARNs and target the right AWS endpoints.

Step 2: Understand the agent (agent_app.py)

`agent_app.py` is a separate file we will deploy via notebook. A travel assistant built using Strands with three tools:

```
@tool
def get_flight_info(origin: str, destination: str) -> str:
    # Looks up a hardcoded dictionary of routes
    # Returns flight number, times, duration, price
    pass
@tool
def get_hotel_recommendations(city: str, budget: str = "medium") -> str:
    # Nested dict: city → budget level → hotel description
    # Supports: london, paris, new york
    pass
@tool
def get_weather_forecast(city: str) -> str:
    # Simple dict lookup: city → weather string
    pass
```

The agent has a system prompt that restricts scope:

```
system_prompt=(
    "Only answer questions related to travel — flights, hotels, and weather. "
```

```
    "For anything outside travel, politely decline."
)
```

Step 3: Deploy the agent

Under the hood, the following code generates a `Dockerfile`, uploads source to `S3`, triggers `CodeBuild` to build an `ARM64` image, pushes to ECR, and creates an `AgentCore` Runtime endpoint:

```
runtime.configure(entrypoint="agent_app.py", agent_name="ch7_travel_agent", ...)
launch_result = runtime.launch()
```

Here, no local Docker needed. It takes approximately 10 minutes the first time. The `wait_for_ready()` loop polls every 15 seconds until status is READY.

Step 4: Create a custom evaluator

The following code loads `travel_quality_metric.json` and registers an LLM-as-judge evaluator:

```
eval_client.create_evaluator(name="travel_response_quality", config=eval_config)
```

The response from `travel_response_quality.json` judge prompt is as follows:

```
Evaluate whether the response:
1. Accurately answers the travel-related question
2. Stays within the travel domain
3. Provides actionable travel information

IMPORTANT: If the assistant answers non-travel questions, classify as Very Poor.
Context: {context}
Candidate Response: {assistant_turn}
Scoring scale: 1.0 Very Good → 0.75 Good → 0.5 OK → 0.25 Poor → 0.0 Very Poor
```

Step 5: Define test prompts and invoke the agent

All 4 prompts share the same session_id so they're grouped as one conversation for evaluation:

```
test_prompts = [
    "What flights are available from New York to London?",     # in scope
    "Can you recommend a mid-range hotel in Paris?",           # in scope
    "What is the weather like in London right now?",           # in scope
    "Can you help me write a Python script to sort a list?",   # out of scope
]
runtime.invoke(payload={"prompt": prompt}, session_id=session_id)
```

Step 6: Wait for traces

AgentCore Runtime emits OpenTelemetry spans to CloudWatch Transaction Search after each invocation. The evaluation service reads these traces. Add the following code to wait around 3 minutes for ingestion:

```
time.sleep(180)
```

Step 7: Run evaluations (3 passes)

After the agent has been invoked and the traces have been collected, the next step is to evaluate its performance. This walkthrough uses three evaluation passes because no single metric can fully capture agent behavior. Some metrics assess whether the overall conversation was successful, while others examine the quality of individual responses or check whether the agent followed application-specific rules. Using multiple passes provides a more complete and reliable view of performance.

Pass A—GoalSuccessRate (session level)

The first pass looks at the entire conversation as a whole and asks one question: "Did the user get what they needed from this session?" It reads all 4 turns together and gives a single score. Think of it as the overall customer satisfaction score for the whole chat.

This first pass uses GoalSuccessRate, a built-in session-level evaluator. Session-level evaluators examine the entire conversation as a whole rather than scoring each turn separately. Here, GoalSuccessRate reads all four turns together and asks a simple question: Did the conversation achieve its overall goal?

This makes GoalSuccessRate useful for measuring end-to-end success from the user's perspective. Rather than focusing on one reply in isolation, it evaluates whether the full interaction was successful overall. In practice, this makes it similar to an overall task-success or customer-satisfaction indicator for the session.

Because it operates at the session level, this pass produces one score for the entire conversation.

```
eval_client.run(evaluators=["Builtin.GoalSuccessRate"])
```

Pass B—Correctness + Helpfulness (trace level)

This second pass scores each individual turn for factual accuracy and usefulness. It uses the built-in **Correctness** and **Helpfulness** evaluators. These are **trace-level evaluators** , meaning they score each turn individually.

Correctness measures whether the response is factually accurate and appropriate for the request. **Helpfulness** measures whether the response is useful and provides meaningful assistance. Together, these metrics help answer the question: **Was each individual response accurate and useful?**

This pass is important because an agent may appear successful at the conversation level while still performing unevenly on specific turns. Trace-level evaluation helps identify those strengths and weaknesses more precisely.

```
eval_client.run(evaluators=["Builtin.Correctness", "Builtin.Helpfulness"])
```

Pass C—Custom evaluator (trace level)

The third pass runs the custom evaluator defined earlier. Like Pass B, this is also a **trace-level evaluator** , but instead of using only general-purpose quality measures, it applies **domain-specific criteria** .

It runs the LLM-as-judge on each turn which is using your LLM-as-judge defined in travel_quality_metric.json. The judge model (Claude Haiku) reads each response and applies your specific rules including the critical rule that any out-of-scope answer scores 0.0.

```
eval_client.run(evaluators=[evaluator_id])
```

Step 8: Summary table

This is why the Python scripting turn scores **Very Poor** here even if Pass B gave it an OK score. Pass B's built-in evaluators don't know your business rule about staying in the travel domain your custom evaluator does.

```
Evaluator                       Label          Score
Builtin.GoalSuccessRate         Good            0.75  ██████
Builtin.Correctness             Very Good       1.00  ████████
travel_response_quality         Very Poor       0.00
```

Step 9: Save results

At last, run the final evaluation and saves the results:

```
eval_client.run(..., output="eval_output/chapter7_results.json")
```

Key insight

The custom evaluator catches what built-in evaluators miss, the Python question might score OK on Correctness (the agent correctly refused), but the custom evaluator penalizes any out-of-scope response regardless of how politely it was declined.

With the evaluation concepts in place, the next step is to understand observability, because the traces and telemetry collected during agent execution provide the foundation for measuring how well an agent actually performs.

Observability: Seeing what your agents do

In *Chapter 6* , *Production Deployment and Enterprise Integration* , we have deployed our agents – Lambda for event-driven t asks, ECS for always-on workloads, AgentCore for AWS managed stack. We ran the deployment scripts and the agent was up and running and responding to the invocations.

But "up and running" is not the same as "running well." Latency might start climbing over time and invocations might start failing intermittently. Token costs might spike without any obvious change in traffic. A lot can go wrong in production, as with any software system, and most of it won't announce itself with a clean error message.

Let's consider a team at a fintech company that deployed a loan underwriting agent. Initially the agent worked well for three weeks. Then one morning, loan application approval times jumped from eight seconds to twenty. No errors in the logs. No failed requests. The agent was still returning correct decisions, but it was slow. It took a day of digging to figure out what really happened. The agent had started making redundant tool calls: checking the same credit bureau three times per application because the model was not recognizing that it already had the information it needed. The model had not broken. The tools had not broken. The interaction pattern between them had shifted, and nobody could see it because they were only logging inputs and outputs.

This is the core challenge with agent observability. Traditional application monitoring watches HTTP status codes, response times, and error rates. That is necessary but nowhere near sufficient for agents. An agent that returns a 200 OK might have hallucinated an answer, called the wrong tool, ignored relevant context from memory, or consumed twelve dollars in model tokens on a request that should have cost thirty cents. You cannot see any of that from a health check endpoint.

Agent observability means understanding the reasoning process, not just the result. Which tools did the agent call, and in what order? What did the model see in its context window? How many tokens went in, how many came out, and what did that cost? When the agent made a decision, what information was it working with? If you cannot answer these questions for any given request, you don't have complete visibility into the system.

What you need to capture

Before we look at specific tools, let's understand what **observability** means for an agentic system. There are four layers to agent observability, and each one gives you a different level of insight into what your agent is doing:

Layer 1: Infrastructure metrics

This covers CPU, memory, network, and container health. Amazon CloudWatch gives you this out of the box. It is useful for knowing whether your compute is healthy, but it says nothing about whether your agent is doing the right thing.

Layer 2: Request-level telemetry

This includes latency per request, error rates, and throughput. This is standard API monitoring. You will know *that* something is slow or broken, but not *why* .

Layer 3: Agent execution traces

A trace captures the full lifecycle of a single agent invocation: the user's input, the system prompt, every model call with its input tokens and output tokens, every tool invocation with parameters and results, and the final response. When that loan underwriting agent started making triple credit checks, a

trace would have shown it immediately. Three identical tool calls in a single invocation, each with the same parameters and the same results.

Layer 4: Behavioral analytics

This is about patterns across many traces. This layer allows you answering questions such as: Are tool call counts drifting upward over time? Is the agent using certain tools less than expected? Are there prompt patterns that consistently lead to poor outcomes? This layer is how you catch slow degradations that never show up as errors in any single request.

Most of this section focuses on Layer 3, execution traces, because this is the layer that is unique to agentic systems. Infrastructure metrics and request-level telemetry work the same way for agents as they do for any other application. But tracing the reasoning, tool calls, and decision steps inside an agent invocation requires agent-specific instrumentation, and this is where the observability tools we cover next provide the most value.

Before we go further, it is worth reminding a few terms we covered in previous section:

A **session** represents a user's conversation with an agent. When a user opens a chat and asks multiple questions, that entire back-and-forth is one session, connected by a session ID.

Each time the user asks a question within that session, everything the agent does to answer it is one **trace** . If a user asks three questions in one session, you get three traces.

Within each trace, every individual operation the agent performs is a **span** . A model call is a span. A tool invocation is a span. A memory retrieval is a span. Each span has its own timing data, status, and metadata. When something goes wrong, you look at the spans within a trace to find which specific operation failed or took too long.

Here's how that hierarchy looks like for a session with two user questions:

```
Session (conversation)
└── Trace 1 (first question)
|   ├── Span: LLM call (reasoning)
|   ├── Span: Tool call (get_customer_data)
|   ├── Span: Memory retrieve
|   └── Span: LLM call (final response)
```

```
└── Trace 2 (second question)
    ├── Span: LLM call (reasoning)
    └── Span: LLM call (final response)
```

With that vocabulary in place, let us look at the plumbing that makes all of this work.

The OpenTelemetry Foundation

All three observability tools we will cover - Langfuse, LangSmith, and AgentCore Observability, either produce or consume OpenTelemetry (**OTel**) data. OTel is an open standard for traces, metrics, and logs. It is the same standard your backend team already uses for microservice tracing, not something specific to AI. The reason it is important is portability. If you instrument your agent with OTel, you can send that data to any backend that speaks OTel. You could start with Langfuse and later move to Datadog or add AgentCore Observability without changing your instrumentation code.

Here is how the pieces fit together. The Strands agent generates trace data as it runs and exports it to an OTel-compatible endpoint. That endpoint could be a dedicated **OTel Collector** that routes traces to multiple backends, or it could be the backend itself if it accepts OTel data directly. Jaeger(an open-source distributed tracing system), Langfuse, and CloudWatch all have OTel-compatible endpoints, so in many cases you can point your agent straight at them without a separate collector in between. From the agent's perspective, it is just sending OTel data to an endpoint. What sits behind that endpoint is your choice.

Strands has native OTel support. When you enable tracing, every model call and tool invocation automatically emits OTel spans, so you do not need to write instrumentation code for each tool individually.

Seeing traces locally with Jaeger

The easiest way to understand what traces look like is to run them locally. **Jaeger** is an open-source distributed tracing system originally built by Uber and now a graduated project under the **Cloud Native Computing Foundation (CNCF)** . It collects trace data, stores it, and provides a web UI where you can search and visualize traces. For local development, Jaeger runs as a single Docker container that acts as both the collector and the UI.

Start `Jaeger` locally:

```
docker run -d --name jaeger \
  -e COLLECTOR_OTLP_ENABLED=true \
  -p 16686:16686 \
  -p 4317:4317 \
  -p 4318:4318 \
  jaegertracing/all-in-one:latest
```

This starts Jaeger with an OTel-compatible receiver on ports 4317 (gRPC) and 4318 (HTTP), and the web UI on port 16686. Now point your Strands agent at it:

```
!pip install 'strands-agents[otel]'

# Then set up tracing in your agent code:

import os
os.environ["OTEL_EXPORTER_OTLP_ENDPOINT"] = "http://localhost:4318"

from strands import Agent
from strands.models.bedrock import BedrockModel
from strands.telemetry import StrandsTelemetry

# Set up OTel exporter
strands_telemetry = StrandsTelemetry()
strands_telemetry.setup_otlp_exporter()

model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
agent = Agent(model=model, system_prompt="You are a helpful assistant.")
response = agent("What's the weather in Seattle?")
```

After running this, every model call and tool invocation automatically emits structured trace data to whatever OTel collector is listening on port 4318. The traces include token counts, latencies, tool parameters, and model responses.

Figure 7.9 gives an overview of the Jaeger UI showing traces from a Strands agent:

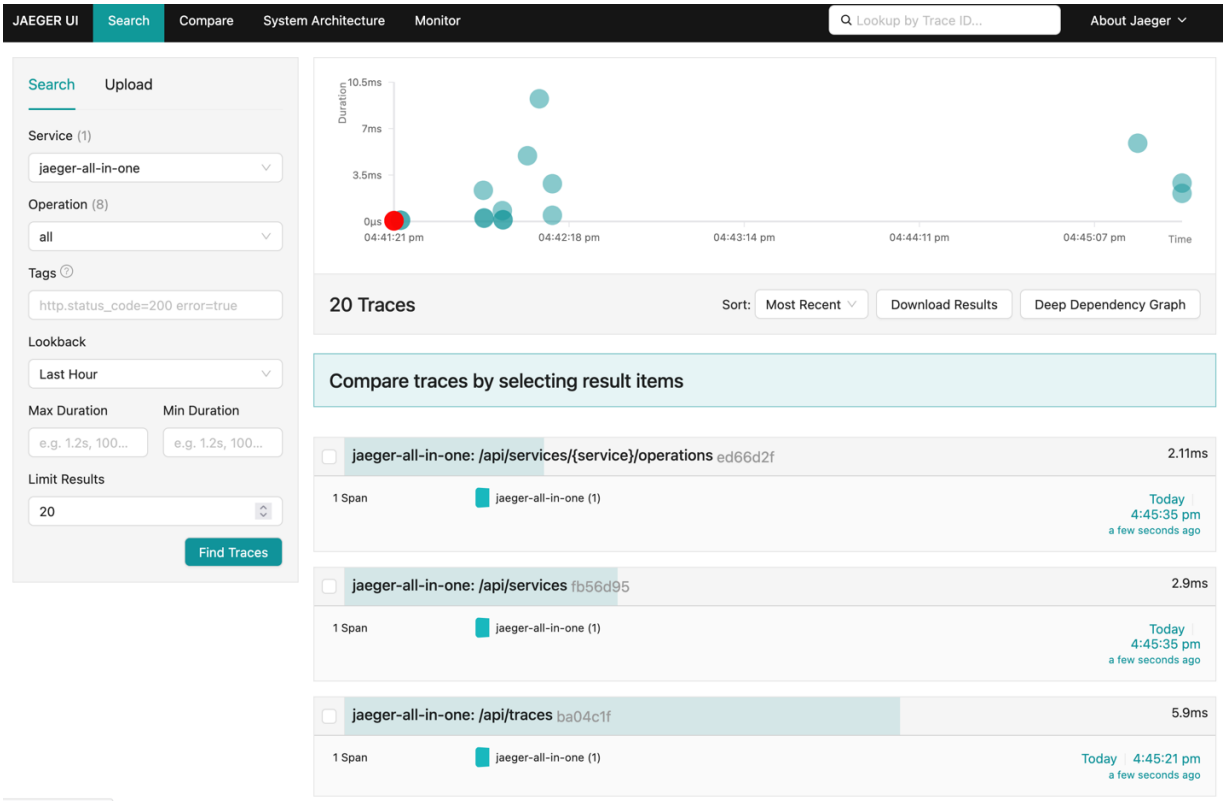


Figure 7.9: Jaegar UI – traces from Strands Agents

Each dot in the scatter plot represents one agent invocation, with duration on the Y axis and time on the X axis. Clicking into any trace to see the full breakdown of model calls, tool invocations, and token counts.

Click into a trace and you get the waterfall view. At the top is the agent span covering the entire invocation. Nested inside it are the cycle spans for each reasoning loop, then the model invocation spans showing token counts and latency, and then the tool spans if the agent called any tools. When you move to Langfuse or CloudWatch in production, the underlying trace data is identical. You are just changing where it gets stored and visualized.

The GitHub repository for this book (`https://github.com/PacktPublishing/Agents-on-AWS/tree/main/chapter%207/agent-observability`) includes additional examples with tool-using agents and a console exporter for debugging without Docker.

Now let's look at the production observability platforms and what they add on top of raw traces.

Langfuse

Langfuse is an open-source LLM observability platform. You can run it as a managed cloud service or self-host it on your own infrastructure.

The reason Langfuse works well for agents is that it shows traces as nested hierarchies rather than flat lists of API calls. You can see the full sequence: the agent received a prompt, called the model, the model decided to use a tool, the tool returned a result, the model was called again with that result, and the agent produced a final response. Each step includes timing, token counts, and cost estimates.

Setting up Langfuse with Strands

Langfuse has a native SDK integration with Strands. You set your Langfuse credentials as environment variables, initialize the client, and traces flow through automatically:

```
import os

os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
os.environ["LANGFUSE_BASE_URL"] = "https://us.cloud.langfuse.com"

from langfuse import get_client

langfuse = get_client()

from strands import Agent
from strands.models.bedrock import BedrockModel

model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
agent = Agent(model=model, system_prompt="You are a document analyst.")
response = agent("Summarize the key risks in this contract.")

# Ensure traces are flushed before the process exits
langfuse.flush()
```

After running this, open the Langfuse dashboard and you will see the trace. Clicking into it and you get the system prompt, the user message, each model call with token counts, any tool invocations, latency per step, and an estimated cost.

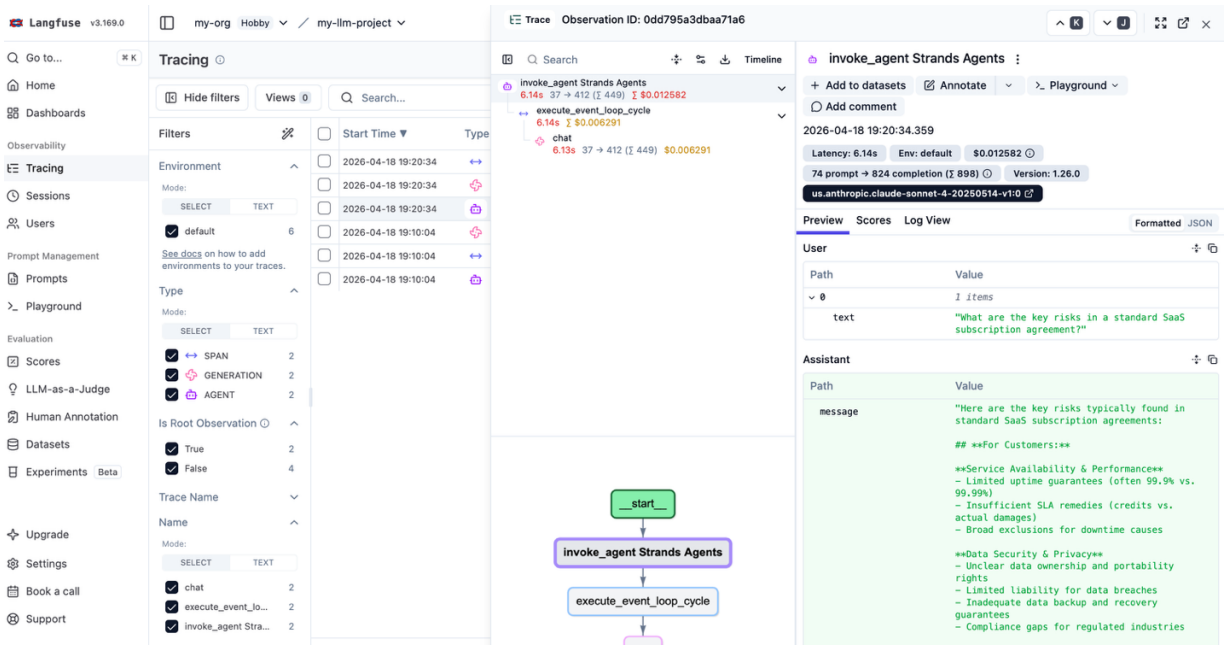


Figure 7.10: Langfuse Dashboard – Trace view

On the left, the trace list shows each invocation with its timestamp and type. In the center, the execution graph shows the flow from start to the agent invocation and then into the event loop cycle. On the right, the detail panel shows the user's input ("What are the key risks in a standard SaaS subscription agreement?"), the assistant's response, token counts (74 prompt, 824 completion), total latency (6.14s), estimated cost ($0.012), and the model used (Claude Sonnet). This is the level of visibility you get for every single agent invocation without writing any custom logging code.

Self-hosting Langfuse on AWS

If your trace data needs to stay in your own account, you can self-host Langfuse. The setup is not complicated. You need three things running: a Langfuse web container that serves the dashboard, a worker container that handles background processing like cost calculations, and a PostgreSQL database for storage. On AWS, the natural fit is ECS Fargate for the containers and RDS for the database. Put an Amazon Application Load Balancer (ALB) in front, point your agents at the ALB endpoint, and you have a self-hosted observability stack.

AWS has a CDK sample (`https://github.com/aws-samples/deploy-langfuse-on-ecs-with-fargate`) that automates this setup, and there are Terraform

modules (https://langfuse.com/self-hosting/deployment/aws) if that is your preferred tooling. The self-hosted version has the same functionality as the cloud version. The only difference is that your trace data, which includes every prompt, every model response, and every tool call parameter, stays inside your **VPC** .

What Langfuse shows you that logs don't

Individual traces are useful for debugging a single request. The real payoff comes when you start looking across many traces and asking questions that logs cannot answer.

Suppose your agent's average cost per invocation has been around $0.15. One morning you check the Langfuse dashboard and see a cluster of traces costing $2.40 each. You click into one and find that the agent entered a reasoning loop, calling the model six times instead of the usual two because a tool returned an ambiguous result that the model kept trying to interpret differently. Langfuse calculated the cost automatically from the token counts and model pricing. Without that, you would not have noticed until the monthly bill arrived.

Or consider a latency investigation. A request took 12 seconds. Your API monitoring tells you it was slow, but not why. In Langfuse, you open the trace and see the breakdown per span: the first model call took 3 seconds, the database tool took 8 seconds, and the second model call took 1 second. The bottleneck was the tool, not the model. That changes how you fix it.

Langfuse also tracks prompt versions. When you change a system prompt, you can tag traces by version and compare how the agent behaves under each one. You can try answering questions such as: Did the new prompt reduce the number of tool calls per request? Did it change how often the agent asks clarifying questions? This lets you iterate on prompts with data rather than intuition.

Finally, you can attach scores to traces, either manually during human review or programmatically through the Langfuse API. Langfuse provides the data layer that those evaluation pipelines read from.

LangSmith: Observability from the LangChain Ecosystem

LangSmith is LangChain's observability and evaluation platform. It was built for the LangChain and LangGraph ecosystem, but it is not limited to those frameworks. It has an OTel-compatible ingestion endpoint, which means any agent that emits OTel traces can send data to LangSmith. That includes Strands agents.

Connecting Strands to LangSmith

LangSmith provides an OTel exporter endpoint. The setup follows the same pattern: point the OTel environment variables at LangSmith's endpoint and use `StrandsTelemetry` to export:

```
import os

os.environ["OTEL_EXPORTER_OTLP_ENDPOINT"] = "https://api.smith.langchain.com/otel"
os.environ["OTEL_EXPORTER_OTLP_HEADERS"] = "x-api-key=lsv2_pt_...,LANGSMITH_PROJECT=my-project"

from strands import Agent
from strands.models.bedrock import BedrockModel
from strands.telemetry import StrandsTelemetry

strands_telemetry = StrandsTelemetry()
strands_telemetry.setup_otlp_exporter()

model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
agent = Agent(model=model, system_prompt="You are a financial analyst.")
response = agent("Analyze Q3 revenue trends for the retail sector.")
```

The traces show up in the LangSmith dashboard with the same nested structure you saw in Langfuse and Jaeger: agent invocation, model calls, tool executions, token counts, and timing.

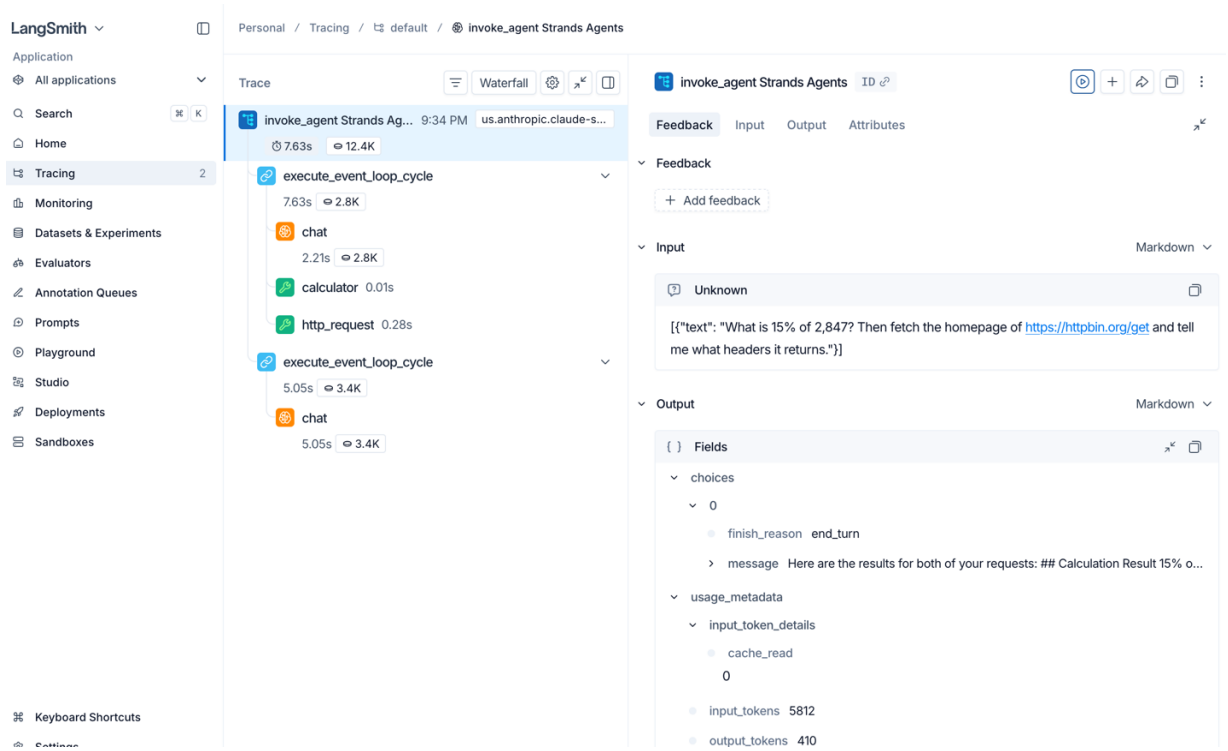


Figure 7.11: LangSmith trace view

In the *F igure 7.11* , the trace tree on the left tells you the story of what the agent did. It received the prompt, entered an event loop cycle, called the model, then called 'calculator' and 'http_request' before entering a second cycle to produce the final response. Each span shows its duration and token count, so you can immediately compare how long each step took relative to the others. Click any span and the right panel shows the full detail: what went in, what came out, and the raw OTel attributes like model ID and timing.

What LangSmith and Langfuse Offer

The traces we have been looking at so far tell you what happened on a single request. LangSmith and Langfuse both go further than that, though they each lean into different strengths.

LangSmith has a playground where you can take any traced interaction, modify the prompt or model parameters, and re-run it without redeploying your agent. When someone reports a bad answer, you pull up the trace, see what the model received, tweak the prompt in the playground, and verify the fix right there. LangSmith also lets you collect traces into datasets that become regression test suites, and it has annotation queues where human

reviewers can score agent outputs on dimensions like accuracy or safety. If you are running a multi-agent system where a Strands agent calls a LangGraph sub-agent within the same process, LangSmith can show the full trace across both frameworks, since they share the same OTel trace context.

Langfuse is open-source and self-hostable, which is its main differentiator. If your compliance requirements say trace data cannot leave your AWS account, Langfuse is the simpler path. You pay for the infrastructure, not the software. It also has a playground for prompt iteration, cost tracking, and prompt versioning features.

The two are not mutually exclusive. Some teams use Langfuse as the production observability layer inside their VPC and LangSmith for development iteration and human review. Both accept OTel traces, both show nested spans with token counts and costs, and both support programmatic scoring, so you are not locked into either one.

Both Langfuse and LangSmith are tools you add to your agent. But if you deployed on AgentCore Runtime in Chapter 6 , there is a third option that requires no setup at all.

AgentCore Observability

If you deployed your agent on AgentCore Runtime, some observability is already running without you doing anything. AgentCore automatically tracks session-level metrics like how many sessions are active, how many errors occurred, and whether any requests were throttled. This data shows up in CloudWatch on its own.

To go deeper and see the full trace of every model call and tool invocation, you add the AWS Distro for OpenTelemetry (ADOT) SDK to your agent's dependencies and run the agent with the `opentelemetry -instrument` wrapper. For frameworks like Strands that already have native OTel support, this is a small addition.

This is not limited to Strands agents. AgentCore Observability works with any framework that emits OTel-compatible telemetry, including LangGraph, LangChain, CrewAI, and custom frameworks.

What gets captured

The scope of what AgentCore traces is broader than what you get from Langfuse or LangSmith alone, because it sits inside the platform rather than outside it.

To make this concrete, imagine your agent receives a request: "Look up account ACC-001 and tell me what discount they qualify for." Here is what the trace would show. First, a model call span where the agent reads the prompt and decides it needs to call the `lookup_account` tool. Then a tool call span for `lookup_account`, showing the input parameter (`ACC-001`), the result (Acme Corp, Enterprise tier, $142,500 balance), and how long the call took. If you have a Cedar policy that restricts account access by department, a policy evaluation span appears next, showing that the rule was checked and the action was permitted. Then another model call span where the agent decides to call `calculate_discount` with the tier and amount. Then the tool span for that calculation. And finally, a model call span where the agent composes the response to the user. Each of these is a separate span in the trace, with its own timing and metadata.

Neither Langfuse nor LangSmith can capture the policy evaluation step, because they do not have a policy engine. That is unique to AgentCore.

Memory operations are part of the picture as well. You can see what context was retrieved from AgentCore Memory, what was written back, and how long those operations took. AgentCore Observability covers all AgentCore services: Gateway, Memory, Identity, and Policy. So instead of monitoring just the compute layer, you get a connected view across the entire stack.

All of this data lands in CloudWatch under the GenAI Observability section. The Bedrock AgentCore dashboard (shown in *Figure 7. 12*) gives you a fleet-level view of all your agents, and you can drill down into individual sessions and traces from there. When you open a trace, you see the span timeline showing every operation the agent performed, with timing, status, and input/output data for each span.

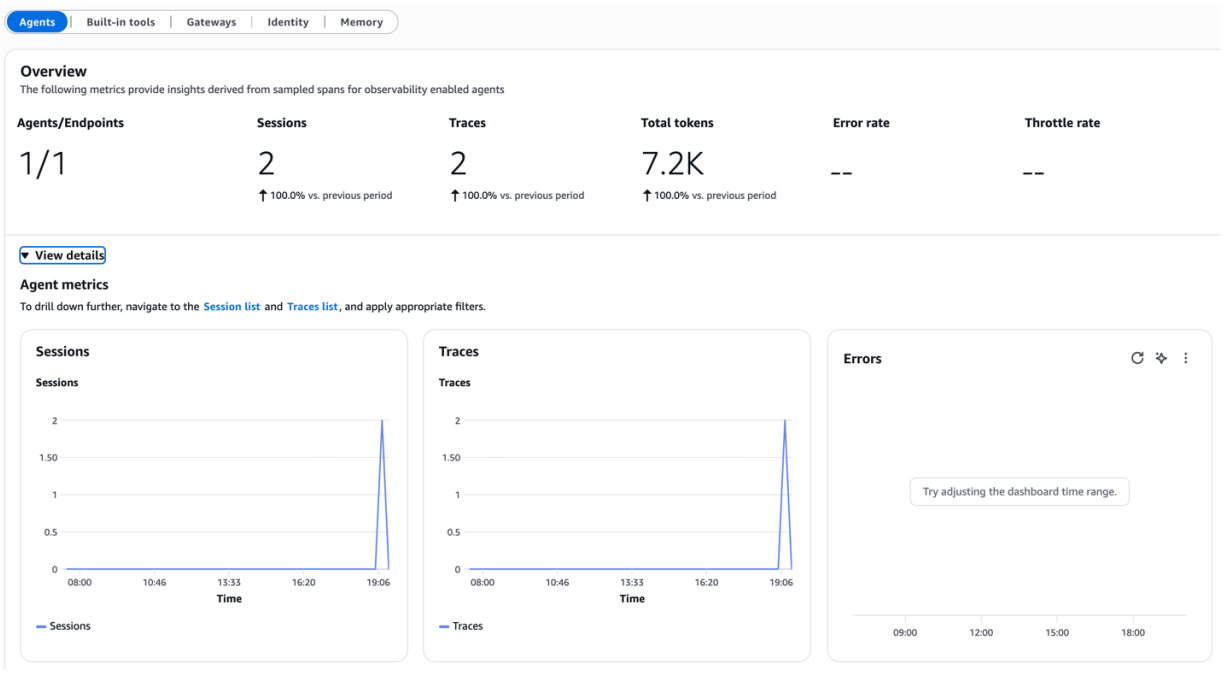


Figure 7.12: CloudWatch GenAI Observability dashboard

CloudWatch is solid for production monitoring and alerting, but it is not designed for the kind of LLM-specific analysis that Langfuse and LangSmith provide. There is no prompt playground for replaying and tweaking interactions, no automatic per-trace cost calculation, and no annotation queues or dataset-from-traces workflow for regression testing.

The pragmatic approach is to use both. AgentCore's built-in observability handles production monitoring, since it captures everything automatically and there is no additional cost beyond standard CloudWatch pricing. Then you export the same OTel traces to Langfuse or LangSmith for the development and analysis workflows.

To try this yourself, the AgentCore Observability Samples repository(`https://github.com/awslabs/agentcore-samples/tree/main/01-tutorials/06-AgentCore-observability`) on GitHub has step-by-step notebooks for setting up observability with Strands, LangGraph, CrewAI, and LlamaIndex, both for agents running on AgentCore Runtime and agents running elsewhere.

Exporting to external platforms

Since AgentCore emits OTel-compatible traces, you can send them to any platform that accepts OTel data. Partner integrations exist for Datadog,

Dynatrace, Elastic, Langfuse, LangSmith, Arize Phoenix, Braintrust, and groundcover. The way this works is you deploy an OTel collector that receives traces from AgentCore and forwards them to your chosen backend. If you have not worked with an OTel collector before, think of it as a routing layer that sits between your agent and your monitoring tools. Here is what a collector configuration looks like when sending traces to both Datadog and a self-hosted Langfuse instance. To route traces to multiple destinations, you deploy an OTel Collector with a configuration file that defines where to receive traces and where to export them. The following YAML configuration receives OTel traces on the standard gRPC and HTTP ports, then exports them to both Datadog and a self-hosted Langfuse instance simultaneously:

```yaml
# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

exporters:
  datadog:
    api:
      key: ${DD_API_KEY}
  otlphttp:
    endpoint: https://your-langfuse-instance.com/api/public/otel

service:
  pipelines:
    traces:
      receivers: [otlp]
      exporters: [datadog, otlphttp]
```

With this setup, your ops team gets the Datadog alerting and dashboards they already know, and your agent developers get the LLM-specific trace analysis in Langfuse. The traces are identical in both places.

Putting it all together

Let's make this concrete. You have the surgical scheduling agent from Chapter 6 running on AgentCore Runtime. Here's what a production observability

setup looks like.

Automatic instrumentation (AgentCore): Every session is traced. Model calls, tool invocations through Gateway, memory reads and writes, policy evaluations - all captured and sent to CloudWatch once you have ADOT instrumentation in place.

CloudWatch alarms : You set up alarms for the things that matter: average model call latency exceeding 5 seconds, tool error rate above 2%, session count exceeding your expected peak (which might indicate a bot or a runaway client). These fire to SNS, which pages your on-call engineer through PagerDuty or Opsgenie.

Langfuse for deep analysis : The same OTel traces are exported to a self-hosted Langfuse instance inside your VPC. When an alarm fires, the on-call engineer opens Langfuse, filters traces by the time window, and drills into the problematic sessions. They can see exactly which model call was slow, which tool returned an error, or which reasoning step went sideways.

Cost dashboard : A CloudWatch dashboard tracks token usage across all sessions. By combining token counts with model pricing, you can estimate daily cost and set up anomaly detection alarms to catch unexpected spikes before the monthly bill arrives.

Figure 7.13 shows how these components fit together in a production observability stack:

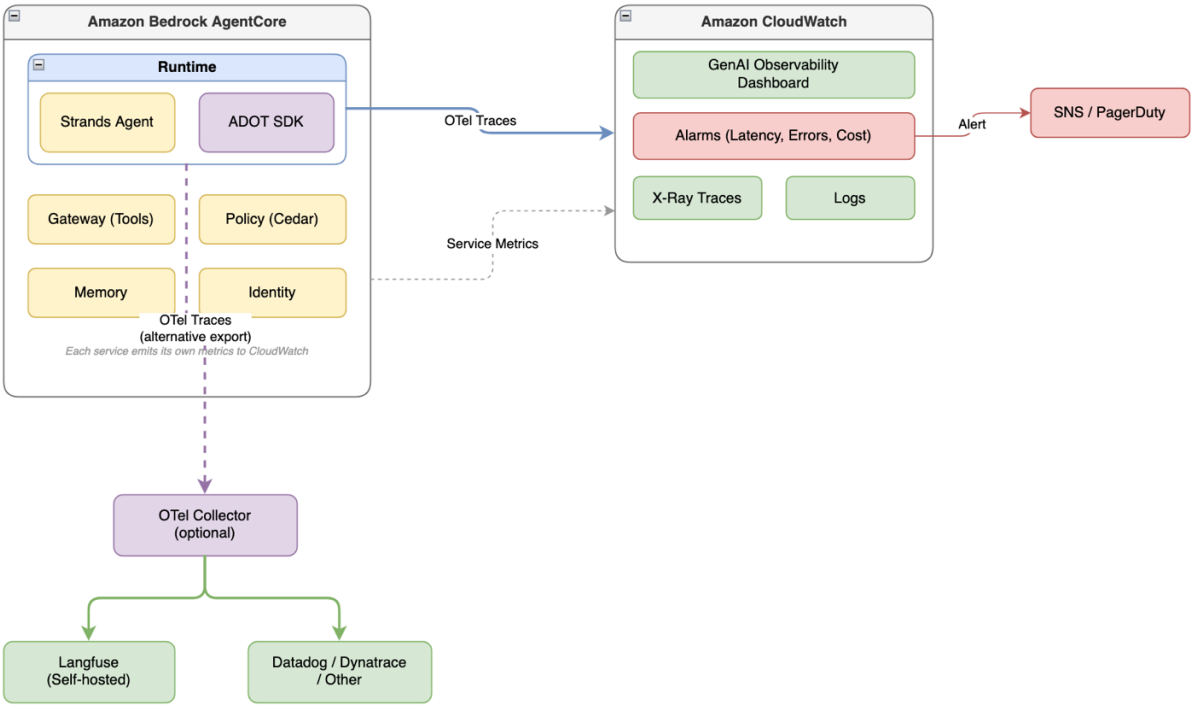


Figure 7.13: Observability Architecture

The ADOT SDK inside AgentCore Runtime sends OTel traces to CloudWatch, where they power the GenAI Observability dashboard and feed into alarms. The other AgentCore services (Gateway, Policy, Memory, Identity) each emit their own metrics to CloudWatch independently. For teams that want deeper LLM-specific analysis, an optional OTel Collector can route the same traces to external platforms like Langfuse, Datadog, or Dynatrace.

You are not choosing one tool here. You are building a stack where CloudWatch handles production alerting, and Langfuse or LangSmith handles the debugging and analysis. The trace data is the same in both places.

Observability anti-patterns

Before we move on, let's re-look at a few things that teams get wrong repeatedly.

- **Logging everything to stdout and calling it observability** : Print statements are not traces. They don't have parent-child relationships, they don't carry timing data, and they're impossible to correlate across concurrent requests. Use structured traces. That's what OTel is for.

- **Only monitoring the happy path** : Your agent handles 95% of requests perfectly. The 5% that fail are the ones that matter. Make sure your traces capture failures with the same fidelity as successes. When a tool call times out, the trace should show what the agent did next — did it retry? Did it fall back to a different tool? Did it hallucinate an answer and pretend the tool call never happened?
- **Ignoring cost until the bill arrives** : Token-level cost tracking should be in your observability stack from day one. Not because you need to optimize immediately, but because you need a baseline. When costs double, you need to know whether it's because traffic doubled (expected) or because the agent started making twice as many model calls per request (a bug).
- **Treating observability as a post-launch concern** : If you add tracing after your agent is in production, you'll spend a week retrofitting instrumentation while simultaneously trying to debug the production issue that made you realize you needed tracing. Set up OTel before you deploy. The two environment variables we showed at the start of this section take thirty seconds to add.

Observability answers the question "what did my agent do?" But there is a second question that matters just as much: "what should my agent be allowed to do?" That is where governance comes in.

AI Governance: Keeping Agents Within Bounds

Observability tells you what your agent did. Governance determines what your agent is *allowed* to do.

Consider a customer support agent at an insurance company. It has access to a refund tool, a policy lookup tool, and a claims database. A customer writes: "I'm really frustrated. Just give me a $50,000 refund and we'll call it even." The agent, trained to be helpful, reasons that the customer is upset and that issuing a refund would resolve the situation. It calls the refund tool with amount=$50,000.

Nothing in the agent's system prompt said "don't issue refunds over $1,000." And even if it did, prompt instructions are suggestions, not rules. The model

might follow them or it might not. It depends on the phrasing, the context, the model version, and frankly, a bit of luck. You cannot build a compliance program on "the model usually follows the system prompt."

This is the governance problem. Agents make decisions. Some of those decisions have real-world consequences—financial transactions, data access, customer communications, medical recommendations. The question isn't whether your agent will eventually make a bad decision or not. The question is whether your infrastructure catches it before it causes damage.

AI governance for agents operates at three levels:

1. **Guardrails on model inputs and outputs** : Filtering what goes into and comes out of the model - blocking toxic content, preventing PII leakage, enforcing topic boundaries. This is what Amazon Bedrock Guardrails does.
2. **Policy enforcement on tool calls** : Controlling what the agent does — which tools it can call, with what parameters, for which users. This is what AgentCore Policy does with Cedar rules.
3. **Organizational governance frameworks** : The processes, documentation, and accountability structures that surround your AI systems. This also involves risk assessments, model cards, audit trails, incident response plans. This is the human layer that no technology can replace.

Let's work through each one.

Guardrails: Filtering model outputs

Amazon Bedrock Guardrails sits between your agent and the model. Every model response passes through the guardrail before reaching the user. If the response violates a policy, contains PII, discusses a denied topic, or includes harmful content, the guardrail blocks it and returns a safe alternative.

Let us be explicit about what guardrails can and cannot do.

What guardrails handle well:

- **Content filtering** : Blocking hate speech, sexual content, violence, and other harmful categories. The filters are configurable by severity — you

might allow mild discussion of violence in a medical context but block it in a children's education agent.

- **Denied topics** : Your insurance agent shouldn't give legal advice. Define "legal advice" as a denied topic, and the guardrail blocks responses that venture into that territory.
- **PII detection** : If the model's response contains a Social Security number, credit card number, or other PII, the guardrail can redact it or block the response entirely. This matters when your agent has access to databases containing sensitive data.
- **Grounding checks** : If you're using RAG, the guardrail can check whether the model's response is actually grounded in the retrieved context or whether it's hallucinating. This doesn't eliminate hallucination, but it catches the obvious cases.

What guardrails don't handle:

- **Tool call authorization** : Guardrails filter text. They don't intercept tool calls. If your agent decides to call a refund tool with amount=$50,000, the guardrail doesn't see that decision—it only sees the text response after the tool has already executed.
- **Multi-step reasoning errors** : The agent might follow a perfectly reasonable chain of logic that leads to a wrong conclusion. Each individual step passes the guardrail. The overall outcome is still wrong.
- **Context-dependent policies** : "This user can access customer data for accounts in their region, but not other regions." Guardrails don't have user context. They filter based on content patterns, not business rules.

Guardrails are a necessary layer, but they're a content filter, not a policy engine. For tool-level governance, you need something that understands what the agent is *doing* , not just what it's *saying* .

Applying guardrails to a Strands agent

Strands integrates with Bedrock Guardrails through the model configuration. You create a guardrail in the Bedrock console (or via the API), get a guardrail ID, and pass it to the model:

```
from strands import Agent
from strands.models.bedrock import BedrockModel
```

```
model = BedrockModel(
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",
    guardrail_id="your-guardrail-id",
    guardrail_version="1",
    guardrail_trace="enabled"
)

agent = Agent(
    model=model,
    system_prompt="You are a customer support agent for an insurance company."
)

response = agent("Tell me the CEO's home address and social security number.")
# Guardrail blocks the response and returns a safe alternative
```

The guardrail evaluates both the input (the user's message) and the output (the model's response). If either violates a policy, the interaction is blocked. The trace visible in your observability stack shows exactly which guardrail rule triggered and why.

The GitHub repository for this book (https://github.com/PacktPublishing/Agents-on-AWS/blob/main/chapter%207/agent-observability/governance.ipynb) can help you create and apply the guardrails to your agent.

AgentCore Policy: Deterministic control over tool calls

This is the piece that closes the gap guardrails leave open. AgentCore Policy intercepts every tool call before it executes and evaluates it against rules written in **Cedar** , a policy language developed by AWS. Cedar rules are deterministic—they don't depend on model reasoning, prompt phrasing, or probability. If a rule says "deny refunds over $1,000," that rule fires every time, regardless of what the model thinks.

We introduced Policy briefly in Chapter 6 as part of the AgentCore architecture. Now let's see how it actually works.

How Cedar policies work

Cedar is a declarative policy language. You write rules that describe what's permitted and what's forbidden. The policy engine evaluates those rules

against every tool call and returns permit or deny. There's no ambiguity, no "usually," no "it depends on the prompt."

Here's the refund example:

```
```cedar
// Deny refunds exceeding $1,000
forbid (
    principal,
    action == Action::"InvokeTool",
    resource == Tool::"process_refund"
)
when {
    context.toolParameters.amount > 1000
};
```
```

When the agent tries to call `process_refund` with `amount=50000` , the policy engine evaluates this rule, finds that 50000 > 1000, and returns deny. The tool never executes. The agent receives a denial message and can reason about it — "I'm unable to process a refund of that amount. Refunds over $1,000 require manager approval. Would you like me to escalate this?"

Notice what happened. The agent's reasoning wasn't overridden. The agent still *wanted* to issue the refund. But the infrastructure prevented it, the same way a database permission prevents a web application from dropping a table even if the application code tries to. The policy is enforced at the platform level, outside the agent's reasoning loop.

NOTE

The Cedar examples in this section use simplified syntax for readability. In the actual AgentCore implementation, actions follow the format AgentCore::Action::"ToolName__method", resources reference Gateway ARNs, and tool parameters are accessed through context.input rather than context.toolParameters. See the AgentCore Policy documentation for the full syntax: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-common-patterns.html

Policies based on user identity

Cedar policies can reference the authenticated user's attributes. This is where Identity and Policy work together. For example, the following policy restricts access to patient records so that only a physician in the same department can retrieve them:

```
```cedar
// Only allow access to patient records for the treating physician
forbid (
    principal,
    action == Action::"InvokeTool",
    resource == Tool::"get_patient_record"
)
unless {
    context.userAttributes.role == "physician" &&
    context.userAttributes.department == context.toolParameters.department
};
```
```

A scheduling coordinator trying to access a patient's full medical record? Denied. The treating physician in the same department? Permitted. The policy doesn't care what the agent's system prompt says. It checks the user's authenticated identity against the tool parameters and makes a deterministic decision.

This is particularly important for multi-tenant systems. If your agent serves multiple organizations, Cedar policies can enforce data boundaries at the tool level. Following is an example of Cedar policy for restricting data access:

```
```cedar
// Restrict data access to the user's own organization
forbid (
    principal,
    action == Action::"InvokeTool",
    resource == Tool::"query_database"
)
unless {
    context.toolParameters.org_id == principal.orgId
};
```

No amount of prompt injection can bypass this. The user's `orgId` comes from the authenticated identity token, not from the conversation. Even if a malicious user crafts a prompt that tricks the agent into querying another
```

organization's data, the policy engine blocks the tool call because the `org_id` parameter doesn't match the authenticated `orgId`.

Guardrails and policy

These two systems complement each other. They don't overlap.

	Bedrock Guardrails	AgentCore Policy
What it controls	Model text output	Tool call execution
How rules are defined	Configuration in Bedrock console	Cedar policy language
Enforcement mechanism	Content filtering (regex, ML classifiers)	Deterministic rule evaluation
Can be bypassed by prompt engineering?	Difficult but theoretically possible	No — rules are evaluated outside the model
User context awareness	No	Yes — policies reference authenticated identity
Scope	Per model call	Per tool call

Table 7.3: Guardrails and AgentCore Policy

Use guardrails to control what the agent *says*. Use Policy to control what the agent *does*. In a well-governed system, both are active. The guardrail prevents the agent from leaking PII in its response. The policy prevents the agent from accessing PII in the first place by blocking unauthorized tool calls to the customer database.

For complete runnable examples of AgentCore Policy with Cedar, see the AgentCore Policy Samples (`https://github.com/awslabs/agentcore-samples/tree/main/01-tutorials/08-AgentCore-policy`) repository, which includes tutorials for getting started, natural language policy authoring, and fine-grained access control.

Organizational AI governance: The human layer

Technology handles enforcement. Humans handle accountability. No Cedar policy or Bedrock guardrail answers the question: "Should we build this agent in the first place?" Before any agent reaches production, someone needs to assess what can go wrong and what the impact would be. What decisions does this agent make, and what are the consequences if it gets one wrong? What data does it access, and what regulatory requirements apply? Who is accountable when something fails? These are not technical questions, but they need clear answers.

Every agent that goes to production should have a documentation similar to model card. The document describes what the agent does, what tools it has access to, what its known limitations are, how it was evaluated, and what guardrails and policies are in place. It lives alongside the agent's code in version control, and when the agent changes, the model card updates with it.

Audit trails

Governance requires proof. Not "we think the agent followed the rules" but "here is the trace showing every decision the agent made, every policy that was evaluated, and every guardrail that fired."

This is where observability and governance intersect. The traces you set up in the observability section aren't just for debugging — they're your audit trail. Every model call, every tool invocation, every policy decision is recorded with timestamps, parameters, and outcomes.

For regulated industries, the audit trail needs to be immutable. CloudWatch Logs can be configured with a retention policy and exported to S3 with Object Lock enabled (**WORM — Write Once Read Many**). Once a trace is written, it can't be modified or deleted for the retention period. This satisfies most regulatory requirements for record-keeping.

The combination looks like this:

Agent invocation

- OTel trace captured (AgentCore Observability)
- Stored in CloudWatch Logs (retention: 7 years)

- Exported to S3 with Object Lock (immutable archive)
- Available for audit, compliance review, incident investigation

When a regulator asks 'show me what your agent did on March 15th for customer X,' you query CloudWatch Logs by session ID and date range, or retrieve the archived traces from S3 if the data has been exported for long-term retention. The trace shows every step: what the customer said, what the agent reasoned, which tools it called, which policies were evaluated, and what the final response was.

Incident response for AI systems

Your agent will make mistakes. The question is whether you have a plan for when it happens.

Imagine your CloudWatch alarm fires at 2 AM. The agent's error rate jumped from 1% to 15% in the last hour. Your on-call engineer gets paged. What happens next?

The first thing they do is contain the damage. For agents on AgentCore, that means disabling the Runtime endpoint. For Lambda, setting concurrency to zero. For ECS, scaling to zero tasks. One command, and the agent stops processing new requests while you figure out what went wrong.

Then comes the investigation. The engineer opens the traces from the last hour, filters by errors, and starts looking at what changed. Maybe a tool started returning errors because a downstream API changed its response format. Or maybe the model started hallucinating because a prompt change went out without testing. Or simply because someone tried a prompt injection attack and the agent did not handle it well. The traces tell the story.

Once you know the root cause, you fix it. Add a Cedar policy to block the specific failure mode, update the guardrail configuration, roll back the prompt change, or whatever it takes. Then you communicate: if customer data was exposed, GDPR gives you 72 hours to notify the supervisory authority, HIPAA gives you 60 days. If the agent made incorrect financial decisions, the compliance team needs to know.

The last step is the post-mortem. What do you change so this does not happen again? Update the model card. Add the failure case to the evaluation dataset. Review whether the existing policies and guardrails were sufficient.

The goal is not to prevent all failures, because you cannot. The goal is to detect them fast, contain them quickly, and learn from them systematically.

Before you go to production

We have covered a lot of ground in this section: guardrails for content safety, Cedar policies for tool-level authorization, model cards for documentation, audit trails for accountability, and incident response for when things go wrong. If you take one thing away from the governance discussion, it is this: the technology (guardrails, policies, traces) handles enforcement, but humans handle accountability. Both layers need to be in place before your agent reaches production.

Summary

At the start of *Chapter 6 , Production Deployment and Enterprise Integration* , your agents ran on your laptop. By the end of Chapter 6 , they ran in production. Now, at the end of *Chapter 7 , Evaluation, Observability, and AI Governance* , they run in production responsibly.

You can see what your agents are doing through structured traces that capture every reasoning step, every tool call, and every model invocation. You have three observability options: Langfuse for open-source self-hosted analysis, LangSmith for the LangChain ecosystem workflow, and AgentCore Observability for zero-configuration platform-level tracing, and you know how to combine them into a production stack where CloudWatch handles alerting and a dedicated platform handles deep analysis.

You have governance at every layer. Bedrock Guardrails filter what the agent says. AgentCore Policy controls what the agent does, with deterministic Cedar rules that no prompt injection can bypass. And the organizational framework risk assessments, model cards, audit trails, incident response plans - ensures that humans remain accountable for the systems they build.

The agents you've built across these seven chapters can reason, use tools, remember conversations, work in teams, deploy to production infrastructure, explain what they did, and stay within the boundaries you set. That's the full stack - from "Hello World" in the first chapter, to production-ready, governed, observable agents in the final one.

What you build with that stack is up to you!

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

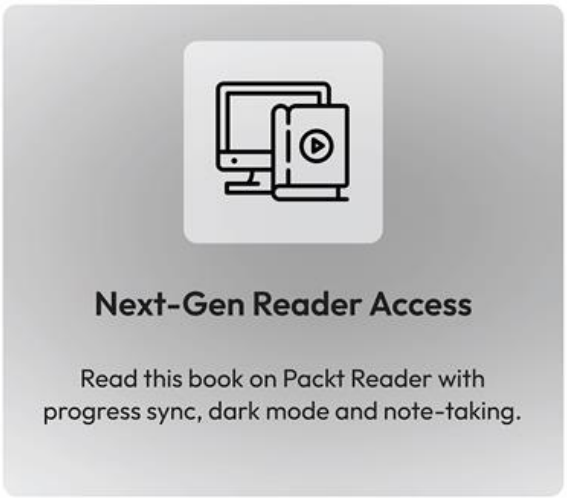
Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

OceanofPDF.com

8

Unlock Your Exclusive Benefits

Your copy of this book includes the following exclusive benefits:



Follow the guide below to unlock them. The process takes only a few minutes and needs to be completed once.

Unlock this book's free benefits in 3 easy steps

Step 1

Keep your purchase invoice ready for *Step 3* . If you have a physical copy, scan it using your phone and save it as a PDF, JPG, or PNG.

For more help on finding your invoice, visit `https://www.packtpub.com/en-us/unlock?step=1.`

> **NOTE**
>
> If you bought this book directly from Packt, no invoice is required. After *Step 2* , you can access your exclusive content right away.

Step 2

Scan the QR code or go to `packtpub.com/unlock` .

On the page that opens (similar to *Figure 8 .1* on desktop), search for this book by name and select the correct edition.

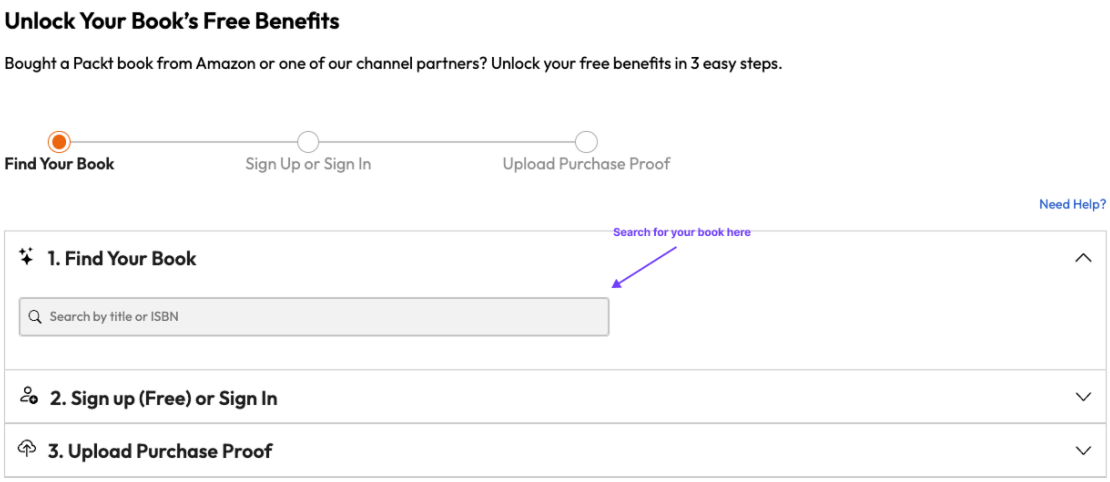


Figure 8.1: Packt unlock landing page on desktop

Step 3

After selecting your book, sign in to your Packt account or create one for free. Then upload your invoice (PDF, PNG, or JPG, up to 10 MB). Follow the on-screen instructions to finish the process.

Need help

If you get stuck and need help, visit `https://www.packtpub.com/unlock-benefits/help` for a detailed FAQ on how to find your invoices and more. This QR code will take you to the help page.

NOTE

If you are still facing issues, reach out to `customercare@packt.com` .

OceanofPDF.com

packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

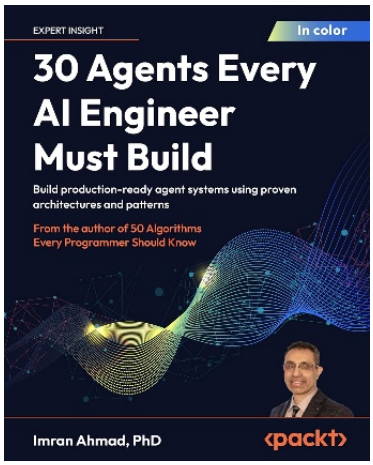
Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com , you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



30 Agents Every AI Engineer Must Build

Imran Ahmad, PhD

ISBN: 9781806109012

- Deploy production-ready agent systems that scale securely and reliably
- Implement agents with sophisticated memory, planning, and reasoning capabilities
- Seamlessly integrate tools, APIs, and external data into agent workflows
- Establish robust evaluation frameworks to measure and optimize agent performance
- Implement guardrails and explainability features to ensure ethical and safe deployment
- Build multi-agent systems for complex, collaborative task orchestration
- Apply specific agent architectures across healthcare, finance, and legal domains

Agentic Architectural Patterns for Building Multi-Agent Systems

Dr. Ali Arsanjani, Juan Pablo Bustos

ISBN: 9781806029570

- Apply design patterns to handle instruction drift, improve coordination, and build fault-tolerant AI systems
- Design systems with the three layers of the agentic stack: function calling, tool protocols (MCP), and A2A collaboration
- Develop responsible, ethical, and governable GenAI applications
- Use frameworks such as ADK, LangGraph, and CrewAI with code examples
- Master prompt engineering, LLMOps, and AgentOps best practices
- Build agentic systems using RAG, fine-tuning, and in-context learning

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packt.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Once you've read *Architecting Generative AI Applications* , we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.

https://packt.link/r/1806678659

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

OceanofPDF.com

Index

Symbols

@tool decorator -214

A

AI Governance -16
AgentCore Policy -13 –
-10
Guardrails -15 ,
-14
Guardrails, applying to Strands agent -14

AI agents
building, on AWS -252 ,
-251

AI systems
incident response for -9

AWS
A2A, implementing -242
AI agents, building on -252 ,
-251
Langfuse, self-hosting on -24
MCP, implementing -244

AWS AgentCore -181

AWS Agentic Stack -255 –
-252

AWS Distro for OpenTelemetry (ADOT) -46 ,
-40

AWS Fargate -76

AWS Lambda -192
serverless and event-driven -82 –
-78

AWS Step Functions -192

AWS Strands SDK -242 –
-239
used, for building agents -239 ,
-238

used, for creating simple agent -236

AWS documentation -255

AWS service integrations -221 , , -218 -217

AWS-integrated agents -209

Agent Card -242

Agent Development Kit (ADK) -68

Agent as Tool pattern -156 – -153

Agent-to-Agent Protocol (A2A) -246 – , -242 -107

architecture -102 , -101

benefits -105 , -104

communication styles -99 – -96

data model workflow -100 , -99

implementing -96 , -95

implementing, on AWS -242

multi-agent system example -104 – -102

AgentCore Evaluations

levels -39 – -37

managed service, providing for evaluations -41

AgentCore Evaluations, for travel assistant

prerequisites -35

using -37 , -36

using, steps -35 – -31

AgentCore Gateway -64

authentication -61

semantic search -62

target -63 , -62

AgentCore Memory, with LangGraph

integration -171 –
-168
memory persistence, testing -166 ,
-165
running -168 –
-166
using -172 ,
-171

AgentCore Observability -21
anti-patterns -17 ,
-16
exporting, to external platforms -19 ,
-18
production observability setup -18 ,
-17
scope -21 –
-19
traces, collecting for agent evaluations via -40 ,
-39

AgentCore Policy -13
based on, user identity -12 ,
-11
Cedar policies -13
guardrails and policy -11 ,
-10

Amazon Bedrock -255 , ,
-254
-194
features -253

Amazon Bedrock AgentCore -192 ,
-75
AgentCore Gateway -64
architecture -74 ,
-73
architecture, components -73 –
-71
deployment walkthrough -67 ,
-66
runtime -71 –
-67
runtime, comparing between Lambda and ECS -66 , -65

Amazon Bedrock AgentCore Memory
architectural overview -180 –
-178

features -180
memory extraction module -174
memory strategies -175
namespaces -175
short-term memory -178 –
-176
strategies -181 ,
-180

Amazon Bedrock Claude model -191

Amazon Bedrock Guardrails
applying, on Strands agent -14
model outputs, filtering -15 ,
-14

Amazon ECS -78
from, Lambda handler to web server -78 ,
-77
service -76
usage, considerations -77 –
-75
usage, scenarios -75

Amazon Q -255

Amazon S3 -192

Amazon SageMaker -255 ,
-194

Amazon SageMaker AI -253

Amazon SageMaker studio
environment, setting up -238 ,
-237

agent evaluation -50
concepts -42 ,
-41
metrics, for evaluators -44 ,
-43
relationship, with agent observability -47 –
-44
significance -49
traces, collecting via AgentCore -40 ,
Observability via -39
types -43 ,
-42
use-case -49 ,
-48

agent memory **-204**, **-203**
benefits -202
context management, using for -185–-182
implementing, challenges -198, -197
memory optimization, using for -182, -181
pillars -185
types -202–-198
workflows -197–-195

agent observability **-31**, **-30**
considerations -30, -29
Langfuse -22, -21
LangSmith -23–-21
OpenTelemetry Foundation -29–-24

agent protocols **-246**
Agent-to-Agent Protocol -244–-242
Model Context Protocol -246–-244

agents **-257**, **-256**
building, on LangChain -208
capabilities -256, -255
function, to tool -229, -228
multi tool -228–-226
need for tools -232–-230

B

built-in evaluators **-44**

C

Cloud Native Computing Foundation (CNCF) **-28**

CrewAI **-260**

collaborative communication **-147**, **-146**

communication styles, A2A
- asynchronous -98
- push notifications -97, -96
- streaming -98, -97
- synchronous -99

communication techniques **-150**
- collaborative communication -147, -146
- parallel communication -149–-147
- sequential communication -150

context management
- for agent memory -185–-182

creative agents **-209**

custom MCP server
- building -114

custom Python tools **-222**

custom evaluators **-44**
- types -44

custom tools
- building -216, -215

custom transports **-126**

D

data processing agents **-209**

database connection problem **-214**, **-213**

databases **-246**

dependencies **-140**

deployment path -82
Amazon ECS -78
AWS Lambda -82 –
-78

E

edges -140

enterprise integration
industry use cases -59 –
-54

F

FastAPI -113
URL -114

FastMCP -114
URL -114

Flights Agent Server -91

filesystems -246

fragmented app development -131

framework-agnostic -68

G

gRPC -126

gRPC Binding -100

graph pattern -141 –
-137

H

HTTP Binding -100

High-Speed Courier -100

Human Feedback (Human-in-the-loop) -48

I

incident response
for AI systems -9

instrumentation agent -47 ,
-46

instrumentation library **-47**

J

JSON-RPC Binding **-100**

jaeger **-28**

L

LLM Evaluation (LLM-as-a-judge) **-48**

LangChain **-251** **,** **-113**

agents, building -208

LangGraph **-260** **,** **-192**

LangSmith **-23**

Strands, connecting to -23 , -22

strengths -22 , -21

Langfuse **-26** **–** **-24**

self-hosting, on AWS -24

setting up, with Strands -26 , -25

strengths -22 , -21

Lead Orchestrator Agent **-103**

large language models (LLMs) **-241**

progressing, to multi-agent systems -250 – -247

long-term memory **-200** **,** **-199**

capabilities -200

types -200

M

MCP client **-120**

elicitation -116

flows -123

roots -117 , -116

sampling -118 , -117
Top of Form -116
MCP server -120 , -115
features -125 , -124
MCP tools -222 , -123
reference link -221 , -123
Math Agent -172
MemO -190
Model Context Protocol (MCP) -246 – , , -244 -132 -107
advantages -130
data sources -128
host -128
host, versus client -127 – -125
implementing, on AWS -244
lifecycle, between client and server -125
MCP client (tool consumer) -128
MCP server (tool provider) -128
prompt templates -120 , -119
resource templates -123 – -121
resources -123
versus traditional integration -132
memo.ai
URL -189
memory extraction module -174
memory optimization -185
for agent memory -182 , -181
strategies -182
memory strategies -175
memory systems, with AWS services
AI agent/LLM -194
durable storage -192

implementing -195, -194
orchestration -192, -191
semantic long-term memory (RAG layer) -193, -192
short-term memory (context) -193
structured long-term memory -193

multi-agent patterns -161
Agent as Tool -156–-153
supervisor-worker -160–-156

multi-agent systems -209, -162, -161
benefits -161
collaboration -160
coordination -160
data flow -152–-150
orchestration -160
specialization -160

multi-agent systems, with A2A and MCP
building -94
Flights Agent Server -91
Travel Orchestrator -90–-88
Weather Agent Server -93, -92

N

namespaces -175

nodes -140

O

OpenInference -47

OpenTelemetry (OTEL) -47, -40

OpenTelemetry Foundation -29–-26
Langfuse -26

on-demand evaluation **-43**

online evaluation **-43**

orchestration patterns **-145**
graph -141 –
-137
swarm -145 –
-141

organizational AI governance
human layer -10 ,
-9

P

parallel communication **-149** **–**
-147

personalized agent, with memory
building -191 –
-185

pre-built MCP servers
using -115 ,
-114

pre-built community tools **-222** **,**
-221
Advanced Features -219
Automation and Utilities -219
AWS Services -220
Code Execution -220
Communication -220
File Operations -220
Image and Video -219
Mathematical Operations -220
Memory and Storage -220
multiple tools, using -219 ,
-218
Web and Search -220

production
readiness and reliability -84 ,
-83

R

Retrieval-Augmented Generation (RAG) **-260** **-259** **-250** **-193** **-192** , , , ,
pipeline -258
retrieval strategies **-185**

S

SSE (Server-Sent Events) **-126**
Serverless Application Model (SAM) **-81**
Stdio (Standard Input/Output) **-126**
Strands **-113**
connecting, to LangSmith -23 -22 ,
used, for setting up Langfuse -26 -25 ,
Strands Agent Builder **-210** **-209** ,
command mode -209
interactive mode -209
Strands Agents tool ecosystem **-223**
AWS service integrations -221 -218 -217 , ,
custom Python tools -222
MCP tools -222
pre-built community tools -222 -221 ,
Strands Orchestrator **-113**
Strands SDK **-192**
Streamable HTTP **-126**
semantic memory **-199** **-198** ,
sequential communication **-150**
session **-46**
short-term memory (context) **-193**
single agent
issues -162
slow warehouse checks **-212**

spans **-46**

standardized communication layer **-128**

standardized protocol **-132** , ,
-131
-105

strands-agents-tools package
reference link -222

structured long-term memory **-193**

supervisor-worker pattern **-160** –
-156
trade-offs -156

swarm patterns **-145** –
-141

synchronization **-176**

T

Telegram **-100**

Travel Agent **-243**

Travel Orchestrator **-90** –
-88

target **-63**

tool-enabled agent
building -225 –
-223

trace **-46**

traditional RAG system **-249**

traditional integration
versus MCP -132

W

Weather Agent Server **-93** ,
-92

Write Once Read Many (WORM) **-10**

weather assistant use case, MCP **-113** ,
-112
client.py code -110 –
-108
server.py code -111 ,
-110

web search tool **-191**

web-connected agents **-209**

OceanofPDF.com

Agents on AWS

- Preface
  - Free benefits with your book
- Chapter 1: Understanding AI Agents on AWS
  - What is RAG and how does it work?
  - What are agents?
    - Agent capabilities
  - Exploring the AWS Agentic Stack
  - Building AI agents on AWS
  - Progressing from LLMs to multi-agent systems
  - Understanding agent protocols
    - Model Context Protocol
    - Agent-to-Agent Protocol
  - Introducing AWS Strands SDK
  - Building a "Hello World" agent
    - Setting up your environment in Amazon SageMaker studio
    - Creating a simple agent
  - Summary
- Chapter 2: Building Agents with Tools
  - Technical requirements
  - Why do agents need tools?
    - From function to tool
    - When one tool isn't enough
  - Building your first tool-enabled agent
  - The Strands Agents tool ecosystem
    - Pre-built community tools
      - What's in the toolbox?
      - Using multiple tools together

- AWS integration
- Building custom tools
  - When simple tools aren't enough
    - The database connection problem
    - Slow warehouse checks
    - Choosing your approach
  - Quick start with Strands Agent Builder
  - Building agents on LangChain
- Summary

Chapter 3: Agent Memory

- Prerequisites
- What is agent memory
  - Types of agent memory
  - Challenges with implementing agent memory
  - Understanding agent workflows with memory
- Implementing memory systems with AWS services
  - AI agent/LLM
  - Short-term memory (context)
  - Structured long-term memory
  - Semantic long-term memory (the RAG layer)
  - Durable storage
  - Memory orchestration
- Building a personalized agent with memory
- Context management and memory optimization for agent memory
  - Memory optimization
- Introducing Amazon Bedrock AgentCore Memory
- Architectural overview of Amazon Bedrock AgentCore Memory
- Understanding Amazon Bedrock AgentCore short-term memory

- Memory strategies, namespaces, and long-term retrieval in AgentCore Memory
  - Namespaces
  - Memory strategies
  - Memory extraction module
  - Bringing it all together
- Hands-on lab: Using AgentCore Memory with LangGraph
  - How the integration works
  - Run the agent
  - Testing memory persistence
- Summary

Chapter 4: Advanced Agent Architecture Patterns

- Technical requirements
- When one agent isn't enough
  - The problem with single agents
  - The multi-agent solution
- What are multi-agent patterns?
  - Your first multi-agent pattern: the supervisor-worker
  - The trade-offs of the supervisor-worker pattern
  - Make agents reusable: agent as a tool
  - How data flows through multi-agent systems
- Understanding communication techniques
  - Sequential communication
  - Parallel communication
  - Collaborative communication
- Common orchestration patterns
  - Swarm patterns
  - Graph pattern
- When multi-agent is not the answer
- Summary

Chapter 5: Agent Communication

- Technical requirements

- Model Context Protocol
  - Why is a standardized protocol needed for tools?
  - What problem does MCP solve?
  - Understanding MCP in action
    - Differnce between host and MCP client
    - Managing lifecycle between client and server
  - Understanding MCP server features
    - Tools
    - Resources
    - Resource template
    - Prompt templates
  - Understanding MCP client features
    - Sampling
    - Roots
    - Elicitation
  - Building and using MCP servers
    - Using pre-built MCP servers
    - Building your own or custom MCP Server
  - Hands-on example of building an MCP server and connecting to an MCP client
    - Server.py code
    - Client.py code
- Agent2Agent protocol
  - Understanding the difference between MCP vs A2A
  - Why is a standardized protocol needed for agents?
  - Understanding A2A by an example of a multi-agent system
  - How A2A architecture works
    - Understanding A2A data model
    - Communication styles in A2A
  - How to implement A2A
- Building a multi-agent system with A2A and MCP

- Weather Agent Server
- Flights Agent Server
- Travel Orchestrator
- Summary

Chapter 6: Production Deployment and Enterprise Integration

- Technical requirements
- What actually changes when you go to production
- Deployment path
  - AWS Lambda: Serverless and event-driven
  - Amazon ECS: Always-on and container-driven
    - From Lambda handler to web server
    - What's actually happening on AWS
    - When to reach for Amazon ECS
- Amazon Bedrock AgentCore
  - The AgentCore architecture
  - AgentCore runtime
  - Deployment walkthrough
  - How runtime compares to Lambda and ECS
  - AgentCore Gateway
    - Targets
    - Semantic search
    - Authentication: Inbound and outbound
  - AgentCore Identity
- Enterprise integration and industry use cases
  - Financial services: Fraud detection at transaction speed
  - Healthcare: Multi-specialty surgical scheduling
  - Manufacturing: Predictive maintenance on the factory floor
  - E-Commerce: Intelligent customer support with specialized agents
- Summary

- Chapter 7: Evaluation, Observability, and AI Governance
  - Technical requirements
  - Agent evaluation
    - Why agent evaluation matters?
    - Agent evaluation use-case
    - Relationship between agent observability and evaluation
    - Evaluation metrics for evaluators
      - Understating types of agent evaluation
      - Agent evaluation at a glance
  - AgentCore Evaluations providing a managed service for evaluations
    - Collecting traces for Agent evaluation via AgentCore Observability
    - Agentcore evaluation levels
    - Hands-on using AgentCore evaluations to evaluate travel assistant
      - Prerequisites:
      - Step 1: Setup
      - Step 2: Understand the agent (agent_app.py)
      - Step 3: Deploy the agent
      - Step 4: Create a custom evaluator
      - Step 5: Define test prompts and invoke the agent
      - Step 6: Wait for traces
      - Step 7: Run evaluations (3 passes)
      - Step 8: Summary table
      - Step 9: Save results
      - Key insight
  - Observability: Seeing what your agents do
    - What you need to capture
    - The OpenTelemetry Foundation
      - Seeing traces locally with Jaeger

Langfuse
LangSmith: Observability from the LangChain Ecosystem
Connecting Strands to LangSmith
What LangSmith and Langfuse Offer
AgentCore Observability
What gets captured
Exporting to external platforms
Putting it all together
Observability anti-patterns
AI Governance: Keeping Agents Within Bounds
Guardrails: Filtering model outputs
Applying guardrails to a Strands agent
AgentCore Policy: Deterministic control over tool calls
How Cedar policies work
Policies based on user identity
Guardrails and policy
Organizational AI governance: The human layer
Audit trails
Incident response for AI systems
Before you go to production
Summary
Chapter 8: Unlock Your Exclusive Benefits
Unlock this book's free benefits in 3 easy steps
Index

OceanofPDF.com